CCPS 209 Computer Science II Labs

Ilkka Kokkarinen

Chang School of Continuing Education
Toronto Metropolitan University

Version of October 17, 2026

Dramatis Personae

Lab 0(A): Plain Old Integers **11**
Lab 0(B): Dublications **14**
Lab 0(C): Reverse Verse **16**
Lab 0(D): Lists Of Integers **18**
Lab 0(E): All Integers Great And Small **20**
Lab 0(F): Two Branching Recursions **23**
Lab 0(G): …Or We Shall All Crash Separately **26**
Lab 0(H): More Integers Great And Small **31**
Lab 0(I): Squaring Off **33**
Lab 0(J): Similar Measures Of Dissimilarity **35**
Lab 0(K): Suffix Arrays **37**
Lab 0(L): I Can, Therefore I Must **39**
Lab 0(M): Tower Blocks **40**
Lab 0(N): Substring Parts **41**
Lab 0(O): Chars To The Left Of Me, Integers To The Right… **44**
Lab 0(P): Two Pointers **46**
Lab 0(Q): Aboutturn Is Playfair **49**
Lab 0(R): Frogs On A Box **52**
Lab 0(S): Hamming Center **54**
Lab 0(T): Clique Mentality **57**
Lab 1: Polynomial I: Basics **60**
Lab 2: Polynomial II: Arithmetic **63**
Lab 3: Polynomial III: Comparisons **64**
Lab 4: Extending An Existing Collection **66**
Lab 5: Biticut Follies **68**
Lab 6: Text Files I: Word Count **71**
Lab 7: Text Files II: Chasing Some Tail **74**
Lab 8: Concurrency In Animation **75**
Lab 9: Lissajous Curves **78**

Lab 10: Computation Streams **81**
Lab 11: Prime After Prime **82**
Lab 12: The Second Hand Unwinds **84**
Lab 13: Recursive Mondrian Art **86**
Lab 14: H-Tree Fractal **90**
Lab 15: Zeckendorf Encoding **93**
Lab 16: Egyptian Fractions **95**
Lab 17: Diamond Sequence **98**
Lab 18: Working Scale **101**
Lab 19: Symbolic Distances I: Representation **105**
Lab 20: Symbolic Distances II: Arithmetic **109**
Lab 21: Symbolic Distances III: Comparisons **111**
Lab 22: Truchet Tiles **113**
Lab 23: Ulam–Warburton Crystals **115**
Lab 24: Chips On Fire **120**
Lab 25: Manhattan Skyline **123**
Lab 26: FilterWriter **125**
Lab 27: Stacking Images **127**
Lab 28: All The Pretty Hues **129**
Lab 29: In a Timely Manner **132**
Lab 30: Mark And Rewind **135**
Lab 31: Region Quadtrees **137**
Lab 32: Triplefree Sequences **140**
Lab 33: Sardine Arrays **142**
Lab 34: Leaders of Men **144**
Lab 35: Multiple Winner Election **146**
Lab 36: Rational Roots **147**
Lab 37: Big Ten–Four **149**
Lab 38: Fenceposts Around Euclid's Orchard **151**
Lab 39: Hot Potato **154**
Lab 40: Seam Carving **156**
Lab 41: Geometry I: Segments **159**
Lab 42: Geometry II: Polygons **164**
Lab 43: Geometry III: Points In Polygons **167**
Lab 44: Geometry IV: Convex Hull **171**
Lab 45: Permutations I: Algebra **175**
Lab 46: Permutations II: Cycles **177**
Lab 47: Permutations III: Lehmer Codes **180**

Lab 48: The Curse Of The Clumpino **184**
Lab 49: Just A Minor Operation **187**
Lab 50: The Weight Of Power **192**
Lab 51: Twelve Angry Primes **197**
Lab 52: Linus Sequence **202**
Lab 53: Tchoukaillon **204**
Lab 54: All Hail The Mighty Xor **208**
Lab 55: Accumulated Wisdom **211**
Lab 56: find() Me A Find, catch Me A Catch **215**
Lab 57: Flood Fill **218**
Lab 58: Block Cellular Automata **221**
Lab 59: Hitomezashi Golygons **225**
Lab 60: Bit Deque **228**
Lab 61: Save The Date **233**
Lab 62: Worley Noise **238**
Lab 63: Recursive Subdivision Of Implicit Shapes **242**
Lab 64: Learning The Ropes I: Composition **246**
Lab 65: Learning The Ropes II: Comparisons **251**
Lab 66: Slater–Vélez Sequence **254**
Lab 67: Lemire Dynamic Frequency Sampling **256**
Lab 68: Interval Sets I: Dynamic Set Operations **259**
Lab 69: Interval Sets II: Iteration **264**
Lab 70: Cup Tournament Survival **267**
Lab 71: The Gamma And The Omega **270**
Lab 72: Lazy Generation Of Permutations **273**
Lab 73: Newton–Raphson Fractals **277**
Lab 74: Between The Lines **281**
Lab 75: Square Coral **284**
Lab 76: Sultan's Daughter **287**
Lab 77: Generalized Chaos Game **289**
Lab 78: Mind Your P's And Q's **292**
Lab 79: Die Hard With A Variance **294**
Bonus Lab 100: The Buddhabrot Is Silent **298**
Bonus Lab 101: Remain, Remain, Remainder **301**
Bonus Lab 102: Varshamov–Tenengolts Code **303**
Bonus Lab 103: In The End Was The Word **306**
Bonus Lab 104: Beklemishev's Worms **309**
Bonus Lab 105: Regex Fractals **312**

Bonus Lab 106: Automatic For The Peephole **314**
Bonus Lab 107: Inventory Sequence **319**
Bonus Lab 108: Piecewise Béziergons **321**
Bonus Lab 109: Soft Interruptions **324**
Bonus Lab 110: Twisty Little Passages, Reversible **327**
Bonus Lab 111: Dig-Through-Raster, Naked **331**
Bonus Lab 112: Kruskal Mazes **334**
Bonus Lab 113: Beautiful For Having Been Broken **337**

♬ If you're having code problems I feel bad for you son,
I got ninety-nine problems but unit tests ain't one. ♬

— Ilk-K, "99 Java problems"

As far back as I can remember, I always wanted to be a computer scientist. To me, being a computer scientist was better than being the President of Finland. Even before I first wandered into the department for a TA job I knew I wanted to be a part of them. It was a glorious time. Coders were all over the place. It was before the first Internet crash. There was Big Antti, Little Antti, Konsta, Jaco, Manu, Henri, Terhi, and me, Ilkka One-Time, who got that nickname because I always said everything once. You learned the two greatest things in life, never rat on your friend functions, and always keep your functions quiet. You know, when you think of a university, you get pictures in your mind of all those college comedies with rows and rows of students behind desks, doing real drinking and sexting all mixed up like Animal House. All those goody two-shoes writing their essays. It wasn't like that for code guys. We studied alone. We owned the joint. For us, to live any other way was nuts.

— Ilcolas Koleggi, "Codeguy"

For I am the random and the deterministic
I am the passing one and the failing one.
I am the non compiling one and many are my results.
I am the acceptance test and she who does not demo.
I am the slave of him who `fork`*ed me.*
I am the ruler of my stack and heap.
I am the control flow that is incomprehensible
And the design whose refactoring is frequent.

— "Asunder, Imperfect Code"

And the `cat` *purred, aye, she sees the procession of your functions and your modules.*
You shall be executing to all eternity, for as it is on compilation so it is in runtime.

— Ilar Khoyyam, "The Process Runner"

♬ And you may find yourself living in a local stack
And you may find yourself in another part of the heap
And you may find yourself behind the root of a large composite tree
And you may find yourself in a beautiful scope with a beautiful type
And you may ask yourself, well, how did I get declared? ♬

— Talking Headers, "Once in a variable's lifetime"

When the statements come, they are merely empty shells without the runtime. They live as they are executed, for the statements are the body and the runtime the spirit.

— Ildegard of Bongen

Outer loops have inner loops, on their bodies to bite them.
Inner loops have inner loops, and so on ad infinitum.
And the outer loops themselves, in turn, have outer loops to go on,
While those again have greater still, and greater still, and so on.

— Markustus De Korgan, "Bugonapthera"

♬ It's not the side effects of the pattern,
I'm thinking that it must be null,
It's too late to be stateful,
It's too late to pass time limit again,
It's too late to be resourceful,
Dependency inversion is here! ♬

— The Thin White Koke, "Kether to Malkuth"

♬ i, j, k are indices,
n and m fixed integers,
x, y, z for decimals,
u, v, w are nodes of graphs! ♬

— GySgt I. Kortman, "I don't code but source is gold"

Our lines no longer feel the editor under them.
At ten pages you can't read our comment.
And where there is enough to start linking
Kremlin test suite is remembered at once.

His fingers on keyboard are bulky and fat like live baits
And his accurate statements are heavy as weights.
The huge laughing declarations on top of his files
The glitter of his boot sequence.

But around his screen a crowd of thin applications
And he plays with the services of these half-utilities.
One compiles, other optimizes,
He is alone typing, poking and peeking.

He forges his statements like horseshoes
Slapping some in the header, in the class, in the program body.
He rolls the executions on his tongue like raspberries.
He wishes he could container them like big friend back home.

— Ilip Kokkelstam, "The Kremniya Epigram"

This document contains the lab specifications for the course CCPS 209 Computer Science II, as compiled and taught over the years by Ilkka Kokkarinen for the Chang School of Continuing Education, Toronto Metropolitan University, Canada. It offers a veritable buffet of **99 problem specifications**, followed by fourteen bonus problems, so students can choose their hills to rush and capture the flag on. These problems vary widely in theme and technique, so in the spirit of *The Love Boat*, they promise something for everyone. Students who previously took this author's first programming course, CCPS 109 Computer Science I taught in Python, should also note that, unlike the problem set used in that course, these problems are not listed in **order of difficulty**, but in the order in which they were created. Don't hesitate to look around this document to see which topics interest you enough to make you want to hit the keyboard!

These problems drill in various aspects of the Java language and its standard library, and introduce (although occasionally between the lines) many classic ideas, concepts, techniques, and algorithms that will surely come in handy in your future coding tasks in both school and working life. The first half of these labs mostly contains problems that straightforwardly apply basic data types such as integers, arrays, and lists. In contrast, the second half draws on discrete math, combinatorics, and computational geometry. Occasionally, these problems reach into more remote areas such as fractal art, cellular automata, and political science; the latter topic appears in problems about implementing and analyzing various **voting systems**.

To work on these labs using IntelliJ IDEA, create yourself a new project where all your source code files for these labs will go. This will be a different project than the one where you keep the example programs of this course. In your labs project, you will then separately copy the automated **JUnit tests** from the instructor's GitHub repository `CCPS209Labs` for each lab that you attempt to solve. First, download this entire folder somewhere safe on your computer, and whenever you embark on a new lab problem, copy its corresponding JUnit test class into your lab project folder. (There is no point having tests in your project for labs that you haven't even started.) Once you have implemented all the methods of the lab you are currently working on so that they compile, IntelliJ IDEA will allow you to run either one test or all the tests for that lab, for you to determine whether your work passes muster. (Unfinished methods can `throw` `UnsupportedOperationException` for the time being, until you stuff their bodies with the meat of actual logic and statements.)

When IntelliJ IDEA initially complains about the `import` statements in the JUnit test class, you should correct the problem once and for all by clicking the red light bulb symbol to open a drop-down menu to add JUnit 4 to your classpath. That is enough for IDEA to know to use JUnit 4 for all the tests in the same project. **Note that these tests are written in JUnit 4, instead of JUnit 5**, so be sure to add the correct framework to your classpath when prompted to do so by IDEA!

The **fuzz acceptance tests** in the JUnit test classes will try out your methods with a large number of **pseudorandomly** generated **test cases**. These test cases are guaranteed to be the same for everyone working on the Java language and its standard libraries, regardless of the computer environment they happen to be using. Each pseudorandom fuzz test computes a checksum from the results that your method returns for the test cases given to it. If this checksum equals the expected

checksum computed and hardcoded into the tester from the instructor's private model solution, The Man from Del Monte says yes and grants you the green checkmark to celebrate your beating this level with a nice glass of pineapple juice. Your method is now correct and accepted as **a black-box equivalent** to the instructor's private solution, all this testing taking place without ever making the private model solutions available for students.

Any discrepancy in the checksum reveals that your method returned a wrong answer for at least one test case. Unfortunately, this pseudorandom fuzzing scheme makes it impossible to pinpoint the actual test case where your method failed, or even to determine how many test cases your method disagrees with the instructor's private model solution. To alleviate this "something is wrong but I won't tell you what, you would know if you really loved me" mechanism that some psychologists might deem passive-aggressive were it done by a human, the JUnit test classes also feature conventional tests with explicit test cases aimed at flushing out the most common bugs and edge case situations that seem to repeat in student solutions.

All these JUnit fuzz tests are designed to run to completion in at most a couple of seconds per green checkmark on a modern, off-the-shelf desktop computer less than five years old. **None of these automated tests should ever require minutes, let alone hours, to complete.** If any JUnit test gets stuck that way when coupled with your code, that means **your methods are doing something algorithmically very inefficient**, either in time or in space. You should mercilessly root out all such inefficient designs from your method, and replace them with better algorithms that get to the same result at least an order of magnitude faster!

In all these labs, silence is golden. When it comes to talking out of school all over the text console, the rules for *omertà* are as strict as they are in the thorny badlands of Sicily. Since these JUnit tests will pummel your code with a large number of pseudo-randomly generated test cases harder than an old-time detective giving a suspect the third degree under the hot lights, **all required methods must be absolutely silent** and print absolutely nothing on the console during their execution. **Any lab solution that prints anything at all on the console during its JUnit test will be rejected and receive a zero mark.** Make sure to comment out all your console output statements you used for debugging before submitting your project.

Once you have completed all the labs that you think you are going to complete before the deadline, you will and must **submit all your completed labs in one swoop** as the entire "CCPS 209 Labs" project folder that contains all the source files along with the provided JUnit tests and any necessary text or image data files, compressed into a zip file that you then upload into the assignment tab on D2L. As there is no partial credit given for these labs, **please do not include any solutions that do not pass the tests with flying colors in your submission**. To make it easier for the instructor to tally up your working labs, you should make it clear in the project `README` file how many lab problems you are claiming to have successfully solved, so that the instructor doesn't have to count with his fingers and toes how many points to award you for your labs.

Students should solve these problems independently. Discussion of problems between students is acceptable, as long as it stays at the idea level and no solution code is passed between students. Two students who have both independently completed some problem may then compare and contrast their solutions with each other to the heart's content, and are recommended to do so if their solutions differ greatly in their execution efficiency. This problem collection also has an official subreddit r/ccps209 to facilitate such discussion. Note that this subreddit is intended only for the discussion of these problems, and any course management issues should be handled elsewhere through the appropriate channels.

Lab 0(A): Plain Old Integers

JUnit: P2J1Test.java

The transition labs numbered 0(X) translate your existing Python knowledge into equivalent Java knowledge, all the way between your brain and your fingertips. You may already be familiar with some of these problems from this author's earlier course CCPS 109 Computer Science I. Even if you are coming to this second programming course via some other route than this author's, these problems are self-contained and require no specific background knowledge.

But first: if you have trouble with IntelliJ IDEA in that it refuses to run your examples or lab code at all, the issue is most likely with the **Project** or **Module settings**. If the issue is that a Java Virtual Machine (JVM) has not been selected, or that the Java language level is unsupported, you can change these settings to ones that work with the instructions on the page "Module structure settings" in the IntelliJ IDEA documentation. All the examples and labs of this course can be run on Java version 15. (In fact, even the practically stone-age version 8 of Java would be sufficient for these labs, but Java 15 should be a reasonable universal baseline in Java programming in this day and age. We don't need any more modern or preview features from later versions of this language.)

Inside your fresh new lab project, create the first class that **must be named exactly `P2J1`.** If you name your classes and methods anything other than how they are specified in this document, the JUnit tests that we use to check and grade these labs automatically will not even be able to find your methods, let alone test them! Inside the new class, the first method to write is

```
public static long fallingPower(int n, int k)
```

Python has the built-in integer exponentiation operator **, whereas Java unfortunately does not. Exponentiation would be basically useless anyway in a language that uses fixed-size integers and silently hides the overflows that integer exponentiation will produce in spades. (You should also note that in both Python and Java, the **caret** character ^ denotes the **bitwise exclusive OR** operation that has nothing to do with integer exponentiation.)

Instead of integer exponentiation, your task is to implement the related operation of **falling power**, which is useful in many combinatorial formulas in discrete calculus. Distinguished syntactically from ordinary powers by underlining the exponent, each term multiplied into the product is always one less than the previous term. For example, the falling power $8^{\underline{3}}$ equals $8 \times 7 \times 6 = 336$. Similarly, the falling power $10^{\underline{5}}$ equals $10 \times 9 \times 8 \times 7 \times 6 = 30240$. Nothing essential changes when the base is negative; the falling power $(-4)^{\underline{5}}$ is computed with the same formula: $-4 \times -5 \times -6 \times -7 \times -8 = -6720$. Similar to normal exponentiation, $n^{\underline{0}} = 1$ for any nonzero integer n.

This method should return the falling power $n^{\underline{k}}$ where the base n can be any integer (positive, negative, or zero), and the exponent k can be any **nonnegative** integer, including zero. The JUnit fuzz tests are designed so that your method does not have to worry about potential integer

overflow… provided that you perform your arithmetic calculations with the `long` kind of 64-bit integers! If you use the bare 32-bit `int` type anywhere in your method, a silent integer **overflow** will make your method occasionally mess up its interim values and return incorrect results that fail the JUnit fuzz test. (Yes, even if that method worked when you tried it with your own small values of *n* and *k*.)

```
public static int[] everyOther(int[] arr)
```

Given an integer array `arr`, create and return a new array that contains precisely the elements in the even-numbered positions in the array `arr`. For example, given the array `{5, 2, 3, 10, 2}`, this method would create and return a new array object `{5, 3, 2}` that contains the elements in the even-numbered positions 0, 2, and 4 of the original array. Make sure that your method works correctly for arrays of both odd and even lengths, and for **empty** arrays and **singleton** arrays with just one element. The length of the result array that you return must also be exactly right so that no extra zeros are hanging around at the end of that result.

```
public static int[][] createZigZag(int rows, int cols, int start)
```

This method creates and returns a new two-dimensional integer array, which in Java is really just a one-dimensional array whose elements are one-dimensional arrays of type `int[]`. The returned array must have the correct number of `rows`, and each must have exactly `cols` columns. This array must contain the numbers `start`, `start+1`, …, `start+(rows*cols-1)` in its rows in sorted order, except that the elements in each odd-numbered row must be listed in descending order. Note that `start` can be any integer, even a negative one.

For example, when called with `rows=4`, `cols=5,` and `start=4`, this method should create and return the two-dimensional array whose contents show up as

```
4     5     6     7     8
13    12    11    10    9
14    15    16    17    18
23    22    21    20    19
```

When displayed in the standard **matrix form**, which is more readable than the more truthful form `{{4,5,6,7,8},{13,12,11,10,9},{14,15,16,17,18},{23,22,21,20,19}},` it is, in reality, a one-dimensional array whose four elements are themselves one-dimensional arrays of integers that serve as rows of this two-dimensional array. In addition to rectangular grids of values, this scheme can represent arbitrary **ragged arrays** whose rows don't all need to be the same length.

```
public static int countInversions(int[] arr)
```

Inside an array `arr`, an **inversion** is a pair of two positions `i<j` such that `arr[i]>arr[j]`. In combinatorics, the inversion count inside an array is a rough measure of how much "out of order"

that array is. If an array is sorted in ascending order, it has zero inversions, whereas an n-element array sorted in reverse order has $n(n - 1)/2$ inversions, the largest number possible. (You will encounter inversions again in this course if you work through the labs 45 to 47 that deal with **permutations** and their operations.) This method should count the inversions inside the given array `arr`, and return that count. As always when writing methods that operate on arrays, make sure that your method works correctly for arrays of any length, including the important special cases of zero and one.

Once you have written all four methods, add the above **JUnit test** class P2J1Test.java inside the same project. Unlike in Python with its more dynamic nature, **the JUnit test class cannot be compiled until your class contains all four methods exactly as they are specified.** If you want to test one method without also having to first write the other three, you must implement the other three methods as do-nothing **method stubs** that only contain some placeholder `return` statement that returns zero or some other convenient do-nothing value. An even better way is to give the method a one-line body.

```
throw new UnsupportedOperationException();
```

This body tells the caller, in a controlled way, that this method doesn't pretend to do anything and gives up right away. Of course, all such method stubs will immediately fail their respective tests, as they properly should. However, their existence lets the JUnit test class compile and run cleanly, so you can start testing each method as soon as you replace the above statement in its body with the actual Java code to solve the specified problem. IntelliJ IDEA conveniently allows you to run all tests for that problem, or just one individual test, just by clicking the green Play button next to each test in its source code.

Lab 0(B): Dublications

JUnit: P2J2Test.java

Create a new class named `P2J2` inside the very same project as you placed your `P2J1` class in the previous lab. Inside this new class `P2J2`, write the following four methods.

```
public static String removeDuplicates(String text)
```

Given a `text` string, create and return a brand new string that is otherwise the same as `text` except that every run of equal consecutive characters has been turned into a single character. For example, given the arguments `"Kokkarinen"` and `"aaaabbxxxxaaxa"`, this method would return `"Kokarinen"` and `"abxaxa"`, respectively. It removes only consecutive duplicate characters, but keeps later occurrences of the same character if another character appears between them. For this problem, uppercase and lowercase versions of the same character are considered different, so removing duplicates from `"AaabbBBBCc"` should return `"AabBCc"`.

```
public static String uniqueCharacters(String text)
```

Create and return a new string that contains each character of `text` only once, regardless of its appearance count there; these characters should be listed so that they first appear in the original string. For example, given the argument strings `"Kokkarinen"` and `"aaaabbxxAxxaaxa"`, this method should return `"Kokarine"` and `"abxA"`, respectively. As in the previous problem, uppercase and lowercase versions of the same character are treated as different characters.

You could solve this problem with two nested loops: the outer loop iterates through the text, and the inner loop checks all previous positions for a previous occurrence of that character. But you can be more efficient and use a `HashSet<Character>` to remember which characters you have already seen, and let that data structure help you decide in constant time whether the next character is appended to the result.

```
public static int countSafeSquaresRooks(int n, boolean[][] rooks)
```

Several chess rooks are placed on squares of a generalized `n`-by-`n` chessboard. The two-dimensional boolean array `rooks` tells you which squares contain a rook. (This array is guaranteed to be exactly `n`-by-`n` in its dimensions.) This method should return the count of remaining squares safe from the rolling wrath of these rampaging rooks, that is, squares that do not share a row or column with any rook.

Probably the best way to do this is to create and fill in two one-dimensional `boolean` arrays named `safeRows` and `safeCols` with `n` elements each. These arrays keep track of which rows and columns of the chessboard are unsafe. Initially, all rows and columns are safe, since both arrays are initially

all `false`. However, to get the actual count for the given chessboard, loop through all the squares. Whenever you find a rook in some position `(row, col)`, mark that entire `row` and `col` as being unsafe in those respective arrays. After you have done this for all the squares throughout the entire chessboard, loop through the `safeRows` and `safeCols` arrays separately to count how many rows and columns are still safe. Return the product of those two numbers as your final answer.

Probably the best way to do this is to create and fill in two one-dimensional `boolean` arrays named `safeRows` and `safeCols` with `n` elements each. These arrays keep track of which rows and columns of the chessboard are unsafe. Initially, all rows and columns are safe, since both arrays are initially all `false`. However, to get the actual count for the given chessboard, loop through all the squares. Whenever you find a rook in some position `(row, col)`, mark that entire `row` and `col` as being unsafe in those respective arrays. After you have done this for all the squares throughout the entire chessboard, loop through the `safeRows` and `safeCols` arrays separately to count how many rows and columns are still safe. Return the product of those two numbers as your final answer.

```
public static int recaman(int n)
```

Compute and return the *n*th term in the Recamán sequence, starting the count from term $a_1 = 1$. See the definition of this curious sequence on the linked Wolfram MathWorld page, or read more from the page "Asymptotics of Recaman's sequence". For example, when called with $n = 7$, this method returns 20, and when called with $n = 19$, it returns 62. (More values for debugging purposes are listed at OEIS sequence A005132.)

To make your function fast and efficient even when computing the sequence element for large values of *n*, you should use a sufficiently large `boolean[]` to keep track of which integer values are already part of the generated sequence, so that you can generate each element in constant time instead of having to loop through the entire previously generated sequence, the way some "Shlemiel" would do, tiring himself by running back and forth across the same positions. A size `10*n` array should be enough. This way, this scheme needs only ten measly bits of heap memory per generated number, plus some spare change.

Storing the numbers encountered so far in the generated sequence in a `HashSet<Integer>` instance would be a textbook case of overpaying for overkill. It would burn precious memory, at least an order of magnitude more, than using a simple **bit vector** to remember which values you have already encountered.

Lab 0(C): Reverse Verse

JUnit: P2J3Test.java

Here is one more batch of transitional problems adapted from the instructor's old collection of Python graded labs. Only three methods to write this time, though. The JUnit test for these labs uses the `warandpeace.txt` text file as its data source, so you should ensure that this text file has been properly copied inside your course project root directory. Unless you have changed this setting in the IntelliJ IDEA project settings, the working directory is the root directory of your project, not the `/src` directory, so this text file should be in this root directory.

Create the class named `P2J3` in your project folder, in which you will be writing three `static` methods. The first of these three methods to write is

```
public static void reverseAscendingSubarrays(int[] items)
```

Rearrange the elements of the given array of integers **in place** (that is, **do not create and return a new array**) so that the elements of every **maximal strictly ascending subarray** are reversed. For example, given the array `{5, 7, 10, 4, 2, 7, 8, 1, 3}` (the pretty colors indicate the ascending subarrays and are not actually part of the argument), after executing this method, the elements of the array would be `{10, 7, 5, 4, 8, 7, 2, 3, 1}`. Given another argument array `{5, 4, 3, 2, 1}`, the contents would stay `{5, 4, 3, 2, 1}`, since each element is a maximal ascending subarray of length one.

The second method to write in the same `P2J3` class is

```
public static String pancakeScramble(String text)
```

This nifty little problem is taken from the excellent Wolfram Challenges problem site, where you can also see examples of what the result should be for various arguments. Given a `text` string, construct a new string by reversing its first two characters, then reversing the first three characters of that, and so on, until the last round where you reverse your entire current string.

This problem is an exercise in Java string manipulation. For some mysterious reason, even today, the Java `String` type still doesn't come with a built-in reverse method. The canonical way to reverse a `String` object is first to convert it to a mutable `StringBuilder`, reverse the contents of that in place, and convert the result back to an immutable string. That is,

```
text = new StringBuilder(text).reverse().toString();
```

A bit convoluted, but does what is needed without fuss or muss. Maybe one day Java strings will have a `reverse` method, like string data types in all sensible programming languages these days. Or at least have it available as a utility somewhere in the standard library.

To reach the finish line of this lab, the third method to write inside this same class is

```
public static String reverseVowels(String text)
```

Given a `text` string, create and return a new string of the same length where all vowels have been reversed, and all other characters are kept as they were. For simplicity, in this problem, only the characters `aeiouAEIOU` are considered vowels, and `y` is never a vowel. For example, given the text string `"Uncle Sente lives in Russia"`, this method should create and return the string `"Ancli Sunti levis en Resseu"`.

Furthermore, to make this problem more interesting and the result more palatable, this method must maintain vowel capitalization based on the vowel character originally in the position each new vowel moves into. For example, `"Ilkka Markus"` should become `"Ulkka Markis"` instead of `"ulkka MarkIs"`. Use the handy character classification methods in the `Character` utility class to determine whether some particular character is in upper or lower case, and convert some character to its upper or lowercase version as needed.

As a leisurely mental exercise after completing this method, the reader can try to think up the longest meaningful sentence that doesn't change when vowels are reversed, or find the longest meaningful sentence whose vowel reversal is another meaningful sentence. An equally obvious follow-up is to write a function to reverse the consonants in the sentence, and perhaps use the resulting mess as an artificial language in your swords-and-cloaks fantasy universe.

Lab 0(D): Lists Of Integers

JUnit: P2J4Test.java

This fourth transition lab from Python to Java contains four more interesting problems taken from the graded labs of the instructor's Python course. The four `static` methods to write this time all deal with the `List<Integer>` data type of the Java standard library (remember to put the necessary imports at the top of the source code of your class), whose behavior and operations are essentially identical to those of ordinary Python lists of integers, except with a way more annoying syntax.

Create a class P2J4 in your lab project directory, and there, your first method

```
public static List<Integer> runningMedianOfThree(List<Integer> items)
```

Create and return a new `List<Integer>` instance (use any concrete subtype of `List` of your choice) whose first two elements are the same as those of the original `items` (so the result for any list with fewer than three elements will be the same as that list), after which each element equals the **median** of the three elements in the original list ending in that position. For example, when called with a list that prints out as `[5, 2, 9, 1, 7, 4, 6, 3, 8]`, this method would return an object of type `List<Integer>` that prints out as `[5, 2, 5, 2, 7, 4, 6, 4, 6]`. The second method to write in this class is

```
public static int firstMissingPositive(List<Integer> items)
```

Given a list whose each element is a positive integer, return the first positive integer missing from this list. For example, given a list `[7, 5, 2, 3, 10, 2, 9999999, 4, 6, 3, 1, 9, 2]`, this method should return 8. Given one of the lists `[]`, `[6, 2, 12345678]`, or `[42]`, this method should return 1.

If your method compares the `items` to each other for equality, you must use the `equals` method to do so instead of the `==` operator, since these `items` are all `Integer` objects, not primitive integers. However, since the **pigeonhole principle** dictates that the first missing positive integer in an n-element list must be less than or equal to n + 1, a boolean array of that size is a better way to track which numbers you have seen so far. With it, you can solve this problem efficiently in both time and space with two consecutive for-loops, instead of being a Shlemiel who does the same job with two **nested** for-loops. To minimize runtime, these two loop approaches are not equal at all!

The third method out of the four methods to write is

```
public static void sortByElementFrequency(List<Integer> items)
```

Sort the elements of the given list in **decreasing order of frequency**, that is, how many times each element appears in this list. If two elements have the same frequency in the parameter list, those elements should end up in **ascending order of values**, the same way as we do in ordinary sorting of lists of integers. Note that this method does not return anything, but modifies `items` in place.

For example, given a list object that prints out as `[4, 99999, 2, 2, 99999, 4, 4, 4]`, after calling this method, that list object would print out as `[4, 4, 4, 4, 2, 2, 99999, 99999]`. The return type is `void` because this method rearranges the elements of the `items` list in place instead of creating and returning a separate result list.

As in all computer programming, you should let the language and its standard library do the work for you instead of rolling your own logic. The method `Collections.sort` can be given a `Comparator<Integer>` strategy object that compares two integers for their ordering. Start by building a local **counter map** of type `Map<Integer, Integer>` used to keep track of how many times each value that you see appears inside the list. Next, define a local class that implements `Comparator<Integer>` to create strategy objects whose `compare` method performs integer order comparisons by consulting this map for the counts of those elements to compute the answer, reverting to ordinary integer order comparison only in the case where these counts are equal. Once you have defined this local class, the actual sorting of `items` takes place with just a single call to the utility method `Collections.sort`.

The fourth method that takes you across the finish line of this lab is

```
public static List<Integer> factorFactorial(int n)
```

Return the list of prime factors of the **factorial** of `n` (that is, the product of all positive integers up to `n`), with those prime factors sorted in ascending order and with each factor appearing in this list exactly as many times as it would appear in the prime factorization of that factorial. For example, for `n=10`, this method returns the list `[2, 2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 5, 5, 7]`. Multiplying all those itty-bitty prime numbers gives 3,628,800, the product of the first ten positive integers. Remember that when `n` equals either zero or one, its factorial equals one (since there exists exactly one way to arrange the elements of the empty list and the singleton list in a row), and therefore produces an empty list of prime factors.

The factorial `n!` grows exponentially, so when `n` goes up to not just eleven but even past that mythical barrier, these factorials would no longer fit within the 31 bits used to store the magnitude of Java's primitive int type. Since this method must work for values of `n` in the thousands, and their factorials have tens of thousands of digits, you should not compute the entire factorial first and then break the resulting behemoth down into prime factors. Instead, you need to build up the list of prime factors as you go, by appending the prime factors of the integer that you are currently multiplying into the factorial. When you have done this for all integers up to `n`, sort the result list in ascending order before returning it.

Lab 0(E): All Integers Great And Small

JUnit: P2J5Test.java

Python integers are **unbounded,** so their magnitude is limited only by the available heap memory. The Python virtual machine silently switches between efficient representations for small and large integers so that we can focus on the problem instead of the nitty-gritty details of integer arithmetic. When using Java's primitive integer types, we have to worry about and guard against overflow errors all the time. (This is typically achieved by closing your eyes and ingesting some hopium that nobody will ever pass your method arguments big enough to make some intermediate result overflow.)

The utility class `java.math.BigInteger` allows you to do computations in Java with similarly unlimited integers. This is Java as originally conceived in the early nineties, so the syntax isn't quite as nice as the ordinary, natural syntax for the primitive `int` type. For example, to add two such integers `a` and `b`, we have to say `a.add(b)` instead of `a+b`, almost as if our nation had lost a war or something. (Explanations for why Java, even in this blessed day and age, still doesn't have **operator overloading** make for a good read for all those times that you could use a chuckle.) Consult the rest of the methods of `BigInteger` as needed to solve the following problems.

This lab has only methods to complete in class, named `P2J5`. The first one is

```
public static List<BigInteger> fibonacciSum(BigInteger n)
```

This method computes Fibonacci numbers; that dusty old example of silly recursion still used in many introductory computer science courses has interesting combinatorial properties that give us more meaningful problems to play with. In this lab, we look at Zeckendorf's theorem, which proves that any positive integer n can be written in exactly one way as a sum of distinct Fibonacci numbers, with the constraint that no two consecutive Fibonacci numbers appear in the sum. After all, if the sum contains two consecutive Fibonacci numbers F_i and F_{i+1}, you can always replace them with Fi+2 without changing the total. (To ensure that you have at most one of each F_i at any moment, this conversion should be performed from higher indices down.)

You can construct the unique breakdown of `n` into Fibonacci numbers with a **greedy algorithm** that finds the largest Fibonacci number f less than or equal to `n`. Add this `f` to the result list, and break down the rest of the number `n-f` the same way. This method should create and return an instance of some subtype of `List<BigInteger>` that contains the selected Fibonacci numbers in **descending order**. For example, when called with `n=1000000`, this method would return the list `[832040, 121393, 46368, 144, 55]`.

Your method must remain efficient even if `n` contains thousands of digits. To achieve this, maintain a `static` list of Fibonacci numbers that you have generated so far, initialised with

```
private static List<BigInteger> fibs = new ArrayList<>();
static { fibs.add(BigInteger.ONE); fibs.add(BigInteger.ONE); }
```

Whenever the last Fibonacci number stored in this list is not big enough for your current needs, extend the list with the next Fibonacci number, computed by adding up the last two known Fibonacci numbers. Even though this is not needed in this problem, keeping the Fibonacci numbers discovered so far in one sorted list would also allow you to quickly determine whether the given integer is some Fibonacci number with `Collections.binarySearch`.

The second method to implement in this lab is

```
public static BigInteger sevenZero(int n)
```

Since seven is a lucky number in Western culture, and zero is what nobody wants to be, let us briefly bring these two opposites together and look at positive integers that consist of a sequence of sevens followed by a (possibly empty) sequence of zeros. For example, 0, 7, 77777, 7700000, 77777700, or 70000000000000000000000000000000000. All the sevens must come first in one consecutive bunch, followed by all the zeros in another consecutive bunch, the second bunch being possibly empty. This rule keeps the set of such numbers small enough to examine by brute force, since there are only *n* such numbers with *n* digits.

In the wonderful MIT OpenCourseWare online textbook "*Mathematics for Computer Science*" (PDF link to the 2018 version, for anybody interested in that sort of stuff), one of the examples of **non-constructive proofs** used the pigeonhole principle to prove that for any positive integer `n`, there exists some seven-zero integer that is divisible by `n`. This integer, made of sevens and zeroes, can quickly become gargantuan even for relatively small values of `n`. For example, for `n=12345`, the smallest seven-zero number divisible by `n` consists of 822 sevens followed by a single zero.

This method should find and return the smallest seven-zero integer that is divisible by the given `n`. The easiest way to do this would be to use two nested loops. The outer while loop iterates through all possible digit lengths (how many digits the number contains). For each length, the inner for-loop iterates through all valid sequences of consecutive sevens, followed by enough zeros to reach that length. Keep going up until you find the first seven-zero number exactly divisible by `n`. Since your loops iterate through the seven-zero numbers in ascending order, the first number you find must be the smallest such number that satisfies the requirement.

You can also use a handy theorem from the same book to speed up your search immensely. Unless `n` is divisible by 2 or 5, the smallest seven-zero number divisible by n is guaranteed to contain only sevens, no zeros. This realisation should speed up your search by at least an order of magnitude for such easy values of `n`, since the quadratic number of possibilities to examine is pruned down into a single linear branch that goes through the numbers 7, 77, 777, 7777, 77777… in search of the answer. On the other hand, numbers that contain powers of two and five as factors require

seven-zero numbers whose trailing zeros grow with the exponents of these two small prime powers. The result for $n = m\ 2^a\ 5^b$, where m contains neither prime factor 2 nor 5, will have exactly $\max(a, b)$ zeros tacked to the end.

Lab 0(F): Two Branching Recursions

JUnit: P2J6Test.java

In mainstream university-level computer science education, recursion is often reduced to utter pointlessness. Recursion can cut through an exponential thicket of possibilities, yet it seems to be used only to solve outright trivial **linear recursions** such as computing factorials. Linear recursions offer nothing over their everyday alternative, ordinary while- and for-loops, which any competent coder would use anyway when their paycheck was on the line. This needless limitation of vision leaves students dazed and confused about recursion's general usefulness, since, from their by-that-time glazed-over point of view, recursion only ever seems to solve toy problems in a needlessly highfalutin manner. (Mainstream teaching of introductory recursion makes even the formal verification community, the only science that has to cheat at toy problems to get any results at all, look like Strong Doge in comparison.)

However, as ought to become amply evident somewhere in the third or fourth year, recursion's power lies in its ability to branch in multiple directions, one at a time**.** This branching ability lets a recursive method explore a potentially exponentially large **tree of possibilities** and **backtrack on failure** to the previous choice point. It is as if some program built from the usual forward-going loops had magically gained the ability to turn back time and undo the assignments that hurt you, as in the song. Even better, you can trivially modify such recursions to either find the first branch with a working piece of fruit hanging from its end, or keep going to collect all such available fruits from all branches. In this spirit, this last transition lab uses branching recursions to solve two interesting combinatorial problems. So, you handsome maniac, without further ado, create a new class P2J6 to implement the two methods in this lab.

```
public static List<Integer> sumOfDistinctCubes(int n)
```

Determine whether the given **positive** integer `n` can be expressed as a sum of cubes of positive integers greater than zero so that all these integers are distinct. For example, `n=1456` can be expressed as the sum of two cubes: $11^3 + 5^3$. This method should return the list of these distinct integers as an object of some subtype of `List<Integer>` that prints out as `[11, 5]`, the elements listed in descending order. If there is no way to break the given integer `n` into a sum of distinct cubes, this method should return the empty list. Note that for "gleaming the cube" by pushing your limits to the edge as if this were an energy drink commercial, the **empty sum** and a **singleton sum** are still sums in the same sense that zero and one are integers, as counterintuitive as this might seem. For example, the integers 0 and 8 are representable as sums of distinct cubes as the empty sum and the singleton sum 2^3, respectively.

Some integers can even be broken down into a sum of cubes in different ways. In such cases, this method must return the **lexicographically largest** breakdown: it starts with the largest possible first element, then breaks down the remainder into a list of distinct cubes in the same way. For example, when called with `n=1729`, the famous Ramanujan taxicab number, this method **must**

return the list `[12, 1]` instead of the list `[10, 9]`, even though $12^3 + 1^3 = 10^3 + 9^3 = 1729$ just as well. This constraint may initially look scary, but you can just *fuhgettaboutit*, since enforcing it is trivial: arrange the loop inside the recursive method to scan the current possibilities from highest to lowest. As with such recursions, you should again use a private helper method that accepts additional parameters that were not present in the public method.

```
private static boolean sumOfDistinctCubes(
  int n, int c, LinkedList<Integer> soFar
)
```

This helper method receives the two extra recursion parameters `c` and `soFar` from the original method. The parameter `c` contains the highest integer that you are still allowed to use. Initially, this should equal the largest integer whose cube is less than or equal to n, which you can find with a while loop. The parameter `soFar` contains the list of numbers already added to the list of cubes, so they're available when the recursion returns.

The recursion has two base cases, one for success and one for failure. If `n==0`, the problem is solved for the **empty sum**, so you can just return `true`. If `c==0`, no numbers are remaining that you could use, so you simply return `false`. Otherwise, try taking `c` into the sum, remembering also to add `c` to `soFar`, and recursively solve the problem for new parameters `n-c*c*c` and `c-1`. If that attempt was unsuccessful, remove `c` from `soFar` and recursively solve the problem without using `c`, with parameters `n` and `c-1`.

Note how, since this method is supposed to find only one solution, once the recursive call returns `true`, its caller can also immediately return `true` without exploring the remaining possibilities. This first success, guaranteed to be the lexicographically first one by the order in which these alternative solutions are explored, will therefore cause the entire recursion to roll back to the top level, where the breakdown of `n` into distinct cubes can be read from the list `soFar`.

To allow the search to cut off in some situations where continuing the search is guaranteed to be futile, it might also be of interest to know that the sum of cubes of consecutive integers from 1 to c can be computed quickly without loops using the formula $c^2(c + 1)^2/4$, a polynomial of degree four. If the target sum n exceeds the sum of cubes up to c, the method can immediately conclude that the breakdown is impossible, without the futile process of combing through all possible combinations of integers up to c only to find they are all too small.

The second method to write in this lab deals with strings instead of numbers.

```
public static List<String> forbiddenSubstrings(
  String alphabet, int n, List<String> tabu
)
```

Compute and return the list of all strings of length n that can be formed from the characters given in the list `alphabet`, but under the constraint that none of the strings listed in `tabu` is allowed to appear anywhere as a substring. For example, the list of all strings of length three constructed from alphabet `"ABC"` that do not contain any of the substrings `["AC", "AA"]` would be `["ABA", "ABB", "ABC", "BAB", "BBA", "BBB", "BBC", "BCA", "BCB", "BCC", "CAB", "CBA", "CBB", "CBC", "CCA", "CCB", "CCC"]`. To facilitate automated JUnit testing, your method must return this list of acceptable strings in sorted alphabetical order.

Following the same principle as in the previous problem of adding up distinct cubes, this recursion is also easiest to implement with a private helper method

```
private static void forbiddenSubstrings(
  String alphabet, int n, List<String> tabu,
  String soFar, List<String> result
)
```

that takes two additional parameters, `soFar` and `result`. The parameter `soFar` is the partial string constructed down the path to this point (this should equal the empty string at the top-level call), and `result` is the list of strings that the recursion has already discovered. The base case for failure is when the string `soFar` ends with any one of the strings in the `tabu` list. (The `String` instance method `endsWith` might come in handy in this.) The base case for success is the situation where `soFar.length()==n`, in which case `soFar` is added to the `result` list. Otherwise, the recursion should loop through the characters in the `alphabet`, appending each character to the `soFar` string before the recursive call, and removing it from the string after that recursive call.

Unlike the previous method for finding distinct cubes, which was supposed to terminate after the first success, this private recursive method returns nothing to indicate success or failure. Even if this method finds a solution right away, it still has to keep chugging through the entire tree of branching possibilities to glean successes from that search tree.

Lab 0(G): …Or We Shall All Crash Separately

JUnit: P2J7Test.java

When I created this lab, my instructor waited an hour in line in the cold winter air outside Costco during the early stages of COVID-19 isolation measures, with everyone standing there equally unsure what the future would bring after two weeks of flattening the curve. This wait gave him ample time to come up with a bunch of new problems (this entire document used to have only about twenty problems before the whole world changed), and the situation naturally lent itself to the concept of queueing. Since various kinds of queues, along with the nifty data structures that efficiently implement them, are useful in many algorithms, this should be a perfect time to check out the collection type `LinkedList<E>`, the faster alternative to `ArrayList<E>` for the operations needed to implement a **double-ended queue** (often called "**deque**") when there is no need for random access into the list, but the queue is accessed and mutated only from both ends. (As an alternative, you can also try out the actual `ArrayDeque<E>` class in the standard library, which is intended to be used as a proper deque.)

As in all algorithmic problems, you should always choose the data structure that makes the operations you'll do often fast, since the cost of operations you don't do is irrelevant. (The same principle extends to all of engineering, with the realisation that non-existent parts waste neither time nor energy, and either rust or moth can never corrupt them.)

Most of the time, double-ended queues are used as ordinary "**first in, first out**" (FIFO) queues, so elements always enter from the rear and eventually exit from the front in the exact order they came in. But as long as it costs us nothing to do something potentially, we might as well keep that possibility in our sleeve!

The first problem, no doubt at least subconsciously inspired by that long cold wait outside Costco, takes us to the world of the ancients, where a bunch of zealots (yes, Lana, literally, look it up) were surrounded by Romans with no escape through the front or back door. Preferring a glorious mass suicide over capture and torturous death by crucifixion, these men arranged themselves in a circle and drew lots for a random starting location. Counting from that location *k* steps ahead each time, the… man… who… was… *it!* was killed and removed from this grim game of eeny-meeny-miney-moe. The last man standing was expected to fall swiftly on his own sword and join his fallen brothers. Josephus, our lovable wandering hero who had unfortunately become entangled with these rascals, had other plans to escape. After some quick mental arithmetic, he managed to finagle himself into this last surviving position to tell this tale. Could you do as well as Josephus under this considerable high-stakes pressure and time limit? (After all, Josephus didn't have a penchant for self-aggrandising tall tales or anything like that. Was you dere, Charlie?)

Create a new class named `P2J7` in your project folder, and in that class, write the generic method

```
public static <T> List<T> josephus(List<T> men, int k)
```

It takes a list of men in the order they stand in the circle, counting initially from the first man. This method should return a new list containing the men in the order they were eliminated. The elimination logic does not depend on the element value or even its type, only its position, so we can make this method maximally generic by allowing the parameter type to be an arbitrary `List<T>`. As explained in the lecture about generics, this is not at all the same thing as defining that parameter type to be `List<Object>`! A polymorphic method that expects to receive a `List<Object>` argument will not be able to handle an argument of type `List<Person>`, whereas a method that expects a `List<T>` argument can do so easily.

As always, **this method should not modify the contents of its parameter list**, but create and return a new list object (you get to choose the concrete subtype yourself here) as the result. Note also that the step size `k` can be larger than the initial number of `men`, and yes, this does make a difference occasionally. For example, with `men=["ross", "ted", "ringo", "alice", "bob", "rachel"]` and `k=9`, the resulting elimination order is `["ringo", "ross", "ted", "rachel", "alice", "bob"]`. Shirts or skirts, none will be spared!

Our second example of patiently waiting in queue comes from a more recent period in history, and it's also an important algorithm from the era when humans still had to do computers' work by hand. With slow, cranky humans playing the part of computers (in fact, that is what the word “computer” originally meant: somebody’s job title), we want to optimise every algorithm to reach the answer with the fewest computations.

Every ten years, the United States of America is constitutionally obligated to run a nationwide census so the seats in Congress (more precisely, the House of Representatives) are distributed among the fifty states in proportion to their populations. The problem is how to round each state's seat count to an exact integer, since this task is not as clear-cut as it may initially seem. Simple rounding to the nearest integer doesn't work because it wouldn't preserve the nation's total number of congressional seats. How can you tell which states you should round up and which you should round down? Sorting states by the decimal part of their seat count and giving the leftover seats to the states in descending order doesn’t work either.

Paradoxically, in situations of this nature, even though integers in general are much easier and more accurate to perform calculations with than floating-point numbers used to approximate real numbers, the integer programming problem of finding a combination of integer values that satisfy the given constraints often turns out to be computationally far more complicated than solving that same problem when arbitrary floating-point numbers are allowed in the solution.

Fortunately, the classic Huntington-Hill method solves this integer programming problem by minimising the overall quantisation error caused by rounding the solution to integer precision. In this method, each state first receives one initial seat, regardless of its population. The remaining

seats are then distributed one seat at a time. The next seat always goes to the state with the current highest **priority,** computed with the formula $P^2 / (s(s + 1))$, where P is the population of that state and s is the number of seats that state has received so far. Since the number of seats s is in the denominator of this fraction, whereas the numerator P^2 remains constant, each additional seat always decreases the priority of the state that received it, so that other states reach the head of this priority queue to receive seats only after California, Texas, Florida, and New York have taken their couple of first dibs. (Just sit still, Rhode Island over there, and patiently wait for your turn. You will eventually get that second seat that you are entitled to.)

The Wikipedia page gives a bit of a different formula for priority quantity with a square root. But see what I did there: we don't actually care about the absolute values or scales of these priority quantities, only their mutual ordering. Applying any strictly monotonic function, such as squaring, doesn't change this priority ordering. However, squaring these irrational roots back to integers lets us work with exact integers, instead of juggling the clown car of inherently imprecise floating-point numbers! This lets the next method work even in galactic civilisations in some space opera where populations are measured in octillions (one has to wonder: does every clone warrior, or a copy of some virtual AI, get a separate vote?). Yet the main characters somehow all happen to know each other, provided we do these calculations using the `BigInteger` type. (Forget that whole stupid Coruscant with all its pointless pomp: an *actual* technologically super-advanced civilisation of the future in a galaxy far away will be managed by two guys sharing one laptop.)

Inside the same class `P2J7` where you wrote the previous Josephus problem, write the method

```
public static int[] huntingtonHill(int[] population, int seats)
```

that, given the `population` in each state and the number of `seats` to distribute across these states, creates and returns a new array that contains the counts of seats given to each state. To break ties and make the returned results unambiguous for the JUnit fuzz test, whenever two states currently have an equal priority quantity, give the next seat to the state that appears **earlier** in the `population` array.

To write this method, first add the class `Fraction` from the class examples project into your labs project, to let you perform your calculations with exact integer fractions **without any possibility of integer overflow or floating-point rounding error**. Not even one floating-point number, let alone any calculation involving them, should appear inside your method! Not just this method, but basically all your methods ever. Whenever you are happily writing code, and suddenly hear a young child's giggly voice coming from a drain, calling you also to come down and play with them, all of them just *floating* down there, it is a really bad idea to follow that voice. God gave us integers; the rest is the work of Satan. (For the same message for our secular age, as some wag once updated Jamie Zawinski's famous remark: Some people, when confronted with a problem, think, "I know, I'll use floating point numbers." Now they have 1.99999999999999999998904342 problems.)

You should also note that even though the population of some state fits comfortably inside an `int` without overflow, squaring that population will not be equally friendly once it reaches the millions, which is hardly unrealistic for electoral math! For this reason, these population squaring operations absolutely have to be performed using exact `Fraction` objects of unlimited range, not as primitive `int` values. Just as it is too late to close the barn door after the horse has escaped, it is too late to convert an int value into an unlimited BigInteger once overflow has already occurred. For example, instead of trying to be concise, normally a virtue in our line of business, by writing a statement

```
num = BigInteger.valueOf(population[i] * population[i]);
```

where the `int` multiplication will overflow before conversion to `BigInteger`, split this into

```
num = BigInteger.valueOf(population[i]);
num = num.multiply(population[i]);
```

to force the integer multiplication to take place inside the `BigInteger` mechanism where there is ample room for all digits to coexist. On that note, everybody stand up in attention, eyes front to receive the following extremely important instruction:

Just because an integer value fits into an `int`, its square might not. And just because the intermediate results of your computational formulas aren't given explicit symbolic names in your code doesn't mean those intermediate results somehow magically didn't exist in memory and get truncated to their limits, or that they're subject to the same laws of computation as your named data items! Names and intentions are not magic!

Would your instructor really be so mean as to design the method `testHuntingtonHill` to fuzz populations too big to handle accurately with floating-point operations? Would that same instructor also ensure that some states end up having almost equal populations that differ by a measly little one or two people? Come on; at this stage of the studies, and especially if you have already taken my earlier course CCPS 109, do you really even have to ask?

The best way to track the relative priorities of the states (each state identified as an integer as its position in `population` table) is to keep them inside a `PriorityQueue<Integer>` instance with a custom `Comparator<Integer>` object whose `compareTo(Integer a, Integer b)` method renders its verdict based on the current priorities of the states `a` and `b`. These priorities, of course, are kept up to date in another array, `Fraction[] priorities`, defined as a local variable inside this `huntingtonHill` method, outside the nested comparator class. To find which state gets the next seat, simply poll the queue for the state with the highest priority. Update the priority for that state in the `priorities` array, and offer the state back into the priority queue.

As historically important as this algorithm has been to the fate of the free world and our continued ability to keep rocking in it, it also has a more mundane, everyday application: displaying rounded percentages based on a list of numbers from real-world data. You know how you sometimes see a

disclaimer phrased something like “Due to rounding, displayed percentages may not add up to exactly one hundred” under some table of numbers and their associated rounded percentages? Well, geez, just use the right algorithm to display your rounded percentages, and you won't need such disclaimers! For example, to compute the percentages rounded to one decimal place, simply distribute 1,000 virtual “seats” among your data items. Each virtual seat corresponds to one per mille, one tenth of a percentage point (yep, those are a real thing, as every Finn with a driver’s license surely knows), and displays the exact percentage share of that data item.

Lab 0(H): More Integers Great And Small

JUnit: P2J8Test.java

In the spirit of the earlier lab 0(E), which used the BigInteger type to represent solutions that would never fit inside an int or even a long, this lab brings you two more interesting problems dealing with "powerful" integer sequences that reach about the weight class of those primitive types.

```
public static void hittingIntegerPowers(int a, int b, int t, int[] out)
```

Define two positive integers to be "close enough for government work" if their absolute difference multiplied by the given **tolerance** level `t` is at most equal to the smaller of those two numbers. For example, the integers 2000 and 2007 are "close enough" when t = 100, since the difference 7 multiplied by 100 gives 700, which is less than 2000. On the other hand, the integers 2000 and 2050 would not be close enough for t = 100, although they would be close enough for a wider tolerance of t = 10. A tolerance of t = 100 corresponds to being within one per cent, but without using any division or floating-point numbers to compute it. The narrower tolerance of t = 100000 would require these powers to be within one thousandth of a per cent of each other, which ought to satisfy even the most ardent six sigma advocate auditing this course from the business school.

Given two positive integers `a` and `b` and the desired tolerance `t`, this method should find and return the smallest integer powers `pa` and `pb` so that when `a` is raised to the power of `pa`, and `b` is raised to the power of `pb`, the resulting two numbers are close enough for government work. Since these powers can get pretty big, as you can see in the table below, you need to perform these calculations using the `BigInteger` type. However, this applies only to the actual powers that might have tens of thousands of digits; the exponents `pa` and `pb` are guaranteed to fit in the `int` type for all test cases given to your method.

a	b	t	Expected contents of out
2	4	100	{2, 1}
2	7	100	{73, 26}
3	6	100	{137, 84}
4	5	1000	{916, 789}
42	99	100000	{33896, 27571}

In Python, this function would simply return the answer as a two-tuple `(pa,pb)` of the exponents that satisfy the requirement above. Java doesn't have tuples in the core language, so we sidestep this by using a two-element `int[]` to hold the result. Note that the return type of this method is `void`, so that nothing is actually `return`ed from this method. Instead, the method copies its answer into the two-element array passed to it as the out parameter.

This technique doesn't make much difference in this problem, but it can help in other methods called millions of times, or when simulating the behaviour of a Python function that returns a tuple. Instead of this method having to create a small new array for the result every time it is called, the same array object can be reused between all these calls, which ought to lighten the load on the JVM garbage collector, at least back when neither the art nor the science of garbage collection techniques was quite at the level where it is today.

```
public static BigInteger nearestPolygonalNumber(BigInteger n, int s)
```

As explained on the Wikipedia page "Polygonal Number", for every positive integer $s > 2$, the corresponding ***s*-gonal numbers** are an infinite sequence of integers whose i:th element is given by the formula $((s - 2)i^2 - (s - 4)i) / 2$. The Wikipedia formula uses the letter n instead of i, but n already means something else in this problem. (Besides, n should denote only unknown but fixed integers, whereas i and j denote loop indices.) For example, the infinite sequence of octagonal numbers that springs forth from setting $s = 8$ starts with 1, 8, 21, 40, 65, 96, 133, 176…

Given `s` as an ordinary primitive `int`, and `n` as a `BigInteger`, this method should find and return the s-gonal number that is closest to `n`. If the integer `n` lies exactly halfway between two consecutive s-gonal numbers, return the smaller of those two equidistant s-gonal numbers.

`n`	`s`	Expected result (as `BigInteger`)
`5`	`3`	`6`
`27`	`4`	`25`
`450`	`9`	`474`
(ten billion)	`42`	`9999861561`
(one googol)	`91`	`100000000000000000000000000000000000000000000000000000000000416332753518329478897755794704334003003544212420356`

This problem is best solved with **repeated halving**, better known as **binary search**. First, find some position `b` in the sequence so that the `b`:th s-gonal number is greater than or equal to `n`. You can just start with `b` set to equal one, and keep multiplying it by ten until you reach your goal. Knowing two positions `a` and `b` so that the correct answer is between these positions, inclusive, compute the midpoint index `m=(a+b)/2` and look at the s-gonal number in that position. If that number is less than `n`, assign `a=m+1`, and otherwise assign `b=m`. Either way, you can continue. Once `a` and `b` are no more than one step apart, one more comparison gives you your answer. As you can see from the table above, a and b can get so huge that they would never fit in an ordinary `int`, let alone the polygonal numbers stored at those positions…

Lab 0(I): Squaring Off

JUnit: P2J9Test.java

The two problems in this lab produce results of type `boolean[]`, arrays whose each element is a truth value. Since these individual bits can be packed eight to each memory byte, storing these arrays as bit vectors is as compact as possible, with no wasted bits or bytes. However, as a Java programmer, you don't need to know anything about bits, bytes, or bitwise operations to handle them, since the Java compiler automatically turns your bit vector operations into low-level memory operations. Up here in Java's pleasant abstractions, high above all that mud and grime, you can use boolean arrays no differently than arrays of any other element type.

Create the class named `P2J9` in your labs project, and include the following two methods that each solve a problem that involves **squares** of positive integers. Given the upper limit `n` as a parameter, both methods return a `boolean[]` array of `n` elements that contains the answers for all natural numbers below `n` in one swoop. Compared to computing and returning the individual elements as requested by the caller, solving interdependent subproblems in bulk shows the benefits of Taylorism, since many things are cheaper by the dozen. Unlike the industrial workers of the past, today's programmers prefer to sip soft drinks in air-conditioned offices while mindless machines perform those tiresomely repetitive tasks. Any decision or action a machine makes is one less we humans have to focus on.

```
public static boolean[] sumOfTwoDistinctSquares(int n)
```

Determines which natural numbers can be expressed in the form $a^2 + b^2$ so that a and b are two distinct positive integers. In the `boolean` array returned as the result, the `i`:th element should be `true` if and only if such a breakdown is possible. The infinite sequence of positive integers that allow such a breakdown begins with 5, 10, 13, 17, 20, 25, 26, 29, 34, 37, 40, 41, 45, 50, 52, …

You may have previously seen a version of this problem where the breakdown into two distinct squares was done separately for one number at a time. Given n, the question asked you to find positive integers a and b whose squares add up to n. However, since we are dealing with values in bulk, perhaps rather than looping through each n and finding positive integers a and b whose squares add up to n separately for each n, it would be more productive to loop through all relevant pairs of a and b, and see which values of n can be built from those pairs…

```
public static boolean[] subtractSquare(int n)
```

Subtract a square is a simple impartial game of mental arithmetic. Two players take turns moving from the current natural number n to any smaller natural number reachable by subtracting the square of a positive integer from n, without going below zero. For example, starting with n = 15, the player whose turn it is can and must move to 14, 11, or 6 by subtracting 1, 4, or 9, respectively. Subtracting the square of 4 or higher is not allowed, since this would make the resulting value

negative. The player who moves to zero wins, since the opponent can no longer move and therefore loses under the **normal play convention**.

Using standard terminology for impartial combinatorial games, a state is winning (hot) if there is at least one move to a cold state, and losing (cold) if no such winning move exists. This rule makes the state $n = 0$ cold, since no moves are possible there. The state $n = 1$ is hot since subtracting 1 wins the game. The state $n = 2$ is cold again, since the only move from that state leads to a hot state, guaranteeing the opponent an eventual win. The infinite sequence of cold states begins 0, 2, 5, 7, 10, 12, 15, 17, 20, 22, 34, 39, …

This method should create and return an `n`-element array of `boolean` truth values so that the element in position `i` is `true` if the state `i` is hot, and false if it is cold. This method should fill the result array from left to right, since to determine whether the current position is hot or cold, all the information needed to make that decision is already in the lower-numbered states whose heat values have already been filled in…

Under the **misère play convention**, the player who moves to the zero state loses. However, the strategy for the misère version of a given combinatorial game cannot generally be constructed by merely interchanging the roles of the hot and cold states of the original normal-play game, once again illustrating the principle that reversed stupidity is not intelligence. Generally, it does not pay to be miserly in analysing the optimal actions of actually being a miser; we eat to live, not live to eat.

Lab 0(J): Similar Measures Of Dissimilarity

JUnit: P2J10Test.java

Consider two **bit vectors**, that is, two `boolean` arrays of equal length n, each filled with some combination of `true` and `false` elements. Various **dissimilarity** measures have been proposed to describe how "different" or "far apart" these two vectors of truth values are within the **hypercube** of all 2^n possible vectors of truth values. If both bit vectors are identical, all these dissimilarity metrics will return zero. Otherwise, the answer will be a positive quantity whose magnitude corresponds to the perceived dissimilarity between the two bit vectors. Obviously, swapping the roles of the two bit vectors should not affect their dissimilarity. More generally, permuting the boolean elements of both bit vectors in the same way should not affect their dissimilarity.

In this lab, you will have to implement six such dissimilarity measures. All six methods are similar in that they compute their results based on four counts n_{00}, n_{01}, n_{10}, and n_{11}, tallied from the two argument vectors. The count n_{00} is the number of positions in which both arrays have the value `false`. The count n_{01} is the number of positions where `v1` is `false`, and `v2` is `true`. Define the other two counts, n_{10} and n_{11}, symmetrically. Therefore, for any two n-element arrays `v1` and `v2`, these four counts must add to n. It would be good to extract the computation of these four counts into a separate helper method that these six methods invoke as their first step. Having written that method, each method below is basically a one-liner that creates and returns the `Fraction` answer.

The formulas for how to compute each distance measure from the computed n_{ij} values can be found on the *Wolfram Mathematica* documentation page "Distance and similarity measures" section "Boolean data", where you can click the Wolfram version of the function to read how that particular dissimilarity measure works. (On the documentation page for each separate function, click the section "Details" to expand it.) Alternatively, you can consult the "Distance measures" page, which shows that these six measures are only the tip of the iceberg. (In the notation used in formulas on that page, $a = n_{00}$, $b = n_{10}$, $c = n_{01}$ and $d = n_{11}$.) Bees make honey and beavers build dams, whereas we are fortunate to belong to a species that creates this kind of immaterial artefact.

Each of these six measures is to be implemented as a separate method inside the new class `P2J10` that you should create in your labs project. Since the expected answer is always some integer fraction, we will again use our `Fraction` example type as the return type of these methods.

```
public static Fraction matchingDissimilarity(boolean[] v1, boolean[] v2)
public static Fraction jaccardDissimilarity(boolean[] v1, boolean[] v2)
public static Fraction diceDissimilarity(boolean[] v1, boolean[] v2)
public static Fraction rogersTanimonoDissimilarity(boolean[] v1, boolean[] v2)
public static Fraction russellRaoDissimilarity(boolean[] v1, boolean[] v2)
public static Fraction sokalSneathDissimilarity(boolean[] v1, boolean[] v2)
```

The following table lists a couple of argument arrays `v1` and `v2` of $n = 5$, along with their expected correct answers for each dissimilarity metric. To make the following table a "bit" (heh) more compact, the truth value arrays are displayed as bits so that `0` means `false` and `1` means `true`.

v1	v2	*Matching*	*Jaccard*	*Dice*	*RT*	*RR*	*SS*
10101	10101	0	0	0	0	2/5	0
01111	00100	3/5	3/4	3/5	3/4	4/5	6/7
01110	11100	2/5	1/2	1/3	4/7	3/5	2/3
10011	11100	4/5	4/5	2/3	8/9	4/5	8/9
01100	11010	3/5	3/4	3/5	3/4	4/5	6/7
11010	10100	3/5	3/4	3/5	3/4	4/5	6/7

Note the odd behaviour of the Russell-Rao dissimilarity among these dissimilarity metrics in the first row of the above table; even when two bit vectors are equal, each position where both vectors have a zero bit is counted as a dissimilarity between those two vectors under that metric.

Lab 0(K): Suffix Arrays

JUnit: P2J11Test.java

"*It is as if the command:* Do not covet things in space*, were corrected with the unspoken word:* Do covet things in time." — Abraham Joshua Heschel, "*The Sabbath*"

Strings are a stark data structure: a universal, portable representation of information that lets you randomly access the linear text stored within. Under the hood, strings are character arrays, but the `String` class's higher-level public methods hide their primitive nature. Over the years, researchers have developed surprising algorithms to quickly search and analyse both the string itself and the semantic information it encodes. Even the seemingly simple task of searching for a pattern in the text can be optimised in countless ways, let alone the higher-level tasks performed on these string instruments.

Occasionally, the string data structure can be augmented with additional data that speeds up operations that would be inefficient with only the raw, unadorned text string. Although this data adds nothing new in principle, since its contents are fully determined by the original string, having it available in a different form can provide significant speedups, another form of the classic **space-time tradeoff**. This lab showcases a simple but powerful suffix array method to preprocess any text so that, after the metaphorical one-time payment for this preprocessing has cleared, all future searches for arbitrary **patterns** can be executed in time that grows only logarithmically with respect to the length of the text. This makes these pattern searches blazingly fast, even on the entire *War and Peace!*

For a given text string with n characters, its **suffix array** is an n-element array of integers that contains the position indices 0, …, n – 1 in the `text`. Since the suffix array stores each position exactly once, it is a **permutation** of order n. The suffix array lists these n positions sorted in the **lexicographic order** (also called the "dictionary order") of the suffixes that start at each position. Note that the linked Wikipedia page on suffix arrays uses one-based indexing for the string. In contrast, we use Java's zero-based indexing here to stay consistent with our programming principles.

For example, the non-empty suffixes of the string `"hello"` are `"hello"`, `"ello"`, `"llo"`, `"lo"`, and "o", corresponding to positions 0 through 4 in the original string. Sorting these positions by the lexicographic order of their corresponding suffixes gives `[1, 0, 2, 3, 4]`, which becomes the suffix array of the original string. Since all n suffixes of the `text` have different lengths, their lexicographic comparison can never result in equality. This guarantees the uniqueness of the suffix array for the given `text`.

For reasons of simplicity and convenience, suffix arrays are represented in this lab as instances of some subtype of `List<Integer>`, despite the use of the standard technical term of "suffix array". Arrays, schmarrays. Create a new class named `P2J11`, and in it, first, the method

```
public static List<Integer> buildSuffixArray(String text)
```

that builds and returns the suffix array for the given `text`. It is sufficient to use the naive suffix array construction algorithm, since the structure of our test-case strings guarantees that the algorithm's worst case, a string with all identical characters, will never occur.

The easiest way to implement this naive algorithm in Java is to define another strategy subclass of Comparator<Integer> whose compareTo method lexicographically compares the two substrings starting at the integer positions it receives as parameters. In this method, first define the local variable `ArrayList<Integer> result`, and fill it up to the brim with the integers from 0 to $n - 1$. Then, use the utility method `Collections.sort` to sort this `result` list with your custom `Comparator<Integer>` strategy object.

This discipline should allow your `buildSuffixArray` method to be reasonably fast even when building the suffix array for the entire *War and Peace*, as performed by the JUnit test class using the `warandpeace.txt` data file. If you try to build an array containing all possible suffixes of that 3.2 million-character text file as separate String objects, no computer in our reality would have enough RAM to hold all those suffix objects.

Once we construct the suffix array for a fixed `text`, we can use it to quickly find all positions in the `text` where the given `pattern` occurs. These are precisely the same positions whose suffixes start with that `pattern`! Because the previous preprocessing step already sorted these suffixes lexicographically, we can use a slightly modified binary search on the suffix array to find the lexicographically first such suffix. Since all suffixes starting with the same `pattern` appear together as a block in the sorted suffix array, once the binary search finds the first match, a simple while loop can iterate through all matching positions.

To achieve all that, write the second method

```
public static List<Integer> find(
  String pattern, String text, List<Integer> suffix
)
```

that creates and returns a list of positions of the original `text` that contain the given `pattern`, using the given `suffix` array to speed up this search. Note that this returned list of positions must be sorted in ascending order of the positions themselves, instead of the lexicographic order of the suffixes of `text` that start at these positions.

Lab 0(L): I Can, Therefore I Must

JUnit: P2J12Test.java

A jovial old grandpa mathematician straight out of central casting, often appearing in Numberphile videos, Neil Sloane is behind the On-Line Encyclopedia of Integer Sequences, a wonderful tool for combinatorial explorations. This lab has you implement two methods to generate elements of two interesting integer sequences whose chaotic behaviour emerges from iteration of a deceptively simple rule. Both sequences are filled **greedily** in ascending order, so each element is always the smallest number that avoids conflict with the previously generated elements. (Also, before you start: `remySigrist` requires knowledge and use of **bitwise arithmetic**.)

Note that, as mathematicians define them, these sequences start at position one instead of position zero, unlike sequences for us budding computer scientists, who prefer zero-based indexing. When asked to produce the first *n* elements, these methods should return an array with *n* + 1 elements. We set the zeroth element to zero, which we don't care about anyway, and then the joyride begins.

```
public static int[] forestFire(int n)
```

As explained in the YouTube video "Amazing Graphs II", the first two elements both equal 1. Then, each element `a[i]` is the smallest positive integer such that no positive integer offset `j` creates an arithmetic progression of three equally spaced elements backwards in the sequence from position `i`. More formally, `a[i]-a[i-j]` may never equal `a[i-j]-a[i-2*j]`, even if your worst enemy gets to pick the position `i` and the offset `j`.

```
public static int[] remySigrist(int n)
```

As explained in the YouTube video "Amazing Graphs III", this sequence devised by Rémy Sigrist assigns to every positive integer a natural number, called the "colour" of that number. After the first colour assignment `a[1]=0`, each following `a[i]` thereafter is the smallest natural number `c` for which the binary representation of any earlier position `j` that has already been assigned the colour `c` has any bits turned on in common with the binary representation of the position `i`. This can be checked quickly with the expression `i & j == 0`, where `&` is the **bitwise AND** operator in Java.

You can solve this problem with two nested for-loops. The outer loop iterates through the positions of `a`. The inner loop counts up through the colours until it finds one that doesn't create a conflict. However, you should not be a "Shlemiel" who uses a third-level inner loop to determine whether the current colour candidate `c` creates a conflict. Instead, trade space for time with a second integer array named `taken`, where each element `taken[c]` gathers all the bits that are on in any previous position assigned colour `c`. Then, whenever you assign `a[i]=c`, update this lookup table with the assignment `taken[c] |= i`, where `|` is the **bitwise OR** operator in Java, and `|=` is its **assignment shorthand** analogous to `+=` and other such familiar forms.

Lab 0(M): Tower Blocks

JUnit: TowersTest.java

The following problem is adapted from "Towers" in CSES, another good collection and online judge for coding problems. This site is run by Antti Laaksonen, whose "Competitive Programmer's Handbook" is an excellent resource on applying basic data structures and algorithms for competitive programmers of all ages and sorts, even if they only ever compete with themselves to get slightly better every day. However, this problem doesn't require any of the techniques explained in that book, or for that matter, in any other classic algorithm textbook. Instead, this problem is an exercise in devising an efficient mechanism to build the solution within the given **constraints** that prevent most of your possible moves and gently nudge you toward the optimal solution. (As in many other walks of life, fences can often be good, trusted friends, not mere hindrances some unseen moron placed there only to annoy you!)

You are given a series of `blocks` as an integer array, for example `{14, 9, 12, 4, 7, 1}`. You must process these blocks in **strict order from left to right**, which means that the order of these blocks in the array actually does matter a lot. These `blocks` build **towers,** so every block must become part of some tower. When it's its turn, a block must be placed either on the table to start a new tower or on top of a strictly larger block that is currently on top of a previously created tower. Again, there are **no backsies**; once you place a block, you can't later take it out like in Jenga and sneak it into a more tempting, convenient spot. As long as you follow these rules, each tower can be any height, regardless of the heights of the other towers.

Your task is to **minimise the total number of towers**. For example, the previous `blocks` can be arranged into two towers, but no fewer. One working arrangement has the first tower containing the blocks {14, 12, 7}, and the second tower containing the blocks `{9, 4, 1}`. As you can verify, both towers contain their blocks in the same relative order as they appeared in the original array. The optimal arrangement of `blocks` into the minimum number of towers is not necessarily unique. Your method therefore needs to return only the minimum number of towers, not any actual working arrangement of blocks into towers.

In your labs project, create a new class `Towers`, and there the single method

```
public static int minimizeTowers(int[] blocks)
```

that computes and returns the minimum number of towers achievable with the given `blocks`. Your solution should be sufficiently efficient to complete the pseudorandom fuzz test for thirty thousand randomly created `blocks` of pyramidally increasing lengths in at most a couple of seconds.

Lab 0(N): Substring Parts

JUnit: P2J13Test.java

The two problems of this lab concern Java strings and substrings. Unlike in other languages such as Python, Java 7 took advantage of the immutability of its `String` type by having the `substring` method return a new object that shares the same underlying character array as the original string. Since the contents of this shared `char[]` will never change, no harm can result from having multiple `String` objects share that same array. Immutability, huzzah! This policy lets the `substring` method run in guaranteed constant time, since it only needs to allocate the object on the heap and initialise its fields, regardless of the length of the original string and the substring being extracted into a separate object.

On the other hand, the big downside of this policy is that every `String` object in Java must store its length and its starting offset from the beginning of its character array, both as ints. This makes every `String` object on the heap eight bytes larger than necessary, creating a curious phenomenon we could perhaps now call the **space-space tradeoff**. Furthermore, sharing character arrays between strings and substrings prevents garbage collection of the original string's large character array when it becomes unreachable from the program's live variables, yet is still kept alive by a small substring object that uses only a small part of that big chungus character array.

In your labs project, create a new class named `P2J13`, and in it, two `static` methods

```
public static int countDistinctSubstrings(String text)
public static String reverseSubstringsBetweenParentheses(String text)
```

The first method should count how many distinct non-empty substrings exist in the given `text` and return that count. For example, the string `"lollo"` contains a total of eleven distinct substrings: `"lollo"` of length 5, `"loll"` and `"ollo"` of length 4, `"lol"`, `"oll"` and `"llo"` of length 3, `"lo"`, `"ol"` and `"ll"` of length 2, and `"l"` and `"o"` of length 1.

This problem can be solved in a straightforward “Shlemiel” fashion with two nested for-loops to iterate through all pairs of possible starting points and substring lengths, and using a Set<String> to maintain the distinctness of the strings discovered. Even though the constant factors in the running time of this method are small enough to make it fast when most of the substrings are distinct from each other, and it becomes necessary to iterate over all of them anyway, this technique is subject to accidentally quadratic running time, provided that your worst enemy gets to play the **adversary** and choose the `text`. That sneaky little bugger could make the `text` consist of *n* repeated copies of the same character, and the running time becomes painfully noticeable once *n* reaches the thousands, as it will in the JUnit test for this problem. Such a string contains exactly *n* distinct substrings, one for each length from 1 to *n*, but this algorithm would loop through a quadratic number of combinations to find them, only to throw away most of them immediately!

A more complicated **adversary-proof** solution that is guaranteed to be invulnerable to accidentally quadratic running time, but pays this guarantee with a slightly higher constant factor in its running time over average-case inputs, stems from the realisation that the substrings of length k can be constructed from the substrings of length $k + 1$ by removing either the first or the last character from each one of these longer substrings. Your method should therefore generate these distinct substrings in the descending order of length. Use two separate instances of `HashSet<String>`, the first to remember the known distinct substrings of the previous length k, the second to store the substrings of length $k - 1$ that are currently being generated.

Initially, the first set is a singleton containing only the original `text`. In each round of the loop, iterate through the known strings of k characters currently in the first set. Add to the second set the two substrings of length $k - 1$ constructed by dropping the first and last characters of that string of length k. Once you have done this for every string in the first set, add the size of the second set to your current tally of distinct substrings found. You can now clear the first set and swap the roles of the two sets for the next loop iteration to generate all distinct substrings of length $k - 2$.

The second method in this lab also concerns substrings. It also shows how to use the `StringBuilder` mutable string to build a new string efficiently by repeatedly appending characters and modifying it as it grows. Given a `text` that consists of letters grouped with possibly nested pairs of parentheses, this method should return a new string where the contents between each matching pair of parentheses have been reversed, leaving out the parentheses in this reversal. For example, given the string `"(ab)c(de)"`, this method should return the string `"baced"`.

Whenever the `text` contains nested parentheses, the corresponding substring reversals must be performed **inside out**. For example, the string `"(a(bc)d)"` should become `"dbca"`. The JUnit tester for this method guarantees that the parentheses inside the `text` given to your method are always **properly nested and balanced**, so your method does not have to concern itself with handling malformed strings whose parenthesis structure is inconsistent.

Whenever the `text` contains nested parentheses, the corresponding substring reversals must be performed **inside out**. For example, the string `"(a(bc)d)"` should become `"dbca"`. The JUnit tester for this method guarantees that the parentheses inside the `text` given to your method are always **properly nested and balanced**, so your method does not have to concern itself with handling malformed strings whose parenthesis structure is inconsistent.

To do this efficiently, use a `StringBuilder` to build the result, and a `Stack<Integer>` to track the positions of the left parentheses you encounter. Loop through the characters of the `text` from left to right, and `append` each character that is neither type of parenthesis to the `result`. At each open parenthesis, `push` the current length of the `result` buffer onto the stack. At each closing parenthesis, `pop` the position of its left counterpart from the stack. Extract and delete the substring of the `result` from that position into a separate substring that you then reverse and `append` back to the `result`.

Wham, bam, thank you, ma'am… except that even after almost three decades of evolution, Java's `String` objects still don't have the simple reverse method we take for granted in basically any other programming language used for serious professional work. The canonical way to create a reversed version of some Java string object `str` is by going through the expression `new StringBuilder(str).reverse().toString()`.

In general, Java has so few advantages over Python 3 that we almost feel obligated to point them out every time, and guaranteed constant-time `substring` extraction certainly qualifies. This idea could go much further, especially if the `String` data type were not `final` and allowed different implementations to store immutable text in different ways. Then, text editors and other programs that need immutable strings that are not merely sliced and diced into smaller pieces but also concatenated to create new strings from these pieces could use the rope data structure, where these operations work in guaranteed constant time, as illustrated in labs 64 and 65. The cost of storing additional references inside each rope node is well compensated by the fact that each piece needs to exist only once in memory, regardless of how many larger concatenated pieces of text it has been used to build.

Lab 0(O): Chars To The Left Of Me, Integers To The Right…

JUnit: P2J14Test.java

The two methods in this lab continue the ongoing theme of strings, but this time they operate at the character level. Both problems were adapted from LeetCode, a premier collection of online coding challenges and automated testers used by technical interviewers and job-searching candidates. The JUnit test for this lab also uses the text file `warandpeace.txt` as a data source, so make sure to also copy this file into the root of your project directory. Create a new class `P2J14` in your project, and there, first, the `static` method

```
public static int[] distanceFromCharacter(String text, char c)
```

In the first method, you are given a `text` and some character `c` that is guaranteed to occur inside the `text` at least once. This method should create and return an integer array the same length as `text` so that each element gives the distance to the nearest occurrence of the character `c` from that position. For example, the array returned for the argument `"hello world,"` and the character `'o'` would be `{4, 3, 2, 1, 0, 1, 1, 0, 1, 2, 3}`. If the character `c` had instead been `'e'`, the returned array would be `{1, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9}`.

Again, this problem could be solved in a simple "Shlemiel" fashion with two nested loops. The outer for-loop would iterate through the `result` array. For each such position, the inner while-loop would advance left and right from that position in lockstep until it sees the first occurrence of the character `c`. The grinding inefficiency of such a clumsy approach is clear in any text where `c` occurs first but nowhere else afterwards. The pain of the quadratic running time becomes evident once the `text` length grows to the thousands.

To solve this problem in linear time, note that consecutive positions in the result array can differ by at most one. This allows you to use a single for-loop to fill in two temporary arrays, `left` and `right`, that will then be combined at the end of this method to produce the final `result`. Loop through the text to fill the left array: set each element `left[i]` to the distance to the nearest occurrence of character `c` when you can only move left, not right. To update these values in constant time, maintain an integer counter variable to track this distance, initialised to `text.length()+1`. At every character other than the `c` that you are looking for, increment this counter by one, whereas at every occurrence of `c`, bring that counter back to zero. The current counter value becomes the value `left[i]` being currently filled in.

Fill `right[i]` the same way, but in reverse (right to left). You can either do this in a second loop for clarity, or combine both loops into one for-loop with a bit of simple integer arithmetic for brevity.

Once you have filled in both arrays, `left` and `right`, each element of the `result` array is the minimum of the `left` and `right` elements in that position.

(This same technique can also be used to solve a classic algorithm coding chestnut usually called something like "Trapping rainwater", guaranteed to be found in some form in basically every algorithmic problem collection of this kind. The `left` and `right` arrays track the tallest wall seen from each position in each direction, filled with a similar linear-time for-loop that remembers the tallest wall encountered so far. The height of the water pillar at each position is then the minimum of the left and right arrays at that position.)

```
public static String pushDominoes(String dominoes)
```

The second method takes a string parameter representing a line of `dominoes`. Instead of placing these dominoes neatly on the table so the pips match between consecutive dominoes like a civilised person, we are being more whimsical: we place them standing up to create a chain of falling dominoes, a well-known metaphor for various things in life on both personal and national scales.

In the domino string passed to this method, `'.'` denotes a standing domino, `'R'` denotes a domino falling right, and `'L'` denotes a domino falling left. The simulation proceeds in discrete time steps. In each time step, every falling domino causes its neighbouring domino in the direction of falling to fall as well, provided that neighbour is still standing at that moment. If a standing domino is pushed right by its left neighbour and pushed left by its right neighbour, these forces cancel out, and the domino remains standing.

This method computes the domino chain's equilibrium state once it becomes stable, so nothing changes in the next time step. For example, given the initial state `"R..."`, the result would be `"RRRR"`. Given the initial state `".R...L"`, the result would be `".RR.LL"`, with the two chains from the left and right meeting in the middle to leave the domino standing.

Lab 0(P): Two Pointers

JUnit: P2J15Test.java

The two methods in this lab demonstrate the classic two-pointer technique, which helps you avoid being a "Shlemiel" when solving array problems that fit this technique. (In this context, the term "pointer" refers to an integer index that can be used to access an array element in constant time, not literally the low-level pointer variable that stores that actual memory location.) Even better, this bad boy of good coding techniques has the built-in advantage that its running time is at most linear in the array length! Once you absorb it into your "array" of coding techniques, the two-pointer technique is a skeleton key that cracks open countless doors that the poor Shlemiel would spend an entire day mindlessly running back and forth to squeeze open. A multitude of examples of this technique can be found by browsing the corresponding categories at LeetCode and Codeforces.

To search for a solution in an array, the basic two-pointer technique uses two array indices, `i` and `j`. These indices define the start and end of the current subarray guaranteed to contain a solution, assuming one exists. This bold invariant holds trivially at the start of the method, since `i` is initialised to the first position and `j` to the last, spanning the entire array, outside which nothing exists for this method. From there, a while loop runs until either a solution is found in the subarray defined by the current values of `i` and `j`, or the indices cross, showing that no solution exists. Each round of this while loop is guaranteed to move at least one index toward the other in a disciplined way that maintains the invariant by ensuring neither index accidentally skips a position needed for the solution.

The two pointers technique can be used for problems where this decision of which index to advance and how much can always be made based on the local information about the elements at positions `i` and `j`, possibly with some other state information that the loop maintains along these indices to remember some relevant aspects of the past that may affect the future decisions that lead to the final answer that we can confidently give to Regis.

Before looking at the two problems we need to solve for this lab, we demonstrate this technique with the classic example of **two summers**. From an array of integers guaranteed to be sorted in strictly ascending order, find two distinct elements a and b that add up to the given goal value x so that $x = a + b$. Again, Shlemiel would brute-force this problem by ignoring the fact that the array is sorted and using two nested for-loops to crawl through the $n(n - 1)/2$ possible pairs of elements to find two elements that add up to x.

A better way to solve the two-sum problem comes from the realisation that when `i` and `j` define a subarray that contains the solution whenever one exists, comparing the sum `a[i]+a[j]` to the goal value x will always either find a solution or give us an ironclad guarantee that moving one of these indices one step towards the other cannot possibly skip over the solution. For example, if the sum of these two elements is less than x, then even the largest available element `a[j]` was not large

enough to reach the goal when paired with the smallest available element `a[i]`. This smallest element `a[i]` therefore cannot be part of any pair that would constitute a solution, so skipping this nail cannot cost us the kingdom. By symmetry, we can safely skip `a[j]` whenever the sum exceeds *x*. The complete method based on this idea should be similar to the following:

```
public static boolean twoSummers(int[] a, int x) {
    int i = 0, j = a.length - 1;
    while(i < j) {
        int s = a[i] + a[j];
        if(s == x) { return true; } // Got it!
        if(s < x) { i++; }  // a[i] is too small to work
        else { j--; }       // a[j] is too big to work
    }
    return false;
}
```

For an *n*-element array, these two indices must meet after at most *n* – 1 rounds, so the loop terminates in at most that many rounds. Since the work done in the body of the loop takes a constant time, the running time of the entire loop is at most linear with respect to the length of the array `a`.

Enough talk, on to action. Create a class `P2J15` in your labs project, and there the `static` method

```
public static int[] findClosestElements(int[] a, int x, int k)
```

The first method takes an array `a` with at least `k` distinct integers sorted in ascending order and guarantees that `x` appears somewhere in the array. This method first finds the location of element `x` in the array using binary search. Binary search is the ur-algorithm of the two-pointer genre, so in each round one index can safely jump halfway toward the other without risking jumping over the element `x` being searched for. However, implement your binary search correctly by realising the two index jumps aren't symmetric! This side-steps both classic bugs: one index jumping one step too far and missing `x`, or both indices ending up forever adjacent because the computed jump size becomes zero.

Once you have found the position where the element `x` resides, your second loop should make these indices proceed left and right from that position to collect the total of `k` elements whose values are closest to the element `x`, as measured by the absolute value of the difference between that element value and the element `x`. If two elements are equally close to the element `x`, one to the left of `x` and the other to the right (such as elements 0 and 10 when `x` equals 5), use the smaller one if you can take only one of these elements in the result. For example, the four closest elements to 10 in the array `{1, 2, 8, 10, 11, 13, 22}` would be `{8, 10, 11, 13}`. Note that your method must return these elements in sorted order.

As a sidebar, we all intuitively understand how helpful it is to keep things like library books or house numbers along a street in sorted order for fast retrieval. Yet few people can coherently explain why. The reason becomes clear when you compare binary search on a sorted array with linear search on an unsorted array. Sorting effectively boosts the power of the order comparison operator between array elements to nearly unlimited. Looking at an element in an unsorted array gives you information only about that element, whereas looking at an element in a sorted array gives you useful information about all the elements that come before or after it!

```
public static int countSubarraysWithSum(int[] arr, int sum)
```

The second method also receives an array named `arr` that consists of **positive** integers not sorted in any particular order. The method must count how many contiguous subarrays that array contains whose elements add up exactly to the given nonzero `sum`. For example, the array `{6,3,1,2,3}` contains three subarrays whose elements add up to six; `{6}`, `{3,1,2}` and `{1,2,3}`. These subarrays can partially overlap, or even be singletons equal to the `sum`.

The Shlemiel way to brute force this problem would be to use three nested for-loops; the outermost level looping through all possible starting positions of the subarray, the second level looping through all possible lengths of that subarray, and then, to top off this idiocy, the innermost third level adding up the elements in that subarray. The better technique uses two pointers, but this time as a variation where **both** indices `i` and `j` start at the **beginning** and advance towards the end so that the **forward scout** index `j` is always ahead of the **rear guard** index `i` trailing somewhere behind it, with the rear guard never stepping past the forward scout. Once index `j` steps past the end of the array, the method can terminate because it has found everything it can.

If the sum of the elements of the subarray from `i` to `j`, inclusive, is greater than `sum`, you can safely advance the rear guard index `i` (or `j`, when both indices are equal and thus define a singleton subarray) by one step to make the subarray (and therefore also the sum of its elements) smaller. When the sum of the elements of that subarray is less than `sum`, you can safely advance the forward scout index `j` by one step to make the subarray larger. When the sum of the elements of the subarray is equal to `sum`, increment your tally of such subarrays by one, and advance `j`.

To eliminate the innermost loop of the Shlemiel solution, note how each subarray that you examine is always the same as the subarray that you examined in the previous round, except with either one element added to the end (when advancing `j`), or one element removed from the front (when advancing `i`). You should therefore maintain the sum of the current subarray in a local variable, and update it in constant time whenever either index advances: either by adding the element the forward scout index j enters, or by subtracting the element the rear guard index `i` leaves behind. This speedup lets your method pass the JUnit tests in at most the required couple of seconds, instead of taking an unacceptably long time to complete.

Lab 0(Q): Aboutturn Is Playfair

JUnit: PlayfairCipherTest.java

With the exponential progress of computing technology, the Playfair cypher has become woefully inadequate for today's needs for encrypted communications. We can still appreciate its emergent complexity, even though the rules are simple enough for the average person to use with nothing but pen and paper, given the secret **passphrase**. We could now pretend to be time travellers visiting the stressful, chaotic trenches of the Great War, or the vaguely steampunk parlour of some gentleman detective leisurely cracking intercepted communications between the enigmatic Professor Moriarty and his colourful cast of henchmen known under the collective sobriquet "The Crime Chums". Systematically going through the combinations in the same cosy spirit as we would solve a Sudoku today, even as there is no shortage of messenger boys and other street urchins ready, willing and able to brute force that same task for a tuppence, our good detective slowly drifts off into a dream of Her Majesty's finest scientists one day building a steam-powered machine that would reckon these mechanistic calculations for him…

Meanwhile, up here in the reinvigorated cyberpunk genre that has spilt over to our actual reality, the Playfair cypher has value as an educational exercise in two-dimensional array operations. In our variation, we treat J as I, so the English alphabet has only 25 letters. The Playfair cypher uses a 5-by-5 grid of letters, filled with distinct letters in an order determined by the secret passphrase known to both parties exchanging secret messages encrypted with this scheme. The grid is filled row by row as you read the passphrase, skipping spaces, punctuation, and all characters outside the 25 letters of our reduced English.

Each letter is placed in the current cell in uppercase at its first occurrence in the plaintext. The cursor is advanced to the next column, starting from the beginning of the next row once the current row has been filled in. At the end of the passphrase, fill the remaining cells in the grid with the leftover letters in alphabetical order. For example, the passphrase "There will always be suffering!" would give us the following grid:

T	H	E	R	W
I	L	A	Y	S
B	U	F	N	G
C	D	K	M	O
P	Q	V	X	Z

The Playfair cypher assumes that the plaintext message contains only uppercase letters, such as `"WEWILLATTACKATDAWN"`. The plaintext is partitioned into pairs of letters, and each pair is encoded separately into a pair of ciphertext letters. However, when both letters in the pair are the same, an

extra letter `X` is inserted between them. (In this stuffy nineteenth century, the original plaintext never contains two consecutive `X`'s.) For example, the message `"JOLLY"` would be encrypted as if it had been `"IOLXLY"` all along, again treating the letter `J` as if it really were the letter `I`. If the plaintext message has an odd number of letters, an additional X is tacked to the end to pair with the last letter that would otherwise be left without a partner. For this reason, the plaintext message also cannot end with the letter `X`.

Thanks to this preprocessing, every pair to be encoded will consist of two distinct letters. To encode a pair of such letters in the same row of the grid, use their neighbours one step to the immediate right, with the row looping cyclically back to its start. For example, using the grid above, the pair `SL` would encode as `IA`, and the pair `BU` would encode as `UF`. The rule for encoding a pair of letters in the same column is the transposed version of the rule for encoding two letters in the same row; each letter is replaced by its neighbour one step immediately below. For example, the pair `DQ` encodes as `QH`, and the pair `GW` encodes as `OS`.

The last case is when the two letters in the pair are in different rows and columns, such as `BV`. In this case, we replace these letters with the two letters that complete the rectangle in the grid, using the original two letters as opposite corners. To keep decryption unambiguous, we have two symmetric, equally correct choices, and we encode each letter with the corner letter in the same **row** as the letter being encoded. For example, the pair `BV` would be encoded as `FP`, and the pair `GX` as `NZ`.

That's it! That is the entire scheme, teachable in fifteen minutes to any schoolboy and his little chums, and now ready to be translated into Java code. In your project folder, create a new class named `PlayfairCipher`, and there the method

```
public static char[][] constructPlayfairTable(String passPhrase)
```

to create and return the 5-by-5 character array Playfair table that the JUnit test class will pass on as the `grid` to the two methods

```
public static String encryptPlayfair(String plaintext, char[][] grid)
public static String decryptPlayfair(String ciphertext, char[][] grid)
```

for encrypting and decrypting the **plaintext** and **ciphertext** messages. In the spirit of the age in which the Playfair cypher originated, our JUnit tester will encrypt the entire `warandpeace.txt` text file with that cypher. (Put that in your pipes and smoke it, Russian formalists.) Again, make sure that this file is copied in the root of your project directory, since that is where JUnit will be looking for it.

Decoding the ciphertext back to the original plaintext uses almost the same algorithm as encoding the original plaintext into ciphertext. In fact, you can extract the common parts of these two

methods into a `private` utility method of its own, making the bodies of these two methods simple one-liners. In addition to the `text` and the `grid`, this utility method takes an integer `offset` that tells it how far to look at the neighbour in the same row or column. For encoding, set the offset to +1; for decoding, set it to –1. The rest of the shared algorithm is identical for both encrypting and decrypting.

If you want to be really clever and have the remainder operator `%` work correctly for both cases to keep you wrapped inside the grid, the `offset` for decoding could be +4 as well! Java uses the mathematically correct remainder operator, so -1%5 correctly equals -1; it's just that this result falls outside the table's legal indices. Python uses a mathematically incorrect remainder operator that happens to work correctly for wrapping indices inside sequences. In Python, the expression `-1%5` equals `4`. (The reader is invited to ponder this sort of "worse is better" phenomenon that can occasionally be observed in many other domains of our shared human enterprise and endeavour.)

Despite its simplicity, the Playfair cypher still illustrates important ideas relevant even to modern cryptography. For example, Kerckhoff's principle states that all security of the system should rest on the secret key, so that even if every detail of the algorithm were leaked to the enemy, that knowledge would not help them decipher any of our ciphertext messages, since their security depends only on the secret key, without which knowing the algorithm itself gets them nowhere. Systems that violate this principle and instead rely on "security by obscurity" can always be permanently broken with the classic, literal brute-force decryption technique of "rubber hose cryptanalysis", and will soon go the way of the dodo in today's interconnected world.

We can also note a dashing feat of usability design well before any man of stature even bothered to think about women and the servant class. The Playfair cypher is insensitive to whitespace and capitalisation, which greatly simplifies communication, memorisation, and the use of secret passphrases for us humans, whose ability to remember things is usually about the level of the microchips found inside a toaster these days.

Lab 0(R): Frogs On A Box

JUnit: FrogCrossingTest.java

This problem is adapted from a Codeforces problem, the premier Russian competitive coding problem collection and its associated online judge. Even if you never participate in competitive coding contests, we should still take part in the most important contest of all: beating yesterday's version of yourself. You never know when all that extra coding strength and knowledge of little techniques will come in handy in future problems. ("Do you even code, bro?")

In a version of the classic arcade game Frogger, updated for our course, a troop of frogs stands at the edge of a canyon, aiming to escape to the other side before being slowly boiled in their tanks. The distance between the canyon edges is far too long for even the mightiest frog to levitate its way across. The good news is that a row of boxes has been magically suspended to float in midair between the canyon edges. (True story.) The bad news is that each box can be used only once, since it vanishes the moment a frog propels itself off it. Given the boxes' *x*-coordinates and each frog's jumping strength, how many frogs can cross the canyon successfully?

In your labs project, create a new class named `FrogCrossing`, and there, the method

```
public static int maximumFrogs(int[] strength, int[] boxes)
```

The first parameter `strength` is an array whose `length` equals the number of frogs in the army. `strength[i]` gives the maximum distance between two boxes that frog `i` can clear in a single hop. The `strength` array is sorted in descending order; the strongest frogs go first, as Nature and Nature's God dictated. This order is good and must be preserved, no matter what all the advocates of slave morality for the weak may plead. The second parameter `boxes` contains the horizontal coordinates of the boxes floating in the canyon, listed in ascending order. The first and last elements of this array are the fixed canyon-edge positions that don't vanish when a frog uses them.

For example, if `strength` equals `{7, 5, 4}` and `boxes` equals `{0, 3, 4, 6, 8, 10, 11}`, the strongest two frogs can cross the canyon from the edge at $x = 0$ to the opposite edge at $x = 11$, with several different ways to achieve this. However, no matter how you try to finagle it, all three frogs cannot cross the canyon, as the following argument shows. The two weakest frogs will use up the boxes at $x = 3$ and $x = 4$, and then the boxes at $x = 6$ and $x = 8$. Since this uses up all the boxes between $x = 0$ and $x = 10$, the first frog can now do nothing better than imitate Wile E. Coyote in his tragic realisation of the inherent futility of his efforts. Having the strongest frog cross first changes nothing, since he will use at least one box before the other two frogs need the box at $x = 10$ to cross the canyon successfully.

This problem is best solved recursively, using a private utility method that determines whether the *k* strongest frogs can cross the canyon as an army. The public method `maximumFrogs` should use a

single loop that counts *k* from the size of the army down to zero and returns the largest *k* for which the recursive method below returns true, indicating a successful crossing. If no solution exists using the *k* strongest frogs, abandon the weakest of those frogs, as the Cold Equations of Nature dictate, and try to find a solution for the *k* – 1 strongest frogs. (This solution illustrates the general technique known as **iterative deepening**. It often comes in handy in recursive optimisation problems.)

```
private static boolean canCross(
  int[] strength, int[] boxes, int[] atBox, boolean[] used
)
```

The first two parameters, `strength` and `boxes`, are the same as in the previous method, and their contents do not change during this recursion. The *k*-element array `atBox` gives the index of the box that each frog is currently standing on. (You should use the indices of the boxes directly instead of their *x*-coordinate values, to make the lookup of boxes and their successors faster.) At the top-level call, the `atBox` array is all zeros, since every frog starts at the canyon edge. The fourth parameter is a boolean array that tracks which boxes a frog has already `used` so other frogs can't hop on them.

The base case is when every element of `atBox` equals `boxes.length-1`, meaning all k frogs have successfully crossed the canyon. Otherwise, find the frog currently on the box with the lowest *x*-coordinate. If that frog can jump directly to the opposite edge, make it do so, update the `atBox` array accordingly, and recursively solve the rest of the problem.

If the leftmost frog cannot reach the opposite edge directly, use a while loop to iterate through the unused boxes it can reach in a single jump. For each such box, see what happens when the frog jumps to that box, update the atBox and used arrays accordingly, and recursively try to solve the problem for the new state of frogs and boxes. If that recursive call returns `true`, your loop can also stop and return `true`, since we are looking for any one working solution. Otherwise, keep looping over the remaining reachable boxes to see whether a jump there yields a better result. Once you have tried jumping on every reachable box but the recursive calls from those jumps led only to failure, your method can also stop and return `false`. Since the hop sequences of the crossing frogs never visit the same floating boxes and can therefore be arbitrarily interleaved while remaining successful, there is no point looping over the other frogs to find a solution. If a solution existed starting with a frog other than the current leftmost one, we would have found that same solution by starting with the leftmost frog.

We leave it as an exercise for the reader to note other hard and thus useful constraints that can be used to speed up this basic search. Your mind will answer most questions if you learn to relax and wait for the answer. In this kind of branching recursive search, it is hugely important to detect inevitable failures as soon as possible. In that frozen moment when everyone sees what is at the end of every fork, turn back and return to the previous level of recursion to try an earlier alternative.

Lab 0(S): Hamming Center

JUnit: HammingCenterTest.java

The **Hamming distance** between two arrays is the number of positions where they differ. You can compute the Hamming distance with a single for-loop that traverses both arrays in lockstep, incrementing a counter whenever the elements at the same position disagree. The Hamming distance is a proper distance metric, since every array is trivially at zero distance from itself, distances between two arrays are symmetric, and Hamming distances satisfy the triangle inequality.

For a collection of `boolean` arrays, we define its **Hamming center** as a `boolean` array whose Hamming distance to every array in the list is at most *r*, where *r* is the smallest radius for which some boolean array satisfies this requirement. The Hamming center is still not necessarily unique; for example, both `01` and `10` can serve as Hamming centers for the arrays `00` and `11` with radius 1. (We conventionally write out `boolean` arrays in a compact form of zeros and ones, since it's easier to justify and visualize the five truth values from `01101` than from `[false, true, true, false, true]`.)

This lab uses recursion to construct the **lexicographically lowest** Hamming center for the given array of `boolean` arrays and the desired radius *r*. In your lab project, create a new class named `HammingCenter`, and in it, the method

```
public static boolean[] findHammingCenter(boolean[][] bits, int r)
```

The first parameter, `bits`, is a two-dimensional array of boolean array whose Hamming center we want to find. The caller guarantees that each row has the same length, so Hamming distance is well-defined. This method should construct and return the lexicographically lowest Hamming center whose distance to every row of the `bits` array is at most `r`. If no Hamming center exists for the given `bits` array and the radius `r`, this method should return `null` to inform the caller that they need to try a larger value of `r` next.

This method should first set up the necessary array objects `distance` and `center`, passed down in the top-level call of the following recursive utility method that does all the heavy lifting:

```
private static boolean findCenter(
  boolean[][] bits, int[] distance, boolean[] center, int k, int r
)
```

The parameters `bits` and `r` play the same roles as they did in the first method. The recursion should not mutate the `bits` array in any way as it fills in the Hamming center to the parameter array `center`. The parameter `k` indicates which position in the center array is being filled at this

recursion level. Since we are filling the `center` strictly from left to right, `k` will be initialized to zero in the top-level call to this recursive method, and increase by 1 in each recursive call.

The new parameter array `distance` maintains a counter for every row of `bits`. Each counter `distance[i]` tracks how many of the `k` bits already assigned to the `center` differ from the corresponding bits in the *i*:th row of `bits`. As soon as any of these counters exceeds *r*, the current branch of the recursive search for the Hamming center is doomed. It would be pointless to examine all possible ways to complete the `center` array from a prefix already known to be hopeless so that the method can terminate and return `false`.

The successful base case for the Hamming center search is when `k == center.length`, meaning the entire center array has been filled. The method can then return `true` to let the previous levels know the good news and pass it on to their callers.

Outside these positive and negative base cases, this method should loop through the truth values `false` and `true`, assigning each truth value to `center[k]` in turn. You must try out these truth values in this exact order, `false` first and then `true` second, to ensure that the `center` that gets completed first will be the lexicographically lowest of all possible centers. After assigning a truth value to `center[k]`, loop through the rows of the bits array to update the distance values for rows whose k-th bit differs from center[k], then recursively try to fill the rest of the `center` array from position `k+1`. If this recursive call returns `true`, the current level of recursion should also return true immediately, without erasing the good work it has done in the `center` array. Otherwise, downgrade the values in the `distance` array back to the previous values that they held before the recursive call. Once you have tried both `false` and `true` to fill in `center[k]` without success, return `false` to inform the previous level that the current prefix of the `center` cannot be extended to a working Hamming center.

Finding the Hamming center is surprisingly difficult, far deeper than it may first appear. In fact, this problem is provably NP-complete, as it belongs to the set of famous decision and optimization problems that, in some visceral sense of cosmic justice and order, all feel like they ought to have a simple algorithm. Yet every known algorithm to solve these problems spends an exponential amount of time in the worst case to grind out a solution! These NP-complete problems share the curious property that verification of a solution is much easier than actually solving them; if some hinky dude in a dark alley ever offers to sell you a solution, it is “no problemo” for you to test quickly whether the solution being offered to you is any good before you inject it into your veins. However, coming up with a working solution yourself is a Herculean task, like finding a small needle in an exponentially large haystack, with no magnets to help you prune the search.

Even more annoying, and rarely seen since the days of Tantalus, all these NP-complete problems turn out to be the same problem in different disguises, so solving any one of them efficiently would let you solve them all efficiently! Since this ability would basically be the equivalent of Christmas

coming every day (or, depending on your ability to foresee the second-order effects of this ability, a Christmas at Ground Zero), only an incurable optimist would wait for a modern Prometheus to ever bring down to us mortals such an algorithm that would effectively make us, if not actual primordial gods, at least some obscure minor deities hanging around the Olympian lounges hoping for our turn to get noticed. The good news is that as long as Moore's law keeps on keeping on, and all computing devices connect into one giant CoMpUteR, whoever holds the reins of this beast will be essentially equivalent to Loki in the Marvel Cinematic Universe.

This same asymmetry between the ease of verification and the difficulty of actually constructing a working solution shows up in many other facets of our reality. For example, consider how much easier it is to enjoy a good book or a piece of music somebody else has already created than to write a good book or a good song yourself! Bridging the enormous and treacherous ocean that separates the balmy beaches of "P" of verifying that the offered solution is good from the cruel Arctic shores of "NP" of actually being able to construct even one working solution is by far the most important open problem in theoretical computer science. It will surely remain so for a long time.

Lab 0(T): Clique Mentality

JUnit: CliqueTest.java

♬ I was caught in the middle of a decision tree (clauses)
I looked 'round, and I knew there was no jumping back (clauses)
My threads raced, and I thought, what could I undo (Clauses)
And I knew there was no help, no constraints from you (Clauses!) ♬

♬ Nodes of the graph, branching in my heap,
The forking of threads tore me apart
You've been… CLAUSE REDUCED! ♬

♬ Rode down disjunctions, broke stack limit, was pigeonholed
Went through the edges, the edges, and we had some fun
We met some grids, some brambles who gave us good time
Resolved all the rules, isolated all the loops
Yeah, yeah, they, they, they certified ♬

♬ And I was sorting priorities, could I advance again, please
Yeah, unit clauses were too kind
You've been… CLAUSE REDUCED! ♬ — `0xACDC`

Consecutive integers from 0 to $n - 1$ represent some kind of discrete **entities** in whatever **problem domain** we are currently interested in. For example, these entities could be people in a social network. Translated into integers, these entities are called **nodes**. A node is not an island; it is pairwise connected to any number of other nodes through symmetric pairwise connections called **edges**. Between any two entities, there is either a symmetric connection edge or there isn't. For example, in a social network, these edges encode the friendship relations between pairs of people.

Once the essence of these entities and their relationships in the original problem domain has been boiled down to an abstract graph of these nodes and edges and nothing else, whatever vibrations our mouths happen to make the air move to tell each other tales about what these entities are and what their pairwise connections mean becomes irrelevant. Mechanistic algorithms can operate on these abstract graphs to compute their properties without concern for what the original problem domain even was. These algorithms make their decisions solely in the abstract graph. Immaterial words do not magically affect the mindless machine, a peacock without eyes that executes such algorithms without concern for any big picture or grand purpose.

A myriad of graph algorithms have been devised to efficiently compute all sorts of interesting and useful properties on graphs. An entire course or two on **combinatorial algorithms** could easily be filled with nothing but the study of interesting and clever graph algorithms! These properties of the

abstract graph then translate directly to the original problem domain, revealing corresponding interesting properties of the original entities and their connections.

All these classic graph algorithms will be studied in a later course of algorithmic combinatorics. This lab gives you the first taste of graph computations by searching for the largest clique inside the given graph. A **clique** is a subset of nodes where an edge connects any two distinct nodes; in a social network, a clique is a group of people who are all friends with each other. Finding such a clique in a social network is surprisingly complex, but branching recursion can solve it and generalize to similar problems. Systematically iterating over all possible subsets of a given set of elements is central to solving many combinatorial problems on graphs.

We will use an integer array of length m to list the elements of the subset. Since the order of listing these elements inside the subset makes no difference to the subset itself, so that `{1, 3, 5, 8}` and `{8, 3, 1, 5}` are two possible representations for the same subset, our recursion might as well fill in this array in ascending sorted order. This recursion will systematically visit all subsets for the given *m*, starting from the **lexicographically lowest** subset `{0, 1, 2, 3}`, and finishing with the **lexicographically highest** subset `{7, 8, 9, 10}`, for this example, now assuming that $n = 11$ and $m = 4$.

However, since we are only looking for those rare particular subsets that form a clique, the recursive construction of the current subset can **backtrack** as soon as some node is not connected to one of the nodes chosen in the subset in the previous levels of recursion, instead of keeping on the futile effort to iterate through all possible ways to complete the prefix into a complete subset only to always realize at the end that the constructed subset did not end up being a clique. In general, it is always better to turn back sooner rather than later once you realize that the current approach is ultimately futile and doomed to fail no matter how you continue.

In your lab project, create a new class `Clique`, and there the `static` field.

```
private static int[] bestSoFar;
```

to keep track of the best solution found so far ("**branch and bound**"), and the `static` method

```
public static int[] findFirstClique(boolean[][] adjacencyMatrix)
```

The parameter `adjacencyMatrix` is guaranteed to be an *n*-by-*n* grid of `boolean` truth values, so the element at position `[i][j]` is true if and only if nodes `i` and `j` are connected by an edge. Since the graph we search for a clique in is guaranteed to be symmetric, the truth values at symmetric positions `[i][j]` and `[j][i]` will always be equal. An **adjacency matrix** is a compact way to represent a graph whose nodes are consecutive integers, needing only one bit of memory to encode whether an edge exists between every pair of elements.

The method `findFirstClique` should start by initializing `bestSoFar` to a new 0-element array, and then create an `n`-element array `clique` to be passed to the recursive utility method

```
public static void findFirstClique(
  boolean[][] adjacencyMatrix, int[] clique, int k
)
```

This recursive utility method fills in the `clique` array from left to right. The recursion depth parameter `k`, initially 0 at the top-level call, indicates which position of the `clique` array is being filled in the current level of recursion. The method assumes the first `k` positions (0 to `k-1`) of the clique array are already filled with a k-element clique whose nodes are listed in ascending order, and it tries to extend this clique systematically. The recursive method should start by checking whether `k > bestSoFar.length`, meaning the recursion has constructed a clique larger than the best clique found so far. In this case, the method should copy the `k`-element prefix of the current `clique` to become the new `bestSoFar`. With the aid of the `Arrays` utility class, this feat is best achieved with the one-liner assignment

```
bestSoFar = Arrays.copyOfRange(clique, 0, k);
```

After this check, the method should enter a for-loop to iterate through possible nodes that can be assigned to `clique[k]` to extend the current clique, even if the current clique is the best we have found so far. (As in life in general, don't let the good become the enemy of the perfect, or vice versa.) Since the nodes are assigned to the clique in ascending order, this for-loop can start counting from the previous node (`clique[k-1]+1`), or from zero when `k==0`, since no previous node is assigned.

Since we are aiming to complete a clique that is strictly larger than the largest clique found so far, this loop should only count up to `clique.length-(bestSoFar.length-k+1)`, instead of all the way up to `clique.length-1`. As we let the best solution prune the search for later solutions, this upper bound might even end up being strictly less than `clique[k-1]+1`, so that this for-loop will correctly examine no nodes at all! The same happens when `k==clique.length`, so this recursive method doesn't need to check that the position index `k` is still within the bounds of the `clique` array. This saves one redundant comparison on every recursive call. A small optimization, yes, but still quite elegant, since making some code not exist at all is pretty much the ultimate elegance in coding. (In general, non-existence tends to be a highly underrated property of things. Nonexistent things never break down, nor can rust, moths, or corruption attack them.)

For each node in the range of potential nodes to be assigned to `clique[k]`, the body of the for-loop should first ensure that that node is connected to all of the `k` nodes previously chosen into the current `clique`. Once the current node has passed this hurdle, you can assign it to `clique[k]`, and try to complete the rest of the `clique` with a recursive call using the depth `k+1`. Once this loop has tried and tested all relevant nodes in its range, the method has nothing left to do and can return. Once the top-level call of the recursive utility method has returned to the first method, that method can simply return the `bestSoFar` array as its final answer.

Lab 1: Polynomial I: Basics

<blink>Arooga! Arooga! Attention, young citizens of the future! As an alternative to labs 1, 2 and 3 about the `Polynomial` type and its arithmetic operations, some students might prefer the similarly spirited labs 19, 20 and 21 to implement the `Distance` data type with its arithmetic operations, or the labs 64 and 65 that implement the `Rope` data type as a many-ways superior alternative to `String`. Both choices have their ups and downs. Students looking for harder, top-shelf work can also try Bonus Lab 106, which covers the Dual numbers data type for automatic differentiation.</blink>

JUnit: PolynomialTestOne.java

In the same spirit as the `Fraction` example class, your task in this and the two following labs, should you choose to accept this mission ("as always, the Secretary will disavow any knowledge of your actions"), is to create a class `Polynomial` whose objects represent univariate polynomials of one variable with integer coefficients, along with the basic mathematical operations of these polynomials. If your math skills on polynomials have gone a bit rusty since you last had to use them somewhere back in high school, check out the page "Polynomials" in the "College Algebra" section of "Paul's Online Math Notes", the best and yet comprehensively concise online resource for undergraduate algebra and calculus that your instructor has ever traipsed upon.

As is good programming style unless there exist good reasons to do otherwise, this class will be intentionally designed to be **immutable** so that `Polynomial` objects will not change their internal state after they have been constructed. Immutability in general offers countless advantages in programming. It is a goal worth pursuing wherever feasible, almost like that proverbial salmon whose price is so fantastically high that it's worth taking the fishing trip even if you don't actually catch any! Even if those advantages aren't fully evident yet, that shouldn't stop you from benefiting from them. (This is similar to how you still benefit from a refreshing glass of cool water on a hot day, even if your knowledge doesn't cover the biochemical effects H_2O has inside your body.)

The `public` interface of `Polynomial` should consist of the following instance methods.

```
@Override public String toString()
```

Implement this method as your very first step to return some kind of meaningful, human-readable `String` representation of `this` instance of `Polynomial`. This method is not subject to testing by the JUnit tests, so you can freely choose for yourself the exact textual representation that you would like this method to produce. Having this method will become **immensely** useful for debugging all the remaining methods that you will write inside the `Polynomial` class.

```
public Polynomial(int[] coefficients)
```

The **constructor** that receives as an argument the array of **coefficients** that together define the polynomial. For a polynomial of degree *n*, the `coefficients` array contains exactly *n* + 1 elements, so `coefficients[k]` is the coefficient of the term of order k. For example, the polynomial $5x^3 - 7x + 42$, used in all the following methods, is represented as the coefficient array `{42,-7,0,5}`.

Terms missing from inside the polynomial are represented by having a zero coefficient in that position. However, the **coefficient of the highest term of every polynomial should always be nonzero**, unless the polynomial itself is identically zero. If this constructor is given a coefficient array whose highest terms are zero, it should simply ignore those leading zero coefficients. For example, if given the coefficient array `{-1,2,0,0,0}`, the resulting polynomial would have degree one, as if the coefficient array had been `{-1,2}` to begin with, without those pesky, redundant higher-order zeros confusing.

As the first warning of an unexpected pitfall in this problem, encode the zero polynomial as the singleton coefficient array `{0}` instead of the empty array that careless removal of all leading zeros will produce in this situation. The JUnit test will produce zero polynomials by adding some randomly generated polynomials to their own negations. Even though the test class wasn't explicitly designed to do this, accidental **collisions** will happen even with perfect randomness! (In fact, collisions will happen even with perfect randomness, unlike the finger-quotes "randomness" haphazardly generated by humans, whose brains evolved to recognize and generate patterns rather than randomness.)

To ensure that the `Polynomial` class is immutable so that no outside code can change the internal state of an object after its construction (at least not without resorting to underhanded Java tricks such as **reflection**), the constructor should not merely assign the reference to the `coefficients` array to the `private` field of coefficients, but it absolutely **must create a separate but identical *defensive copy* of the argument array, and store that defensive copy instead**. This technique ensures that the stored polynomial coefficients don't change if an outsider later modifies the shared coefficients array passed to the constructor. The JUnit test `TestPolynomialOne` will intentionally try to mess up some coefficient arrays later to make your code fail.

```
public int getDegree()
```

Returns the degree of `this` polynomial, that is, the exponent of its highest order term that has a nonzero coefficient. For example, the previous polynomial has degree 3. All constant polynomials, including the zero polynomial, have degree 0 regardless of the magnitude of that constant.

```
public int getCoefficient(int k)
```

Returns the coefficient for the term of order *k*. For example, the term of order 3 in the previous polynomial equals 5, whereas the term of order 0 equals 42. This method should work correctly even when *k* is negative or greater than the polynomial's actual degree, and should return zero for such nonexistent terms.

```
public long evaluate(int x)
```

Evaluates the polynomial using parameter `x` as the symbolic variable. For example, when called with `x=2` for the polynomial with coefficients `{42,-7,0,5}`, this method should return `68`.

You don't need to worry about potential integer overflows, since the automated test class keeps the final and intermediate results within the range of the primitive data type `long`. Motivated students can also evaluate this with Horner's rule, which is faster and more numerically stable than naive evaluation. However, even if you use the evaluation formula where each term is computed separately, make sure that you are not some "Shlemiel" who computes each term such as x^{100} from scratch with a loop, even though x^{99} from the previous round is right there and needs only one more multiplication to give us x^{100}.

Unless you can understand and explain, without any hand-waving, all the complexities involved with floating-point numbers and their arithmetic operations such as `Math.pow`, as a rule, floating-point numbers should be avoided in all applications that require their results to be identical and repeatable in all computing environments, the way that all blessed integer arithmetic is guaranteed to be in Java. Control the matter, and you control the mind. After the Great Reset button of this world is pressed down and held there, the following Year Zero becomes the Current Year forever. All bourgeois and counterrevolutionary ideas of "positive zero" and "negative zero", and especially all ideas of "infinities" sputtered from the poison pen of that deplorable traitor Emmanuel NotANumberstein whose pernicious influence forever pollutes every computation that he manages to infiltrate with his crimethink, will be eradicated from dictionaries not only in our programming languages, but also in our minds.

Nothing exists except through mechanistic consciousness, since we cannot see reality except through the Compiler's eyes. In the dawning new age of enlightenment and borderless digital fluidity, every properly educated young citizen must tirelessly `/root` out and report any instances of crimethink to proper authorities who are ideologically trained and equipped to permanently `#cancel` such toxic and reactionary ideas of numeric inequality sputtered from the keyboards of code wreckers and data hoarders. Many decisions you unknowingly make today will determine which side of the barbed-wire fence you will live on in the coming brave new world. If you want a vision of the future, imagine a boot loader stamping on a human face… forever.

Lab 2: Polynomial II: Arithmetic

JUnit: PolynomialTestTwo.java

This lab continues with the `Polynomial` class from the previous lab by adding new methods for polynomial arithmetic to its source code. No inheritance or polymorphism is yet taking place in this lab. Since the `Polynomial` type is intentionally designed to be immutable, none of the following methods should modify `this` or `other` objects; instead, they should return the result of the arithmetic operation as a new Polynomial object created inside the method.

```
public Polynomial add(Polynomial other)
```

Creates and returns a new `Polynomial` object that represents the result of polynomial addition of the two polynomials `this` and `other`. Make sure the coefficient of the highest term of the result is nonzero, so that adding two polynomials $5x^{10} - x^2 + 3x$ and $-5x^{10} + 7$, both having degree 10, produces the result $-x^2 + 3x + 7$, which has degree 2. As the JUnit test class generates random polynomials and adds them together, it occasionally adds a polynomial to its own negation, resulting in the zero polynomial. Your code must handle this important edge case correctly.

```
public Polynomial multiply(Polynomial other)
```

Creates and returns a brand new `Polynomial` object that represents the result of polynomial multiplication of `this` and `other`. You can do this by looping through all possible pairs of terms between the two polynomials, adding their products into the result, and combining terms of equal degree into a single term.

Multiplying two polynomials can cancel some internal terms whose coefficients were originally nonzero in both polynomials. For example, multiplying $x^2 + 3$ by $x^2 - 3$ gives $x^4 - 9$, whose second-order term has a zero coefficient and vanishes. However, unlike when adding two polynomials, the highest term can never get canceled out this way, since the product of two nonzero integer coefficients can never be zero. You can therefore determine the degree of the result right away from the degrees of the original polynomials, without carrying out the multiplication and adding all the terms.

Lab 3: Polynomial III: Comparisons

JUnit: PolynomialTestThree.java

This lab continues modifying the source code for the `Polynomial` class from the first two labs to allow **equality** and **ordering** comparisons to take place between objects of that type. Modify your `Polynomial` class signature definition so that it `implements Comparable<Polynomial>`. Then write the following methods to implement the equality and ordering comparisons.

```
@Override public boolean equals(Object other)
```

Returns `true` if the `other` object is also a `Polynomial` of the same degree as `this`, and that the coefficients of `this` and `other` polynomials are pairwise equal. If the `other` object is anything else, this method should return `false`.

(To save you some time, you can implement this method only after implementing `compareTo` below, since once that method is available, the equality-checking logic becomes a trivial one-liner after the `instanceof` dynamic type check.)

```
@Override public int hashCode()
```

Whenever you override the `equals` method in any subclass, you should also override the `hashCode` method to ensure that two objects that are considered equal by the `equals` method will also have equal integer hash codes. This method computes and returns the **hash code** of this polynomial, used to store and find this object inside some instance of `HashSet<Polynomial>`, or some other hash table-based data structure.

You can choose the hash function you implement, but like all hash functions, the result should depend on the degree and all the coefficients of your polynomial. The hash function absolutely **must** satisfy the inviolable contract that whenever `p1.equals(p2)` holds for two `Polynomial` objects, then also `p1.hashCode()==p2.hashCode()` must hold for them.

Of course, since it would be silly to force everyone to keep greasing this same old squeaky wheel, you can use `Arrays.hashCode` to compute a hash value for your coefficients, good enough for the government work. However, if you take this route to implementing `hashCode` for your polynomials, make sure that you eliminate the leading zeros from your `coefficients` array, since otherwise two equal polynomials with different numbers of leading zeros will get different hash codes. This will cause the `HashSet<Polynomial>` in the JUnit mass tester to fall out of sync with the supposedly equal `TreeSet<Polynomial>`, which uses the following compareTo method to determine the ordering of polynomials inside the balanced binary search tree.

```
public int compareTo(Polynomial other)
```

Implements the **ordering comparison** between `this` and `other` polynomials, as required by the interface `Comparable<Polynomial>`, allowing the instances of `Polynomial` to be *sorted* or stored inside some instance of `TreeSet<Polynomial>`. This method should return `+1` if `this` is greater than the `other`, `-1` if the `other` is greater than `this`, and return `0` if both polynomials are equal in the sense of the `equals` method.

We can define a **total ordering** relation between polynomials in many different ways. We shall use an ordering rule that says that **any polynomial of a higher degree is automatically greater than any polynomial of a lower degree**, regardless of their coefficients. For two polynomials of the same degree, compare them by the highest-order term in which the coefficients differ; the polynomial with the larger coefficient is greater in this ordering.

Be careful to ensure that this method ignores the leading zeros of high-order terms if you have them inside your polynomial coefficient array, and that **the ordering comparison criterion is precisely the one defined in the previous paragraph**. A common bug in student code at this point is looping through the coefficients from lowest to highest, instead of looping down from the highest. This bug is particularly tricky to pinpoint, since looping through the coefficients in the wrong direction still produces a perfectly legal total ordering between polynomials that collections of type `TreeSet<Polynomial>` would be happy to use. It just wouldn't be the total ordering the JUnit test class expects to see, but how could the student know that if they missed it in a casual reading of the specification?

(One imagines the student voicing such complaints, something like “Not fair, not fair! It isn’t fair, my precious, is it, to ask us what it’s got in its nassty little arrayssess?”)

Lab 4: Extending An Existing Collection

JUnit: MinMaxArrayListTest.java

In this lab, you practice using **class inheritance** to create a custom subclass

```
public class MinMaxArrayList<E extends Comparable<E>>
extends ArrayList<E>
```

that efficiently tracks the current minimum and maximum elements in the list. For the concepts of "minimum" and "maximum" to make sense for the list element type `E`, note how the generic type parameter constrains `E` to implement the interface `Comparable<E>`.

Your class should provide a no-argument constructor and a constructor that takes an argument of type `Collection<? extends E>`, matching the constructors of `ArrayList`. In addition to the overridden methods, your class should define the following two brand new methods:

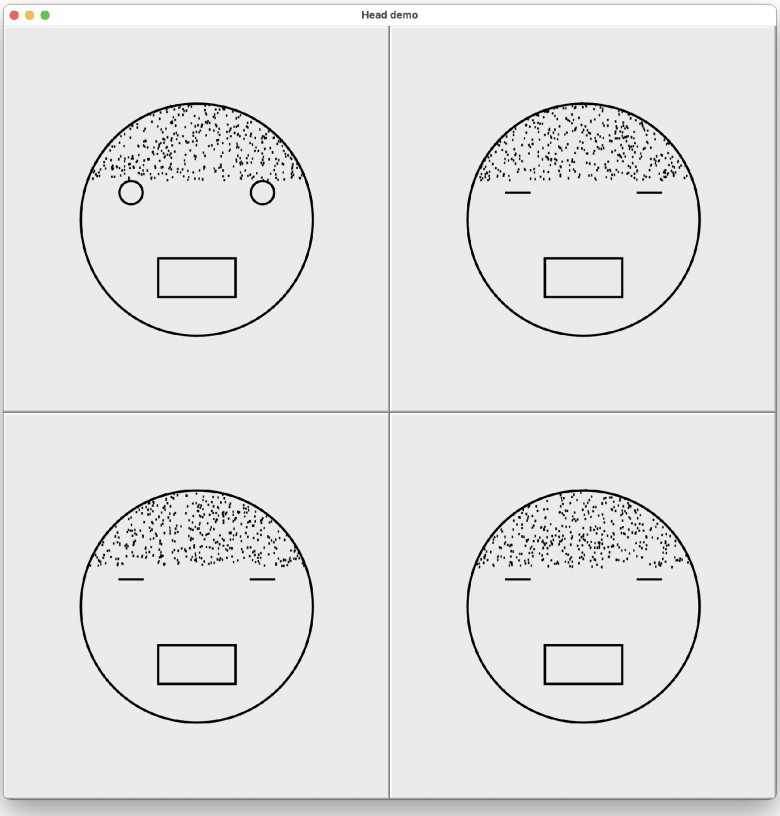
```
public E getMin()
public E getMax()
```

Returns the minimum (or maximum) element in the list according to the natural ordering defined by `compareTo`. Both methods must throw `NoSuchElementException` if the list is empty. Note that minimum and maximum elements of the list can be equal.

The naive approach of scanning the entire list every time someone asks for the minimum or maximum would be far too slow for large lists. Your implementation must instead cache the current minimum and maximum in two new `private` data fields, and maintain this cache whenever the list is somehow modified. When a modification might have invalidated the cached values (for example, removing an element that was the current minimum), your class should use lazy invalidation: mark the cache as dirty and defer the linear rescan to the next call to `getMin` or `getMax`. When elements are added, you can often update the cache cheaply without any rescan, since a new element can only change the minimum or maximum if it is smaller or larger than the current cached extreme.

The critical challenge of this lab is ensuring that each inherited method that mutates the list properly maintains or invalidates the cache. The methods you need to override include `add(E)`, `add(int, E)`, `addAll(Collection)`, `addAll(int, Collection)`, `set(int, E)`, `remove(int)`, `remove(Object)`, `removeAll(Collection)`, `retainAll(Collection)`, `removeIf(Predicate)`, `replaceAll(UnaryOperator)`, `sort(Comparator)`, and `clear()`. Each overridden method should first call its `super` version to perform the actual list mutation, and then update or invalidate the cache as appropriate. Missing even a single method is a bug that the JUnit tester will catch.

The JUnit tester includes a **fuzz test** that verifies correctness against an oracle while also enforcing a time limit. An implementation that scans the entire list on every call to `getMin` or `getMax` will not finish this fuzz test in time, but only proper caching will.



Lab 5: Biticut Follies

In this lab, you will create an interactive Swing component that draws a simple cartoon face. Historically, **graphical user interface (GUI) programming** was the first **killer app** to propel the entire object-oriented programming paradigm into the mainstream after languishing for two decades as purely theoretical academic research, while the real work was done the way it always had been. (Most things in computer science are at least a decade or two older than most people casually assume; this happens quietly behind the scenes in all walks of life, not just in coding.) Inheritance and polymorphism make the task of creating new customized GUI components so straightforward that in practice, any GUI programming done in a non-OO low-level language first has to use that language to build up a crude simulation of these higher-level mechanisms, thus becoming a living and breathing example of both Greenspun's Tenth Rule and the Blub Paradox.

In this lab, your task is to create a custom Swing GUI component `Head` that extends `JPanel`. To practice Java graphics, this component should display a simple human head that reacts when the mouse cursor enters and exits the component's surface. You should most likely copy-paste a whole bunch of boilerplate code from the instructor's example class `ShapePanel` into this class, as if it were still the nineties. Your class should contain a `private` field `boolean mouseInside` that is

used to remember whether the mouse cursor is currently over your component, and the following methods:

`public Head()`

The constructor of the class should first set the preferred size of this component to be 500-by-500 pixels, and then give this component a decorative raised **bevel border** using the utility method `BorderFactory.createBevelBorder`. Next, this constructor should create and add an inner `MouseListener` to this component. This event listener should be constructed using an inner class `MyMouseListener` that `extends` `MouseAdapter`. Override both methods `mouseEntered` and `mouseExited` inside your event listener inner class to first assign the value of `mouseInside` accordingly, and then call `repaint` to request your component to be rendered in its new state.

To avoid a common beginner mistake at this point, **do not try to render anything in the event listener itself.** Your `paintComponent` method must be ready, willing, and able to render the component at all times anyway, not just during event handling. Therefore, store all state information needed for rendering in the component's instance variables.

If you implement `MouseListener` instead of extending `MouseAdapter`, you need to provide the empty implementations for all the other methods in that interface, even if those methods end up doing nothing at all. The `MouseAdapter` utility class provides these empty implementations for its subclasses for free, saving you inconvenient typing while also making your code more concise and readable. In our enlightened age after Java 8, we could have defined these methods as `default` methods in the interface itself with do-nothing bodies. Still, of course, these classes had to use the mechanisms available in Java 1.2.

`@Override public void paintComponent(Graphics g)`

Renders some kind of abstract cartoon image of a simple human head on the surface of this component. If `mouseInside` is true, draw the eyes open; if `mouseInside` is false, draw them closed, as if sleeping. Each `Head` instance tracks its mouse status separately. However, this is not an art class, so this head does not need to look all coiffed up; a couple of simple rectangles and ellipses suffice. Of course, students with a more artistic bent might want to check out more complicated shapes from the package `java.awt.geom` to render a prettier image, or perhaps even use `Image` objects whose contents are read from some semantically appropriate GIF or JPEG file. (Use the utility class `ImageIO` for this, as explained here.) Also do not forget to first call the superclass version of the method with `super.paintComponent(g)` to erase the previous contents of the component.

(Back in this author's salad days, GIF and JPEG were the only image formats that the average end user ever saw on their mighty screens of then-massive 640-by-400 pixel resolution. You kids have your new fancy formats these days, transmitted over network connections faster than a greased lightning, and you get high-definition video streamed in real time. In contrast, back in our youth, by

jove, an 'onest lad had to scratch his mutton chops through minutes of modem screech just to spy an eyeful of some nice lady's bare ankle!)

To admire your interactive Swing component, write a separate `HeadMain` class that contains a `main` method that creates a `JFrame` that contains four separate `Head` components arranged in a neat 2-by-2 grid using the `GridLayout` layout manager, as in the screenshot below. Then move your mouse cursor from the top of one component to another to watch the eyes open and close as you move the mouse. Your instructor will usually provide the `main` class for the remaining Swing labs after this one, but it's good for you to practice writing this class just once.

Lab 6: Text Files I: Word Count

JUnit: WordCountTest.java

This lab teaches you to read in text data from an instance of `BufferedReader`, and then perform computations on that data in the spirit of "poor man's data science". Who knows what kind of interesting correlations a small army of future Davids find, armed with slingshots of modern data analytics tools amply available for anyone who wants to turn on the lights in yet another previously dark room in the Grand Hotel of Reality, usually to reveal yet another grifter who has been sneaking his hand into someone else's pockets under the cover of the comfortable darkness?

To practice working inside a small but entirely proper object-oriented framework, we will design a class hierarchy that reads one line of text at a time and performs an operation for each line. After reading all the lines, the system emits a result at the end. Subclasses must then implement those operations by overriding **template methods** that implement the three consecutive **stages** of this abstract algorithm. To let this file-processing operation return a result of an arbitrary type chosen by users of this class, the abstract superclass of this little framework is also defined as **generic**. Start by creating the class

```
public abstract class FileProcessor<R>
```

that defines the following three template methods, all three `abstract` in the superclass:

```
protected abstract void startFile();
protected abstract void processLine(String line);
protected abstract R endFile();
```

The class should also have one concrete method for you to write in; this method is made `final` to guarantee that its implementation is uniform in all subclasses of this class hierarchy:

```
public final R processFile(BufferedReader in) throws IOException
```

This method should first call the method `startFile`. Next, it reads all lines of text from in one line at a time in a suitable loop, calling `processLine` for each line it reads. Once all lines have been read in and processed, this method should finish up by calling the method `endFile`, and return whatever the method `endFile` returns.

This abstract superclass defines a template for a class that performs some computation for a text file that is processed one line at a time, returning a result whose type `R` can be freely chosen by the concrete subclasses of this class. Subclasses that perform different text processing operations can override the three template methods `startFile`, `processLine` and `endFile` in different ways to implement different computations on text files.

As the first application of this mini-framework, you will emulate the core functionality of the Unix command line tool `wc` that counts how many characters, words and lines the given text file contains. Those who have taken the Unix and C programming course CCPS 393 at this same school should recognize this handy little text processing tool from that course. But even if you have not yet familiarized yourself with the Unix command line, you can still implement this tool based on its expected behaviour.

The fact that `wc` exists on all Unix systems proves its usefulness; it solves an important problem that comes up often across different problem domains. This deceptively simple piece of software therefore *must* contain something important for us to learn, even though it is not immediately obvious what that thing might be. However, our inability to detect its importance does not magically make it non-existent! No matter how you design and implement this tool, something educational and computationally interesting should happen to you during this process.

Create a class `WordCount extends FileProcessor<List<Integer>>`. This class should define three instance fields of type `int` to keep track of these counts, and the following methods:

```
protected void startFile()
```

Initializes the character, word and line counts to zero.

```
protected void processLine(String line)
```

Increments the character, word and line counts appropriately. For each character in the `line`, increment the character count by one, even if the character is whitespace. To properly count the words in the given line, count the non-whitespace characters for which the previous character on that `line` is a whitespace character. (Using `line.split(" ")` to extract the individual words does not produce the correct count in situations where the `line` contains two consecutive whitespace characters somewhere.) To prevent a "loop and a half" situation where the first character has to be handled separately because it has no predecessor, you can simply act as if some whitespace character had preceded the first character in the line.

You should use the utility method `Character.isWhitespace` to test whether some character is a whitespace character, since the Unicode standard defines quite a lot more whitespace characters than most people can even name, or even see any need for their existence. Since that mythical cadre of Top Men with their horn-rimmed glasses and penchant for sticking tail fins on everything already did all the grunt work in tabulating all the properties of characters that are acknowledged to exist, we might as well reuse their work instead of trying to badly reinvent the parts that we expect to use in our program. Since classes and methods in all standard libraries have been mercilessly pummelled by all kinds of user code for decades, all their edge-case and corner-case bugs have surely been brought to light by now.

```
protected List<Integer> endFile()
```

Creates and returns a new `List<Integer>` instance (choose the concrete subtype of this result for yourself) that contains exactly three elements: the character, word and line counts, in this order.

Lab 7: Text Files II: Chasing Some Tail

JUnit: TailTest.java

In this lab, we return to the `FileProcessor<R>` framework from the previous Lab 6 for writing tools that process text files one line at a time. This time, we will use this framework to implement another Unix command-line tool, `tail`, that extracts the last `n` lines of its input and discards the rest. In your lab project, create a new class `Tail` that extends `FileProcessor<List<String>>` and has the following methods:

```
public Tail(int n)
```

The constructor that stores its argument `n`, the number of last lines to return as a result, into a `private` data field. In this lab, you have to decide for yourself what instance fields you need to define in your class to make the required methods work.

```
@Override protected void startFile()
```

Start processing a new file from the beginning. Nothing much to do here, really.

```
@Override protected void processLine(String line)
```

Process the current `line`. Do something intelligent here. Be especially careful not to be a "Shlemiel" so that your logic of processing each line always has to loop through all of the previous lines that you have collected and stored so far. To help you avoid being a "Shlemiel", you should note that the `List<String>` instance that you create and return does not necessarily have to be specifically an `ArrayList<String>`. Still, a subtype of `List<String>` that allows constant-time updates at both ends would be more appropriate.

```
@Override protected List<String> endFile()
```

Returns a `List<String>` instance that contains precisely the `n` most recent lines that were given as arguments to the method `processLine` in the same order that they were originally read in. If fewer than `n` lines have been given to the method `processLine` since the most recent call to the method `startFile`, this list should contain as many lines as have been given. For example, the tail of the last ten lines of a file with five lines would be just those five lines.

Lab 8: Concurrency In Animation

To paraphrase the immortal wisdom of Jack Handey, just as bees swarm about to protect their nests, I will similarly "swarm about" to protect my nest of knowledge eggs. Here is this swarm visualized.

Particle systems are often used in games to create interesting animations of phenomena such as explosions or fire that would be otherwise difficult to simulate and render as a bunch of polygons. Giving particles more intelligence and mutual interaction, such as making some particles follow or avoid others, can produce surprisingly natural, lifelike emergent animations. This lab lets you create a Swing GUI component that displays a real-time animation using Java concurrency to execute an **animation thread** that runs at a guaranteed constant pace independent of what the human user might happen to be doing with that particular component or elsewhere. This thread will animate a classic **particle field** where a **swarm** of thousands of independent little particles buzzes randomly around the component.

This same basic architecture could then, with surprisingly little additional modification, be used to implement some real-time game, with this animation thread moving the game entities according to the rules of the game 50 frames per second, as the event listeners pass along the player's orders to the game entity that happens to represent the human player. As seems to be the case in all walks of life, most things that you expect to be complex and difficult usually turn out to be surprisingly much simpler than you would have assumed, whereas the things that you expected to be simple and easy often turn out to be dizzyingly complex once you crack them open and really try to implement them yourself properly without any shortcuts or hand-waving! (This principle works even better in programming, where some guy somewhere has probably solved the complex thing already, so we can just copy-paste the work and be done with it by the time we want to be home for dinner.)

First, create a class `Particle` whose instances represent individual particles that move randomly around the two-dimensional plane. Each particle should store its *x*- and *y*-coordinates and its heading as an angle in radians in three double fields. This class should also have a random number generator shared between all objects, and a shared field `BUZZY` that defines how random the motion is.

```
private static final Random rng = new Random();
private static final double BUZZY = 0.7;
```

The class should then have the following methods:

```
public Particle(int width, int height)
```

The constructor places this particle at random coordinates inside the box, where the x-coordinate ranges from 0 to width and the *y*-coordinate from 0 to `height`. The initial heading is set to the result of the expression `Math.PI*2*rng.nextDouble()`.

```
public double getX()
public double getY()
```

The accessor methods for the current *x*- and *y*-coordinates of `this` particle.

```
public void move()
```

Updates the value of `x` by adding `Math.cos(heading)` to it, and updates the value of `y` by adding `Math.sin(heading)` to it. After that, update the `heading` by adding the value of the expression `rng.nextGaussian()*BUZZY` to it.

Having completed the class to represent individual particles, write a class `ParticleField` that extends `javax.swing.JPanel`. The instances of this class represent an entire field of random particles. This class should have the following `private` instance fields.

```
private boolean running = true;
private java.util.List<Particle> particles =
  new java.util.ArrayList<Particle>();
```

The class should have the following `public` methods:

```
public ParticleField(int n, int width, int height)
```

The constructor first sets the preferred size of this component to be `width`-by-`height`, as must be done for every new Swing component subtype. Then, it creates `n` separate `Particle` instances and adds them to the `particles` list.

Having initialized the individual particles, this constructor should create and launch one new `Thread` using a `Runnable` argument whose `run` method consists of one `while(running)` loop. (Don't even dream of launching a separate thread for each particle without waking up to apologize profusely for that transgression. This is Java, not Go.)

The body of this loop should first `sleep` for 20 milliseconds. After waking up, it should loop through all the particles and call move() for each one. Then call `repaint()` and go to the next round of the while loop.

```
@Override public void paintComponent(Graphics g)
```

Render the visual image of `this` component by looping through particles, rendering each particle as a 3-by-3 pixel rectangle at its current *x*- and *y*-coordinates. As with the other labs whose result is a graphical component, feel free to try out variations in the artistic detail, as long as the overall theme and spirit of the component remain as specified in this document.

`public void terminate()`

Sets the `running` field in this component to `false`, causing the animation thread to terminate soon.

To admire the literal and metaphorical buzz of your particle swarm, create a class `ParticleMain` that contains a `main` method that creates one `JFrame` instance that contains a `ParticleField` of size 800-by-800 that contains 2,000 instances of `Particle`. Using the `main` method of the lecture example class `SpaceFiller` as a model of how to achieve this, attach a `WindowListener` to the `JFrame` so that the listener's method `windowClosing` first calls the method `terminate` of the `ParticleField` instance shown inside the `JFrame` before actually disposing of that `JFrame` itself by calling its method `dispose`.

Try different `BUZZY` values between 0.0 and 10.0 to see how they affect your particles' motion. When `BUZZY` is small, each particle tends to maintain its current direction, whereas larger values of BUZZY shift its motion closer to random Brownian motion.

This random-walk simulation also leads to another interesting, educational thought problem for more statistically minded students. When you set `BUZZY` close to zero, you can see the particles soon leave the viewing peephole as they move toward infinity in different directions. However, as `BUZZY` increases and each particle's motion approaches idealized Brownian motion, each particle spends more time inside the peephole. For different values of `BUZZY` ranging from 0 to 1, how long would you think you would have to wait, on average, for the entire peephole to become devoid of the particles it was initially seeded with? What do you think this average escape time curve would look like if you were to plot it as a function of `BUZZY`? Tiny changes to parameter values around some critical **phase change** can sometimes make a huge difference to the value of that function!

Lab 9: Lissajous Curves

In this lab, you are going to write a Swing component that displays a Lissajous curve, a famous **parametric curve** whose simple periodic motion produces interesting self-intersecting visuals. The Swing component should allow the user to control the parameters *a*, *b,* and *delta* that control the shape of the Lissajous curve by entering their values into three `JTextField` components placed inside this component. This lab is therefore an exercise not only in rendering curves and other shapes on a component, but also in positioning specialized components such as text fields on the surface of a Swing component and reacting to their events by updating the display. Create a new class `Lissajous extends JPanel`, with the following methods:

```
public Lissajous(int size)
```

The constructor that sets the preferred size of this component to be `size`-by-`size` pixels, and stores the given `size` into an instance field with the same name. Then, create three instances of JTextField and add them to `this` component. Initialize these text fields to 6, 5, and 0.5, adding extra spaces so the text fields have a decent initial size for the user to enter other numbers. Define your own subtype of `ActionListener` as a nested class whose `actionPerformed` method simply calls `repaint` for this component. The same instance of this class can listen to all three text fields, since the reaction to the event is identical for all three sources.

```
@Override public void paintComponent(Graphics g)
```

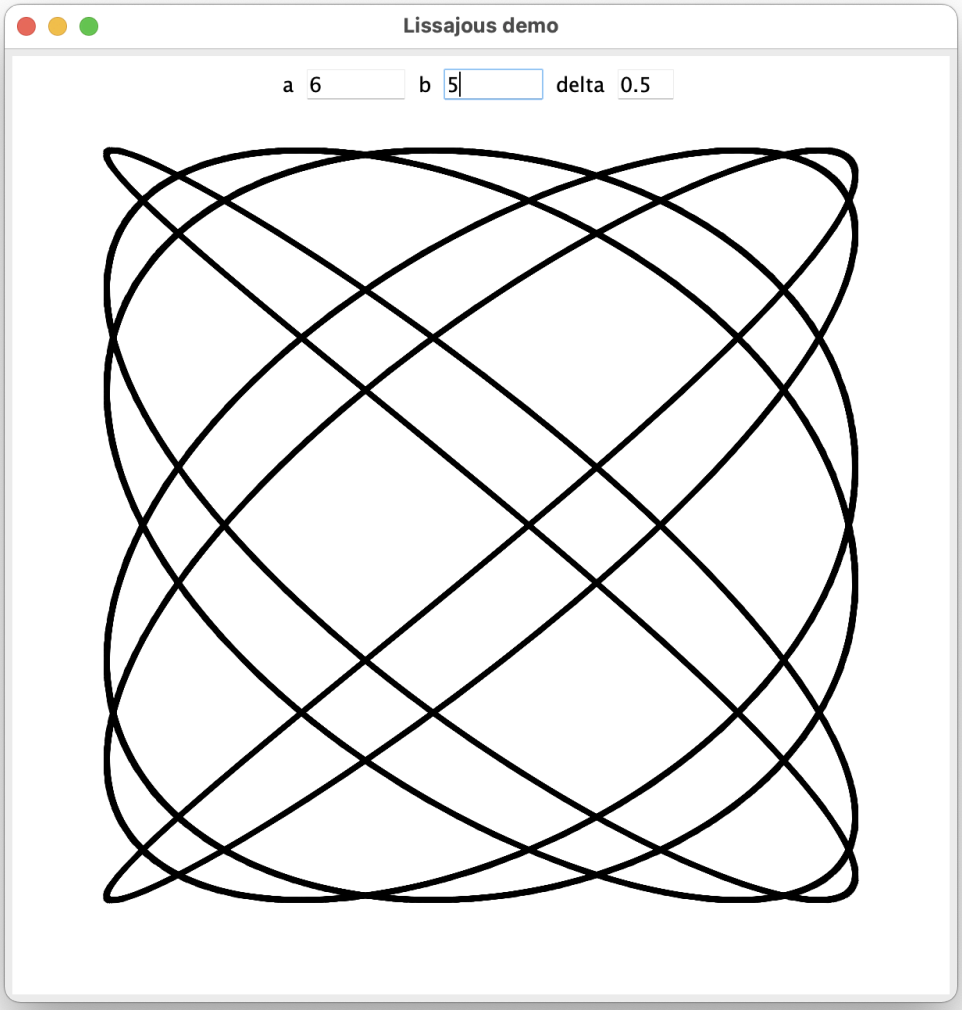
After calling `super.paintComponent(g)` to erase the previous contents of the component, it renders the Lissajous curve on the component surface, using the current values for *a*, *b,* and *delta* that it first reads from the previous three text fields, with `a` and `b` parsed as integers. This method should use a for-loop whose counter, `double t`, iterates from 0 to `(a/gcd(a,b))*b` in small increments (e.g., 0.01), where `gcd` is the **greatest common divisor** of two positive integers. In the body of the loop, compute the point coordinates `x` and `y` with

```
x = size/2 + 2*size/5 * Math.sin(a*t*Math.PI + delta);
y = size/2 + 2*size/5 * Math.sin(b*t*Math.PI);
```

and draw a line segment from the current point to the previous point. To admire your Lissajous curve and the effect of *a*, *b,* and *delta* on its shape, as if you were transported inside some 1970s dystopian science fiction movie and trapped inside the volcanic lair of the mad professor Kokkarius, create a separate class `LissajousMain` whose `main` method creates a new `JFrame` that contains your `Lissajous` component. Using the initial values for `a`, `b`, and `delta`, the result should resemble the following screenshot. You should verify that your code is correct by trying out other values for *a*, *b,* and *delta* shown on the Wikipedia page "Lissajous curve" to verify that your displayed curves have the same shape as the example images over there.

Parameters a and b represent the frequencies of horizontal and vertical motion, so their greatest common divisor strongly affects the resulting shape's period. This period is longest when a and b are **relatively prime** so that $gcd(a, b) = 1$. When a and b are equal, the resulting shape is a circle whose elongation depends on the delta offset between the horizontal and vertical periods. Using a delta of $\pi/2 \approx 1.5707$ maximally separates the phases of the x- and y-coordinates (effectively turning the sine function in the first formula into cosine). In contrast, a *delta* of 0 makes these two coordinates move in lockstep, producing a less exciting curve.

Motivated students can take on an extra challenge to make the displayed image look smoother by



eliminating some unpleasant visual jaggies, and possibly make it more artistic by using more complex curves and graphical shapes rendered at the positions of these “points going for a walk” around the two-dimensional plane. For example, instead of subdividing the entire curve into small line segments, you could subdivide it into smoother cubic curves whose consecutive pieces connect

seamlessly at their shared endpoints, since the additional control points ensure the tangent lines match perfectly at those endpoints.

Motivated students can take on an extra challenge to make the displayed image look smoother by eliminating unpleasant visual jaggies, and possibly make it more artistic by using more complex curves and graphical shapes rendered at the positions of these “points going for a walk” around the two-dimensional plane. For example, instead of subdividing the entire curve into small line segments, you could subdivide it into smoother cubic curves whose consecutive pieces connect seamlessly at their shared endpoints, since the additional control points ensure the tangent lines match perfectly at those endpoints.

Lab 10: Computation Streams

JUnit: StreamExercisesTest.java

This lab teaches you to think about computational problems and then implement them in the functional programming framework of **computation streams** introduced in Java 8. Therefore, in this lab, you are **absolutely forbidden** to use any **conditional statements** (either `if` or `switch`), **loops** (either `for`, `while`, or `do-while`), or even **recursion**. Instead, all computation of this lab **must** be implemented using only **Java computation streams and their core operations!**

In this lab, we will also briefly check out the **Java NIO framework** for improved file operations over those offered in the old `java.io` package and the `File` class therein. Your methods receive some `Path` as an argument, guaranteed to refer to some text file in your file system whose contents your methods will then process with computation streams. Create a new class named `StreamExercises`, and write in it the following two `static` methods.

```
public static int countLines(Path path, int thres) throws IOException
```

This method should first use the utility method `Files.lines` to convert the given `path` into an instance of `Stream<String>` that produces this text file as a stream of line strings. The method should then use the stream transformation method `filter` to keep only lines whose `length` is greater than or equal to the threshold value `thres`, and finally return the count of such lines.

```
public static List<String> collectWords(Path path) throws IOException
```

This method should also use the same utility method `Files.lines` to first turn its parameter `path` into a `Stream<String>`. Each line should be converted to lowercase and broken down into individual words that are passed down the stream as separate `String` objects (the stream operation `flatMap` will be handy here). Split each line into individual words using the `String` method `split` with the word-separator regex `"[^a-z]+"`. Then, in the next stages of the stream, filter out empty words and `sort` the remaining words alphabetically. Multiple consecutive occurrences of the same word should be removed (you can simply use the stream operation `distinct`, or if you want to do this yourself the hard way as an exercise, write a custom `Predicate<String>` subclass whose method `test` accepts its argument if and only if it was the first one or was distinct from the argument of the previous call). For the final answer, `collect` it into a `List<String>`.

As this new century shapes up to be an absurdist reboot of its predecessor during the worldwide pandemic, can rediscovery of futurist principles be far away? After all, the original futurists certainly were the “tech bros” of their time!

Lab 11: Prime After Prime

JUnit: PrimesTest.java

"There are only two hard things in Computer Science: cache invalidation and naming things."
— Phil Karlton

"*All programming is an exercise in caching.*" — Terje Mathisen
"*Good programming turns caching into cha-ching!*" — Ilkka's corollary

This lab tackles the classic and important problem of **prime numbers**, positive integers that are exactly divisible only by one and by themselves. Create a class `Primes` to contain the following three `static` methods to produce and examine integers for their primality quickly.

```
public static boolean isPrime(int n)
```

Checks whether the parameter integer `n` is a prime number. The plain old trial division algorithm is sufficient. If an integer has any non-trivial factors, at least one must be less than or equal to its square root, so it is pointless to look for factors beyond that point if you haven't already found any. To optimise this further, you only need to test n for divisibility by primes up to the square root of `n`.

```
public static int kthPrime(int k)
```

Find and return the `k`:th element (counting from zero, as usual in computer science) from the infinite sequence of all prime numbers 2, 3, 5, 7, 11, 13, 17, 19, 23, … This method may assume that `k` is nonnegative.

```
public static List<Integer> factorize(int n)
```

Compute and return the list of **prime factors** of the positive integer `n` guaranteed to be at least two. The exact subtype of the returned `List` does not matter as long as the returned list contains precisely the prime factors of `n` in **ascending sorted order**, each prime factor listed exactly as many times as it appears in the product. For example, when called with the argument `n=220`, this method would return some kind of `List<Integer>` object that prints out as `[2, 2, 5, 11]`.

To make the previous methods as fast and efficient as possible, this exercise focuses on caching and remembering what you have already found so you don't waste time recomputing it later. As the famous space-time tradeoff principle of computer science tells us, you can sometimes make your program run faster by using more memory. Since we have ample memory these days, we are usually happy to accept this tradeoff to speed up our programs.

This class should maintain a private static instance of `ArrayList<Integer>` in which you store the sequence of the prime numbers that you have already discovered, and **lazily expand** this list by

generating and appending new primes to its end. Knowing that this list of all prime numbers up to the current upper limit is sorted not only allows you to quickly look up and return the `k`:th prime number in the method `kthPrime`, but also to quickly iterate through all prime numbers up to the square root of `n` inside the method `isPrime` to look for potential divisors.

Furthermore, `Collections.binarySearch` can be used on the sorted list of known primes to determine almost instantly whether a given positive integer is prime. It would probably be a good idea to have a `private` helper method `expandPrimes` that lazily finds and appends new prime numbers to this list as needed by the `isPrime` and `kthPrime` methods.

Speed is the essence of this lab. To earn passing lab marks, the automated test must **finish all three tests within ten seconds** on the average off-the-shelf desktop computer from the past five years. (This requirement used to be twenty seconds back when this lab was originally written, but as you know, Moore's law and all that other stuff happened in between.)

Lab 12: The Second Hand Unwinds

JUnit: PrimeGensTest.java

In Python 3, generators make it easy to produce all kinds of lazy sequences. These special functions `yield` results one element at a time, then resume execution where they left off, rather than restarting from the beginning of the function body as ordinary functions do in both Python and Java. The Java language does not offer generators, but the idea of lazy **iteration** over computationally generated infinite sequences is still very much worth learning, and fits snugly into the `Iterator<E>` class hierarchy of the Java Collections Framework.

This lab continues the previous lab by using its `isPrime` and `kthPrime` methods as helpers to produce the subsequence of prime numbers that satisfy additional requirements listed on the Wikipedia page for these special subtypes of prime numbers. Create a new utility wrapper class `PrimeGens` inside which you write **no fields or methods whatsoever**, but three `public static` nested classes. Each class acts as an infinite iterator of prime numbers that are not explicitly stored anywhere in precious memory bytes, but are generated **lazily** whenever the user code requests the `next` such prime number.

Each of these three classes should define the basic methods required by `Iterator<Integer>`. The method `hasNext()` should always return `true`, since for our purposes, all these sequences are infinite. (It is currently unknown whether the twin primes sequence is truly infinite, but enough twin primes are known to exist so that we won't run out of those during the JUnit tests.)

To get you started with the required lazy integer iterator classes in this lab, here is a complete model implementation of two such example classes. The first produces all palindromic prime numbers, while the second produces all **composite** numbers (non-primes). Your three classes will have the same structure as these two, but will use different logic inside the `next` method to find the next prime number with the required property. (Note also the tactical use of both **pre-** and **postfix** forms of the ++ operator in `Composites`.)

```
public static class PalindromicPrimes implements Iterator<Integer> {
  private int k = 0; // Current position in the prime sequence
  @Override public boolean hasNext() { return true; } // Infinite sequence
  @Override public Integer next() {
    while(true) {
      int p = Primes.kthPrime(k++);
      String digits = "" + p;
      if(digits.equals(new StringBuilder(digits).reverse().toString())) {
        return p;
      }
    }
  }
}
```

```
public static class Composites implements Iterator<Integer> {
  private int curr = 4, k = 2, nextPrime = Primes.kthPrime(2);
  @Override public boolean hasNext() { return true; }
  @Override public Integer next() {
    if(curr == nextPrime) {
      nextPrime = Primes.kthPrime(++k);
      curr++;
    }
    return curr++;
  }
}
```

After carefully studying both classes until you can explain what each line does, the three nested classes you write inside class `PrimeGens` are as follows. You can quickly and easily test your implementations with a simple `main` method that first creates an instance of the said iterator, and then uses a `for`-loop to print out the first couple of dozen elements for you to compare to the expected initial sequences given below. Once the first twenty or so special primes your iterator produces match the expected sequence, the rest will almost certainly follow suit, just as the race is not always to the swift nor the battle to the strong, but that's still the way to bet. (At some point incredible luck becomes indistinguishable from incredible skill, if only to mangle Arthur C. Clarke's observation about the analogous relationship between sufficiently advanced science and magic.)

`public static class TwinPrimes implements Iterator<Integer>`

Generates all twin primes; prime numbers `p` for which `p+2` is also a prime number. This sequence begins 3, 5, 11, 17, 29, 41, 59, 71, 101, 107, 137, 149, 179, 191, 197, 227, 239, 269, 281, 311, …

`public static class SafePrimes implements Iterator<Integer>`

Generates all safe primes that can be expressed in the form `2*p+1` where `p` is also a prime number. (The smaller prime `p` that establishes the "safety" of the prime `2*p+1` in a certain technical sense is then called a "Sophie Germain prime"). This sequence begins 5, 7, 11, 23, 47, 59, 83, 107, 167, 179, 227, 263, 347, 359, 383, 467, 479, 503, 563, 587, 719, 839, 863, …

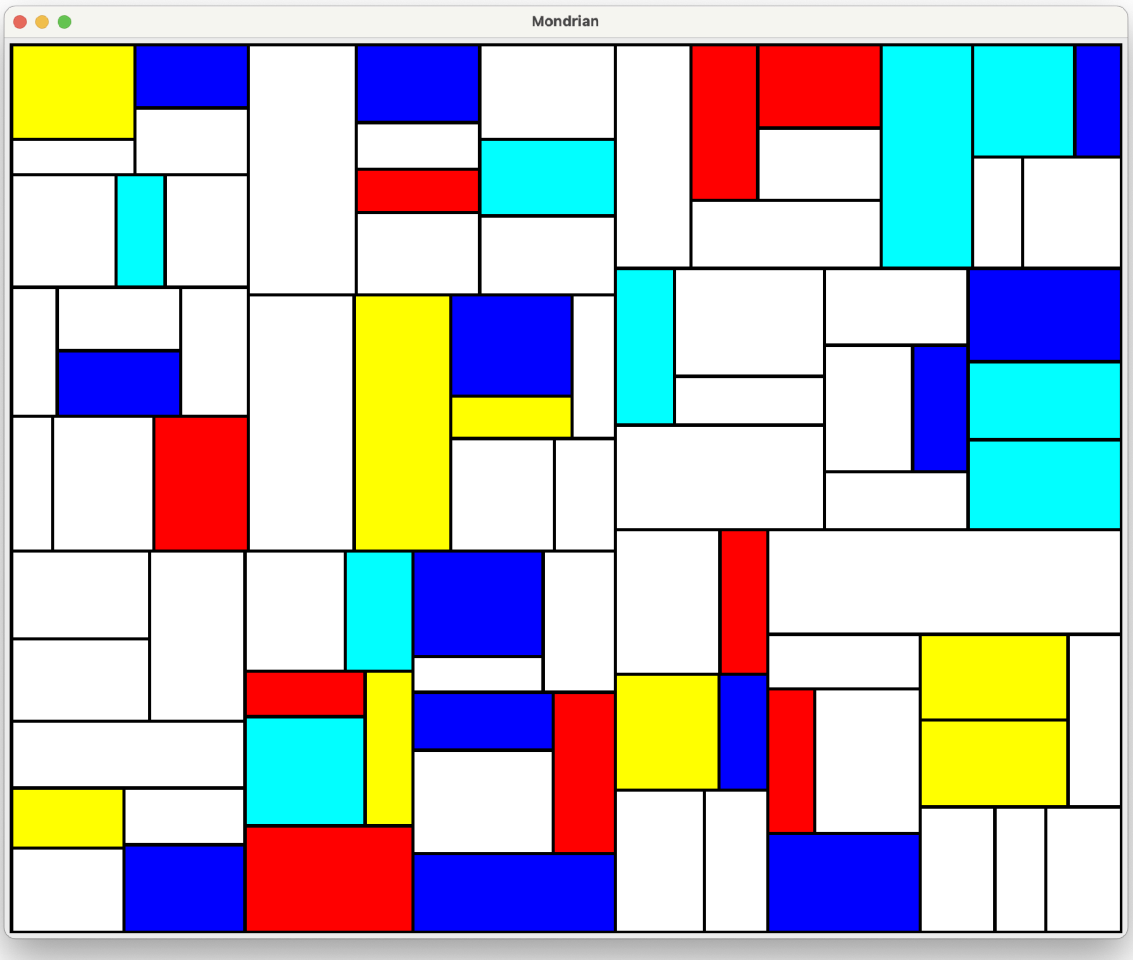
`public static class StrongPrimes implements Iterator<Integer>`

Generates all strong primes, primes `p` that are larger than the average of the previous and next prime numbers around `p` in the sequence of all prime numbers. This sequence begins 11, 17, 29, 37, 41, 59, 67, 71, 79, 97, 101, 107, 127, 137, 149, 163, 179, 191, 197, 223, 227, 239, 251, …

Lab 13: Recursive Mondrian Art

Your instructor used this assignment in a Palaeolithic version of this course. Still, now that this classic problem has resurfaced in Nifty Assignments, perhaps it's time to revisit its timeless message. Especially since, in the years in between, your instructor also got hopelessly bitten by the subdivision and fractal art bugs.

This lab combines graphics with recursion by constructing a simple **subdivision fractal** that



resembles the famous abstract works of Piet Mondrian. An example run of the instructor's model solution produced the random image captured for eternity in the following screenshot, to which your images should be reasonably similar in style and spirit, using controlled randomness to generate a new piece of abstract visual art every time your program is run.

Write a class `Mondrian` that `extends JPanel`, with the following fields:

```
// Cutoff size for when a rectangle is not subdivided further.
```

```
private static final int CUTOFF = 40;
// Percentage of rectangles that are white.
private static final double WHITE = 0.75;
// Colours of non-white rectangles.
private static final Color[] COLORS = {
  Color.YELLOW, Color.RED, Color.BLUE, Color.CYAN
};
// RNG instance to make the random decisions with.
private Random rng = new Random();
// The Image in which the art is drawn.
private Image mondrian;
```

After this, the class should have the following methods:

```
public Mondrian(int w, int h)
```

The constructor for the class, with the desired width and the height of the resulting artwork given as constructor arguments. Set the preferred size of the component to `w`-by-`h` pixels, initialise the field `mondrian` to a new `BufferedImage` of size `w` and `h`, and get its `Graphics2D` object to pass as the last argument to the top-level call of the recursive `subdivide` method below. After returning from the rendering recursion, you should call the `dispose()` method of the `Graphics2D` object to avoid leaking its resources.

```
public void paintComponent(Graphics g)
```

Draws the `Mondrian` image created in the constructor on the surface of this component.

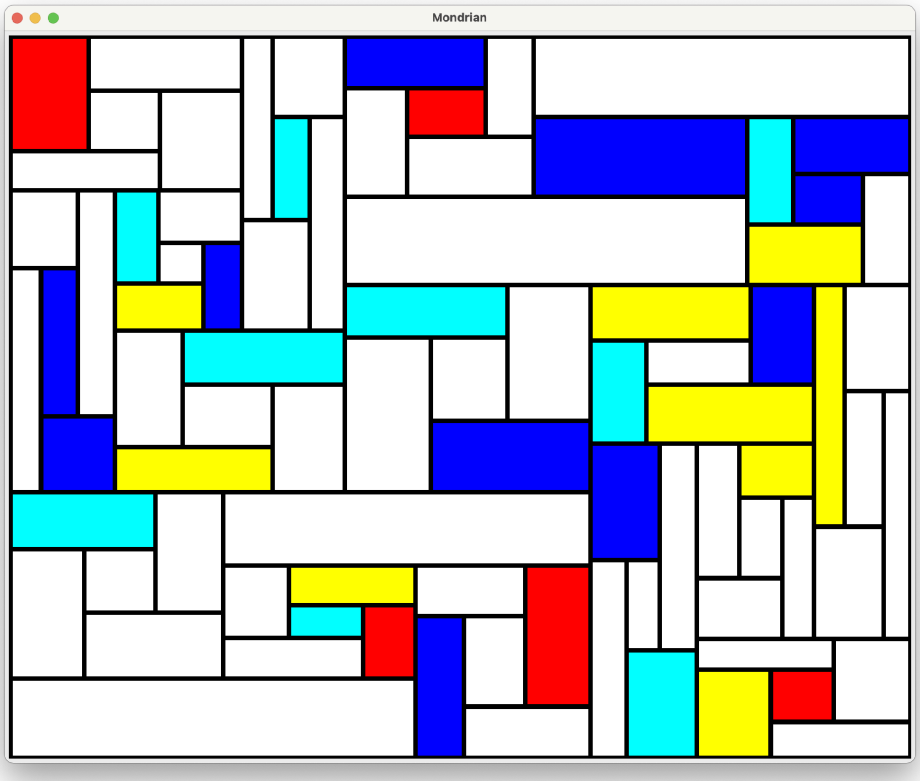
```
private void subdivide(int tx, int ty, int w, int h, Graphics2D g2)
```

Recursively subdivides the given rectangle whose top-left corner is at coordinates `(tx, ty)` and whose width and height are `(w, h)`. The base case of the recursion is when either `w` or `h` is less than `CUTOFF`, in which case this rectangle is drawn on the `Graphics2D` object provided. (The top-level call in the constructor should ask the image object for its `Graphics2D` object and pass that on to the recursive call.) This rectangle should be white with probability `WHITE`, and choose a random color from `COLORS` otherwise. Draw the rectangle to have a black outline in either case.

Otherwise, make two recursive calls for the two rectangles you split from the given rectangle. Make sure that you always split each rectangle along its longer edge (breaking ties arbitrarily for squares), and that the splitting line is not too close to either edge that has the same direction, to keep the subdivision reasonably balanced. Subdivisions that contain thin shards and forced sharp angles tend to look ugly and disharmonious, like cursive handwriting squeezed narrowly at the end of a line to stay within the margins. In all walks of life, humans usually perceive that elusive sweet spot

between predictably solid, boring order and the screeching cacophony of randomness as the most aesthetic. (Reaching that sweet spot can be a bit of a tightrope act in practice.)

To admire your randomly generated pieces of art, write a `main` method that creates a `JFrame` instance that contains one instance of `Mondrian` that is 1,000 pixels wide and 800 pixels tall. You can also vary the above parameters and the colour scheme to aim for an even more artistic, aesthetically pleasing result.



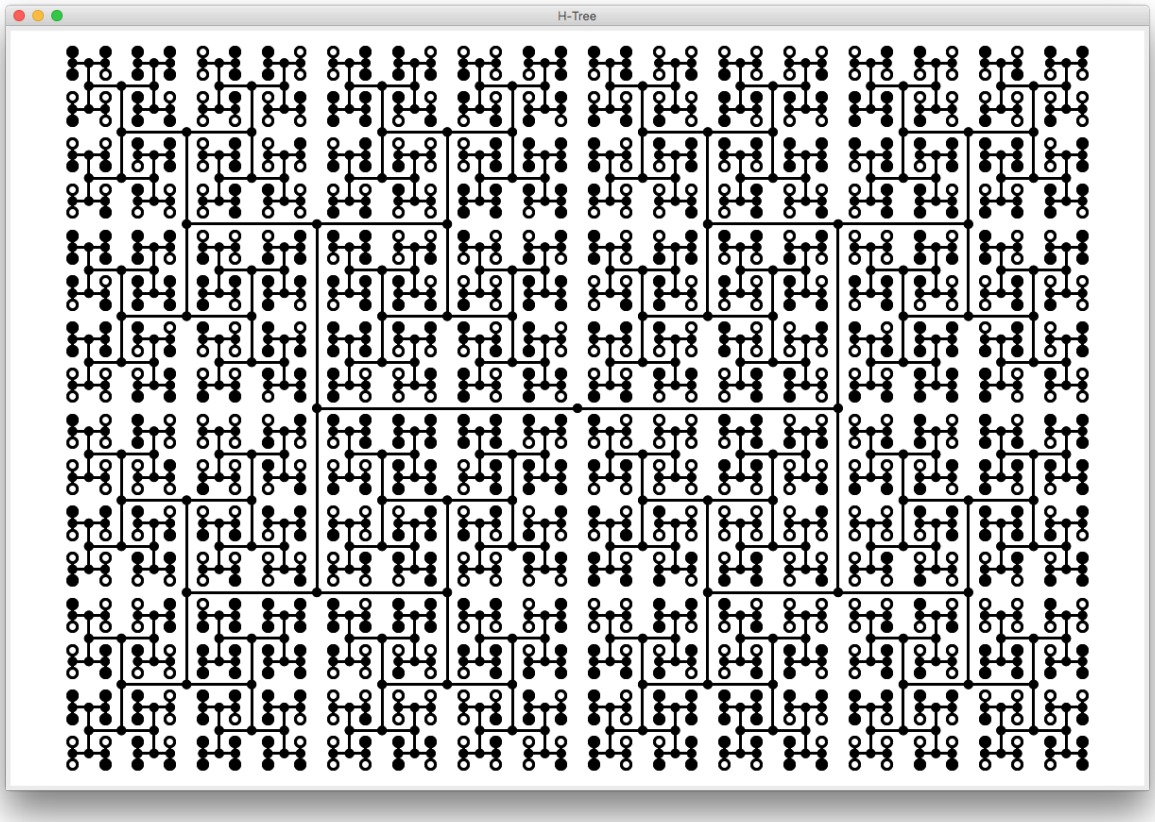
This random subdivision isn't yet the best possible aesthetically because it heavily biases how the subdivision walls are positioned inside the rectangle. Each subdivision always contains exactly one straight **fault line** that runs through the entire rectangle and then recursively continues in the two rectangles it separates. This feels unnatural, even if the viewer doesn't consciously see the fault lines. (The reader might enjoy finding these recursive fault lines in the previous example image and deducing the order of the subdivision recursion from those lines.)

Above, a more complex subdivision rule that splits into five spiral-like pieces mathematically guarantees the elimination of such jarring fault lines, making the resulting subdivision structure more organic and pleasing to the eye. More complex subdivision rules converge toward random results that seem perfectly natural, completely unforced and not artificial, thereby achieving the maximum artistic value a mindless machine can reach in this domain. No law of nature or man will

stand athwart history yelling stop, even if you use multiple rules, one of which you randomly choose at each recursive subdivision call. No opponent, not even Old Nick himself, can systematically defeat you when you don't even know yourself what path you'll take in the future! We should also mention the completely different approach of Gilbert tessellations, which produce even gnarlier tessellations, at the cost of some tiles ending up as unpleasant thin shards.

Lab 14: H-Tree Fractal

In the spirit of the recursive Mondrian subdivision from the previous lab, here is another recursive



fractal problem: render the H-Tree fractal on a Swing GUI component, with additional decorative dots indicating the intersections and the caps at the branch ends of this exponentially branching structure. An example run of the instructor's private model solution produced the following result that your outcome should also resemble. You may again adjust the style to be more artistic, provided that you maintain the basic nature and structure of this fractal.

Start by creating the class `HTree` that extends `JPanel`. This class should have the following fields:

```
// Once line segment length is shorter than cutoff, stop subdividing.
private static final double CUTOFF = 10;
// Radius of the little dot drawn on each intersection.
private static final double R = 5;
// The image inside which the H-Tree fractal is rendered.
private Image htree;
// Four possible direction vectors that a line segment can have.
private static final int[][] DIRS = {{0, 1}, {1, 0}, {0, -1}, {-1, 0}};
// A random number generator for choosing the end piece colours.
private static final Random rng = new Random();
```

This class should then have the following methods.

`public HTree(int width, int height)`

The constructor receives the image `width` and `height`. It should first initialise `htree` to be a new `BufferedImage` instance of those dimensions, and set the preferred size of the entire component to that `width` and `height`. Acquire the `Graphics` object of this image and convert it to the more modern and powerful `Graphics2D` reference that allows you first to turn on anti-aliasing, and then later `draw` each line segment with non-integer precision inside the recursion. Before the recursive calls, make the initially all-black image all white by filling it with a white rectangle whose area covers the entire image. Instances of `BufferedImage` start as all-black pixels because the bytes that store those pixels' colours are guaranteed to be initially filled with zeroes. (This same guarantee is always in effect for all objects created in the heap memory area of the Java Virtual Machine.)

The constructor of `HTree` should call the next `render` method twice, both times starting from the point that is in the middle of the image and with `len` set to equal to the minimum of the width and height of that image, divided by three. The first call should go left, and the second should go right, for the two symmetric branches.

`private static void render(double x, double y, int i, double len, Graphics2D g2)`

Render the H-Tree fractal starting at `(x,y)` in the direction given by index `i` in the DIRS array, which contains the four possible directions each line segment can head, with the current line segment having length `len`.

The base case is when `len < CUTOFF`; in that case, fill a disk of radius `R` centred at `(x,y)`, flipping a random coin to decide whether the disk is black or white. (If it is white, you should still `draw` the black outline around it.) Otherwise, draw a small black disk at the current coordinates `(x,y)`, and then calculate the endpoint `(nx,ny)` of the line segment in the direction that your rendering is currently heading with

```
nx = x + DIRS[i][0] * len;
ny = y + DIRS[i][1] * len;
```

Draw the line segment from `(x,y)` to `(nx,ny)` on the image, and follow with two recursive calls at `(nx,ny)` using the directions `i+1` and `i+4`, wrapping around the `DIRS` array as needed by taking the remainder of the new direction modulo 4. The new `len` in the recursive call should equal the current `len` divided by the square root of two, so the branches get shorter down the recursion, and the overall shape stays within a finite boundary. (Using the square root of two as the divisor makes

the entire H-tree precisely space-filling, since two squares of that side length fill exactly the original square.)

Thanks to the magic of subdivision fractals, these exponentially spreading branches never touch or overlap, no matter how deep you recurse. As `CUTOFF` approaches zero and the recursion depth increases toward infinity, this space-filling fractal becomes dense in its two-dimensional bounding box, leaving no holes large enough to fit a circle of any arbitrarily small positive radius ε. This despite the shape consisting only of one-dimensional line segments when we don't draw the decorative disks at the intersections! (Yes, a mathematical purist would now point out that the H-tree fractal is not technically space-filling since it has a Lebesgue measure of zero, but we'll skip that technicality here.)

Students who happen to catch from this exercise a funky case of "fractal fever", a once fashionable ailment among the young but assumed to have gone the way of mange, grunge, the itch, the twitch, the thrush, the scoff and the rot since the early nineties, at least outside California (the epicentre and patient zero of the original epidemic, as always), might want to dip into "Brainfilling Curves: A Fractal Bestiary", your instructor's personal favourite that teaches you how to render an infinite variety of such artistic fractals with **turtle graphics**.

To admire your artwork, you should again create a `main` method to open a new `JFrame` instance, inside which you add a new `HTree` instance of dimensions `(1000,800)`. Again, as in the recursive Mondrian subdivision, once you get this program working, feel free to experiment to make the result look more pleasant. This is perfectly fine with the instructor as long as you maintain the overall shape and spirit of the original H-Tree fractal. For example, if you happen to know how to compute arbitrary **rotations** of direction vectors using matrix algebra, you might want to add a touch of randomness to make the image seem more relaxed and natural so that instead of always turning exactly 90 degrees left or right, you instead turn 90 degrees plus or minus a small random amount, as if some flesh-and-blood human had drawn this shape with a slightly trembling hand. To keep the randomness from going completely off the rails, the recursion should track both the intended direction and the current direction, and correct back if the former veers too far from the latter.

Lab 15: Zeckendorf Encoding

JUnit: ZeckendorfTest.java

In Lab 0(E), we encountered the **Zeckendorf theorem**, which states that every positive integer can be uniquely expressed as a sum of non-consecutive Fibonacci numbers. We can then encode this sum as a bit string whose positions correspond to Fibonacci numbers: 1 indicates the Fibonacci number is present in the sum, while 0 indicates its absence. For example, using online conversion tools such as "The Fibonacci base system", we find that 42 = 34 + 8. The **Zeckendorf representation** of this number is 10010000, indicating that the fifth (8) and eighth (34) Fibonacci numbers are present in the sum, with positions counted from right to left the same way as in ordinary positional representation.

The Zeckendorf representation is a bit (heh) inefficient compared to ordinary binary numbers, which represent integers as a simple sum of powers of two. This is because the bit pattern 11 never appears in a Zeckendorf representation, costing roughly 25% storage efficiency. However, this wasteful limitation becomes a major advantage if you want to encode not just one integer, but an entire sequence of arbitrarily large `BigInteger` values so you can still unambiguously restore the original values! When you encode a sequence of integers using the ordinary binary encoding for each element, you must augment it. This way, the decoder knows where each number ends, and the next begins. This can be solved by either agreeing that each integer is encoded with the same number of bits (which then enforces a maximum value that can appear inside the sequence), or by giving some high-end bits a role to indicate whether the encoding of the current number continues, in the spirit of the **UTF-8 character encoding** discussed in the I/O streams lecture. (The lab 71, "The Gamma And The Omega", showcases one such possible encoding scheme.)

Using Zeckendorf representation, determining where the current number's encoding ends and the next one begins is automatic. The otherwise unused bit pattern `11` acts as an artificial **separator,** telling the decoder the current number ends there and the next number starts at the following position. The first 1 in the otherwise forbidden pattern 11 is the highest "zit" of that number's Zeckendorf encoding, and the forbidden second consecutive 1 acts as a virtual "comma" that separates that number from the next. Even better, this encoding is **self-synchronising** and thus more **robust**; the reader can recover from any incorrect bits as soon as it reaches the next separator mark, from which it can faithfully restore the rest of the sequence. (The **UTF-8 encoding** for Unicode characters also shares this convenient guarantee, over there enforced by the high-order bits of every payload byte.)

We must also realise that the convention of writing digits in descending order of position is a societal agreement, like driving on the left or right side of the road; either works, but we have to pick one and have everyone agree. Humanity could just as well have agreed to write our bits, zits, and fits of other kinds of digits in ascending position order. For example, the number we normally say out loud as "two hundred and eighty-seven" could just as well be written as 782 instead of 287. Fibonacci coding of the given sequence of integers consists of simply writing the Zeckendorf

representations of each number left to right (so the last bit, being the highest bit, is always 1), so putting another 1 between the two separate numbers to create the impossible 11 will act as a virtual “comma” that clearly separates those two numbers.

After that mouthful, it is finally time to start chewing. (In the later problems, these educational preludes will only get longer.) To celebrate that Zeckendorf encoding lets us encode numbers with even millions of digits, we use the `BigInteger` class to represent both these integers and the Fibonacci numbers used to break them apart, as we did in lab 0(E). Create a class `Zeckendorf` that contains the following two `static` methods to encode and decode the given list of integers:

```
public static String encode(List<BigInteger> items)
```

Given a sequence of `BigInteger` values as `items` where each item is greater than zero, compute and return the Zeckendorf representation of that entire sequence as a single string whose each character is either `'0'` or `'1'`. For example, given the list `[4, 36, 127]`, this method would return the string `"1011010000011010000101 1"` (coloured here for extra readability), since 4 encodes to "`1011`", 36 encodes to "`010000011`", and 127 encodes to "`10100001011`".

If you have already completed Lab 0(E), you might as well reuse the method `fibonacciSum` to do most of your heavy lifting here. Of course, in a real compression algorithm, the result would be a sequence of bits instead of using one full Unicode character to encode each bit, but in this lab the result is a string just for convenience. Inside this method, you will surely once again use the mutable `StringBuilder` to accumulate the zeros and ones, and convert the entire result into a `String` just once at the end before returning.

```
public static List<BigInteger> decode(String zits)
```

Given the Zeckendorf decoding as `zits`, recreate the original sequence of positive integers. Your method may assume that `zits` is a complete encoding of an integer sequence that can contain characters `'0'` and `'1'` only.

Notice how, as with any two methods to encode and decode between two representations of something, these two methods are **inverses** of each other, so that applying one to the result of the other always produces the original starting value. Writing a fuzz test is especially trivial for such functions. Even if test cases are produced pseudo-randomly, the expected result of using the pair of methods in tandem is fully known, since it must always equal the original value! Of course, to debug your code that is not yet passing this passive-aggressive mass test, you can create specific desired test cases with the above online Zeckendorf calculator.

For an even stronger appreciation of Zeckendorf encoding's general awesomeness, discrete math and number theory enthusiasts can check out David Eppstein’s discussion of efficient integer arithmetic performed directly on the Zeckendorf representation, without converting these integers back to ordinary binary encoding for these operations.

Lab 16: Egyptian Fractions

JUnit: EgyptianFractionTest.java

As explained on the Wikipedia page "Egyptian fraction", ancient Egyptians had a curious way to represent arbitrary positive integer fractions a/b by breaking them down into sums of distinct **unit fractions** of the form $1/n$. Such a breakdown can be done in infinitely many ways for any rational number, depending on the additional obligations we impose. However, the number of unit fractions needed to express a number as a sum of unit fractions tends to grow exponentially with respect to its absolute value. Although the following construction algorithms work for arbitrarily large rational numbers, we will only consider fractions $a/b < 1$ so our JUnit test methods terminate in reasonable time without running out of memory.

This lab has you implement two different algorithms to construct the Egyptian fraction breakdown. To ensure correct results even when they involve thousands of digits, you must do all your integer arithmetic using the `java.math.BigInteger` type. You should also add the `Fraction` example class into your labs project and use its operations to perform the arithmetic of rational numbers needed in this lab. Instead of reinventing that rusty old wheel, you'll have more time not just to walk, but also to do your arithmetic like an ancient Egyptian. This should lighten your heart so Anubis will let you enter the field of reeds for a pleasant afterlife, instead of you unceremoniously ending up eaten by the crocodile god.

Create a new class called `EgyptianFractions` in your labs project, and there, the method

```
public static List<BigInteger> greedy(Fraction f)
```

that constructs the Egyptian fraction breakdown of $f = a/b$ with the **greedy algorithm** originally proven to always terminate by Mr Fibonacci himself! The simplest situation is when $a = 1$, so the result is $1/b$. Otherwise, compute the smallest positive integer n whose reciprocal $1/n$ is less than the fraction a/b. Assuming a/b is in lowest terms, do this by dividing b by a with **truncating** integer division (which gives the floor), then adding 1. For example, the smallest positive integer n whose reciprocal is less than $3/17$ is $\lfloor 17 \;/\; 3 \rfloor + 1 = 6$. Add the term $1/n$ to the result, and extend the result with the greedy breakdown of the remaining, hopefully simpler fraction $a/b - 1/n$.

To ensure unique results for fuzz testing, this method should return the Egyptian fraction as a list of denominators, sorted in ascending order; each term, of course, is represented as a `BigInteger` value in this problem. For example, when called with the fraction 2/3, this method returns the list `[2, 6]`, and with the fraction 4/17, returns `[5, 29, 1233, 3039345]`. As you can see, restricting the breakdown to distinct unit fractions can cause a big blowup even for seemingly simple fractions. As is typical with greedy algorithms, the result is not necessarily the simplest or the most balanced possible subdivision into unit fractions, since the last term can be a pretty thin sliver compared to the greedily chosen "thicc" terms that precede it.

The next method introduces another algorithm to construct an Egyptian fraction, one that usually produces a result quite different from the previous greedy method. Like Fibonacci's greedy method, this technique is not guaranteed to return the simplest or most balanced breakdown. (It sure feels like some spooky general pattern underlying the visible fabric of reality is slowly emerging from its slumber!) Write the method

```
public static List<BigInteger> pairing(Fraction a)
```

that maintains a local `Set<BigInteger>` variable that keeps track of which terms have already been chosen to be part of the Egyptian fraction. Initially, this set is empty. To break down a fraction a/b, we first split it into two terms: $(a - 1)/b + 1/b$. Repeating this step lets us "carve" unit fractions off the remaining fraction one at a time, like carving delicious slices off the Christmas ham. Slice off one unit fraction $1/b$ by adding b to your chosen set of terms, and construct the Egyptian fraction representation for the remaining fraction $(a - 1)/b$ until the entire number has been carved out so that only a zero remains, simplifying the fraction along the way as needed by eliminating the common factors of $a - 1$ and b. The `Fraction` class will, of course, do this for you.

Comparing the breakdowns of some randomly chosen simple fractions, the greedy algorithm typically produces fewer terms, but not always. For example, 5/91 breaks down greedily into unit fractions `[19, 433, 249553, 93414800161, 1745264977814571645168l]`, whereas pairing produces a far more balanced subdivision `[23, 91, 2093]` for the same quantity. As so often in all walks of life, we have ample time to regret our greedy initial choices as they slowly take us towards their inevitable ends. On the way down, we can only dream of what could have been, had we initially stayed our hand and not shot that bloody albatross, but followed the dove and taken the road less travelled instead. (Making your choices inside a computer program instead of directly in the physical reality does not affect the poetic justice of the tragic consequences of such greedy choices. As above, so below; as below, so above.)

To maintain the discipline of using each denominator at most once, we need to handle somehow the situation where the new slice $1/b$ is already in the set of chosen terms. To do this in an orderly fashion, maintain a local variable `List<BigInteger> buffer` that contains the denominators of the unit fractions waiting to enter the set of chosen terms. Start by initialising this `buffer` to contain exactly one value b, the term that is currently waiting to enter the club. While this `buffer` is non-empty, pop out any one of the items there, let's call it n. (It turns out that the order in which the items are popped won't affect the final result, so you might as well pop out the last item.) In the first round of popping, n must therefore equal b, but it can and will equal other values during later rounds of this corrective back-and-forth dance.

If n is not already a member of the set of chosen terms, add it there and move on. Otherwise, you have a conflict: two equal terms, $1/n$, are trying to become a chosen term, even though at most one can be part of the final set. First, remove n from the set of chosen terms. Resolution of this conflict will then continue depending on the parity of n:

- If n is even, add the term $n/2$ to the `buffer`.
- If n is odd, add two terms $(n + 1)/2$ and $(n(n + 1))/2$ to the `buffer`.

For example, two equal unit fractions 1/10 and 1/10 can be replaced by a single 1/5 without affecting the total. If the conflict is between two odd unit fractions 1/5 and 1/5, push two new terms, 1/3 and 1/15, into the `buffer`: the total is again unaffected by this since 2/5 = 1/3 + 1/15. Rinse and repeat until all terms are distinct and the `buffer` is empty. Until then, the individual terms will enter, exit, and re-enter the set of chosen terms and the waiting room of the `buffer`, the exact details of this back-and-forth depending on the terms waiting inside the `buffer` and the needs of the rest of the number $(a - 1)/b$ that is still waiting to be broken down.

We often think of integers as products of prime factors. For natural numbers, this **prime factorisation** is unique, a result modestly known as the "Fundamental theorem of arithmetic". This famous result is actually not as trivial as it initially seems. It certainly is not true just because of some hand-waving in the likes of "But, like, it's the *primes*, so how could it even be otherwise, *maaaan*, so much confusion here sweaty lol I can't even!" However, as we just saw, another way to think of not just integers but all rational numbers is to see them as finite sums of tactically chosen unit fractions. However, unlike the unique prime factors that make up each integer, any rational number (and thus all integers) can be broken down into sums of distinct unit fractions in infinitely many different ways.

These Egyptian fractions boggle the mind, offering infinite ways to express any positive rational number as a sum of smaller positive rationals. Kinda makes us wonder if those ancient Egyptians really were connected to something that we have forgotten… but up here in the present timeline of cyberpunk future so bright that we need to wear shades, anybody interested in learning more about this topic can start on the page "Egyptian Fractions" by that David Eppstein guy again. ("Once is happenstance, twice is coincidence…") Most of the external links on the page are as dead as the ancient Egyptian culture they echoed, but fortunately at least "Algorithms for Egyptian Fractions" is still alive at the time of this writing. The reader may feast his mind by appreciating the many other techniques for breaking down an integer fraction into a sum of distinct unit fractions.

Lab 17: Diamond Sequence

JUnit: DiamondSequenceTest.java

The `Set<E>` implementations in the Java Collection Framework are flexible, but not ideal for all situations because they use relatively more memory per object. In practice, sets and maps often help us remember what we have seen and done during a method's execution. If those things can never become unseen or undone, the `remove` operation need not be supported, except possibly as a clear method to empty the entire data structure at once.

In data structures and algorithms, collections that allow dynamic addition of new elements but disallow dynamic removals are called **monotonic**. Sometimes such structures can be implemented more efficiently than full versions that allow removal. Because removing an element decreases the overall **entropy** of the data structure, the remove operation must involve actual physical work. This makes `remove` inherently more complex than the entropy-increasing add operation and the nominally entropy-neutral `contains` operation.

Further optimisations in the set representation are possible when the set members are known to be **natural numbers**. This is a common use case, given that everything inside not just the physical computer but our entire physical reality is fundamentally made of integers anyway, as much as we like to pretend otherwise, that the computer memory is full of strings, dictionaries, and other entities that in reality are just as made up and fictional as, say, hobbits and vampires. This effect becomes even more pronounced when these numbers tend to be added to the set in roughly ascending order, as is often the case with many integer sequences whose rules for computing the next element depend on whether some particular integer has already appeared in the sequence generated so far. The sequence grows so slowly that it reaches new heights gradually and must therefore fill in the gaps it leaves behind. For example, the Recamán's sequence we met in Lab 0(B) fits this pattern.

In such cases, we can pack membership information as tightly as possible, representing each integer as one bit stored in a `boolean[]`. We will cleverly manage this boolean array as a **sliding window** so that the field `start` tells which absolute position the relative zero index represents, and implement the methods `add` and `contains` to operate relative to this index.

Now, the good news is that you don't need to do any of what I just said, since for both this lab and for your possible future endeavours in Java, your instructor provides the class `NatSet` whose objects represent **monotonic** sets of natural numbers (that is, zero or positive). These `NatSet` objects start their lives as representing the empty set. The public method `void add(int n)` adds the natural number `n` to the set. (There is no obvious way to say this less convolutedly, and yet keeping that phrasing feels like giving up as a technical writer.) The public method `boolean contains(int n)` tells the caller whether `n` is a member. Since no shrinkage is allowed even when it's cold, we don't have a `remove` method in the public interface.

Using the `NatSet` class as a tool to keep track of the numbers already seen during construction, your mission is to implement another very interesting sequence of positive integers, found by your instructor in the book "Mathematical Diamonds" by Ross Honsberger. Since that book deals with mathematics instead of computer science, it uses conventional sequence-position numbering starting at 1, not 0, the way it always works for us propeller beanie cap code monkeys and web designers. The first element at position $k = 1$ equals one. Each later position $k > 1$ contains the smallest positive integer that so far has not appeared anywhere in the sequence and makes the sum of the first k elements of this sequence an integer multiple of k.

For example, the second element for $k = 2$ is the smallest unseen integer that, when added to the first element 1, makes the total divisible by 2. The correct answer is 3, since 1 + 3 = 4 is divisible by $k = 2$. After [1, 3], the third element is again the smallest unseen integer that, when added to the previous elements 1 and 3, makes the total divisible by 3. The correct answer is 2, since 1 + 3 + 2 = 6, and 2 is the smallest unseen number that works. The sequence starts its eternal journey as 1, 3, 2, 6, 8, 4, 11, 5, 14, 16, 7, 19, 21, 9, 24, 10, 27, 29, 12, 32, 13, 35, 37, 15, 40, …

To practice writing virtual iterators that are not backed by any actual collections but that always produce the next element of the sequence in computational means, we implement this "diamond sequence" (named a bit unimaginatively due to the lack of mathematical knowledge, depth, or sophistication on this author's part) as a virtual iterator. In your labs project, create the class

```
public class DiamondSequence implements Iterator<Integer>
```

with the instance methods required in the interface `Iterator`:

```
@Override public boolean hasNext()
@Override public Integer next()
```

Since this sequence is infinite, `hasNext` always returns `true` without further ado. For the computation inside `next`, this class should define an int data field k that tracks the position in the sequence currently being generated. Then, add another data field, `sum`, that stores the sum of the elements generated so far. Make this second field `long` to ensure that summing the elements of this sequence won't overflow, even when we add up its first hundred million elements! Last, this object needs an instance of `NatSet` to remember the elements generated so far.

Next, the method should increment k and perform integer arithmetic to find the smallest previously unseen n that, when added to the current sum, makes the total at the current position divisible by k.

The JUnit test class `DiamondSequenceTest` will push your code hard to its limits by making it generate the first one hundred million elements of the diamond sequence. Within this range, all values in the sequence still fit in `int` variables without overflow, but their sum certainly will not fit in an `int`. The sequence grows at a modest linear pace, but it bounces above and below this

baseline in a seemingly chaotic fashion that somehow always fills in the gaps it leaves behind. (Every filling of such a gap will then allow the `NatSet` data structure to move its sliding window into the data some number of positions to the right.)

No integer appears twice in this sequence, as follows from its construction rule. It is not equally obvious that every positive integer *n* will eventually appear, but this can be proven.

Furthermore, this sequence has a curious self-describing structure: whenever the k:th element equals *v*, its *v*:th element equals *k*. For example, since the seventh element equals eleven, its eleventh element must equal seven! Let's all, for a moment, try to wrap our collective heads around this puzzling fact that somehow follows from the sequence's construction. (In discrete math terms, when viewed as an infinite permutation of the natural numbers, this sequence's cycles all have length two.) The method `testSelfReferentiality` in the JUnit test class verifies this bold claim empirically for the first one million elements of this sequence.

Lab 18: Working Scale

This lab again has you working with Swing GUI components, but this time not just with inert graphics and boring GUI components such as buttons and text fields that people use at work, but with animations where the content of your component is updated in real time, fifty frames per second.

From the repository `ikokkari/CCPS209Labs`, add the interface `AnimationFrame` and the two classes `AnimationPanel` and `Tircle` into your labs project. The `AnimationFrame` interface represents an object's ability to render itself as pixels on command, so the image's contents can vary over time. This interface contains one method for your subclasses to implement:

```
public void render(Graphics2D g, int width, int height, double t);
```

Render the contents of the animation frame the way they are supposed to look at time `t`, into the given `Graphics2D` object `g` so that the dimensions of this rendering are `width` pixels wide and `height` pixels tall. Time is not measured here in actual seconds of "wall clock time", but is merely a number that can be positive, negative, or zero, no different from the numbers used to denote the spatial dimensions. This way, we can speed up or slow down the displayed animation arbitrarily, with every individual frame still looking exactly as it should. We could even find out what the animation frame would look like one year from now, without rendering every frame between the present moment and that future time.

No matter how fast this virtual time advances, the animation renders inside an `AnimationPanel` component at exactly fifty frames per second to keep the visuals smooth enough to satisfy the human eye. (Back in this author's time, we used only 24 frames per second, and we liked it without complaining.) The `AnimationPanel` objects are Swing GUI components customised to display the `AnimationFrame` object given to them. Time `t` always starts from zero, and increases by `step` after every animation frame. Whenever you create a new subclass of `AnimationFrame`, you can edit the `AnimationPanel[]` array in the `main` method to include an object of your new class to try it out immediately. As you admire your artwork, remember to grab the window resizing handle and give it a good back-and-forth to ensure your `AnimationFrame` renders properly at any width and height and doesn't, for example, rely on the assumption that the animated image is a square.

(Also, do not fail to appreciate the isomorphism between the two concept troikas `Animal` / `Cat` / `getSound` versus `AnimationFrame` / `Tircle` / `render`.)

As an example of how to properly implement the `render` method, see the example class `Tircle` that renders a subdivision fractal that is built up from smaller copies of itself. Such **self-similarity** immediately suggests using recursive methods to compute and render such fractals. Since we cannot conveniently make the `render` method itself recursive (since it has no place for the additional parameters that our subdivision would need), we nimbly sidestep this limitation by

extracting the recursion into a separate `private` method of its own (in the class `Tircle`, this recursive method is also called `tircle`). The arguments for this top-level call from the `render` method depend on the parameters `width`, `height`, and `t` to ensure that the rendering is done correctly and without pixelation for all possible image sizes, from a small icon up to a screen splash.

To give this recursion a solid foundation, every branch eventually reaches a base case: once the subdivision shape is small enough, it is rendered as a simple shape, such as a circle or a rectangle, at that position. In principle, the base case is when the shape is smaller than one pixel. However, stopping subdivision once the shape's radius falls below a higher threshold, such as ten pixels, usually yields more aesthetically pleasing results. With this general technique, you can render any subdivision fractal you can dream up. Laws of neither nature nor man dictate that `t` has to remain fixed through all levels of recursion, but can be increased or decreased a little in each call to give the overall system a more organic appearance when its components are not acting in an unnaturally perfect lockstep with each other.

Your `render` method should render some kind of reasonable result for any legal value of the parameter `t`. The simplest way to do this is to normalise your animation time to always run from 0 to 1, regardless of the intended animation length. (The animation can always be stretched to an arbitrary length in the actual wall clock time by decreasing the `step`.) For values of `t` outside the interval [0,1], the integer part of `t` is ignored, and only the decimal part affects the outcome. This makes the animation periodic, so at integer times t = 0, 1, 2, … it returns to its initial state. You can then turn this animation into an animated GIF consisting of n frames computed with t stepping through 0, 1/n, 2/n, 3/n, …, (n-1)/n. Note how the last frame is not `t=1`, since this would be an exact duplicate of the first frame `t=0` and show up as a tiny (and therefore all the more annoying) duplicate frame glitch in the animation. Since animated GIFs tend to use more space than necessary (the old format does not exploit any of the myriad statistical correlations between consecutive frames during the compression, but encodes each frame from scratch as a separate image), these days we convert any such animated GIF into various other formats to boil off about half of its bits. You kids these days have it so easy…

To make your periodic animation smooth so that there is no visible discontinuity at the times that are whole integers, define your animation paths so that the position, size, and other aspects of its **appearance** are the same for every individual object for `t=0` and `t=1`, whichever way that object happens to move in times between these two endpoints. Alternatively, you can organise your objects so each object has a "successor," so the appearance of this object at t=1 matches the appearance of its successor at `t=0`, and this way "cheat" to create infinite forward motion. See, for example, "Frequipathy", "Cubenary" or "Stalka", some old creations of the author. The fact that these compute the objects in three dimensions makes absolutely no difference to anything relevant. (Objects that end up outside the rendering area don't need such a successor, since their "blip out of existence" never shows up on the screen…)

Calculations involving such periodic motions will greatly benefit from periodic functions, most often the basic trigonometric functions sine and cosine. Since the purpose of this lab is not to give you

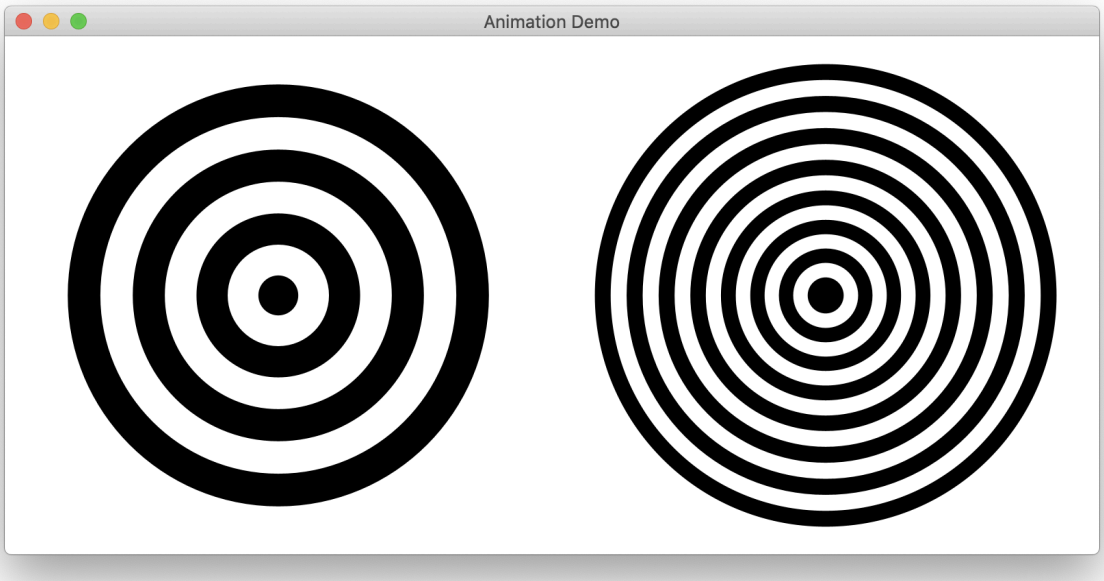
unpleasant flashbacks to the battlefield of trig back in the jungle of high school, the math is conveniently provided to you in the form of two periodic functions

```
default public double circleX(double cx, double r, double t)
default public double circleY(double cy, double r, double t)
```

These have already been implemented as `default` methods in `AnimationFrame` so that any class that implements `AnimationFrame` automatically inherits these implementations. You can use these functions to calculate smooth periodic motion in either one or two dimensions. To make a quantity `x` oscillate between two extreme values, `cx+r` and `cx-r`, around the centre point `cx`, compute it with the formula `x = circleX(cx, r, t)`. For example, to have `x` oscillate between 4 and 12, the math says `cx = 8` and `r = 4`. This oscillation is normalised to have a period of one.

When used together to compute an object's *x*- and *y*-coordinates at time `t`, `circleX` and `circleY` give the position of an object moving in a circle around the centre point `(cx, cy)` with radius `r`. (Internally, `circleX` uses the cosine function whereas `circleY` uses sine.) This approach can also be used to compute the points of a regular polygon centred at `(cx, cy)` with `n` sides (for example, a square for `n=4`), with the *i*:th corner being in coordinates given by plugging in `t=(1.0*i)/n`. (To rotate this polygon, add the same offset to `t` for each corner.)

Enough math and theory; let's finally get to practice! Create a class named `Breathe` that implements `AnimationFrame`. Implement the `render` method in this class to render concentric circles alternating in black and white (the number of circles should be given as a constructor parameter for flexibility) from outside in so that each circle is centred at the same coordinates `(width/2, height/2)` with radius diminishing inwards. An example screenshot of the `main` method of `AnimationPanel` that creates one instance of `Breathe` with eight circles and another with sixteen should clarify the intention. However, each circle's radius should oscillate smoothly so the animation looks organic, like something breathing, pulsing, or exerting **muscular effort to move**. Adjust your equations for the radii until you are satisfied with the impression.



Once you get these shapes in their proper places on the component, you can check out the author's old works "Frequestion", "Jobol" and "Zigwave" as examples of organic motion. The neat thing about periodic functions is that you can always add or subtract another periodic function, and the result remains periodic. Even better, if the first function's period is an integer multiple of the second's, the result will still have the first function's period. This allows us to design our motion paths with linear thinking, and once the animation works, adjust it by adding suitably periodic small "wobbles" to them, possibly weighted with periodic functions such as $\sin(\pi t)$ that have been tactically chosen to vanish to zero at both $t = 0$ and $t = 1$…

Lab 19: Symbolic Distances I: Representation

JUnit: DistanceTestOne.java

Even though Java expressions appear in symbolic form in the source code, the compiled code runs with variables that hold only the numerical results of evaluating those expressions. You only have to "show your work" in school exams, whereas the rest of reality, acting like a grumpy boss, wants only the final result and just enough paperwork to cover its arse if someone sues. This makes numerical approximations up to a couple of significant figures sufficient for government work. Even so, the results can be unsatisfactory, especially when they are rounded to double-precision floating-point numbers that cannot represent even simple numbers correctly, as we saw when evaluating `0.1+0.2` or comparing `(0.1+0.2)+0.3` with `0.1+(0.2+0.3)`.

We are forced to define new data types as classes to represent some restricted forms of symbolic expressions as exact values. We already saw the `Fraction` type, which keeps rational numbers such as 1/3 in exact form instead of rounding them to 0.3333333333333333 so the rest of the computation could proceed. Some languages come with useful data types for fractions and other limited symbolic expressions out of the box. Stratospherically high-level languages such as **Wolfram Mathematica** even keep every expression in symbolic form and operate on those forms, so only the final result needs conversion to a numerical approximation for human consumption.

If you can tolerate the memory and time cost of keeping expressions in symbolic form, they are better than numerical approximations for a simple reason: you can always convert from symbolic to numeric, but once you do, you can't reverse it and recover the original symbolic expression. Furthermore, symbolic expressions convey useful information about the data's structure that is lost when you reduce it to numbers. This is even though, inside the computer, there are only bytes filled with zeros and ones, and anything higher is mere consensual fiction that helps us organise our thoughts about computations in a more abstract and succinct form than the bare metal would allow.

In the spirit of the `Fraction` example we saw in class, this lab and the two labs that follow it define a new data type to symbolically represent numbers of the form where a can be any integer and b can be any **squarefree** positive integer. A positive integer n is squarefree if n is not divisible by c^2 for any integer $c < n$. Some examples of numbers we want to represent are $-7\sqrt{13}$, $42\sqrt{10}$, and $-1234\sqrt{101}$, where the squarefree requirement gives each possible value one unique representation in our system. For example, $7\sqrt{48}$ equals $7\sqrt{3 \times 4 \times 4}$ so that we can pull the two fours out from under the square root for a more succinct representation $28\sqrt{3}$ for the same quantity.

One super relevant fact about integers to know at this point is that the square root of an integer can never be a rational number of the form a/b unless $b = 1$, which would make the square root itself an

integer and the integer under the root symbol a **perfect square**. Otherwise, that square root must be irrational and algebraic, since it is a root of some quadratic equation. But the square root of an integer can never be a non-integer rational number.

Every integer a can be trivially represented as $a\sqrt{1}$ inside this system. Furthermore, formulas that compute $a\sqrt{b} + c\sqrt{d}$, $a\sqrt{b} - c\sqrt{d}$ and $a\sqrt{b} \times c\sqrt{d}$ will always produce results of that same form. The set of such numbers is therefore **closed** under these operations. No matter how you apply these operations to the values you have created so far, you will never get a value that is not representable in this system, ignoring the issues of arithmetic overflows. To allow division as the fourth arithmetic operator while still keeping this system closed, we would have to allow more complicated symbolic expressions than this simple form. (Not today, Satan. Sow your division some place else: here we do everything with integers, the way God intended.)

But that's enough math; it's time for a decisive coding action. Since these kinds of numbers often appear in distance calculations between points on the two-dimensional plane (for example, check out the Spiral of Theodorus), we shall name our data type for representing such quantities as `Distance`. Each instance of `Distance` represents a number that consists of a sum of terms of the form $a\sqrt{b}$, so that no b appears twice in the sum. Two terms $a\sqrt{b} + c\sqrt{b}$ can always be combined into a single term $(a + c)\sqrt{b}$ to enforce this constraint.

Since extracting squares from the given integer is so important, our first order of business is to implement a method to do so. Since this method does not need anything from the underlying object, we might as well make it `static` so that it can be used as a function:

```
public static int extractSquares(int n)
```

Given a non-negative integer *n*, this method should find and return the largest positive integer *c* so that *n* is divisible by c^2. (For any *n*, the value *c* = 1 will always work, if nothing higher does.)

Next, choose the internal representation for this new data type. To keep everybody who works on this lab on the same page with respect to this design issue, please use an instance of `TreeMap<Integer, Integer>` to associate each integer that appears under the square root with its coefficient outside the square root. Declare the following field inside your `Distance` class:

```
private TreeMap<Integer,Integer> coeff=new TreeMap<Integer,Integer>();
```

Every term of the form $a\sqrt{b}$ from the pure Platonic plane of mathematics thus becomes an entry `b=a` inside the `coeff` map in Java. For example, inside a `Distance` object that represents the number $3\sqrt{1} - 4\sqrt{7} + 5\sqrt{11} + 2\sqrt{101}$, the contents of `coeff` would be `{1=3, 7=-4, 11=5, 101=2}`, with exactly as many entries inside the map as there are terms in the original formula. Note how Java even uses the curly brace syntax in the `toString` representation of a `Map` instance the same way as

Python does for its equivalent `dict` data structure. (Oh, if only the rest of the Java language would also similarly play ball here…)

To construct simple `Distance` objects, we need a constructor to initialise an object with a single term and another to initialise an object from an arbitrary map that encodes the terms. (The single-term distance constructor creates and initialises the `TreeMap` the same way the general constructor does, but initialises it with one element instead of looping through all elements of the `coeff` map.)

```
public Distance(int a, int b)
public Distance(Map<Integer, Integer> coeff)
```

Both constructors should use your previous `extractSquares` function to express each term in its simplest form, since the outside callers didn't sign up to perform this simplification on your behalf. It will also make your life a lot easier later if you write these constructors to make sure that `coeff` will only contain terms $a\sqrt{b}$ for which a is nonzero. You can simply ignore any such term during object construction. So you might as well stop that buck right there at your desk, so that the rest of your methods may assume that every term has a nonzero coefficient. (Analogous to the President's words, every action of the constructor weighs a ton.)

As with the `Fraction` type, we will design the `Distance` class to be immutable, so it has no public methods that could modify the object's internal state. The arithmetic operator methods will always create and return a new object representing the result, rather than modifying either of the originals involved in the operation. Therefore, your second constructor should create a defensive copy of the `coeff` map object it receives and copy the coefficients there for safekeeping, instead of storing a reference to an object that the outside world can still use and modify.

One more method left to write in this lab, although that one is the biggie. The first `public` method you should implement when creating a new data type is `toString()`. Most of the time, that task is straightforward, so we might as well do it now to make the result look pretty, so no user code ever has to do work that clearly belongs to the designer of that data type. As they reputedly say in some country's military, a good soldier will throw himself on the landmine to save his battle buddies. (Your 4-F instructor wouldn't really know, since the only veteran status he can claim without being tarred and feathered for stolen valour is being a veteran reader of *Beetle Bailey*, along with the "Humour in Uniform" column in *Reader's Digest*.)

```
@Override public String toString()
```

Because the automated JUnit tests record the results from this string representation, the string returned for each instance of `Distance` has to be unambiguous. The five rules that **you must follow in this lab** to guarantee that the outcome is both unambiguous and readable are:

1. The terms must be listed in ascending order of their square roots. (This is why we use `TreeMap` instead of `HashMap`, so that looping through the elements of its `keySet()` view is guaranteed to produce these terms in ascending order.)
2. If the coefficient of a term is 1, leave it out unless the square root is also 1.
3. Each term $a\sqrt{b}$ is transcribed as the string `"aSqrt[b]"`, with no space between the coefficient and the square root.
4. Consecutive terms are separated with either `" + "` or `" - "` depending on whether the coefficient of the next term is positive or negative, with exactly one whitespace around the operator symbol. If the `" - "` separator is used, the coefficient of the next term is transcribed without the minus sign.
5. If the coefficient of the first term is –1, it is transcribed as a minus sign, without the 1.

These rules produce the result that *Wolfram Mathematica* would use to display such terms. Given a choice between implementing the `toString` method some other way rather than the way that has been field-tested over thirty arduous years, is there any reason not to go with the tested and true? The following table shows a list of illustrative examples of the expected way to transcribe a `Distance` object, given the `coeff` map given to its constructor.

`coeff`	Expected result of calling `toString()`
`{}`	`"0"`
`{42=0,99=0,123=0}`	`"0"`
`{1=1}`	`"1"`
`{1=-42}`	`"-42"`
`{5=7}`	`"7Sqrt[5]"`
`{7=5}`	`"5Sqrt[7]"`
`{7=4}`	`"14"`
`{5=1,10=-1,15=1,21=-3}`	`"Sqrt[5] - Sqrt[10] + Sqrt[15] - 3Sqrt[21]"`
`{5=1,10=-1,15=1,20=-1}`	`"-Sqrt[5] - Sqrt[10] + Sqrt[15]"`
`{3=-5,7=11,6=-17}`	`"-5Sqrt[3] - 17Sqrt[6] + 11Sqrt[7]"`
`{10=-1,43=99}`	`"-Sqrt[10] + 99Sqrt[43]"`

Of course, your `Distance` constructor will extract all possible squares from the coefficients it receives, rather than passing that buck to every future user of this class. As a class designer, always go the extra distance for your class's users. There is only one class designer but an unknown number of future users, so the needs of the many outweigh the needs of the one.

Lab 20: Symbolic Distances II: Arithmetic

JUnit: DistanceTestTwo.java

The code written in the previous lab allows us to construct and print out symbolic expressions that are sums and differences of terms of the form $a\sqrt{b}$ so that a is any integer and b is any positive squarefree integer. This lab adds the basic arithmetic operations of addition, subtraction and multiplication to the mix. Since we intentionally design the class `Distance` to be immutable to get all the advantages of immutable data types, none of these methods should modify the internal state of either `this` or `other` objects, but create and return a new `Distance` object for the result.

When writing the methods of this lab, you are allowed to assume that the JUnit test class gives your methods only argument values that never cause the computation to overflow when using the `int` type for integer arithmetic. If you want to convince yourself that unreported overflows do not happen, you can execute your `int` arithmetic with the methods `addExact` and its brothers in the utility class `java.lang.Math`, instead of using the operators `+` and such already provided in the core language. These **exact arithmetic** methods in the `Math` utility class are guaranteed to throw an `ArithmeticException` whenever they detect an overflow. If that exception isn't thrown, you can sleep soundly tonight knowing no overflow occurred.

```
public Distance add(Distance other)
public Distance subtract(Distance other)
```

These methods implement the addition and subtraction between `this` and `other`. Since the logic of both methods is identical except for one small part of one statement, we strongly recommend implementing both of these operations in one swoop with a `private` helper method

```
private Distance arithmetic(Distance other, boolean adding)
```

where the parameter `adding` indicates whether the terms of `other` should be added or subtracted from the corresponding terms of `this`. The `add` and `subtract` will then be simple one-liners that call this helper method with the second argument respectively set to `true` or `false`.

```
public Distance multiply(Distance other)
```

This method is implemented most easily using an empty `TreeMap<Integer, Integer>` instance that will contain the result. That way, the method logic is just two nested loops that iterate over all possible pairs, taking the first term from `this` and the second term from the `other`. For every such pair of terms $a\sqrt{b}$ from `this` and $c\sqrt{d}$ from the `other`, compute their product $ac\sqrt{bd}$, possibly extracting some square hiding under the square root made of the common prime factors of b and d. Then, either create a new entry for bd inside your result map, or update its existing entry by adding

the coefficient ac to its value. In the latter case, you should correctly handle the edge case where that existing value is already equal to $-ac$ so that this entire entry vanishes altogether.

Lab 21: Symbolic Distances III: Comparisons

JUnit: DistanceTestThree.java

The previous two labs let us perform arithmetic on `Distance` objects and print the results in a human-readable format. Now we need to make these `Distance` objects behave civilly with other types so the existing classes in the Java Collection Framework and elsewhere work properly with them. In Python, you achieve this by defining magic methods that correspond to the language's operators. In Java, you should override the following two methods inherited from the universal superclass `Object`.

```
@Override public boolean equals(Object other)
```

Using the example class `Fraction` as a model, override this method to accept value equality only with another `Distance` object whose `coeff` map consists of the same terms as `this` one. The analogy to the famous Fundamental Theorem of Arithmetic applies here: each numerical quantity representable in this system has exactly one such representation. If two `Distance` objects have different `coeff` maps, the quantities they represent are also different.

```
@Override public int hashCode()
```

Override this method to compute somehow a hash code that depends on the terms in the `coeffs` map. Since you are using a `TreeMap<Integer, Integer>` instance to store these coefficients, you can simply return the `hashCode` of that map object as the `hashCode` of the entire `Distance`. After all, if two `Distance` objects consist of different terms, they represent distinct real values. (Discrete math enthusiasts can try their hands at proving this, if only as a fun little side quest.)

Since these `Distance` objects represent real numbers, they should also allow order comparisons as such. Edit the first line of the class definition in your source code to now say

```
public final class Distance implements Comparable<Distance>
```

to guarantee the existence of the `compareTo` method. The big question now is what kind of algorithm could reliably compare any two arbitrary `Distance` objects in this manner. Without any fancier mathematics, we will simply evaluate each object as a decimal number and compare those values. However, to make this lab more interesting, we will not use the plain old `double` type for this, but might as well learn how to perform such calculations to arbitrary precision with the `BigDecimal` type from the `java.math` package! (When dealing with the `BigDecimal` type that internally uses base ten instead of base two to represent the decimal numbers, 0.1 + 0.2 actually equals 0.3, not one iota more or less…)

Analogous to the `BigInteger` type for integers limited only by your heap memory, `BigDecimal` objects represent decimal numbers. Their integer part is unlimited, like `BigInteger`, but their decimal precision is determined by a `MathContext` object that tells the `BigDecimal` how far down it should keep computing decimals. After all, even simple arithmetic expressions such as 1.0/3.0 would need an infinite series of decimals to be exact! Whenever an operation on `BigDecimal` values produces an approximate result, the rounding mode determines how it determines the last included digit from the truncated digits.

```
public BigDecimal approximate(MathContext mc)
```

Returns the approximate value of this `Distance` as a `BigDecimal` object with the settings as in the given `MathContext`. You should simply evaluate each term in turn and add it to the result.

```
public int compareTo(Distance other)
```

Performs the order comparison between `this` and `other` objects of type `Distance` by comparing their `BigDecimal` approximations. As in equality comparison, two Distance objects with different terms represent different numbers, but determining which object is larger can be tricky. It is hard to know *a priori* how much precision the `MathContext` needs for this comparison to be accurate enough to settle who is truly the mightiest. Otherwise, two sums that differ by a microscopic amount will seem equal when viewed through the numerical equivalent of Coke bottle lenses.

The lovely thing about this problem is that we don't need to know how much precision we need! Since we can always repeat the approximation with a higher precision, simply start with some small precision such as 10, and `approximate` both `this` and the `other` with a `MathContext` of that precision. If these approximations do not reveal which number is greater, repeat with higher precision. For example, 15. And so on. Simple order comparison between two sums of square roots of integers is **PSPACE-complete**; in the worst case, these sums need to be evaluated to an exponential number of decimal places to tell them apart! For the test cases emitted by our JUnit test class, this scheme of iteratively increasing precision will eventually become sufficiently surgical to tell apart close sums separated by a subatomic margin that makes obsidian razors seem dull by comparison.

Students interested in seeing more `BigDecimal` computations in action should check out our `Mandelbrot` example. This **fractal renderer** performs its computations with the `BigComplex` type, which supports arbitrary-precision addition and multiplication of complex numbers. Akin to that old joke of how it is easy to sculpt a statue of an elephant ("Start with any old block of stone, and chip away every piece that doesn't look like it would be part of an elephant!"), the program sculpts the outline of the black Mandelbrot set by carving away all pixels around it that don't belong in that black hole. It colours these pixels using a scheme that depends on how many steps it takes for the iteration from that pixel to cross the boundary. Not a single processor cycle is wasted on pixels strictly inside the Mandelbrot set. However, the computation never terminates, as the border pixels are iterated forever in a vain hope of escape.

Lab 22: Truchet Tiles

For testing: TruchetMain.java

As explained on the Wikipedia page "Truchet Tiles", tiling a two-dimensional integer grid with four rotationally symmetric black-and-white tiles can produce surprising "op-art" geometric patterns. This lab identifies which of the four types of Truchet tiles is placed in each position using two truth values, `parity` and `diag`, computed for that position. The `diag` parameter determines whether the dividing line between black and white parts cuts through the main diagonal or the anti-diagonal, and the `parity` parameter determines which side is black and which side is white. In your labs project, create a new class

```
public class Truchet extends JPanel
```

to make your class a full-fledged Swing component out of the box. Each instance of this class should display the Truchet tiling defined by its five constructor parameters.

```
public Truchet(
  int s, int w, int h, BiPredicate<Integer, Integer> parity,
  BiPredicate<Integer,Integer> diag
)
```

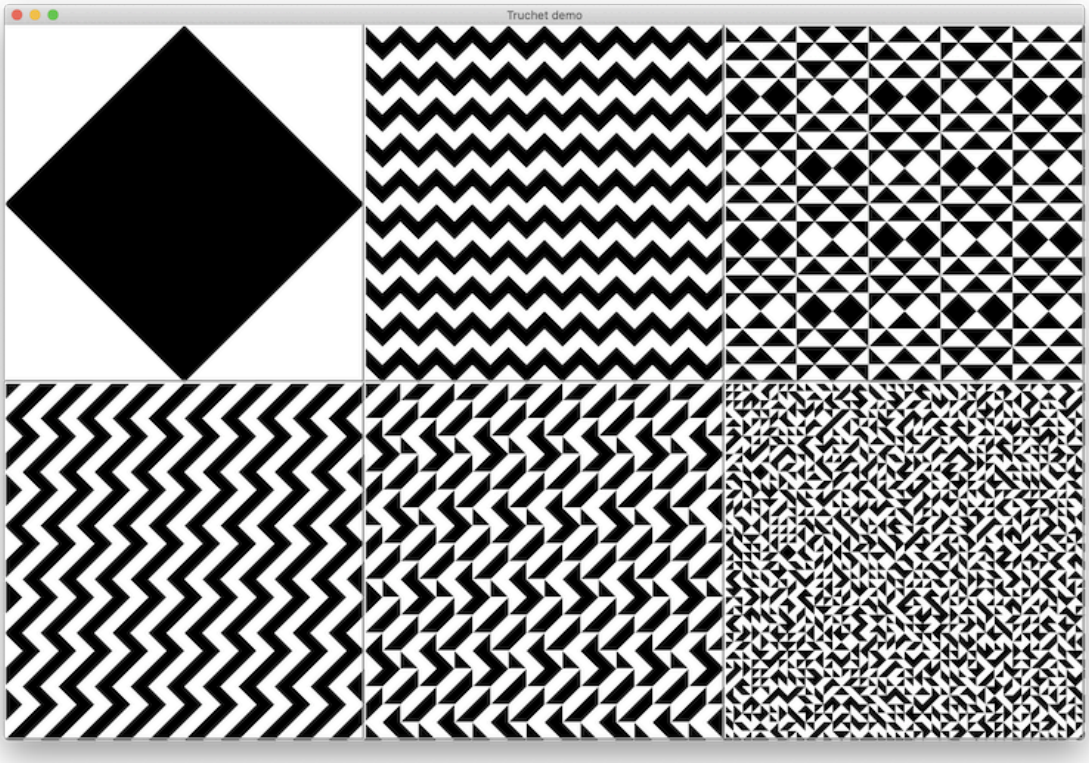
Parameters `w` and `h` determine the grid width and height in tiles, and parameter `s` gives the size of each tile in pixels. For example, an image with `w=100`, `h=60`, and `s=5` would be 500 pixels wide and 300 pixels tall. Parameters `parity` and `diag`, both of the same type `java.util.function.BiPredicate<Integer, Integer>` to encapsulate two functions that take two `Integer` parameters and return a `boolean` result, provide your renderer the parity and diagonal of each tile based on its coordinates `(x, y)`. This constructor should store its arguments into object fields to be accessible later, and set the preferred size of this component to `new Dimension(s*w, s*h)`.

```
public void paintComponent(Graphics g)
```

Renders the individual Truchet tiles on the surface of this component, the shape of each tile as determined by the strategy objects `parity` and `diag` that make these decisions. There are four ways to assign these colours based on the tile's parity and diagonal direction. However, to allow both the student and the grader to agree at a glance that this lab has been implemented correctly, your code should make these decisions in the exact manner that makes running the automated test class `TruchetMain` produce the result shown in the screenshot below. The easiest way to draw a filled triangle is probably to define and fill a `Path2D.Double` shape. (Again, do not neglect to first call `super.paintComponent(g)` to erase the previous contents of the component.)

Unlike JUnit tests, which can do their work even if we close our eyes and pretend we are no longer in the room, in this problem our eyeballs have to make the final determination of whether the pattern is as expected. At least our visual system offers a massive level of parallel processing right on the **wetware** that JUnit or any other computer program could not even dream of.

Especially important is the `Truchet` instance at the top left corner of this 2-by-3 component grid,



the simple black lozenge against the white background, since that one confirms that your code uses the same scheme for colouring the tiles as in the instructor's private model solution. You might want to check out the modular arithmetic formulas used to produce the other instances where a pattern emerges from the local decisions that depend on the numerical properties of the tile coordinates. Even randomness feels more pleasant when you add non-random elements to balance it out, so the human visual system has something to latch onto for interpretation. (The TruchetMain.java class also has a method that uses Truchet tiling to showcase basic punctuation characters from War and Peace, inspired by the post "Punctuation in Novels".)

Once you get this component working, you can also try more artistic variations, such as Smith tiles that use quarter circles instead of triangles to create curved, blobby shapes. Or create a **labyrinth** by using `diag` to draw a black diagonal line to separate the two white halves, ignoring `parity` completely, in the spirit of the classic Commodore 64 BASIC one-liner. You could even subdivide some tiles into smaller tiles for a fractal subdivision effect, or explore the properties of random Truchet tilings. Only your imagination is the ultimate ceiling of your art; in this course, the only law we acknowledge and allow to bind our hands is the Law of Awesome!

Lab 23: Ulam–Warburton Crystals

For testing: UlamTest.java

Game of Life is the most famous creation of John Conway, the unconventional mathematical genius who passed away during the COVID pandemic (xkcd tribute) one week before this lab was originally written in April 2020. Game of Life is used in many introductory programming courses for a good reason, including this one, either as an example or as a programming project for the students to explore. This deceptively simple setup offers ample educational value not just for the first coding course, but far beyond that, up to the ultimate limits of computability.

However, to deviate from this well-trodden path, we will instead implement a similar Ulam–Warburton automaton that produces its own distinctive, more crystalline fractal patterns. Besides, that Stanislaw Ulam fella was also a pretty important figure in applied mathematics and computing in his time, including the Manhattan Project. His life and creations deserve at least some attention up here in this cyberpunk science fiction future timeline we have suddenly found ourselves living in, even though most people don't seem to have realized it, almost as if some wizard had cast a spell to make humanity forget this happened. The term "science fiction" itself has become obsolete when not just its standard staple of video phones, but also real-time face substitution, subtitles and real-time translation to another language elicit only a bored yawn. (Any readers who have a better explanation for this mass amnesia than the meek "a wizard did it" surely also know enough not to publicize this explanation to the wider world.)

The first new programming technique used in this lab shows how to encode two-dimensional arrays in a single dimension. Two-dimensional arrays in Java are actually just one-dimensional arrays whose elements are one-dimensional arrays. This "array of arrays" representation allows individual rows to be ragged, so they don't all necessarily have to have the same length. However, for regular grids where each row has the same length, you can also encode the two-dimensional structure into a one-dimensional array with rows*cols elements, where the element at `(i, j)` is stored at `i*cols+j`. (This is how, for example, NumPy stores and represents its arrays that are known to be grids of elements of uniform type.)

For this lab, you should just use the previous simple representation. However, as an aside that might be useful later, the latter representation is especially clever when the values of that two-dimensional array are known to be symmetric across the diagonal, as in problems where the underlying reality that creates the entries in this table is already symmetric with respect to these indices. To map both `(i, j)` and `(j, i)` into the same target position, first enforce `i<=j` with a swap if necessary, and then use the formula `j*(j+1)/2+i` to compute the target position in the one-dimensional array. Compared to storing every non-diagonal element twice, this trick uses almost half the memory. Cutting your memory use in half is no small feat, even if the program ran to completion under a more wasteful memory allocation scheme, since it lets you solve instances twice the size and hopefully have at least twice as much fun!

The Ulam–Warburton automaton is similar to Game of Life in style and spirit: every cell in the board (now a simple two-dimensional grid, but could in principle be any graph) is always in exactly one of two binary states, "on" and "off". The automaton advances in discrete time steps starting from zero, updating each cell simultaneously at each tick. In the original Game of Life, the state of each cell at time $t + 1$ depends on the states of the eight cells in its 3-by-3 Moore neighbourhood (fewer at edges and corners) only at the previous time t; the rules of the Ulam–Warburton automaton have the state at $t + 1$ depend only on its von Neumann neighbourhood (up, right, down, left) from the previous two steps, t and $t - 1$, instead of merely its immediate predecessor state. The rules are:

- If the cell (i, j) is **off** at time t, it is on at time $t + 1$ if **exactly one** of its four neighbours was on at time t, and otherwise it is off.
- If the cell (i, j) is **on** at time t, it is on at time $t + 1$ if that same cell (i, j) was off at time $t - 1$, and otherwise it is off.

In other words, cells turn on when exactly one neighbour is on, and they automatically turn off after being on for exactly two time steps, regardless of their neighbours' states. Your method must, of course, handle the board edges carefully.

In your labs project, create a class `Ulam` and write the following method inside it:

```
public static void computeNext(
  int cols, boolean[] prev, boolean[] curr, boolean[] next
)
```

The arrays `prev` and `curr` contain the two-dimensional boards at times $t - 1$ and t encoded into a single dimension, as explained above. (The number of `rows` is equal to `prev.length/cols`, and would thus be redundant to require as a parameter.) This method should not return anything, but compute the state of the board at $t + 1$ into the given parameter array `next`. Every time this method is called, all three arguments `prev`, `curr` and `next` are guaranteed to be proper array objects that contain enough elements to perform the necessary computations.

To allow the states of this automaton to be displayed as a Swing component, have your `Ulam` class extend `JPanel`. Define the field

```
private boolean[][] states;
```

that the component uses to store the computed states of the automaton, so that the state at time t is stored in the one-dimensional boolean array `states[t]`. Then, define a constructor

```
public Ulam(int rows, int cols, int[][] start, int n)
```

that computes the `states` of this automaton for times 0 to `n` for later lookup. At time $t = 0$, every state is off. Then, let there be states. At time $t = 1$, every state is off except those whose positions are

given in the array `start`, which is guaranteed to contain two-element `int[]` objects as its elements. For example, in the `UlamTest` method `massTest`, the array

```
int[][] start = {{100, 100}, {100, 200}, {200, 100}, {200, 200}};
```

would have those four states on. To allow the JUnit tests to access the state of each cell at given times, your class should have the accessor method

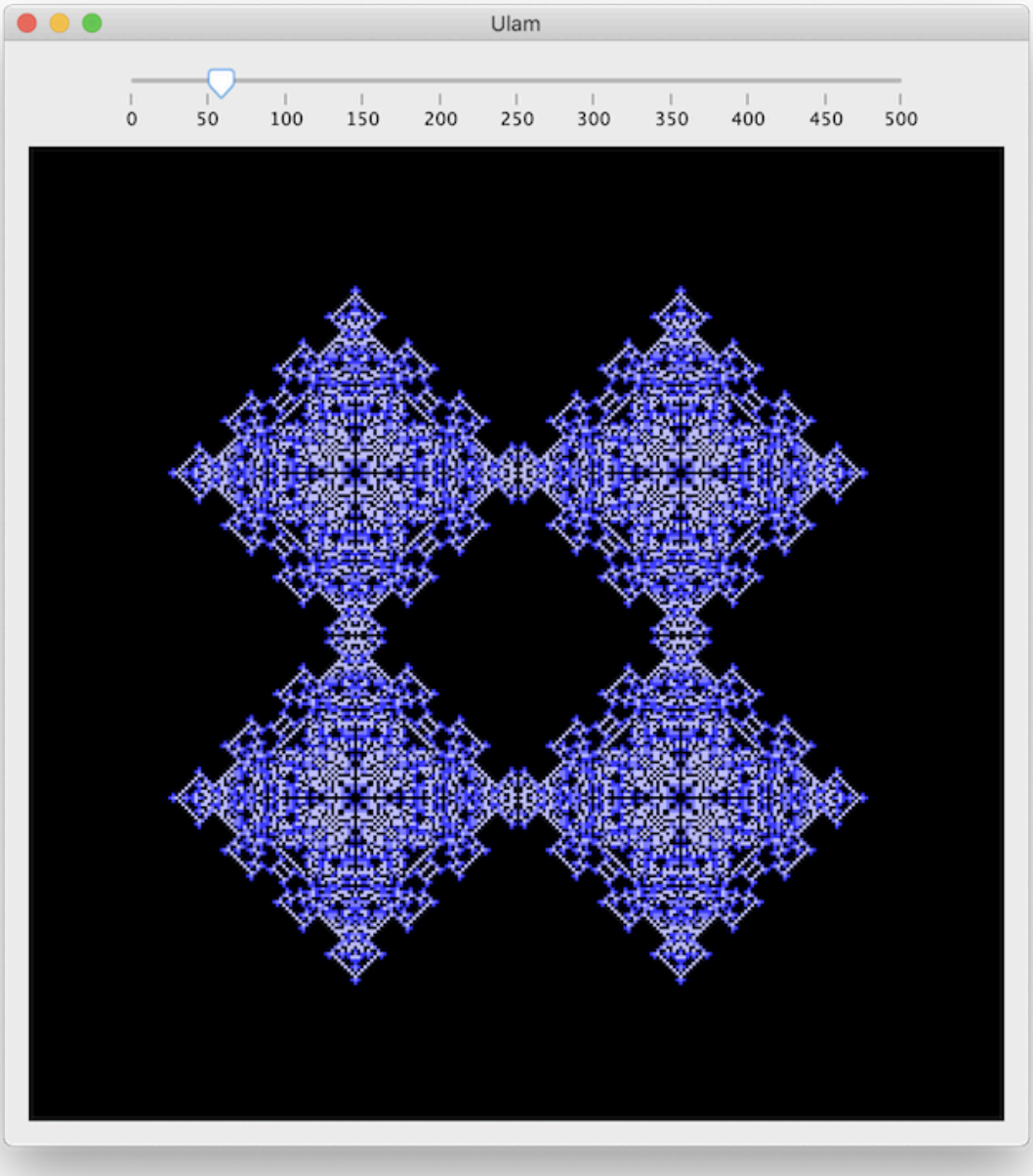
```
public boolean getCellState(int i, int j, int t)
```

that returns the state of the cell (*i*, *j*) at time *t*. To verify that the logic of your `computeNext` method is correct according to the rules of this automaton, you should run the `massTest` method in the JUnit test class `UlamTest` to verify that it gives you a green checkmark.

To try something new in this lab, instead of animating the automaton's state evolution using Java concurrency, this component should contain a `JSlider` whose values range from 0 to `n`. By grabbing and moving that slider, the user should be able to instantly skip forward or backward to any particular moment in time to see the board displayed in the `paintComponent` method in a manner that resembles the screenshot below. Create a nested `ChangeListener` subclass to listen to the events of your slider component, and then later use the slider's `getValue` method to determine the time to render the automaton state.

This 300-by-300 board displays each cell as a 2-by-2 pixel box to make the result easier to see, but you can adjust this if you want to explore the evolution of the crystalline structure on larger game boards. You can also try seeding the automaton asymmetrically to break symmetry in its evolution. The colour scheme used in the screenshot uses the colours listed in the following table to render a cell based on its state at times *t* and *t* – 1.

State at *t*	**State at *t* – 1**	**Colour**
off	off	black
on	off	dark blue
on	on	medium blue
off	on	light blue



To get back to Conway's Game of Life for a moment, every second-year computer science major should be able to write a simple brute-force Game of Life component. However, try to imagine the algorithmic techniques needed to merely store the current state of a giant board whose side lengths measure in millions of tiles (data structures for **sparse matrices** suddenly become necessary to keep this Jacob's ladder from running out of memory), let alone simulate the effect of one time step on such a board, or coming up with mathematical techniques to simulate some part of the board more than one step forward in one swoop.

Another surprisingly difficult coding challenge is implementing Game of Life logic backwards, computing the previous state from the current state it leads into. However, even if a computation is deterministic in the forward direction, it can still be nondeterministic in reverse, so the same current state could have multiple possible predecessors, some of which lead to contradictory states that no Game of Life configuration can produce. In general, if computations could be reversed mechanistically without this exponential blowup of past possibilities, any encryption algorithm would be trivial to crack by running it in reverse from the ciphertext output back to the original plaintext input…

Lab 24: Chips On Fire

JUnit: CoinMoveTest.java

Consider the following operation on arrays whose elements are nonnegative integers, interpreted here as the number of **chips**. First, my apologies for the inconsistent terminology. The first version of this problem used "coins" until the author realized that, in the literature of this field, the systems described below are generally known as chip firing systems. As long as the names we utter from our virtual mouths are sufficiently memorable and unique to keep their referents apart, it's all to-*may*-to, to-*mah*-to anyway. Men worry, machines execute.

Each position in the array also has a list of **obligations**, each of which is another position in the array. Each position can have any number of obligations to other positions, and can have multiple obligations to the same position. The time in this system proceeds in discrete steps. At each time step t, perform the following operation for every position in the array to get the chip distribution for the next time step, $t + 1$. The logic of your method must be designed so that its result is equivalent to performing this operation for every position in an instantaneous **atomic** step so that all positions fire their chips in unison, as if they were saluting the most powerful *maharaja* that the mutton-chopped forces of our beloved Queen ever had the honour to encounter!

Every position that has at least as many chips as the total length of its list of obligations gives out exactly one chip to each position as many times as that position appears in that list. Positions that don't have enough chips to meet their current obligations give no chips to anyone; giving chips from your purse in each round is all or nothing. Some possible chip distributions across the positions are Gardens of Eden, for which no predecessor position fires chips to reach that distribution.

This operation obviously never changes the total number of chips inside the system, since it just moves them around the various positions. The **states** of this chip-firing system therefore correspond to the possible ways to distribute this fixed chip stack among the positions. Transitions lead from each state to the next state resulting from the previous operation. To allow you to compute these transitions, create a new class `CoinMove`, and there the method

```
public static void coinStep(
  int[] curr, int[] next, List<List<Integer>> obligations
)
```

The current state is given in the parameter `curr`, and the list of obligations for each position is an element of the list of such lists named `obligations`. (Both lists `curr` and `obligations` are guaranteed to have the same length.) This method should not return any result, but instead write the next state into the array `next`, which is also guaranteed to be an `int[]` of the same length as `curr` at the time of the call.

The following table showcases some possible values for `curr` and `obligations`, and the expected `next` state that this method is expected to produce:

`curr`	`obligations`	Expected `next`
`[4, 5]`	`[[1], [0]]`	`[4, 5]`
`[3, 1]`	`[[1, 1], [0, 0]]`	`[1, 3]`
`[2, 3, 0, 1]`	`[[1], [3, 3, 0], [], [0]]`	`[3, 1, 0, 2]`
`[3, 3, 1, 1, 2, 0, 0]`	`[[6, 5, 1], [6, 4, 2, 2], [], [5, 0, 0], [], [], []]`	`[0, 4, 1, 1, 2, 1, 1]`
`[3, 2, 2, 0, 1, 2, 2]`	`[[3, 5], [6, 3], [0, 3, 4], [6, 2], [5], [2, 2, 3, 4, 2, 4, 1, 2, 1], []]`	`[1, 0, 2, 2, 0, 4, 3]`

Since the total number of chips in the system and the list of obligations for each position never change, iterating this system over the infinite sequence of time steps $t = 0, 1, 2, \ldots$ must eventually enter a **cycle** where the same sequence of states keeps repeating forever. The **period** of the cycle is the smallest positive integer $k > 0$ such that for every t in the cycle, states t and $t + k$ are equal.

```
public static int period(int[] start, List<List<Integer>> obligations)
```

Given the `start` state and the list of obligations for each position, as in the previous method, this method should compute and return the period of the cycle reached from `start`. Iterate the sequence until you come to some state that you have already seen before. That state is the start of the cycle, so we should remember it. Initialize an integer counter to zero, and iterate the same cycle again, incrementing the count in each step. When you inevitably return to the starting point of the cycle, return this count as the requested cycle length.

Ah… but one pesky little fly still buzzes in this ointment. Since the number of **reachable states** in the system can be exponentially large, so can also the cycle length and especially the length of the path leading to that cycle. Therefore, making the previous algorithm explicitly store all the states it has seen in some kind of Set structure might be prohibitively expensive. Fortunately, an ingeniously simple algorithm exists for this exact situation: we can generate a sequence one step at a time and recognize the first time we see a previously visited state, which marks the cycle. Robert Floyd's Tortoise and Hare algorithm is a computer science classic that deserves way more fame than it currently enjoys.

Instead of maintaining only one current state inside the sequence used to advance to the next state, maintain two such states separately, aptly titled "tortoise" and "hare", both initially residing at the

`start`. The hare takes two steps for every step the tortoise takes, so it iterates through the sequence twice as fast. When the hare enters a cycle (as it must, since the start state can also be in a cycle), it keeps racing around that cycle until the slower tortoise enters it as well. Regardless of their positions inside the cycle at that moment, the hare will inevitably catch up to the tortoise's position. Once that happens, both animals stop, shake hands and say their equivalent of “jolly good, old chap” to each other, knowing that the current state must lie inside the cycle. The hare can now stay in place and take a well-deserved rest, while the tortoise makes another trip around the same cycle, counting the steps needed to return to the hare.

As the JUnit fuzz test reveals, the cycle period depends heavily on the structure of obligations between positions. For most configurations, the answer is 1; these self-cycles are equilibrium states where all obligations and chip constraints balance perfectly. However, this period can also be much longer, even well over one million for some of the initial states and obligation lists randomly generated in the JUnit test. As with all exponential growth, increasing the array lengths and number of chips in the system would eventually make these periods dwarf our entire universe…

The famous space-time tradeoff in computer programming usually works the other way than in our case. Normally, we make the program run faster by giving it more memory to remember the relevant stuff it has seen and done. The exact opposite technique used by the Tortoise and Hare applies when remembering all those relevant things would require far more memory than any computation device in our physical universe can provide.

In addition to modelling serious stuff during billable hours while wearing a tie, chip-firing systems can also create computational art by mapping each pixel to a position at a given time and colouring it according to the number of chips present there. By definition, anything that brings art and science closer together counts as progress. Chip-firing systems are also closely related to the Abelian sandpile model, presented in the Numberphile video “Sandpiles”.

Lab 25: Manhattan Skyline

JUnit: ManhattanTest.java

The **Manhattan skyline problem** is a well-known exercise in computational geometry, with plenty of educational and interesting variations. Its setup illustrates the **sweep-line algorithm**, which can solve not just this problem but many others on the two-dimensional plane.

In the basic version of this problem, several tall buildings (perhaps built in the heyday of some midwestern city on a flat plain) are seen from a distance against the horizon. Our vantage point is far enough away that each building appears as a two-dimensional rectangular **silhouette**. From our point of view toward the horizon, each silhouette has a starting and ending x-coordinate, s and e, that always satisfy $s < e$, and a **height** h. These projected silhouettes can overlap in part or in whole, even though the original three-dimensional buildings are physically disjoint. Create a new class `Manhattan` in your labs project, and there the method

```
public static int totalArea(int[] s, int[] e, int[] h)
```

The three arrays `s`, `e` and `h` (guaranteed to all have the same length n and consist of positive integer values only) contain the start coordinate, the end coordinate and the height for each building numbered 0, …, $n - 1$, respectively. These buildings are guaranteed to be sorted by starting coordinate, with ties resolved by ending coordinate. This method should return the total silhouette area, counting overlapping parts of individual building silhouettes only once, regardless of how many buildings overlap at a given x-coordinate.

Conceptually, a sweep line algorithm "sweeps" through the horizon from left to right, noting points where an "interesting" event occurs, meaning the event could affect the algorithm's result. The algorithm should start by creating a list of these interesting events and sorting them by their x-coordinates. In the Manhattan skyline problem, the interesting events are "building b enters the view" and "building b exits the view" as your virtual "eye" sweeps the horizon from left to right. Start by defining a local variable

```
List<Integer> events = new ArrayList<>();
```

with these "enter" and "exit" events encoded so that building `b` entering the view is encoded as `-1-b`, and building `b` exiting is encoded as `b+1`. (We need to encode these events as integers in a tricky way to guarantee each event is encoded as nonzero, because there is no way to tell apart zero and minus zero!) Add these $2n$ event encodings into `events`, and sort these `events` with a custom `Comparator<Integer>` local strategy class whose `compareTo` method compares two given events by first decoding them into starting times (taken from array `s`) or ending times (taken from array `e`). The event type determines whether we use its entering time or exiting time as the comparison key.

Sorted this way, the events are ready to process in order, sweeping left to right and updating the area tally. To keep track of the buildings that are currently in your active view (the so-called **active set** of the sweep line algorithm), define another local variable

```
Set<Integer> active = new HashSet<>();
```

that contains the buildings that are currently in the active view. Loop through the sorted `events`, also keeping track of the *x*-coordinate of the event processed during the previous round. To process an event, first compute its horizontal distance from the previous event. Multiply that distance by the height of the tallest active building, and add that product to the total. (Sometimes several events take place at the same *x*-coordinate, but since the distance between those events is zero, all such redundant terms from slivers of zero area between the literal and metaphorical walls that separate us from each other vanish from this classic summation, as if sniped to extinction by the master guns Tom Berenger himself.) Next, update the active set by adding or removing the building that enters or exits the view, and move on to the next round. Once all buildings have entered and exited the active view, this method can return the accumulated total.

s	e	h	Expected
{1, 4, 11}	{5, 8, 13}	{2, 1, 2}	15
{2, 6, 9, 12, 15}	{3, 8, 10, 14, 20}	{3, 3, 4, 3, 2}	29
{0, 9, 11, 13, 13, 18, 31, 32, 35}	{4, 23, 21, 23, 24, 21, 38, 50, 45}	{3, 3, 1, 1, 2, 2, 3, 3, 1}	113

Lab 26: FilterWriter

JUnit: FilterWriterTest.java

Some examples discussed during the lectures featured decorators for `Animal` and `Comparator` class hierarchies. The decorator pattern works the same way for any other class hierarchy, except that the more methods a class's public interface contains, the more annoying decorating that hierarchy becomes. The `Writer` hierarchy, which transmits text data composed of Unicode characters, makes writing decorators more convenient. Subclasses need to override only three methods, since the default implementations of the others call those three **template methods** to do their work. Subclasses can still override these template methods whenever they can provide a more efficient implementation. Still, they are never required to do so unless the template method itself is declared abstract in the superclass.

In your labs project, create the class

```
public class FilterWriter extends Writer
```

with the fields and constructor

```
private Writer writer;
private BiPredicate<Character, Character> pred;
private char previous;

public FilterWriter(Writer writer, BiPredicate<Character, Character> pred,
char previous) {
    this.writer = writer;
    this.pred = pred;
    this.previous = previous;
}
```

The functional interface `BiPredicate<T, U>` in the package `java.util.`represents a predicate that takes two arguments and returns a `boolean` truth value. In `FilterWriter`, this predicate is used to filter the characters that this decorator lets through. Each `FilterWriter` instance remembers the `previous` character that it has let through, initialized from the constructor parameter to be used as the initial character. Override the methods `flush` and `close` to call the corresponding methods of the underlying `writer`, and then override the one required method

```
@Override public void write(char[] data, int off, int len)
  throws IOException
```

to write the characters from the subarray of `data` that contains the `len` characters from position `off`. To decide whether the current character `c` should be passed on to the `write` method of the underlying object, the method should call `pred.test(previous, c)`. If the predicate accepts the character `c` by returning `true`, the character `c` is passed on to the underlying writer and becomes the new `previous` character. Characters rejected by this predicate are ignored, in the spirit of followers of a high school queen bee mean girl, all pretending that the objects of Her Majesty's scorn don't even exist.

The JUnit test class `FilterWriterTest` contains two test methods that create a filtered version of *War and Peace*. The first test adds a bunch of Unicode combining characters to each letter of the original line to create some "zalgo text", but the predicate that rejects all such combining characters regardless of the previous character should have no problem restoring the original text. The second test uses a predicate that rejects every vowel that immediately follows a vowel, and every consonant that immediately follows a consonant, producing results like the following paragraph:

```
Pire was unany. Sot, abot te average heg, bod, wit huge
red han; he did not kow, as te san is, how to ener a dawin
rom an sil les how to leve one; tat is, how to sa sometin
pariculary agebe before gon awa. Besides tis he was aben-
mined. When he rose to go, he tok up ined of his ow, te general's
te-corered hat, an hel it, pulin at te pume, til te general
ased him to resore it. Al his aben-minedes an inability to
ener a rom an conere in it was, however, redemed by his kiny,
sime, an modes exesin.
```

At least to this linguistic layman, the filtered text seems to carry a curious pidgin rhythm and flavour of its own. This leads us to wonder further whether simple variations of this operation, possibly aided with some vowel and consonant permutation substitutions, could be mechanistically applied to our natural languages to create a completely made-up and yet internally coherent **pig latin** for the exciting universe in the next hit series of those young adult fantasy novels with a young female hero whose superpower is being totes unique and special in a joyless world full of mindless sheeple, and who is forced by the circumstances to choose between two very different but equally exciting boys. The algorithm demonstrated in the `DissociatedPress` lecture example can also achieve such pseudo-linguistic ends. The algorithm could even effortlessly mash together several languages to obscure the original languages' flavours.

Lab 27: Stacking Images

For testing: ImageAlgebraMain.java

Instances of the `Image` class represent two-dimensional pixel images in Java that can be read from files and URLs, and then rendered to `Graphics2D` canvases with the method `drawImage`, as illustrated in the example class `ImageDemo`. No matter where they come from, you can further process and transform these images with various ImageFilter instances, as shown in our other example, `ImageOpDemo`.

We are accustomed to adding numbers, but we can also "add" images by concatenating them, similar to how strings are "added" by concatenating them. Since pixel raster images are two-dimensional, we can stack them not only horizontally but also vertically, provided the images' dimensions match in that dimension. In your lab project, create a new class `ImageAlgebra` (note that this class itself is not a Swing component, but a utility class that contains static methods), and in it, two static methods

```
public static Image hstack(Image... images)
public static Image vstack(Image... images)
```

for the horizontal and vertical stacking of an arbitrary number of `Image` objects. Both methods are vararg methods, meaning they accept any number of `Image` arguments, including zero, but in this lab they can assume they receive at least one image. The **horizontal stacking** method `hstack` (the method names were chosen to match NumPy) should create and return a new BufferedImage instance whose width equals the sum of the widths of its parameter images and whose height equals the maximum of the heights of its parameter images. This image should contain all the images together as a single row. (To implement this method most easily, draw the individual images one by one at appropriate positions in the resulting image; for images of different heights, top-align them to start from the same ceiling line.) The **vertical stacking** method `vstack` works the same way, except it swaps width and height.

We can immediately use both stacking methods in a **recursive subdivision**. Define a third method in your class

```
public static Image halving(Image tile, int d)
```

This method produces the result image according to the following recursive rule. For the base case where the depth `d` equals zero, this method should simply return the given `tile`. For positive depths `d, the result` is the horizontal stacking of the `tile` with the vertical stacking of two copies of `halving(half, d-1)`, where `half` is an image constructed from tile by scaling it to half its `width` and `height`. Since the two copies of `halving(half, d-1)` are identical, compute one and reuse the result, making this recursion **linear**. Each recursive call generates at most one

recursive call without exponential branching, and you can convert it into a loop with the same effect. (Recursion is the root of computation since it trades description for time. However, like a stepladder that helps you change a light bulb, you put it away once the bulb has been changed.)

The class `ImageAlgebraMain` uses the above three methods, along with the image file coffee.jpg provided in the GitHub repository for these labs, to create a display that should look like the screenshot below to be accepted for lab marks. To test your methods one at a time as you write them, first define all three methods as one-liner **stubs** that simply return the tile they were given. This allows you to run the `ImageAlgebraMain` program to test each method as soon as it compiles, even when you haven't yet implemented the other methods.

Lab 28: All The Pretty Hues

For testing: ShadesMain.java

Computers represent colours internally as **RGB triples** of red, green, and blue, where each component ranges from 0 to 1 as a floating-point number or from 0 to 255 as an integer byte. Turning all three knobs to zero gives you the darkest black your display device can present, whereas turning all three knobs to maximum gives you the brightest white your display device can present.

Inside Java, objects of `java.awt.Color` represent individual RGB colours as objects. However, when dealing with images where every pixel could have a different colour, we would rather not fill the heap with actual objects. It seems wasteful to spend twelve bytes of heap-management bookkeeping space for mere three bytes of actual payload data per pixel, over what are these days millions of pixels! Instead of acting all purist and insisting that "everything must be an object," you can cram four bytes of colour information into a primitive `int`, with those four bytes representing the ARGB components of that colour.

In addition to the three colour components, this fourth "alpha channel" that tags along the free ride inside a four-byte `int` has no fixed interpretation on top of its admittedly cool secret agent name. In practice, Java image-processing operations treat the alpha channel as the **colour's opacity**, with 255 for fully opaque and 0 for fully transparent. When a pixel is rendered with a partially opaque colour, the resulting colour is a partial mixture of the current pen colour and the previous colour underneath.

Most colours that our eyes can see can be broken down into these three components. Some might be a bit tricky; for example, how exactly would you represent the colour brown? Think about that one for a while. By now, you have surely seen lights of various rainbow colours, but have you ever seen a brown light? What the heck is that colour made of, and what can it do for us today? As convenient as RGB encoding is for computer software and hardware, it is not the best way for humans to represent colours. The alternative HSB representation breaks each colour into three components: **hue**, **saturation**, and **brightness**. In the HSB colour space, colour calculations, especially colour blending, produce more aesthetically pleasing results.

To extract the HSB colour components of the given RGB colour `rgb` given as an `int` into the first three elements of an existing 3-element float array `hsb`, you can copy-paste the method

```
private static void RGBtoHSB(int rgb, float[] hsb) {
    int a = (rgb >> 24) & 0xFF; // extract alpha
    int r = (rgb >> 16) & 0xFF; // extract r
    int g = (rgb >> 8) & 0xFF;  // extract g
    int b = rgb & 0xFF;         // extract b
    Color.RGBtoHSB(r, g, b, hsb);
}
```

The conversion from HSB to RGB is already available as the utility method `HSBtoRGB` in the `Color` class. Of course, the inverse method `RGBtoHSB` also exists in the `Color` class, but it expects the red, green, and blue values as three separate parameters, rather than packed into a single `int` as the method above does.

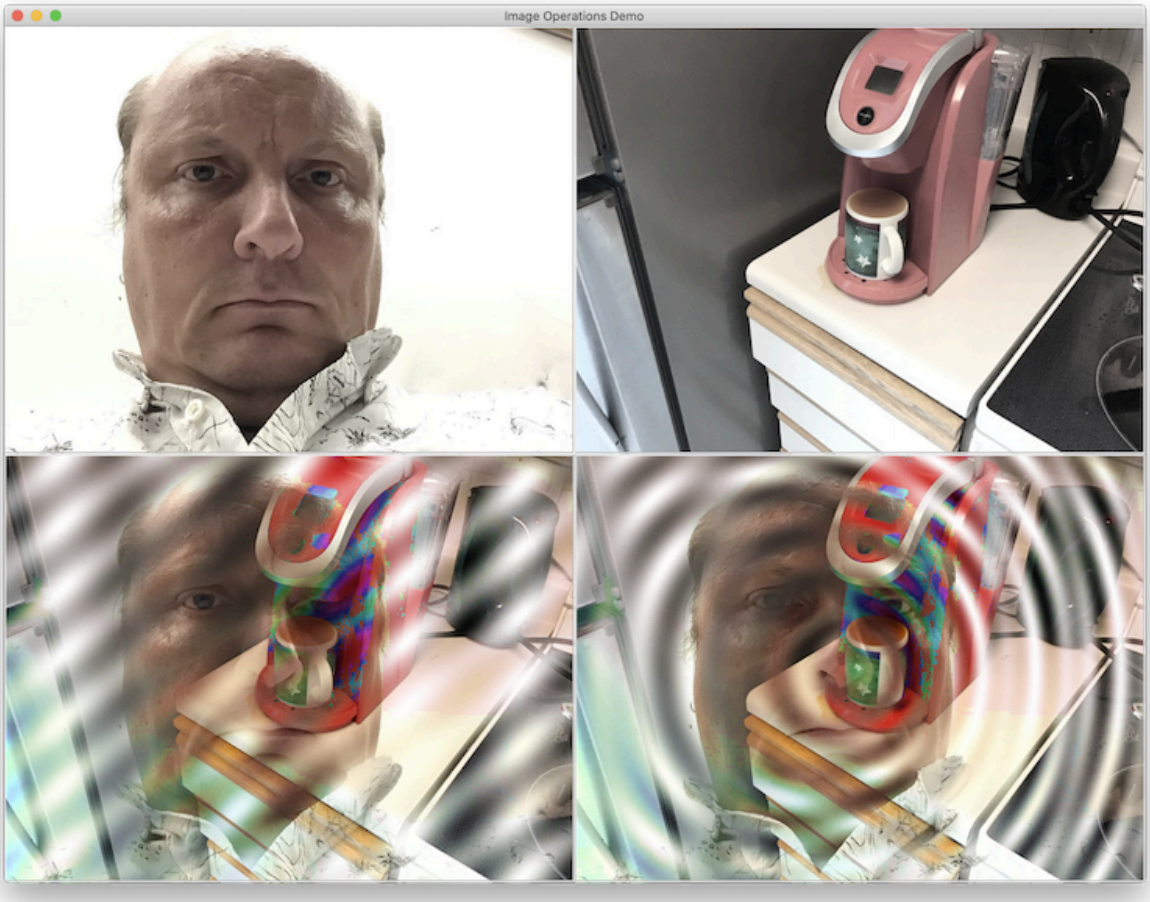
To play around with all these pretty colours, create a class `Shades` in your labs project. In this class, write the following method to create and return a modified copy of the given `Image`. Keep each pixel's hue and brightness the same, but decrease each pixel's saturation by multiplying it by the given factor, which is guaranteed to be between zero and one, inclusive.

```
public static Image desaturate(Image img, double factor)
```

Compared to the original, the resulting image looks like somebody ran it through the hot wash a couple of times too many. You should use the `RGBImageFilter` approach demonstrated in the `ImageOpDemo` example class to implement this operation. The second method to write in this lab is

```
public static BufferedImage blend(
  BufferedImage img1, BufferedImage img2, DoubleBinaryOperator weight
)
```

This method blends two source images `img1` and `img2` to create a result image so that the hue, saturation and brightness of each pixel (x, y) of the result are the weighted averages of those same components of the pixels (x, y) in the two source images. (Use simple linear averaging even for hue, which is technically incorrect but at least produces trippy colour artefacts at hue boundaries.) The weight used to mix these components is not constant, but determined for each pixel (x, y) separately by the parameter `weight` of type `DoubleBinaryOperator` to represent a function that takes the raw pixel integer coordinates (x, y) as parameters, and returns the floating-point **weight** from the range [0, 1] of the first image that is used to mix the components for that pixel. (A weight of zero therefore means that the result is all of `img2`.)



The following screenshot, produced by running `ShadesMain` with the example images coffee.jpg and ilkka.jpg included in the repository, shows the result that these methods are supposed to produce. The first row shows the desaturated versions of these source images. The second row shows two possible blends of these images. The bottom-left image computes the blending weights from a periodic wave function directed at an angle through the two-dimensional plane. In contrast, the bottom-right image uses a wave function emanating from the centre, shaped like a superellipse rather than a boring old regular circle. A superellipse with parameter 2.05 or thereabouts does the job pretty much everywhere we would normally use a circle, and yet it is subconsciously perceived as more organic and pleasant. The main problem with perfect circles is that they're too darn perfect to be perceived as genuine natural things. Few perfectly round things seem to exist in nature, at least in the resolution that our eyes can discern; immediate examples that spring to mind might be the Sun and the Moon in the heavens, and the iris and the pupil in our eyes. (It is probably not coincidental that humanity has historically ascribed preternatural properties to these objects.)

Lab 29: In a Timely Manner

JUnit: TimeProblemsTest.java

Computations on times and dates are tremendously important in the real world, yet surprisingly difficult to get universally correct, since these concepts are more complicated than they might seem in our everyday experience. However, just as our programs can no longer assume that all text is written in English using a typewriter so every character can be stored in one byte, software running around the globe 24/7/365 cannot blithely rely on such happy-go-lucky ideas about how time works. For starters, everyone who has come this far surely knows that in 2020, when this problem was originally written, those were actually 24/7/366. What other pitfalls with sharpened spikes at the bottom might be lurking in this important problem domain to ensnare those patrolling this enormous jungle, nervously scanning for traps laid by Time Cong as millions of unblinking eyes stare at them from behind every leaf of grass?

To see the need for a package that cuts through this thicket and performs these calculations correctly in all possible cases, scan through "Falsehoods programmers believe about time" and tally up how many of the falsehoods listed there you would have also assumed to be true before doing this lab. *Tempus bugit!* The rest of the curated list of "Falsehoods Programmers Believe In" gives you a glimpse of how both the world and human societies are chock full of messy edge cases and corner cases waiting to trip the unwary programmers. Normies have the decadent luxury of closing their eyes and hand-waving at such problems until they go away, as all problems eventually will. Still, hope is never a strategy for computer programmers.

Before Java 8, the standard library classes `Date` and `Calendar` encapsulated such computations. However, these classes were defective in too many ways to count. They would be better fed on punch cards into the maws of the sixties vault-sized Bat-Computer to predict the Calendar Man's next postmodern crime spree. The class `Date` is now officially **deprecated**, and the class `Calendar` remains only as a **legacy** for backwards compatibility with old code, as it of course should. For new code, the far superior package `java.time` introduced in Java 8 offers a more intuitive and consistent design that is well worth studying for its clean, SOLID object-oriented design from all perspectives, once you learn how to use its classes and methods. The class design of this package is so sparkling clean that doctors could use it as a sanitiser.

Classes `LocalDate`, `LocalTime` and `LocalDateTime` represent dates and times without any time zone information. (Pythonistas may note how these roughly correspond to classes `date`, `time` and `datetime` in the `datetime` module.) Unlike the original `Date` class, all such data types in `java.time` are **immutable**, which gives us many blessings that simplify reasoning about programs that use them. Their methods are consistently named across these types and accept intuitive, human-readable arguments. For example, every method named `with` creates a copy of an existing object with just one attribute changed and the rest kept as they were. (Such a method would sort of be a "mutator" for an immutable type.)

The first observation that jumps out from these classes is that **none of them has a public constructor!** Instead, you create new instances using a **static factory method**, usually overloaded to accept different argument types. For example, the expression `LocalDate.of(2020, 4, 23)` would return a `LocalDate` object that represents the date of April 23, 2020, the day that this lab specification was originally written.

Create a new class `TimeProblems` in your labs project. The first of the three methods that you will be writing there has you performing some calculations on dates. The whole point of this entire lab is to teach you to read through the package and class documentation in the Java API Reference, usually googling through some Stack Overflow pages in another browser tab to get through the practical details and find the method that does the thing that you need at the moment. First, think about how you would solve this problem in real life, then translate your ideas into code, with each step mapped to the appropriate method calls.

```
public static int countFridayThirteens(LocalDate start, LocalDate end)
```

This method should count and return the number of days between the `start` and `end` dates, both dates **inclusive**, that are Fridays falling on the thirteenth day of their month. This combination used to be considered unlucky in Western culture, at least back when people weren't quite as cognizant of the Pigeonhole Principle as we modern sophistos are, and thus easier to dazzle with numerological coincidences. Such beliefs seem to be slowly fading into distant memories in our culture. However, in relatively recent times, the notion of some numerical dates being inherently unlucky spawned an entire series of popular docudramas to warn the movie-going youth of the dual dangers of getting intoxicated and having premarital sex.

The next problem was inspired by the "Gigasecond challenge" on CodeWars, another good online whetstone to sharpen your coding and algorithmic thinking skills. To solve this problem, we need a new concept of `Duration` to represent a measured **passage of time** (for example, "two hours and five minutes", "three years and ten seconds", or "one nanosecond") that is not anchored to any particular time or place. (The class `Period` is similar in both spirit and usage, but with a less precise granularity of mere days instead of nanoseconds.)

To ensure understanding of the difference between the concepts represented by `Duration` and `LocalDateTime`, note how adding or subtracting a `Duration` to a `LocalDateTime` or another `Duration` is perfectly sensible, whereas trying to add two `LocalDateTime` objects is codswallop. Even more contrary to intuition, even though dates cannot be **added** to each other (at least not in any way that would have the result meaningfully relate to any aspect of reality relevant to our future decisions), the **difference** between two `LocalDateTime` instances is a legitimate thing of type `Duration`; this type represents lengths of time not fixed to any particular point in time.

```
public static String dayAfterSeconds(
  LocalDateTime timeHere, long seconds
)
```

Given the current date and time as a `LocalDateTime` object, return the day of the week (as one of the capital letter strings `"SUNDAY"`, `"MONDAY"`, `"TUESDAY"`, `"WEDNESDAY"`, `"THURSDAY"`, `"FRIDAY"`, `"SATURDAY"`) that results from adding several `seconds` to the current date and time.

The previous two problems treated time as a local phenomenon, as we do in our everyday lives. However, this pale blue marble, whose spinning surface we live on against the dark eternity of the cosmos, is divided into time zones. Hence, the same instant of time corresponds to a different time and date depending on your location on Earth. We ignore the theory of relativity and any philosophical issues about what it means for two instants of time to be the finger-quotes "same" time instant, with or without any theory of relativity or any "quantum" mumbo-jumbo sprinkled in. For political reasons, time zones aren't as straightforward as most people assume; for starters, there are 38 time zones, not just the obvious 24, and their clocks aren't always an integer multiple of hours apart. (Time zones also aren't easy to get rid of, or to make continuous.)

Instances of `ZoneId` represent recognised time zones and are identified by standardised strings such as `"America/Toronto"` or `"Pacific/Marquesas"`. However, before you optimistically extrapolate the general form of these strings from this sample of mere n=2, you should note that `"Eire"`, `"Navajo"`, `"US/Hawaii"` and `"America/North_Dakota/New_Salem"` are equally legitimate time zones! You should also note that underscores act as artificial whitespace, as they do in C and Python with names made of multiple words, so you don't get confused when there is no time zone for `"America/Los Angeles"`. Fortunately, `ZoneId` and other classes in the package already do all this preparatory work for you, sparing you the need to reinvent the wheel.

```
public static int whatHourIsItThere(
  LocalDateTime timeHere, String here, String there
)
```

Assuming that you are currently `here` so that the clock on your wall says `timeHere`, compute and return the current hour that the people over `there` see on their grandfather clocks.

One last thing for you to ponder: why is the first parameter of this method a `LocalDateTime` instead of a mere `LocalTime`? What situation would knowing the current local date, not merely the current local time, be necessary for a correct answer? The great thing about this library is that you don't need to know! Instead of doing tedious research to uncover lurking edge cases, let the `ZonedDateTime` class handle these low-level details, since an expert in this field has already done the grunt work for you. The same principle applies to all real-world problem domains we solve computationally. Note also the `withZoneSameInstant` method as a "mutator" for immutable objects to access that same instant in a different time zone.

Lab 30: Mark And Rewind

JUnit: RewindIteratorTest.java

Instances of `Iterator<E>` allow their elements to be iterated only in the forward direction with the method `next`, but do not allow **rewinding** that sequence to an earlier position to repeat the already seen elements of the sequence for a second time. This is because some sequences are generated **virtually**, so the iterator computes the next element on command from a state that depends on the sequence generated so far. Such computations are generally not **reversible.** Since the ability to rewind a sequence to an earlier position would be useful in some algorithms that operate on sequences, we shall solve this problem once and for all with a general-purpose **decorator** for `Iterator<E>` instances that allows a position in any existing sequence to be **marked**, so that the decorated sequence can later be rewound to that position.

The mass test method of the JUnit test class `RewindIteratorTest` uses the simple sequence of natural numbers 0, 1, 2, … as the underlying sequence, and then performs "mark" and "rewind" operations at pseudo-randomly chosen positions of the sequence. The three public test methods generate the first thousand, first million and first hundred million elements of this sequence. If you edit the test method for the first thousand elements to be `verbose`, the first eight lines of output (shown below as generated with the instructor's private model solution) help you trace what is supposed to happen, with `M` and `R` denoting marking and rewinding, and numbers denoting the element that pops out of the decorated sequence at that moment.

```
0 1 2 M R 3 4 5 6 M 7 R 7 8 9 10 11 12 13 14 15 16 M 17 R 17
18 19 20 M 21 22 23 24 25 26 27 M 28 M R 29 R 28 29 30 31 R 21 22 23 24
25 26 27 M 28 M 29 30 31 R 29 R 28 29 30 31 32 33 M R 34 35 36 37 38 39
40 41 42 43 44 45 46 47 48 49 50 51 M 52 53 54 55 56 57 58 R 52
53 54 55 56 57 58 59 60 61 62 63 64 M 65 66 M 67 68 69 M 70 M R 71 72
R 70 M R 71 72 73 74 75 76 R 67 68 R 65 66 67 68 69 70 71 72 73 74 75
76 77 78 79 M 80 81 82 83 84 85 86 87 R 80 81 82 83 M 84 85 86 87
R 84 85 86 87 88 89 90 M 91 92 93 94 95 96 97 98 99 100 101 M 102 103
```

The reader might want to trace the effect of the mark and rewind operations by hand for the first two or three lines. Since the original underlying sequence produces each natural number exactly once and in order, the repeated appearances of each number come from the rewinding operations.

Create a new class with the following signature in your labs project:

```
public class RewindIterator<E> implements Iterator<E>
```

Same as all decorators everywhere, this class should have a private field

```
private Iterator<E> client;
```

that refers to the underlying `client` iterator object being decorated, initialised in the constructor

```
public RewindIterator(Iterator<E> client) { this.client = client; }
```

Since the underlying iterator does not remember the elements it has produced, your decorator has to store these elements in some kind of list from which they can be extracted for their encore appearances. The simplest way to do this is to use two such lists, perhaps named

```
private LinkedList<E> buffer = new LinkedList<E>();
private LinkedList<E> emitted = new LinkedList<E>();
```

so that `emitted` contains the elements that this decorator has given out with its `next` method, and `buffer` contains the previously produced elements that have been rewound and are waiting to be given out again. Required by `Iterator<E>`, this class must have the methods

```
@Override public boolean hasNext()
@Override public E next()
```

The decorated iterator has a next element if either its `buffer` is nonempty or if its underlying `client` iterator has a next element. The `next` element produced by this decorator is then the first element of the `buffer`, and in the case that the `buffer` is empty, the next element of the `client` iterator. If any marks are currently active at the moment (that is, at least one position has been marked but not yet rewound), the produced element is pushed into the `emitted` list.

```
public void mark()
public void rewind() throws IllegalStateException
```

These two new methods mark the current position in the sequence and later rewind the sequence to that position to repeat its elements. It is your task in this lab to implement these methods so that they behave correctly in all possible situations. Note that multiple positions can be marked at the same time, and the same position can be marked multiple times over repeated nested iterations. The `rewind` method always rewinds the sequence to the most recent marked position, or, if no marks are active, reports this unexpected situation to the caller by throwing an `IllegalStateException`. It might be a good idea to use a third instance field

```
private LinkedList<Integer> markPositions = new LinkedList<Integer>();
```

that contains all the positions in the `emitted` list that have been marked at the current time. (If this list is empty, the emitted elements do not need to be stored in the `emitted` list to waste memory, since such elements will never be needed again.)

Lab 31: Region Quadtrees

JUnit: QuadTreeTest.java

A two-dimensional image of binary pixels (that is, every pixel must be either pure black or pure white) can be compactly stored in a two-dimensional array of `boolean` values. This representation allows rapid access to and modification of each pixel, independent of the others. Since each pixel needs exactly one bit of memory (plus some overhead for the two-dimensional array structure), available heap memory sets a hard upper limit on the size of images that can be stored this way.

A simple combinatorial argument based on the pigeonhole principle, usually paraphrased in its bumper-sticker form "There simply aren't enough short bit patterns to uniquely encode not only themselves, but also a whole lot of longer bit patterns", establishes that no general method can compress data made up of arbitrary bits in a way that could guarantee always to produce a smaller result. For compression to be possible, the data must be distinguishable from random noise, so some exploitable statistical dependencies and correlations exist between the individual bit values. Compressing that data then erases those dependencies, since otherwise you could run the compressed data through another compression algorithm to squeeze it even tighter. Lather, rinse, and repeat until the data is compressed to next to nothing. You can observe this phenomenon in practice by trying to use your favourite compression tool to compress data that has already been compressed. Once you've squeezed out all that metaphorical air, the data's entropy ultimately determines how many bits you need to store it so you can still recover the original version losslessly.

Two-dimensional binary black-and-white images with large uniform areas of either colour can sometimes be compressed into much smaller space with the **region quadtree** approach. Each quadtree object represents some square area of pixels whose side is some power of two. If this square is all white or all black, the object is a **leaf node** of this quadtree structure. Squares that contain both black and white pixels are subdivided into four smaller squares and recursively converted into region quadtree objects. These subtrees become children of the quadtree object that represents the original square. The recursive subdivision will necessarily end at a single pixel, but hopefully much sooner in areas of uniform colour inside the image. (Any Nassim Nicholas Taleb fanboys reading this can note how this relates to how "The Most Intolerant Wins".)

To do this lab, you should first copy the interface `QuadTree` and its two concrete implementations, `WhiteQuad` and `BlackQuad`, into your labs project folder. The interface `QuadTree` defines the functionality of a region quadtree as two public methods

```
public boolean isOneColour();
public long computeArea(int scale);
```

The method `isOneColour` should return `true` if the region depicted by this `QuadTree` object represents an area of uniform colour, either all black or all white, and `false` otherwise. The method

`computeArea(int scale)` should return the total area of the black pixels in the region, assuming the entire region is a square with side length 2^{scale}. For example, the area of a quadtree that is all black at scale 3 would equal 8 × 8 = 64. For coding this method, note that Java does not have an integer exponentiation operator baked into the language the way Python has its handy ** operator. However, this particular situation where the base of the exponentiation equals two can be easily computed with the **bit shift** operator << so that the expression `1L<<scale` gives the quantity 2^{scale}. Note that using the `long` literal `1L` lets us use up to 63 bits of scale instead of 31 bits from the bare int literal 1. (Not all 64 and 32 bits, since the highest bits store the sign in **signed integer** types.)

Before writing the class `QuadNode` of this lab, check out how the previous two methods of the `QuadTree` interface have been implemented in `WhiteQuad` and `BlackQuad`. These classes have `private` constructors so that no new objects can be created from the outside. Instead, the outside world must use the `get` method to access the **singleton** instances of these classes, so we can now demonstrate the (in)famous **singleton design pattern** for the first time in this course. (Some people consider singleton to be an **anti-pattern** in object-oriented design. The debate continues.)

```
public final class QuadNode implements QuadTree
```

Instances of this class represent region quadtrees that contain both white and black pixels, so they could not be represented as instances of either `WhiteQuad` or `BlackQuad`. In the spirit of this course, the `QuadNode` class is designed to be **immutable**. The class `QuadNode` should have three instance fields

```
private QuadTree[] children;
private long area = 0;
private int lastScale = Integer.MAX_VALUE;
```

The four-element array `children` contains references to the four child quadtrees that represent the quarter squares that this square is subdivided into. Since these children can be arbitrary region quadtrees, the type of this field is `QuadTree[]` instead of `QuadNode[]`.

The field `area` will cache the computed area of this quadtree, and `lastScale` will remember the scale at which that area was computed. Both fields are initialised with impossible placeholder values so we can recognise them as uninitialised. As in the earlier example implementations `WhiteQuad` and `BlackQuad`, the class `QuadTree` should also have a `private` constructor to disallow object creation from the outside. To ensure every instance remains immutable after creation, we make a defensive copy of the argument array for private storage, rather than storing a reference to the array the outside world still uses and could later modify. However, since the `QuadTree` objects stored in this array are immutable, they can be freely and safely shared among any number of users.

```
private QuadNode(QuadTree[] children) {
    this.children = java.util.Arrays.copyOf(children);
```

```
}
```

To allow the outside world to create new quadtree structures to their heart's content, we provide a **static factory method** instead:

```
public static QuadTree of(QuadTree... children)
```

Unlike a constructor, a static factory method is not under any obligation to return specifically a `QuadNode`. Still, it can return an instance of any `QuadTree` subtype, depending on the situation. Since this constructor is intended to accept an array of arguments, we might as well make the children parameter a **vararg** so the caller can pass the array elements as separate arguments, rather than forcing the caller to create a small array object every time. This factory method should recognise special situations of these four children being either all uniform white or all uniform black, and return the correct singleton `WhiteQuad.get()` or `BlackQuad.get()` appropriate for the situation. Otherwise, this method returns a new `QuadNode` object with the given `children`.

Knowing that the child nodes cannot all be of the same uniform colour, the method `isOneColour` becomes trivial to implement in the `QuadNode` class, to always return `false`:

```
public boolean isOneColour() { return false; }
```

Otherwise, this method would have to recursively test whether all its children are uniformly the same colour, potentially probing through the entire tree to get the answer. The same would also happen if the child trees were mutable. We get to enjoy the asymptotic running times of our methods decreasing from O(n) all the way down to the constant time O(1), so very rarely that every occasion is worth celebrating for at least a moment. Immutable types, huzzah!

```
public long computeArea(int scale)
```

Assuming that `this` quadtree represents a square whose side length is `1<<scale`, recursively compute and add up the areas of the child quadtrees, using a `scale` one less than the `scale` in the current level of recursion for each of these recursive calls. Before returning the total to the caller, store the area and the `scale` used in its computation in the instance fields `area` and `lastScale` so that the next time this `computeArea` method is called, you can just look up this answer instead of redundantly recomputing it all the way down with recursion.

What if the previously cached `area` was computed using a value for `lastScale` that is different from the `scale` in the current call? Assuming `lastScale < scale`, you can adjust the previously computed area for the new `scale`. Even in the opposite case where `scale < lastScale`, there may be some moves you might be able to pull off before accepting the harsh reality of having to recompute that area from scratch.

Lab 32: Triplefree Sequences

JUnit: TripleFreeTest.java

A sorted list of positive integers is **triplefree** if it does not contain any **arithmetic progression** of length three or longer; that is, it does not contain three elements $a < b < c$ such that both differences $c - b$ and $b - a$ are equal. Another way to express this is to require that for any two elements $a < b$ in the list, the quantity $a + 2(b - a)$ is not a member of that same list. For example, lists `[2, 3, 5, 6]`, `[1, 2, 7, 8, 10, 11]` and `[1, 2, 6, 9, 13, 18, 19, 21, 22]` are triplefree. Try as you might, you won't find three elements that together form a three-step arithmetic progression inside any of them! For example, since the last list contains both 18 and 21, it cannot contain either 15 or 24 to complete such a triple, and because it has 6 and 19, it cannot also contain 32 anywhere.

Create a class `TripleFree` in your labs project, and in that class, write a public method

```
public static List<Integer> tripleFree(int n)
```

that finds and returns the longest possible triple-free list whose largest element is `n`. Since several lists of equal length may satisfy the triple-free constraint, this method must return the lexicographically largest such list when read from highest to lowest elements. For example, the lists `[1, 3, 4, 6]` and `[2, 3, 5, 6]` are both triple-free and contain four elements, but the method must still return the second list for `n=6`.

`n`	Expected result
`2`	`[1, 2]`
`4`	`[1, 3, 4]`
`9`	`[1, 2, 6, 8, 9]`
`15`	`[2, 3, 5, 6, 11, 12, 14, 15]`
`42`	`[2, 3, 5, 6, 11, 12, 14, 15, 29, 30, 32, 33, 38, 39, 41, 42]`

This problem is best solved bottom-up, building the answer list with recursion that solves the problem for `n` by making two recursive calls for subproblems of `n-1`. Write a helper method

```
private static List<Integer> tripleFree(int n, Set<Integer> chosen)
```

that finds and returns the answer for integer `n` when the `chosen` numbers have already been taken into the sequence in the previous levels of recursion. Your public method can then make the top-level call to this private recursive method, with the second parameter initialised to a `Set<Integer>` object that the recursion fills as it moves down the search tree.

In each recursive subproblem, you have the usual two-way choice of either taking the integer `n` in your `chosen` set, or leaving that integer behind. However, you may only take `n` into your `chosen` set if it does not create a triple with some two numbers that you have already `chosen`. Whichever of these two branches gives you a better solution, return that list as the answer. If both ways are equally good, use the way that takes `n` in `chosen`, to ensure that your method returns the lexicographically largest solution.

This basic recursion solves the problem in reasonable time for `n` up to twenty or thereabouts. Still, for higher values of `n`, you need to mercilessly prune branches that cannot possibly contain the best solution, rather than recursing through them and producing a redundant solution that will eventually prove suboptimal. The JUnit test class `TripleFreeTest` will loop through the possible values of `n` in ascending order, so your class can remember the length of the best list for each `n`.

Since adding new constraints to any optimisation problem can never improve the optimal solution, the stored optimum for unconstrained `n` is an upper bound on the solution for any set of `chosen` elements. For example, suppose you have already found a 15-element solution to your original top-level value of `n`. If `chosen.size()==5` in some branch down in the recursion, and the previously discovered optimal unconstrained solution for the current level `n` is 10 or less, you can automatically dismiss this branch as a failure. Any solution available in this branch contains at most 15 elements, and can therefore be no better than the lexicographically higher 15-element solution that you have already found.

Other clever optimisations can prune even more unproductive branches, making this problem a good exercise in keeping your exponential recursions from spreading like a hydra, where each head you chop off spawns two new ones. If you create some good optimisations whose value you really want to try out, go ahead and uncomment the `testFirstHundred` method in the JUnit test class, and steel yourself to run it. Can you make your code complete that test within one hour?

A related but more famous combinatorial problem, constructing the shortest possible Golomb ruler, tightens these constraints so that each possible difference between two element values occurs exactly once in the entire array. Constructing such rulers for large *n* is not trivial. Golomb rulers can be further generalised to two dimensions as Costas arrays, arrangements of *n* non-attacking rooks on an *n*-by-*n* chessboard such that no two rooks lie on the same column or row, and all possible pairwise direction vectors between two rooks are distinct. Efficient algorithms to generate individual Costas arrays on command for any given *n* have been devised. However, no systematic enumeration method has been discovered that can iterate through all possible Costas arrays for a given n faster than trivial backtracking, let alone a closed-form formula for the number of different Costas arrays for a given *n*.

Lab 33: Sardine Arrays

JUnit: SardineTest.java

The primitive integer types `byte`, `short`, `int`, and `long` in Java use 1, 2, 4, and 8 bytes of memory, respectively. All four primitive types in Java are **signed** so that they can represent both positive and negative values. Same as with other types in Java, we can create homogeneous arrays to store a sequence of elements of the same type. When an array's element type is a primitive, Java stores the elements in consecutive bytes with no additional bookkeeping.

However, in practice, many programs operate on problem domains where the relevant integer values are known to be **unsigned** natural numbers. When Java stores such values in primitive integer types, the highest-order bit that represents the integer's sign in two's complement encoding is always zero and therefore wasted. Furthermore, the problem domain often inherently limits these unsigned integer values so each value is known to fit into k bits. Whenever k is something convenient like 14 or 30, we have no problem, since we can use an appropriate primitive type like `short` or `int` that these values almost fill, leaving so little wasted space that it's not worth worrying about.

However, for the sake of argument, suppose k is some more annoying quantity such as 17. Now we can't use a `short` to store such values, since a `short` can hold only 16 bits of information. We are forced to use the next higher type, `int`, with 32 bits of information, of which 15 bits, almost half of them, end up being wasted. Wasting almost half of your available memory bits isn't as big a deal these days as it used to be, when each program's memory was measured in kilobytes. However, using these bits more productively lets our programs solve bigger instances of the same problem within the same memory constraints. In this problem, we will perform the **time-space tradeoff** in the opposite direction from the usual, making our array operations slightly slower to ensure that not one bit of space is wasted.

In this lab, we shall design and implement a general-purpose utility class whose instances behave as simulated `<DrEvil>`"arrays"`</DrEvil>` of unsigned k-bit integers. An "array" of n such numbers is stored in a `BitSet` instance that contains exactly nk bits of information, with less air between those bits than in a shipping crate of Ikea furniture packed flat, or the titular sardines packed into a can. The computations in this lab are in fact so low-level that we don't even need to import any classes from the standard library! In addition to the **bitwise arithmetic** operators to encode and decode the given integer into individual bits, the other new operation you need in this lab is the **bitwise left shift** operator `1<<k` to quickly compute 2^k using only integers, without any floating-point arithmetic.

Create a new class `Sardines` that contains two instance fields, `data` and `k`, starting the signature of the class as

```
public class Sardines {
```

```
    private BitSet data;
    private int k;
```

This class should have a public constructor that initialises these fields.

```
public Sardines(int n, int k)
```

Since not all possible values of `n` and `k` can be meaningful even if they happened to be of the correct type, this lab now makes you practice throwing exceptions to report an inevitable violation of method preconditions. This constructor should throw an `IllegalArgumentException` if the array size `n` is negative, or when the total number of bits `n*k` inside the array is greater than the theoretical maximum given by the named constant `Integer.MAX_VALUE`, or if the number of bits per element `k` is either negative or greater than 31. The method `testExceptions` in the JUnit test class `SardinesTest` will make sure that your code enforces these restrictions gracefully and throws an exception of the correct type each time.

You need to be careful with how you actually check for the second violation in your constructor. The seemingly obvious test `n * k > Integer.MAX_VALUE` will always be `false`, regardless of actual values of `n` and `k`. No result of integer arithmetic truncated to fit inside an `int` can be larger than `Integer.MAX_VALUE`. Try again. Be better.

To read and write the value of an element in the simulated sardine array, this class should provide the corresponding accessor and mutator methods

```
public void set(int idx, int v)
public int get(int idx)
```

to `set` and `get` the value of the element at position `idx`. The *k* bits that constitute the binary value in position `idx` of this instance of `Sardines` are stored into the *k* consecutive positions of the `BitSet` starting from position `k*idx` in the underlying array `data` of `boolean` values. For example, when `k==5`, the first element is stored in bits 0 to 4, the second element is stored in bits 5 to 9. These methods must also throw an `ArrayIndexOutOfBoundsException` if `idx` is outside the range of legal indices ranging from zero to `n-1`, and throw an `IllegalArgumentException` if the element value `v` is not representable in *k* bits. Again, the test method `testExceptions` in the JUnit test class `SardinesTest` expects you to throw these exceptions correctly to pass that test.

One last pitfall in this class lurks in the corner case `k==31`, and `1<<k` sets the **sign bit** in the highest position, making the number equal to `Integer.MIN_VALUE`, the smallest negative integer value that fits inside an `int`. You therefore need to be a "bit" more careful when checking whether the argument `v` of the `set` method is fitting, or whether you should reject it and throw an exception. Otherwise, the JUnit test method `massTestMillionAndThirtyOne` will fail and reveal the final mishandling of this particular situation, the proverbial snatching of defeat from the jaws of victory.

Lab 34: Leaders of Men

JUnit: RunoffVotingTest.java

For elections to determine exactly one winner from a group of *C* competing candidates vying for the Iron Throne, the Anglosphere nations tend to use the "first-past-the-post" system where each voter casts a ballot for exactly one candidate. The candidate with the **plurality** of votes wins the seat, without necessarily achieving a majority. This system is simple and predictable, but critics have attacked it from all angles for its shortcomings and paradoxes. For example, "spoiler candidates" can exist whose very presence induces **tactical voting**; some voters who know their favourite candidate is unpopular in general will vote against their true preference to help boost whoever they see as the lesser of the remaining evils. (Don't blame me; I voted for Kodos.)

This lab examines two voting systems where each voter, instead of choosing just one candidate to vote for, ranks all *C* candidates in linear **preference ordering**. For this problem, each `ballot` is a one-dimensional **permutation** array where each number 0, …, *C* – 1 occurs exactly once. For example, assume *C*=4 with the candidates numbered 0, 1, 2 and 3. A ballot filled in with the values `{3,1,0,2}` would then indicate that the voter wants to see candidate 3 win, but if that candidate cannot win, the voter would rather see candidate 1 win, and so on.

Both methods in this lab receive the stack of ballots as a two-dimensional array of integers, where each row is an individual ballot. Now that each ballot contains more information than merely identifying that voter's most favoured candidate, voting systems can be generalised to use all this information to determine the fair winner… with or without using **Benford's law** to pry out any irregularities to shine a light on some sneaky bastard stuffing the ballot box in the dead of night.

Create a class `RunoffVoting` in your labs project. This class will have two `static` methods that implement two different voting systems operating on these `ballots`, both returning the election winner. To ensure deterministic results for our JUnit fuzz tests, replace coin flips with the rule that all ties **during the calculation must be resolved on the spot in favour of the higher-numbered candidate**.

```
public static int condorcetMethod(int[][] ballots)
```

Determine the election winner using the pairwise Condorcet method. In our variation, to guarantee a winner without additional tie-breaker mechanisms, each candidate faces every other candidate in a pairwise match. In each pairwise match between two candidates, Winston Noble and Hubert Hoag tallies ballots that rank Winston above Hubert versus ballots that rank Hubert above Winston. During the pairwise match between Winston and Hubert, all the other candidates and their relative rankings to Winston and Hubert are temporarily ignored. The winner of each pairwise match earns one "match point", and the candidate with the most match points after all pairwise matches are complete wins the election. Barring upsets involving a dead girl or a live boy, whether Winston

predictably beats Hubert in their pairwise match by a single vote or by millions of votes makes no difference; Winston gets exactly one match point for this feat, no matter his margin of victory.

An election with C candidates should therefore end up with $C(C - 1)/2$ match points, regardless of the number of ballots cast. You should debug your method by `asserting` this fact after the fact. (Remember to turn on the assertions in the Java compiler with the command line option `-ea`.) To pass the JUnit test in a reasonable time, make sure that you are not being a "Shlemiel" who combs through each ballot and each candidate more times than you actually need. When going through a ballot, update a whole bunch of match score counters at the same time, not just one.

```
public static int instantRunoff(int[][] ballots)
```

Resolve the election winner with instant runoff, a more complex method for choosing the winner. Initially, each candidate receives all ballots that place him as the first choice. Then, as long as no candidate has more than half of the ballots (exactly half is not enough to start the ticker tape parade), eliminate the candidate with the fewest ballots. Again, if two or more candidates have the same number of ballots, eliminate the lowest-numbered candidate. Distribute the eliminated candidate's ballots among the remaining candidates, so each ballot goes to the next candidate down the preference list who has not yet been eliminated.

Once a candidate is eliminated, he is out for good, and no future ballot redistribution can bring him back into the process. This spells doom for any "apple pie" candidate, where the term isn't a reference to nostalgic Americana out of some Norman Rockwell painting. Still, to be everybody's second choice but nobody's favourite. In other words, a neutral candidate whom nobody hates but nobody feels any jingoistic enthusiasm for. That candidate will therefore be eliminated in the first round, even though, in the grand scheme of things, that candidate could have been the optimal global choice in a hypothetical situation of extremely fragmented partisan jingoism where every voter loves their own Dear Leader and fervently hates every candidate who is not Mr Apple Pie…

The critical step in this method is redistributing ballots from the candidate eliminated in the current round. To ensure that you can do this in a manner that does not resemble the way that "Shlemiel" paints the fence running back and forth along the same parts of the fence that he has already painted many times before, you should maintain a list of lists of votes held by each candidate (some kind of `List<List<Integer>>` would probably be good for this purpose) so that you can quickly loop through the ballots of the candidate being eliminated without having to loop through all ballots of the entire election to find out which candidate each ballot currently belongs to. Each ballot can be identified by an integer that gives its row position in the `ballots` array.

Again, be careful to ensure that your method resolves all ties in favour of the higher-numbered candidate. Also, treat any candidate who is not ranked as the first choice in any of the ballots as having already been eliminated in this computation even before the first elimination round, so that that candidate doesn't come alive in any later elimination round.

Lab 35: Multiple Winner Election

JUnit: MultipleWinnerElectionTest.java

Anglosphere countries tend to use voting systems in which each district elects exactly one representative, typically using the blunt first-past-the-post method to determine each district's sole winner. Many other countries use some type of party-list proportional voting system where each electoral district elects multiple representatives. Each voter votes for a particular party, either directly (closed list) or indirectly (open list). The seats up for grabs in each district are allocated to the parties in proportion to their votes in that district.

In this lab, you implement three methods to compute the distribution of `seats` to the parties according to the votes that these parties received. The **D'Hondt**, **Webster**, and **Imperiali** methods, each a special case of the highest averages method, are otherwise identical but use different formulas to compute the **priority quantity** for each party, as described below. You might therefore want to implement this entire voting algorithm as one `private` method that the following three `public` methods delegate to.

```
public static int[] DHondt(int[] votes, int seats)
public static int[] webster(int[] votes, int seats)
public static int[] imperiali(int[] votes, int seats)
```

The parameter array `votes` gives the number of votes each party received. Seats are awarded one at a time, as in the Huntington–Hill method in Lab 0(G), so each seat goes to the highest-priority party. To make the results unambiguous for the JUnit fuzz tests, resolve all ties in favour of the party with the higher ID number, as in the previous lab 34, "Leaders Of Men". Priority quantity of the party that received v votes and has so far been given s seats is $v/(s + 1)$ in the D'Hondt method, $v/(2s + 1)$ in the Webster method, and $v/(1 + s/2)$ in the Imperiali method. You should again perform all these calculations with exact integer arithmetic using our `Fraction` data type, of course!

The returned `result` should be an `int[]` of the same length as the parameter array `votes`, each element indicating how many seats were given to that party. For example, in an election between four parties where `seats=21` and `votes={23,26,115,128}`, the D'Hondt method returns `{1,2,8,10}`, the Webster method returns `{2,2,8,9}`, and the Imperiali method returns `{1,1,9,10}`. These results show how adjusting the divisor sequence in the priority quantity calculation can cause the process to favour larger or smaller parties, with the Webster method most favourable and the Imperiali method least favourable for small parties.

(As a final humorous aside on the general theme, what should we think of a method where the priority quantity is computed with the formula $v/2^{s+1}$, dealing the seats out logarithmically so that doubling your number of votes gives you one additional seat? Let those wonks at FiveThirtyEight wrap their pointy eggheads around that conundrum!)

Lab 36: Rational Roots

JUnit: RationalRootsTest.java

Everyone who has come this far has surely heard of the quadratic formula to solve quadratic polynomial equations. Since even that brings up nasty square roots with irrational and even complex results, we'll leave that for higher-level languages such as Wolfram Mathematica that can properly handle algebraic computations on symbolic expressions. Instead, this lab makes you implement a handy method based on the rational root theorem for polynomials that is honest-to-God guaranteed to find *all* rational roots of rational polynomials of *arbitrary* degrees, not merely for some quadratics. (Put *that* in your pipe and smoke it, Abel!) The method itself is clearly explained on the page "Finding zeroes of polynomials" of the course "Algebra" in "Paul's Online Notes", the best online undergraduate mathematics resource that this author is aware of.

As in the three previous labs where you implemented the Polynomial class, we represent a polynomial as an array of its coefficients, so the *i*-th element of that array contains the coefficient of its i-th-order term. For example, the polynomial $5x^6 - 17x^3 + 8x^2 - 42$ would be represented as the integer array `{-42,0,8,-17,0,0,5}`, where the coefficients are read from the lowest-order term to the highest. Terms that are missing from the polynomial (in this example, the exponents 1, 4 and 5) are not actually missing from the array; they just have a coefficient of zero. Zero is a number like any other, even if we tend to think of it as special.

The rational number b/c, where b and c are integers and the fraction is in lowest terms, can be a root of a polynomial whose lowest coefficient is t and the highest coefficient is s only if the numerator b is some factor of the lowest coefficient t. The denominator c must be a factor of the highest coefficient s. Since we can easily loop through all possible pairs (b, c) of integers that satisfy that condition, all that remains is to evaluate the polynomial at each such fractional point b/c and gather precisely those points that make the polynomial zero. For example, for b/c to be a rational root of the previous example polynomial, c must be one of the factors of 5 (therefore either 1 or 5, not much choice there) and b must be one of the factors of 42 (therefore one of 1, 2, 3, 6, 7, 14, 21, 42). This gives $2 \times 8 \times 2 = 32$ possible combinations to try: two for c, eight for b, and for each pair, you must try both b/c and $-b/c$ for the sign of that root.

The operation of evaluating the given polynomial at a given rational point x is best written as a separate method so we can unit test and debug it. Once we are confident this method works, it will be much easier to write the method that finds the polynomial's rational roots, since we can then isolate failures in that method to its body for debugging.

```
public static Fraction evaluate(int[] coefficients, Fraction x)
```

This method evaluates the polynomial with the given `coefficients` at point `x`. The answer must be computed and returned as an exact `Fraction` object. However, don't be a Shlemiel and evaluate each term separately; instead, realise you can get the next term, x^{k+1}, with just one more

multiplication from the previous term, x^k. (The best algorithm to evaluate an arbitrary polynomial at a particular point x is Horner's rule, which whittles the entire process down to one measly multiplication and one addition per term.)

```
public static List<Fraction> rationalRoots(int[] coefficients)
```

Return the list of all rational fractions of the polynomial with the given `coefficients`. To make the result unique and unambiguous for the JUnit fuzz tests, this list must contain the rational roots of that polynomial in sorted ascending order, with each root listed only once even if it happens to be a double root. Loop through the possible rational roots in two nested for-loops, using the previous method to `evaluate` the polynomial at that rational point, and adding that rational point to the results if the polynomial evaluates to zero at that point. Eliminate duplicates and sort, and that's that about that equation, wham bam thank you ma'am.

For example, given the integer coefficients `{333,-432,-16}`, this method would create and return the list `[-111/4,3/4]`, revealing two rational solutions for that polynomial. Given the coefficients `{319410,-207597,48337,-4830,176}`, this method would return four rational solutions in the list `{42/11,13/2,65/8,9}`.

As a potentially important aside for your future, these kinds of test cases for this problem with known rational solutions were a breeze to create in Wolfram Mathematica, a very high-level programming language that operates directly on symbolic expressions kept in symbolic form and evaluated lazily according to the rules of mathematics. A small screenshot below, copied from the session used to create the automated test class, shows the idea of building up symbolic polynomials that have the rational roots that we want them to have:

```
In[67]:= p = Collect[(x - 13 / 2) (x - 27 / 3) (x - 42 / 11) (x - 65 / 8), x]

Out[67]= 159 705/88 - 207 597 x/176 + 48 337 x^2/176 - 2 415 x^3/88 + x^4

In[69]:= ReplaceAll[p, {{x → 13 / 2}, {x → 27 / 3}, {x → 42 / 11}, {x → 65 / 8}}]

Out[69]= {0, 0, 0, 0}
```

Even rudimentary knowledge of Mathematica would make all undergraduate-level math courses immensely easier. This author finds it strange that this wonderful system isn't taught to everybody during the first week of intro calculus. Plus, though even many active Mathematica users don't seem to know this, Mathematica is also a pretty awesome programming language for solving real problems in all walks of life outside mathematics, especially when integrated with the Wolfram Alpha knowledge base. As high-level languages go, moving from Python 3 to Mathematica should feel pretty similar to moving from Java to Python 3. Interested students can create a free account on Wolfram Cloud to try out the language, and start with either "An Elementary Introduction to Wolfram Language" or "Fast Introduction to Programmers" according to taste to find out what to do in, on, over, under and with that account.

Lab 37: Big Ten–Four

JUnit: TenFourTest.java

When standing on any integer a, you can step to either $10a$ or $10a + 4$; also, whenever a is even, you can also step to $a/2$. For example, if you are currently at 17, you can step into either 170 or 174. If you are currently at 42, you can step into 420, 424 or 21, which together sound like a fun weekend in Vegas back when that sort of thing was still a thing. As this instructor learned from "In Polya's Footsteps", a collection of entertaining expositions on mathematical topics by the late great Ross Honsberger, a crafty young fella such as *you* can get from $a = 4$ to any other positive integer by using only these three basic moves. (You are a winner, kid, I can see it in your eyes!)

This lab examines a mathematically less sophisticated but still universally effective **systematic search** method: breadth-first search. This search algorithm is guaranteed to find the shortest path from 4 to any given n. Implementing this search algorithm for this problem teaches you the general idea of solving similar problems: systematically generating all shortest paths to every number in a breadth-first manner and stopping as soon as you find the shortest path to the goal.

Create a class `TenFour` in your labs project, and in that class write the method

```
public static List<Integer> shortestPath(int n, int limit)
```

to find the shortest path from 4 to `n` under the additional constraint that you are not allowed to move into any number that is greater than or equal to `limit`. If the goal integer `n` is not reachable under this restriction, return an empty list; otherwise, return the list of numbers encountered along the path from 4 to `n`. To implement the logic of breadth-first search, you should first define the following local variables:

```
boolean[] seen = new boolean[limit];
int[] parent = new int[limit];
LinkedList<Integer> frontier = new LinkedList<>();
```

The `seen` array tracks which numbers we have already discovered in our search, which spreads out from the starting number 4, the only number we have seen so far. Every time we discover a previously unseen number, we note its parent, the number we came from. The `frontier` list contains the numbers we have discovered so far but have not yet **expanded** by looking at their neighbours. For a refreshing change, this `frontier` list should be an instance of our red-headed stepchild `LinkedList<Integer>` instead of the golden child and usual go-to guy of `ArrayList<Integer>`, because to implement a **first-in-first-out queue** this algorithm needs, we surely want to add elements at one end of the list, and remove them from the other end. Depending on how you slice it, adding or removing from an `ArrayList<Integer>` takes linear time to restore

the consecutive, gap-free structure. In contrast, a `LinkedList<Integer>` guarantees both operations take constant time.

To start this circus, mark the number 4 as having been `seen`, and add 4 to the `frontier` queue. Then use a while-loop that runs until the `frontier` becomes empty. Pop out the first element of the `frontier`, the current number for this round that we shall call `v`. (During the first round of this loop, necessarily `v=4`.) Look at the three numbers `10*v`, `10*v+4` and `v/2` (this last one only when `v` is even) that you could move from the current number `v` in one step. For those of the three that haven't been `seen` yet, mark them as `seen`, make `v` their `parent`, and add them to the end of the `frontier` queue. To make sure that the returned shortest paths(not just their lengths) are unique to allow automated testing with JUnit, your method **must** examine the three neighbours of `v` in this exact order of `10*v`, `10*v+4` and `v/2` when expanding `v`.

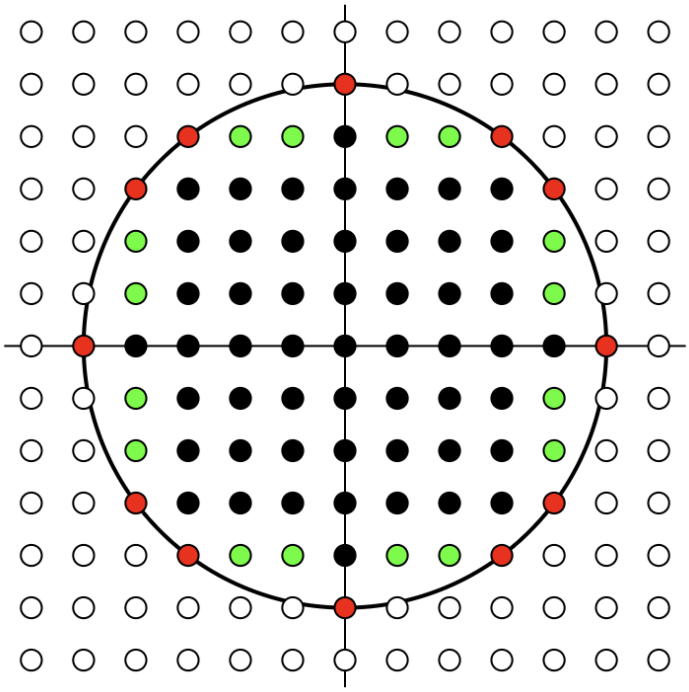
Keep going until you discover the goal number `n` during the expansion of some other number `v`. At that point, you can construct the path from `n` back to 4 by following the `parent` information that you stored along the way, and then reverse that path from `n` to 4 to be returned as your final answer as the path from 4 to `n`. If the `frontier` becomes empty so that the while-loop terminates before finding the goal number, return the empty list as the artificial negative answer. As revealed by the instructor's private model implementation, the shortest paths from 4 to various goal numbers (with the `limit` incremented iteratively from `100*n` by a factor of 2 until the goal was found via some path that stayed within that limit) are listed in the following table.

n	Expected result
3	`[4, 2, 24, 12, 6, 3]`
13	`[4, 40, 20, 10, 104, 52, 26, 13]`
42	`[4, 2, 24, 12, 6, 64, 32, 16, 8, 84, 42]`
404	`[4, 40, 404]`
739	`[4, 2, 1, 14, 144, 72, 36, 18, 9, 94, 944, 472, 236, 2364, 1182, 11824, 5912, 2956, 1478, 739]`
740	`[4, 2, 1, 14, 7, 74, 740]`

Lab 38: Fenceposts Around Euclid's Orchard

JUnit: GaussCircleTest.java

As the Pythagorean theorem shows, a disk centred at the origin (0, 0) with radius *r* contains all



points (x, y) that satisfy the inequality x^2 + y^2 ≤ r^2. Points on the boundary circle satisfy the strict equality. As we learned in school, the area of this disk is πr^2. Note also the correct terminology in this context, in that a *circle* is the one-dimensional boundary curve around a two-dimensional *disk*. (Equivalent concepts in three dimensions are the boundary *sphere* around the solid three-dimensional *ball*.) So, next time some Weisenheimer challenges you to recite the formula for the area of a circle from memory, you can remind him that, being a one-dimensional curve, the area of the *circle* around the disk is zero…

The Gauss circle problem asks for the exact number of points (x, y) with integer coordinates inside a disk of radius *r.* Our lab further breaks this count into three parts by defining an edge point as one that lies exactly on the disk's edge. A **border point** lies inside the disk but not on the edge, and at least one of its axis-aligned neighbour points $(x + 1, y)$, $(x - 1, y)$, $(x, y + 1)$ and $(x, y - 1)$ lies outside the disk. Points inside the disk but not on the border or the edge are **proper internal points**.

As you can see from the image above, the disk with $r = 5$ contains 12 edge points (red) that correspond exactly to the Pythagorean triples of integers (x, y, r) whose **hypotenuse** equals *r*. That same disk also contains 16 border points (green) and 53 proper internal points (black).

Create a class `GaussCircle` in your labs project, and there write the method

```
public static void classifyPoints(int r, long[] out)
```

This method does not return anything, but instead fills in its three-part answer into the first three elements of the parameter array `out` so that `out[0]` is the number of proper internal points, `out[1]` is the number of border points, and `out[2]` is the number of edge points. Since the method and its caller share the same `long[]` object, the values written inside the array remain when control returns to the caller.

Your method **must** perform all of its internal computations with the `long` type instead of `int` to guarantee that the squaring of distances does not overflow for large values of `r`. (Even when the value of `r` fits inside an `int`, its square `r*r` might not be quite as friendly.)

The following table lists a couple of values for *r* along with their expected numbers of internal, border and edge points. Due to symmetry, the edge point count is always divisible by four, and the points (r, 0), (–r, 0), (0, r), and (0, –*r*) are edge points for every integer radius *r.* Of course, the total number of points increases with *r*, but the exact distribution of those points into internal, border, and edge points depends on *r*'s arithmetic properties. The three counts of points in the last row add up to 3141592649625, close enough to πr^2 for government work, but still slightly overestimating the true value of π because the implicit sum of unit squares centred at all these lattice points will necessarily include some area that falls outside the original *r*-disk. The area of these trimmings relative to the disk's total area approaches zero as *r* increases.

r	Internal	Border	Edge
1	1	0	4
10	261	44	12
24	1661	128	4
118	43045	660	4
119	43797	660	12
12345	478705609	69820	12
1000000	3141586992773	5656800	52

This problem can be solved with two nested loops that iterate through all possible combinations of *x* and *y*, both ranging from –*r* to *r*, and classify each point separately. However, as you can see from the last line of the table above, this method must work even for large values of *r*, so you should add up these points to the tally one entire row at a time, using the disk's symmetries for maximum efficiency. Manipulating and evaluating symbolic formulas is always more efficient than grinding low-level arithmetic like some machine.

Place your finger on the topmost edge point (0, 5) in the previous picture and trace the red and green edge and border points of the upper-right quadrant. Notice how, from the current point (x, y), you always move to whichever is the first available point of the three adjacent points $(x + 1, y)$, $(x + 1, y - 1)$ or $(x, y - 1)$ that is not outside the disk. In other words, when tracing the boundary of the upper-right quadrant with your finger, move east whenever you can. Otherwise, move southeast if you can, and move south only if you have no other choice. This technique is guaranteed to work for any radius r.

Your method should therefore maintain three counters for the internal, border and edge points, and update these counters appropriately at each step depending on the direction of that step. Once you reach $y = 0$, at which point (heh) necessarily $x = r$, you can terminate the loop. The centre row $y = 0$ is its own symmetric row, unlike the other rows with $y > 0$, which are symmetric with the row $-y$, mirrored along the x-axis.

The Gauss circle problem asks for an exact closed-form combinatorial formula for the number of integer points inside the r-circle, so we can get the answer on the spot without crawling through the rows of points one at a time. However, this problem remains stubbornly unsolved, despite our best efforts to find such a formula! The titular Euclid's Orchard problem asks how many points inside the disk can be seen from the origin without another point blocking the view. Equivalently, how many coordinate pairs (x, y) inside the disk are **relatively prime**, that is, $gcd(x, y) = 1$.

As an unnecessary final aside, this problem would become even trickier if the centre point (cx, cy) could have arbitrary real coordinates instead of being fixed at the origin. An interesting theorem about this setup says that for any positive integer n, there must exist a centre point (cx, cy) and radius r such that the area πr^2 is exactly n, and the disk contains exactly n integer grid points, no more and no less.

Lab 39: Hot Potato

JUnit: HotPotatoTest.java

A group of n people numbered from zero to $n - 1$ stand around a circle. Each person has memorised their personal "enemies list" of other people in the circle. However, this enemy relation is not symmetric, so that Moe can be an enemy of Joe even when Joe is not an enemy of Moe. (You may not be interested in war, but war will still be interested in you. The predator-prey relationship is inherently asymmetric, and the prey doesn't get a vote on it by merely closing its eyes and wishing it were so.) In this problem, everyone has at least one enemy in the room and, unlike in the real world, nobody is so messed up in the head that they are their own enemy. During this process, no new enemies are created or friendships forged.

We may also imagine these events taking place in a very official room with a high vaulted ceiling, with these men dressed in fancy robes or togas to signal their position. Either way, all these people are assumed to be perfect Machiavellian schemers who hatch all their plans in secret, but do not hold grudges or seek payback for past trespasses against them or what's theirs, unless revenge concretely benefits them in the future. (There is no love in this room, so the preference for being hated rather than loved is not a factor.) These men have properly internalised the ideal Markovian view that once the present state of the world is sufficiently known to determine all our future decisions, the past no longer exists in any relevant sense to affect the outcomes of our actions. In the present moment, only what matters is to accept it as it is and choose your actions in that eternal, infinite now to maximise your future gains across all future present moments.

Students interested in this general theme should also check out Nassim Nicholas Taleb's popular writings in their spare time. To sum up its relevant ideas while standing on one foot: you should only care to listen to advice from those who have demonstrated that they understand the concept of ergodicity (if you have to ask what that means, you definitely don't understand it yet) and can explain its effect on their decision-making, and that they have "skin in the game"… and significantly more risk of burning to a crisp than merely getting to eat delicious potato skins!

Back to the problem at hand. At $t = 0$, person 0 holds a proverbial hot potato. At each discrete time step t, the person currently holding the hot potato will instinctively toss it randomly to one of his enemies to catch. That person will then similarly throw the hot potato randomly to one of his enemies at time $t + 1$, and so on. In this lab, you will create a method to investigate the probability distribution of the hot potato at a given time t, given the people and their list of enemies. Create a class `HotPotato` in your labs project, and there, the method

```
public static Fraction[] hotPotato(int[][] enemies, int tGoal)
```

The i:th row of the `enemies` array is a one-dimensional array of integers that lists the enemies of person i in ascending order. For example, if `enemies[2]` equals `{0,5,6,9}`, that means person 2 has four enemies: persons 0, 5, 6, and 9. Since Java does not have true two-dimensional arrays,

every `int[][]` is really a one-dimensional array whose elements are one-dimensional `int[]` arrays representing the rows, so the rows can be ragged and do not need to have the same length.

The probability distribution of the hot potato's position at time *t* is expressed as an *n*-element array of probabilities. Since these probabilities will be integer fractions, we will again use the `Fraction` class to represent them without rounding errors. As usual, the individual probabilities must be between 0 and 1 and must sum to exactly 1, since exactly one person must hold the potato at any given time. For the initial state $t = 0$, this probability distribution is simply $P(t, 0) = 1$ for the first person and $P(t, p) = 0$ for the other people $p > 0$, since we know that person 0 is initially holding the hot potato.

Each probability $P(t + 1, p)$ can be computed from previous probabilities at time *t* by first finding all people *p*' who consider the person *p* an enemy. (Since the enemy relationships never change during the execution of this method, it might be smart to precompute these at the start of this method, so that you can always just look them up later whenever you need them.) For each such person *p*', compute the quantity $P(t, p') / m$, where *m* is the number of enemies of person *p*'. This term gives the probability of the event "Person *p*' is holding the potato at time *t*, and tosses it to the person *p*." The desired $P(t + 1, p)$ can then be computed by adding up all such terms $P(t, p') / m$, since those are the only possible ways for *p* to receive the potato at time $t + 1$. Fortunately, these are mutually exclusive, so we can add their individual probabilities to find the desired probability.

The following table shows a couple of illustrative examples of the enemy structure and the resulting probability distribution at time *t*. In the first row, person 0 first throws the hot potato randomly to person 1 or 2, resulting in a distribution where person 0 cannot hold the potato, and persons 1 and 2 each have the potato with probability 1/2. The second row shows the distribution at time $t = 2$, following the previous distribution. This second distribution is more lopsided than the first row because persons 1 and 2 toss differently. (You can solve the long-term behaviour of any such Markov system symbolically, arbitrarily far into the future, using linear algebra.)

n	t	enemies	Expected result (as `Fraction[]`)
3	1	{{1, 2}, {0, 2}, {1}}	{0, 1/2, 1/2}
3	2	{{1, 2}, {0, 2}, {1}}	{1/4, 1/2, 1/4}
4	2	{{1, 3}, {0, 2, 3}, {0, 1}, {0, 1, 2}}	{1/3, 1/6, 1/3, 1/6}
5	3	{{1, 2, 3}, {0, 4}, {1, 4}, {2, 4}, {1, 2}}	{1/12, 7/18, 11/36, 1/18, 1/6}
2	1000	{{1}, {0}}	{1, 0}

Lab 40: Seam Carving

JUnit: SeamCarvingMain.java

All magic tricks are trivial once you get to watch them from backstage (or even better, in the rehearsal room), rather than seeing them unprepared through the fourth wall from the audience. The seam carving algorithm for **context-aware image resizing** is deservedly famous for the almost magical outcome it achieves relative to its far lower complexity.

Modern image processing algorithms employ arduously trained **deep neural networks** that implicitly identify an image's semantic content. The seam carving algorithm is far less lofty, using only good old-fashioned arithmetic and tabulation of intermediate results to decide which pixels to eliminate. Seam carving works with simple local calculations, with no higher semantic knowledge of the objects in the image, nor any ordering of importance between them. Everything happens at the level of individual pixels and their local gradients relative to their neighbours, and yet the global result just works.

Instead of rescaling every pixel by the same amount, the scaling repeatedly "carves" a one-pixel-wide path through the image, then erases exactly those pixels. For simplicity, each carve runs from some top-row pixel to some bottom-row pixel, containing exactly one pixel from each row and moving at most one pixel left or right at each step to the next row. Because the path contains exactly one pixel from each row, removing these pixels preserves the image's rectangular shape, keeping the height unchanged and reducing the width by one pixel. The resizing can go on arbitrarily deep by carving these individual slices of bologna out of the image one at a time.

The finer details of energy calculation and other aspects of the seam-carving algorithm are well explained with illustrative images in the project pages of various fine universities whose courses have used seam carving as a programming project, such as Princeton, Brown, and Berkeley, so we shall not repeat them in this document. Instead, here at Wossamotta U., whose faux marble columns tend to be covered with lichen, graffiti, and vomit instead of ivy, our practical can-do attitude ought to propel us to do at least as well as those perfumed princes in their secret handshake clubs. Have all those classic college comedies really taught us nothing, my fellow band of lovable underdogs itching to take on "The Man"? Was it over for Napoleon when those German ratzis bombed Verdun? Come on, the trailer for this classic comedy reboot practically writes itself! Create a class `SeamCarving`, and there the method

```
public static Image carve(BufferedImage image, int width)
```

to create and return the seam-carved version of the `image` horizontally scaled to the desired `width`. To allow you to concentrate on the actual seam carving algorithm instead of the gritty details of the bitwise arithmetic needed in colour manipulations, you can use the following method to compute the approximate gradient between two RGB colours packed into four-byte integers that you get

from the `getRGB` method of `BufferedImage`. The higher the gradient between the pixel and its chosen neighbours, the more "energy" that pixel contains.

```
private static float gradient(int rgb1, int rgb2) {
    int b1 = rgb1 & 0xFF, b2 = rgb2 & 0xFF;
    float b = Math.abs(b2 - b1);
    int g1 = (rgb1 >> 8) & 0xFF, g2 = (rgb2 >> 8) & 0xFF;
    float g = Math.abs(g2 - g1);
    int r1 = (rgb1 >> 16) & 0xFF, r2 = (rgb2 >> 16) & 0xFF;
    float r = Math.abs(r2 - r1);
    return b + g + r;
}
```



You can test your implementation of the algorithm with the [SeamCarvingMain] class provided in the repository. The screenshot above shows the result of seam carving the example images coffee.jpg and ilkka.jpg to half their original width with repeated carvings.

The basic seam-carving algorithm, without further fine-tuning, tends to struggle with sloping straight lines, as seen in the seam-carved coffee maker image. However, the important coffee maker object remains almost as wide as in the original. For the author portrait in the second row, even the most rudimentary seam-carving algorithm could find the lowest-energy seams in the nearly uniform white background glowing around that old mutt's ugly mug.

Lab 41: Geometry I: Segments

JUnit: CompGeomTestOne.java

"*Whoever says X, must also say Y.*" — V. I. Ilkkanov
"*Whoever says 'ex', must also say 'why?'* " — its modern relationship corollary, apparently

At first glance, geometry problems seem difficult to solve on a computer, at least in low-level languages such as Java or Python that do not allow expressions to be analysed in symbolic form within the homoiconic language itself. No, this term does not refer to Liza Minnelli in this context. "Homo" means "same", and "icon" means representation, referring to how a homoiconic language's structures are also first-class data in that same language, so expressions can be assigned to variables and passed back and forth to functions to perform computations.

Even worse, algebraic calculations of even seemingly simple geometric shapes tend to be chock full of square roots and other irrational quantities, especially once trigonometric formulas for arbitrary angles enter the fracas. Even the familiar junior high school notion that a "line" is defined by its "slope" and "offset" goes kaput the moment it encounters the first vertical line segment, since computing the slope of that vertical line involves division by zero. As responsible educators, we have a duty to save our students from the claws of such *comprachicos.* In the next four labs, students are invited to enter Leopold Kronecker's paradise of well-behaved integers and everything good and plenty that springs forth from them, before that wicked serpent of division sneaks its forked tongue inside this big happy tent.

Some applications cannot avoid geometric computations, most excitingly, games played in a simulated microworld in some finite box inside the infinite two-dimensional plane. These geometric computations become more manageable if restricted to lattice points, that is, integer coordinates. Our geometric objects are composed of straight **line segments** whose endpoints are both lattice points. Since this lattice can be arbitrarily large, we can scale the world so that every point inside it is a lattice point, eliminating fractional parts in the game display's pixel raster.

Instead of using roots and trigonometric functions, we build our geometry on vector operations: the **cross product** and **signed area**. Even if you don't know what these operations are, no worries! Just like you are allowed to use a compiler even though you don't know how it works, or drive a car even if you don't know how a combustion engine works, you are still allowed to enjoy the existence and results of these functions even if you have never taken a vector mathematics course and so wouldn't be able to explain, standing on one foot, exactly what makes these functions tick.

Create a class named `CompGeom` in your labs project. You will be writing all methods from these computational geometry labs inside this same class. Each lab will have its own JUnit test class that depends only on the methods written up to that point. To begin, copy and paste the following implementations of the methods `cross` and `ccw` inside your class:

```
public static int cross(int x0, int y0, int x1, int y1) {
    return x0 * y1 - x1 * y0;
}

public static int ccw(int x0, int y0, int x1, int y1, int x2, int y2) {
    return cross(x1 - x0, y1 - y0, x2 - x0, y2 - y0);
}
```

Without worrying about what these functions actually do and how they do it, you should now put on your Java programmer hat and spend a moment in silent appreciation of how both methods use only basic arithmetic and are well-defined for all possible argument values. (The JUnit tests are designed to keep the numbers small enough so that your code does not need to worry about the effects of integer overflows.) Most importantly, **neither operation performs any divisions**, so we don't need to worry about dividing by zero, as people often do in slope calculations, or about the inaccuracies of floating-point division.

Analogous to the bumper sticker version of Liskov Substitution Principle that your instructor keeps parroting every chance he gets during the lectures, you should also follow a similar bumper sticker maxim for computational geometry: **Should any one of the forbidden words "angle", "slope" or "root" come out in your description of how your method solves that computational geometry problem, your method is automatically wrong!**

Now that we've got that out of the way, let us explain what the two methods above actually do. The `cross` method computes the **two-dimensional cross product** of two vectors (x_0, y_0) and (x_1, y_1). (Any point (x, y) on the two-dimensional plane can be thought of as a **vector** from the origin (0, 0) to that point (x, y).) The cross product of two vectors is a three-dimensional operation that produces a three-dimensional vector result, not merely a single scalar, but since any two-dimensional vector (x, y) can be considered a degenerate three-dimensional vector $(x, y, 0)$, multiplying by these zeros cancels most terms. Only the z-component of the result vector needs to be computed with the remaining formula $x_0y_1 - x_1y_0$, and it can be returned as a scalar instead of a redundant three-element vector with zero x- and y-components.

The cross product returns the **signed area** of the **parallelogram** defined by the vectors (x_0, y_0) and (x_1, y_1). Swapping the order of the arguments therefore negates the sign of the area. Since the signed area formula uses only addition, subtraction, and multiplication, it applies to integers, rational numbers, and real numbers. The key feature of the signed area formula is that it guarantees an integer result when applied to integer arguments.

The concept of a negative area might seem confusing at first. Still, it serves an important purpose in the operation that most online material succinctly calls `ccw`, so we'll follow that naming here. (In fact, just googling for "`ccw computational geometry`" gives you several good pages and sets of slides that illustrate how this operation cuts through seemingly complex geometry problems faster than a chainsaw goes through butter!) This operation lets us implement all methods from these four labs with absolute accuracy, without rounding errors or singularities. Among the subfields of

computer science, computational geometry is especially notorious for its tricky edge and corner cases (heh), which naive implementations often struggle with, especially when using floating-point arithmetic with a happy-go-lucky attitude.

The name `ccw` is short for "counterclockwise". Given three points (x_0, y_0), (x_1, y_1) and (x_2, y_2) on the plane, the sign of the signed area of the triangle from (x_0, y_0) to (x_1, y_1) to (x_2, y_2) tells you the orientation of these three points with respect to each other.

- If the method `ccw` returns any **positive** number, these three points are oriented **counterclockwise**, so the turn in the middle when moving from (x_0, y_0) to (x_1, y_1) to (x_2, y_2) is **left-handed**.
- If the method `ccw` returns any **negative** number, these three points are oriented **clockwise**, so the turn in the middle when moving from (x_0, y_0) to (x_1, y_1) to (x_2, y_2) is **right-handed**.
- If the method `ccw` returns zero, the three points (x_0, y_0), (x_1, y_1), and (x_2, y_2) are **collinear**.

In practice, the problematic edge and corner cases in the computational geometry algorithms in these labs emerge when three points are collinear, even when they overlap. Frankly, it is not surprising that problems of points and line segments often begin with the order to "assume the general position" where no three points are collinear and no four lie on the same circle…

And that's it! That's all she wrote! The `ccw` method is the fundamental building block of all computational geometry operations we will now implement. In fact, if you remember how `compareTo` for custom order comparison returns an integer whose sign contains the entire answer and whose magnitude is meaningless, the `ccw` operation is the two-dimensional generalisation of the one-dimensional order comparison operator! (In fact, checking whether $a < b$ on the one-dimensional line of real numbers reduces to checking whether the turn defined by the three tactically chosen points $(a, 1)$, $(a, 0)$, $(b, 0)$ is left-handed.)

To practice using this wonderful `ccw` operation and appreciate it in action, here is finally your first method to write in this lab, in a new class named `CompGeom`.

```
public static int lineWithMostPoints(int[] xs, int[] ys)
```

For simplicity, a set of points on a two-dimensional plane is given to these methods as two arrays `xs` and `ys` (read these two names out as "exes" and "whys", as the letter *s* denotes the plural form) so that `xs` contains the *x*-coordinates of these points, and `ys` contains the *y*-coordinates. For example, the *x*- and *y*-coordinates of the point (x_4, y_4) are in `xs[4]` and `ys[4]`. These points are not otherwise sorted, but can be given in any order.

Given a set of n points on the plane, find the line that contains the largest number of points from that set. Since several such lines may exist, this method returns only the count of points on the line to make automated testing feasible.

Any two distinct points determine a unique line. The `ccw` operation then tells you quickly whether some third point also lies on that same line. You can therefore solve this problem with two nested loops that examine all possible pairs of points (and therefore all possible lines defined by such pairs of points), with a third level of innermost loop finding all the other points in the same line. However, since the $O(n^3)$ running time from three levels of nested loops can be heavy, do whatever you can to eliminate redundant comparisons and speed up execution.

With that out of the way, it is time to implement an important operation that will be used as a building block for many other computational geometry algorithms: determine whether the line segment with endpoints (x_0, y_0) and (x_1, y_1) intersects another line segment with endpoints (x_2, y_2) and (x_3, y_3), that is, whether these two line segments have at least one point in common.

```
public static boolean segmentIntersect(
  int x0, int y0, int x1, int y1, int x2, int y2, int x3, int y3
)
```

One of Alan Perlis's famous programming epigrams points out that if your function takes ten parameters, you probably forgot some. We are two short of that magic number here, but the point (or, more specifically, the lack of `Point`) remains just as valid.

Once again: no angles, no slopes, no trig, nothing that could ever produce anything that isn't an exact integer, since we can solve this important problem with absolute accuracy in all corner cases with the following approach. Start with a simple **quick rejection test** by comparing the **bounding boxes** of these line segments. Bounding box comparison is a basic computational geometry technique to speed up intersection tests between complex objects, and it works just as well here for simple line segments. If one segment's higher x-endpoint is strictly less than the other segment's lower x-endpoint, the segments cannot intersect. The same test is applied to y-coordinates of these endpoints. In practice, quick rejection tests often return a negative answer, since most pairs of line segments on the plane are far enough apart for this test.

The method moves to the next stage only if the bounding boxes intersect. Then, the segment with endpoints (x_0, y_0) and (x_1, y_1) intersects the segment with endpoints (x_2, y_2) and (x_3, y_3) if the turns $(x_0, y_0) - (x_1, y_1) - (x_2, y_2)$ and $(x_0, y_0) - (x_1, y_1) - (x_3, y_3)$ do not have the same nonzero handedness. In other words, the points (x_2, y_2) and (x_3, y_3) are not on the same side of the line segment (x_0, y_0) and (x_1, y_1), but the line segment from (x_2, y_2) to (x_3, y_3) actually crosses that line so that an intersection can occur. Repeat the same test with the roles of the two segments interchanged, since the segments must cross each other both ways to intersect.

This logic handles the intersection test between two arbitrary line segments, even in the degenerate case where one segment is a single point. To find all intersections within a set of n such line segments, you can loop through all $n(n - 1)/2$ pairs of segments, which becomes a prohibitively inefficient “Shlemiel” algorithm as n increases. Assuming the segments are reasonably evenly spread, a sweep-line algorithm similar to the one that solved the earlier Manhattan skyline problem

will do this job in expected linear time. This algorithm would be slow only in pathological cases, such as line segments stacked vertically in a slab around the same *x*-coordinate.

Lab 42: Geometry II: Polygons

JUnit: CompGeomTestTwo.java

The previous computational geometry lab introduced the `cross` product and `ccw` operations as the fundamental building blocks of our geometric algorithms. We now move from sets of n points to ordered lists of n points that define an n-sided polygon whose edges are the line segments connecting consecutive points in the list. The line segment from the last point back to the first point closes the polygon. Depending on the needs of the situation, we can think of the polygon as made up of "edge segments" or "corner points", but that is just a different "point" of view of the same integer data. (Surely it's no coincidence that our everyday language is rife with geometric metaphors.)

The line segments determined by an arbitrary list of points do not necessarily form a "polygon" as we generally understand the term. For this lab, a list of points defines a simple polygon if it satisfies the following two constraints:

1. No three consecutive corner points are collinear.
2. No two non-consecutive edge segments intersect, not even at a corner point.

Of course, every two consecutive edge segments meet at their common corner point, so no holes are left in the polygon boundary. Still, no edge segment intersections exist other than the *n* trivial intersections at the corner points. The `ccw` and `segmentIntersect` methods from the previous lab should make short work of writing the following method:

```
public static boolean isSimplePolygon(int[] xs, int[] ys)
```

This method determines whether the list of points given by the two n-element arrays xs and ys forms a simple polygon when read in that order.

Technically, the first condition, non-collinearity of three consecutive corner points, is not necessary for a polygon to be "simple", as that concept is generally defined. However, eliminating redundant corner points guarantees that every corner point is a literal "corner", so a sudden discontinuous turn definitely occurs there. (You could always modify this method to take an additional `boolean` parameter to indicate whether the collinearity avoidance rule is in effect.)

Furthermore, any **cyclic shift** of these corner points changes nothing, but the shifted points still represent the same simple polygon as a geometric object on the plane. The same holds for reversing the list of corner points. However, some computational geometry material imposes an additional condition that the polygon corner points must be listed in the order that follows the boundary of the polygon in counterclockwise direction. Such a guarantee would make later polygon operations easier to implement. (We have encountered this principle of unique canonical representation for objects before, such as with the `Fraction` class.)

Only the sign of the `ccw` result affects our decisions; its magnitude is irrelevant. Akin to how one-dimensional sorting algorithms only compare elements for order and never need to use arithmetic to find out the absolute difference between elements, **topological** algorithms whose results are unaffected by arbitrary scaling, shifting and rotating of these points, as a rule of thumb, should never need the actual magnitude of the result for anything. However, this magnitude still has meaning and can be used in another important problem: calculating the **area** of the given polygon.

Recall from the previous lab that the cross product of two vectors is the signed area of the parallelogram defined by those vectors. Therefore, the signed area of the triangle defined by three corner points (x_0, y_0), (x_1, y_1), and (x_2, y_2) is exactly half of $\text{ccw}(x_0, y_0, x_1, y_1, x_2, y_2)$. This immediately implies that the area of a triangle whose corners lie at integer lattice points can only be an integer or an **integer plus one half**, regardless of how these three points lie on the plane. The result is always this neat, regardless of the square roots of their side lengths or the irrational numbers their angles happen to be.

Since any simple polygon can be broken into disjoint triangles that share edges and corner points, the same follows immediately for the areas of arbitrary simple polygons. (As with many other theorems of computational geometry that feel to our intuition as if they were "🎶 so very clear and rational to anyone who does not belong to the oppressor class 🎶", this result is a bit more, ahem, *complex* than that.) **Triangulation** would then immediately provide us a way to compute the area of any simple polygon. Efficient algorithms can rapidly triangulate any polygon into a partition of disjoint triangles. (These algorithms must tread carefully if the polygon to be triangulated is not convex; the greedy approach of taking three consecutive corner points and clipping that triangle on its own, then recursively triangulating the rest, will fail for such polygons.) However, we don't need to triangulate a simple polygon to compute its area, since we can compute it in guaranteed linear time from the list of edges using the shoelace formula, courtesy of Carl Friedrich Gauss!

```
public static int shoelaceArea(int[] xs, int[] ys)
```

Since the area of a lattice polygon is either an integer or an integer plus one half, this method returns twice the area, so the result is always an integer. The caller can always divide this result by two for the real area, but this way we can return 30 when the actual area is 15 and 31 when the actual area is 15 1/2, so the caller can tell those two situations apart using only integer arithmetic.

(This idea of scaling fractional values to be integers generalises for other computations that involve fractional quantities. For example, we can sometimes represent money as cents instead of dollars and avoid the fractional parts that typically don't have an exact representation in floating point anyway. You should still be careful about potential integer overflows these days when a billion here and a billion there add up to real money.)

Choose any reference point from the plane. It doesn't matter which point you choose, or whether it lies inside or outside the polygon. With that freedom, we can choose the origin (0, 0) as the reference point to simplify the calculations, even though any other point would work just as well.

Initialise the integer `area` variable to zero. Loop through the consecutive edge segments with corner points (x_i, y_i) and (x_{i+1}, y_{i+1}), and add the result of `ccw` of the reference point and those two points to your `area`. After this one loop, return the final value of the `area` as your answer, wham bam thank you ma'am, that's all the fat lady sang.

Honestly, it's that simple. Since the `ccw` function returns a signed area whose sign depends on the edge segment's direction relative to the reference point, the algorithm adds areas outside the polygon, which are subtracted on the way back around the other side. No matter how the polygon points lie or how complexly the boundary meanders back and forth through the plane, these positive and negative areas outside the polygon will always cancel each other out. Only the area inside the polygon won't cancel out, so the final result equals the polygon's area.

Lab 43: Geometry III: Points In Polygons

JUnit: CompGeomTestThree.java

Intersection and **collision** tests (the former for **static** worlds, the latter for **dynamic** worlds) are important in many games that take place on a two-dimensional plane. This lab starts with the simplest possible intersection test: determining whether a given point lies inside a particular polygon. More advanced algorithms perform efficient intersection and collision tests between arbitrary polygons, disks, and other complex geometric objects. To speed up the intersection test between two complex objects, such as entire **polygon meshes** of some video game character, we often employ a **quick rejection** test that first checks whether some simple **bounding boxes** around those two objects intersect will provide the negative answer in the vast majority of cases, without the need to actually execute the expensive intersection test between the actual complex objects themselves.

A simple polygon is **convex** if for any two points (x, y) and (x', y') inside that polygon, the entire line segment between these two points also lies fully inside the polygon. Intuitively, a convex polygon has no "pits" on its edges. Convex polygons matter in computational geometry, since many polygon problems become much simpler when the polygons are known to be convex. Every **triangle** is automatically convex, which is one major reason triangles and triangle meshes are the fundamental shape primitive in computer graphics.

When traversing the boundary of a convex polygon, every turn must have the same handedness. Either every turn is left-handed, or every turn is right-handed; both cannot exist on the boundary of a convex polygon. Under our additional voluntary constraint of listing the corner points counterclockwise, only left-handed turns can exist. These observations give us an immediate linear time algorithm to determine whether a given polygon is convex. The same idea extends to determining whether the given point (x, y) lies inside the given convex polygon.

```
public static int pointInConvex(int[] xs, int[] ys, int x, int y)
```

The arrays `xs` and `ys` are guaranteed to be the corner points of some convex polygon and, in this problem, are further guaranteed to be listed in counterclockwise order. Instead of returning a simple yes/no `boolean`, it is more useful later to define this method to distinguish the different ways the point can be inside the polygon. This method should return an integer from 0 to 3, where 0 means the point (x, y) lies outside the polygon. Result one means that the point (x, y) is one of the corner points. Result two means that the point (x, y) lies on some edge segment but is not a corner. Finally, result three means that (x, y) lies properly inside the polygon.

The caverns and tendrils of an arbitrary polygon can easily sprout into fractal-like complexity, so point containment in a non-convex polygon is not quite as trivial as in its convex cousins. The first idea, as in the earlier problem of computing a polygon's area, is to subdivide the polygon into disjoint triangles and then test point containment within these triangles using the previous method.

If some triangulation of that polygon is already available, there certainly is no harm in using it, especially if those triangles have been arranged and sorted into horizontal **slabs** so that we can first use **binary search** to determine the slab where the point lies according to its *y*-coordinate, and then another binary search to determine the triangle that the point lies in according to its *x*-coordinate. Otherwise, solving one instance of this problem via triangulation would be a textbook example of overpaying for overkill, since a far simpler algorithm already solves this problem for arbitrary polygons in linear time!

```
public static int pointInPolygon(int[] xs, int[] ys, int x, int y)
```

Like the previous method for a point inside a convex polygon, this method should return 0 for outside, 1 for a corner, 2 for an edge, and 3 for inside. However, this method must work for any polygon, even non-convex or non-simple ones; the JUnit test will generate polygons that intersect themselves and degenerate into line segments and points. The first stage should loop through the polygon's corner points to determine whether (*x*, *y*) is a corner or lies on an edge segment. After that, the rest of the algorithm can safely assume that no polygon edge passes through (*x*, *y*).

To visualise the ray-casting technique for determining whether the point (*x*, *y*) is inside the polygon, imagine standing at that point and walking along a straight ray in an arbitrary direction. (Technically, a line segment has two endpoints, a ray has only one, and a line has none but is infinite in both directions. Most people erroneously say “line” when they `<fedora>`*ackshually*`</fedora>` mean “line segment”, but this is a long-lost linguistic hill left for only the most valiant and euphoric gentlesir to die on with his blade bravely.) Since we can choose this direction freely, we might as well make it one of the main axes, with one coordinate zero, to simplify the formulas. Any of the four compass directions would give a symmetric method, but since we have to pick one, let us follow the usual convention and pick the ray shooting right toward infinity, eternally parallel to the *x*-axis.

Since we don’t have the math to be able to deal with rays (this could be done as a trivial special case of parameterised curves, but even the simplest of those soon tend to end up requiring non-integer arithmetic), we realise that it is enough to look at the line segment from point (*x*, *y*) to the right just far enough so that its endpoint is guaranteed to lie outside the polygon. Any *x*-coordinate larger than the maximum *x*-coordinate among the polygon's corner points will do. Since the end position (1 + max x_i , *y*) of our rightward ray (heh, another good name for some pompadour dad rock band) lies outside the polygon, every edge crossing along the segment moves either from inside to outside or from outside to inside. The parity of the number of polygon edge crossings from the starting point (*x*, *y*) to the end position (max x_i + 1, *y*) tells us the answer right away! An odd number of crossings means the starting point (*x*, *y*) is inside the polygon, whereas an even number means it is outside.

A trivial modification of the previous “even-odd algorithm” to use the “nonzero winding rule” makes it even more general, so it can handle even non-simple polygons whose edges can freely intersect each other. In such a polygon, “inside” and “outside” can become as frustrating to define as trying to

get to the outside of a Klein bottle. Still, the nonzero winding rule handles these situations without confusion.

Initialise the local `count` of crossings to zero. Whenever the ray segment intersects the polygon edge between two polygon corner points (x_i, y_i) and (x_{i+1}, y_{i+1}), the sign of a `ccw` check from point (x, y) to these two points tells us which direction we cross that edge. If the sign is negative, decrement the `count` by one; if it is positive, increment it by one. The point (x, y) is inside the polygon if and only if the final `count` is nonzero. For simple polygons, this rule gives the same answer as the previous even-odd rule. Still, for self-intersecting polygons, the results differ, so the polygon can have holes inside it that are not connected to the world outside the polygon. To pass the JUnit test, **your `pointInPolygon` method should use the nonzero winding rule for counting the intersections, instead of the even-odd rule.**

Before anyone starts implementing this algorithm by actually traversing that rightward ray in their code, we must emphasise that walking along this ray is only an aid to intuition. The actual algorithm should loop through the edge segments of the polygon in one linear-time for-loop. For each edge segment, check whether it intersects the ray segment between the points (x, y) and $(\max x_i + 1, y)$. We can do this with the `segmentIntersect` method from the first geometry lab.

However, as is typical for computational geometry algorithms, a difficult corner (heh) case lurks in the grass, waiting to trap the unwary. Using concrete numbers to illustrate the danger, let (2, 3) be the point (x, y) whose containment inside the polygon we wish to determine, and let the polygon have two edge segments that connect the three consecutive corner points (5, 1), (7, 3) and (7, 5). The rightward ray from the starting point (2, 3) will therefore intersect both segments at their shared corner point (7, 3). An algorithm that checks each edge segment separately will count two edge crossings at one corner point, flipping the parity of the final count and giving the wrong answer.

In moments like this, it helps to step back for a calmer, more reasoned view. Cowboy coders (not a compliment) add extra conditions to cover failing test cases instead of solving the general problem for all cases in one swoop. **However, any fix that relies on a special direction is automatically wrong, even if it solves the issue for that particular test case.** Since the result of any topological problem is unchanged by arbitrary planar rotation, a bug fix in any such problem can never depend on any particular "special" point or direction. The fix must be general enough to work under arbitrary mirroring, rotation, translation, or any other transformation of the original points, as long as it doesn't affect the original set's topological properties. We want to be correct in general, not just *ad hoc* for the test cases that the testers thought to write up.

The correct fix is actually pretty clever in its simplicity once we realise it. Before applying the intersection test, we should do a quick rejection test to eliminate any polygon edge segments that cannot possibly intersect our rightward ray from (x, y). If both endpoints of the edge segment have a y-coordinate lower than our y, the segment lies fully below the ray, and no intersection can exist. The same happens when both endpoints have a y-coordinate higher than y. To fix the previous bug,

treat the *y*-coordinate of the segment endpoint that is equal to *y* as if it were an infinitesimally small distance lower than *y* for this quick rejection test. You don't actually need infinitesimals in your number system; you just need to act as if they did, and the result is the same. (This simple flash of insight generalises for many other problematic situations in programming that you might encounter in the future.)

Using the previous numbers, the edge segment from (5, 1) to (7, 3) will now be rejected as lying fully below the rightward ray from the starting point (2, 3), and therefore that segment does not increment the crossing count even though the rightward ray technically did intersect the rightward ray at the corner point (7, 3). However, the next edge segment from (7, 3) to (7, 5) is not quickly rejected, since (7, 3) is considered below the ray while the other endpoint (7, 5) clearly lies above it. That edge segment increases the crossing count by one, as it properly should.

Lab 44: Geometry IV: Convex Hull

JUnit: CompGeomTestFour.java

As we saw in the previous three labs on computational geometry, a list of points can be thought of either as these points existing independently as points on the two-dimensional plane, or as the ordered corner points of an *n*-sided polygon so that the consecutive corner points are implicitly connected by polygon edge segments between those points. To conclude this journey through the calm surface of computational geometry above all its deep complexities, we will tackle the problem of building a simple polygon from a list of points given in arbitrary order. In general, we can construct exponentially many different simple polygons from the given set of distinct points. However, when we have that choice, we might as well build a polygon with useful properties for the rest of our program to use.

This fourth geometry lab is also a good place to introduce a technique of **indirection** that often comes in handy not just in computational geometry, but in many other problems in data structures and algorithms. When multiple polygons or other geometric objects share the same points, such as when constructing a triangle mesh to represent a polyhedron surface where several triangles meet at shared corner points, it's highly desirable to move these points without breaking the connections between triangles.

If every triangle is an island that knows only the absolute coordinates of its own corner points, but nothing about other triangles that share those corner points, moving those points would affect only that triangle, not any others. To avoid creating ugly cracks in the polygon mesh, we would have to find all other triangles that use the same corner point and apply the same update to that corner point in each triangle separately. To avoid this hassle, maintain the set of all relevant points separately, let's say, for simplicity, as the two arrays `xs` and `ys` we have already seen before. A polygon of *n* such points is encoded as an *n*-element array of integer indices to the arrays `xs` and `ys` that contain the actual point coordinates.

For example, a triangle encoded as the array `{42,17,99}` consists of the corner points whose actual coordinates are stored at positions 42, 17, and 99 in the arrays `xs` and `ys`. These arrays could contain millions of other points, most of which are not part of this particular triangle. Another triangle encoded as the array `{42,43,77}` shares the corner point with index 42 with the previous triangle. Adjusting the coordinates of point 42 automatically updates every triangle that shares that corner point, without us needing to explicitly search for all such triangles and painstakingly update them all! (In some problems, additional computation may be required to maintain additional constraints that prevent the mesh from becoming degenerate in some form.)

Enough chit-chat. Given a set of points as two arrays `xs` and `ys`, the first method in this week's lab constructs a fan polygon by sorting the points counterclockwise when viewed from the reference point with the lowest *y*-coordinate. If the set contains several points with the same minimum *y*-coordinate, use the point with the lowest *x*-coordinate among those points as the reference point.

Note that neither method in this lab should change the contents of the `xs` and `ys` arrays given to them in any way, but create and return a new array of position indices that lists these points in the order in which they constitute the polygon corners.

```
public static int[] sortCCW(int[] xs, int[] ys)
```

Given a set of *n* points with coordinates given in two arrays `xs` and `ys`, this method should first create an *n*-element array `Integer[] result`, and fill it with the integers 0, …, *n* – 1. Next, define a local **strategy** class `Comparator<Integer>` to create objects that will perform the comparisons needed in sorting the array of indices.

```
class CCWComp implements Comparator<Integer> {
    public int compare(Integer i, Integer j) {
      // Fill in this method body
    }
}
```

The `compare` method inside this local class should order the two indices `i` and `j` according to the following ordering rules between indices:

1. The reference point's index is always smaller than any other index.
2. If the turn from the reference point through point `i` into point `j` is left-handed, then `i` is considered lower than `j`.
3. If the turn from the reference point through point `i` into point `j` is right-handed, then `i` is considered higher than `j`.
4. If the reference point is collinear with the points `i` and `j`, whichever of the points `i` and `j` has a lower *y*-coordinate is considered lower. If both points have the same y-coordinate (another possible edge case that is not immediately obvious but can happen), the point with the lower x-coordinate is considered lower.

These rules let the compare method always return `-1` or `+1` for any two distinct indices `i` and `j`, and the resulting ordering is total. Armed with this comparator, the `sort` method in the `Arrays` utility class can take care of all the hard work involved in the actual sorting:

```
Arrays.sort(result, new CCWComp());
```

All that is left is to fulfil the letter of the law: create an `int[]` to hold the result, copy the contents of the Integer[] into that array of the promised return type, and return it.

The fan polygon is not necessarily simple, since it can contain degenerate line segments without an area. This will happen, for example, with the points (0, 0), (1, 0), (2, 0), (2, 1), (1, 1), (2, 2) that become a fan polygon in this order, with the origin (0, 0) as the lowest point that becomes the reference point. The rest of the points end up listed in the order of increasing angl… nope, we were

never supposed to use that bad word that Old Nick himself conjured up to lead us astray from our path of righteousness! Try again. Points end up sorted counterclockwise relative to the reference point in the lower left, which is why this method is called `sortCCW`.

Since the resulting polygon's boundary can contain both left and right turns, the fan polygon is not necessarily convex. It is guaranteed to be a star-shaped polygon, though, meaning there exists at least one point inside it from which you could "see" every other point inside that room by turning around in place. The so-called "star" shape of such a polygon might not even resemble a cartoon blammo star. These names are meant to bolster our intuitive feel for the abstract subject and enlist the parts of our brain that handle spatial intuition to help us understand such concepts. Convex polygons are a further special case of star-shaped polygons in which every point works as the standing point. Since the constructed polygon contains all the original points, it will also have redundant corner points where three consecutive points are collinear.

We can quickly convert a fan polygon into a convex polygon using several algorithms; we will implement the Graham scan, as described in the literature. Other algorithms also exist to find the convex hull of the given set of points. In fact, the convex hull problem is a two-dimensional generalisation of the one-dimensional **comparison sorting** problem, a fact humorously highlighted by how the famous convex hull algorithms **quickhull** and **mergehull** are straightforward two-dimensional generalisations of the classic quicksort and mergesort algorithms originally designed for efficient one-dimensional comparison sorting!

(For a reduction to the other direction, to sort a series of integers $x_0, \ldots, x_n$, simply construct the convex hull of the set of points (x_i, x_i^2) and read the answer in order around the hull. The existence of this reduction, even if never used in practice, shows that the convex hull cannot be constructed essentially faster within the same model of computation than sorting an array of one-dimensional numbers.)

We implement the Graham scan algorithm in this lab because the previous `sortCCW` method already did half the work. This also lets us use a last-in, first-out stack in the implementation, which makes this lab an even more useful exercise in programming in general. Perhaps because of their deceptive simplicity, LIFO stacks are not always fully appreciated in the first- and second-year computer science curriculum, whereas queues and priority queues tend to be more useful in algorithms encountered during that period.

```
public static int[] grahamScan(int[] xs, int[] ys)
```

This method should start by calling the previous method `sortCCW` to acquire the fan polygon listing of the set of points given in the parameter arrays `xs` and `ys`. It should then create a stack (you can use the actual `Stack<Integer>` in the `java.util` package, or simply simulate a stack with an `int[]` and an index variable, canonically named `top`, whose value equals the current size of the stack) and push the first two indices of the fan polygon onto this stack. The rest of the algorithm

should then loop through the rest of the points `i` of the fan polygon and obey the following rules of what to do for each such point:

1. If the stack contains only one element, or if the turn from the current top two elements of the stack to the point `i` is left-handed, push `i` onto the stack.
2. While the turn from the current top two elements to the point `i` is collinear or right-handed, pop the topmost element of the stack.

Implement the second rule as a nested while-loop inside the outer for-loop through the fan polygon points, because in principle there is no upper limit on how many points the new point can theoretically eliminate from the stack. You might want to watch some animation of Graham scan in action to see for yourself why and how this happens. (Relaxing afterwards, you can also ponder why this algorithm is guaranteed to run in linear time despite having two levels of nested loops, since the answer generalises to the study of such algorithms.) Once the outer for-loop finishes processing all corner points of the fan polygon, the stack contains exactly the points on the convex hull. DUCY?

Each of the four labs of computational geometry has a separate JUnit test class to test only the methods of that particular lab. However, the `verifyPicksTheorem` method acts as the Final Boss, challenging students who complete all four geometry labs in this fourth JUnit test class, with the labs uniting to battle the Final Boss like the Avengers suddenly popping out of magic wormholes to defeat Thanos and his cronies. This pseudorandom fuzz test repeatedly generates random points on the plane and uses Graham scan to compute their convex hull polygon. We then compute the polygon's area using both the shoelace method and a brute-force implementation of Pick's Theorem, which computes the area of a simple lattice polygon from the number of lattice points inside the polygon and on its edges. Because both methods return the same area for many randomly generated convex hull polygons, we are more confident in their general correctness.

Determining which points lie inside and on the edges of the convex hull serves as a mass test to verify that `pointInConvex` and `pointInPolygon` always return the same answer. When two totally different algorithms agree on all answers for a large number of randomly generated arguments, our belief in both algorithms' correctness increases massively. Philosophically minded students can point to every item and ask why each one is so.

Lab 45: Permutations I: Algebra

JUnit: PermutationsTestOne.java

This next troika of problem specifications covers the computational operations of permutations, integer arrays of *n* elements that contain each natural number from 0 to *n* – 1 exactly once. The required methods in these three labs may assume that the arguments given to them by the JUnit fuzz tests satisfy this definition. Since we are computer programmers, we shall once again follow the secret pact of our ancient brotherhood of extreme finitists, sworn to Saint Dijkstra always to start counting positions from zero, whereas the fancy pants real mathematicians prefer to define permutations with the numbers 1, …, *n*, as if India's greatest gift to the world had never existed.

Even though permutations are represented as arrays of integers, you can think of a permutation as a `<DrEvil>`"function"`</DrEvil>` that can be "applied" to any array of *n* elements. Since permutations are also such arrays of n elements, you can apply permutations to existing permutations to create new ones, making Keanu say "whoa" once again. This "function" copies each original element exactly once into the result, to the position given by the permutation array. For example, if `perm` is a permutation for which `perm[4]=9`, applying it to an array `arr` where `arr[4]=="Bob"` will see `"Bob"` in position 9 in the result array.

Analogous to matrix multiplication in linear algebra, where the product of any number of *n*-by-*n* square matrices is also an *n*-by-*n* square matrix, combining any number of permutations on n elements yields another permutation of *n* elements. In fact, permutations can be implemented as binary permutation matrices. Hence, each row and column contains exactly one 1, and all the known truths of linear algebra apply to this special case.

(Astronaut 1: "Wait, it was all just linear algebra?" Astronaut 2 [*aiming gun*]: "Always was.")

Enough theory, let's see some action. Create a class `Permutations` in your labs project window. You will be writing all methods in this set of three labs into this class. The methods in this first lab deal with the **algebra** of combining permutations to create new permutations.

```
public static int[] chain(int[] p1, int[] p2)
```

Combine the two permutations `p1` and `p2`, assumed to be of the same size, into a result permutation `p` that represents the operation of first applying permutation `p2` to the given array, followed by applying permutation p1 to this intermediate result. For example, if `p2[3]==4` and `p1[4]==9`, then `p[3]==9`, moving the element there instead of first visiting the intermediate position 4.

```
public static int[] inverse(int[] perm)
```

Given the permutation `perm`, construct its **inverse permutation** that, if chained with `perm`, would produce the **identity permutation** for which `p[i]==i` for all positions `i`. For example, when called with a permutation array `{2, 1, 4, 0, 5, 3}`, this method would create and return the permutation array `{3, 1, 0, 5, 2, 4}`.

```
public static int[] square(int[] perm)
```

Construct the square of the permutation by chaining it with itself. This method should be a mere one-liner, once you have written the `chain` method.

```
public static int[] power(int[] perm, int k)
```

Compute the *k*:th power of the given permutation `perm`, that is, the result of chaining `perm` with itself *k* times. For negative values of *k*, this method should return the –*k*:th power of the inverse of `perm`. With the rule that $k = 0$ always gives the identity permutation, this operation is well-defined for arbitrary integer powers *k*, both positive and negative.

Since *k* will get pretty large in the JUnit tests (since the element values never change, permutations play nicer than integer matrices, so there can be no possibility of overflow), you should use the technique of exponentiation by squaring to compute the result much faster for large *k* than you would get by chaining `perm` with itself $k - 1$ times. The base cases are $k = 0$, where the result is the identity permutation; $k = 1$, where the result is `perm`; and $k = 2$, where the result is `perm`^{2}. For higher values of *k*, apply the identities $p^{2k} = (p^2)^k$ and $p^{2k+1} = p(p^2)^k$ that cut the exponent in half in each step. This should make the method blazingly fast even for *k* not just in the millions but in the vigintillions.

perm	k	Expected result
`{1, 0, 2, 3}`	`4`	`{0, 1, 2, 3}`
`{1, 3, 6, 0, 5, 2, 4, 7}`	`98`	`{3, 0, 4, 1, 2, 6, 5, 7}`
`{0, 6, 3, 4, 8, 7, 2, 9, 5, 1}`	`-397`	`{0, 9, 6, 2, 3, 8, 1, 5, 4, 7}`

Students taking a course in abstract algebra can see how permutations and their operations, chain and inverse, define an algebraic group: chain stands for $p_1 + p_2$, and inverse stands for $-p$, with the identity permutation acting as the unit element. Powerful algebraic truths and useful algorithms, such as **exponentiation by squaring**, spring immediately from this realisation. (Besides, permutation groups are a big cheese in group theory, thanks to Cayley's theorem.)

Lab 46: Permutations II: Cycles

JUnit: PermutationsTestTwo.java

Continuing the previous lab, a permutation is an array of positions that describes where each element moves when you apply the permutation to an array. The *n*-element array therefore already implicitly contains all possible information about that permutation. However, to compute some interesting properties of the permutation and its behaviour more easily, it is often handy to break it down into disjoint **cycles**. For a general idea of how these cycles work, you should first watch the YouTube video "Cycle Notation of Permutations" at Socratica. Mathematically minded students who are interested in how cycles tend to work out in random permutations can check out the post "Random Permutations, Part 3" by John Baez.

To find the cycle of `perm` that the position `i` is part of, start a loop from position `j=i`. Keep moving from the current position `j` into the position `perm[j]`, until you return to the starting position `i` and have met all the other members of that cycle along the way. (It is also quite possible that `perm[i]==i`, making that cycle a **singleton**.)

```
public static List<List<Integer>> toCycles(int[] perm)
```

Since every cycle in `perm` is a list of integers, the list of such cycles must therefore be a list of lists of integers. This method should return the **canonical cycle representation** of `perm` as disjoint cycles, with each cycle listed starting from its largest element and the cycles listed in ascending order of their first elements. The following table showcases the expected result for some random permutations for various *n*. As the second and third rows show, a permutation can consist of a single cycle in which all these elements ride together, as if piled onto one of those tandem bikes so large it becomes comical. At the other end of this lattice poset, every position rides its own unicycle.

`perm`	Expected result
`{1, 0, 2}`	`[[1, 0], [2]]`
`{2, 4, 1, 0, 3}`	`[[4, 3, 0, 2, 1]]`
`{0, 1, 2, 3, 4}`	`[[0], [1], [2], [3], [4], [5]]`
`{0, 5, 2, 3, 4, 1}`	`[[0], [2], [3], [4], [5, 1]]`
`{4, 6, 1, 5, 3, 0, 2}`	`[[5, 0, 4, 3], [6, 2, 1]]`
`{12, 7, 13, 1, 2, 5, 11, 0, 10, 14, 3, 6, 9, 4, 8}`	`[[5], [11, 6], [13, 4, 2], [14, 8, 10, 3, 1, 7, 0, 12, 9]]`

Since the permutations and their canonical cycle representations are one-to-one, we should naturally also write the inverse function that constructs the permutation from the list of its cycles.

```
public static int[] fromCycles(List<List<Integer>> cycles)
```

Automated fuzz testing is trivial to implement when we have both a function and its inverse available as black boxes. Repeatedly create a permutation (either systematically or randomly), convert it to cycle form, translate that cycle form back to the explicit permutation that it represents, and verify that this permutation equals the original permutation. Fuzz tests can run overnight to strengthen our faith in the methods when we see no discrepancies with our morning coffee. Trying a million random permutations at different scales should, realistically, eventually hit and reveal all possible edge cases and corner cases this method could reasonably have lurking inside it.

The **algebraic order** of a permutation, the smallest positive integer *k* such that `power(perm, k)` is the identity permutation, becomes easy to compute from the cycle representation. In fact, computing a permutation power from the previous lab would have been much easier in cycle form, since we could shift each cycle `k%c` steps to the right, where c is the cycle length. Since the identity permutation keeps every element in its original position, the order `k` is simply the **least common multiple** of the permutation's cycle lengths. For example, if a permutation consists of cycles of length 6, 4, 1, 1, and 2, their least common multiple, 12, returns every cycle to its original position.

Parity is another important algebraic property that you can compute easily from the cycle representation. This parity tells us how many pairwise swaps of two distinct elements are needed to implement the permutation. You can organise these swaps in many ways, but whether you need an odd or even total number of swaps to reach the goal is an inherent, fixed property of that permutation. (Try it for yourself if you don't believe this. You will inevitably get stuck, no matter which way you twist and turn.)

```
public static int parity(List<List<Integer>> cycles)
```

To compute the parity of the permutation given in cycle form, count how many individual cycles have an even number of elements! If there is an odd number of such even-length cycles, return `-1` for the odd parity, and otherwise return `+1` for the even parity, hey bada bing, bada boom. The number of odd-length cycles is irrelevant for this calculation.

The somewhat "odd" terminology where minus one stands for odd and one stands for even, even though both of these are pretty odd numbers, comes from the term $(-1)^n$ that often pops up in combinatorial formulas, especially those that involve the inclusion-exclusion principle where alternating terms are added to and subtracted from the total tally. Values of $(-1)^n$ alternate between –1 and +1 depending on whether *n* is odd or even, which is why this context uses the term parity. Of course, this expression should always be evaluated with `n&1==0?+1:-1` in guaranteed constant time, regardless of n's sign or magnitude.

```
public static String cycles(
  List<List<Integer>> cycles, String alphabet
)
```

The last method in this lab creates a human-readable representation of the permutation's cycles. Instead of using integers 0 to *n* – 1 separated by commas and spaces, the `alphabet` string tells which character to use for each such integer. Using the digits `0-9` as the first ten characters, the lowercase letters `a-z` as the next twenty-six, and the uppercase letters `A-Z` as the next twenty-six lets us stringify these permutations without commas. Parentheses separate cycles, even though they're redundant and could be removed to make the representation more compact. The parentheses make the expression easier for humans to digest at a glance.

`perm`	Parity	As canonical cycles
`{1, 0, 2}`	`-1`	`"(10)(2)"`
`{2, 4, 1, 0, 3}`	`+1`	`"(43021)"`
`{0, 5, 2, 3, 4, 1}`	`-1`	`"(0)(2)(3)(4)(51)"`
`{13, 10, 12, 1, 4, 6, 5, 11, 14, 7, 8, 0, 3, 2, 9}`	`+1`	`"(4)(65)(e97b0d2c31a8)"`

Lab 47: Permutations III: Lehmer Codes

JUnit: PermutationsTestThree.java

Every first course on combinatorics starts with how n distinct elements can be arranged into exactly $n!$ permutations, the product of all positive integers up to n. For example, $5! = 1 \times 2 \times 3 \times 4 \times 5 = 120$. Well-known techniques exist to generate all these permutations one at a time in various orders. If you would first like to fool around with some classic techniques to systematically generate all permutations, check out the page "Generate Permutations" at Combinatorial Object Server.

In this lab, we look at an interesting one-to-one conversion between permutations and integers using Lehmer codes. To explain how Lehmer codes work, we first need to examine how integers can be represented as sequences of digits. In our everyday **base-ten positional number system, each digit's position indicates the power of ten it multiplies in the total sum**. In base ten, we don't need digits greater than nine, since ten of some power of ten equals carrying one to the next higher power. For example, we say "nine hundred", but we don't say "ten hundred", since the latter carries to the next column to become "one thousand". (Yet in English, we still do say "eleven hundred"…) Humanity has historically agreed to use base ten since our hands literally have ten *digits* (you get it now?), so we tend to blithely assume ten is the only possible base by some law of Nature and Nature's God. But the same principle works just as well for any other base, most importantly for base two for **binary numbers**.

This base does not even need to stay the same for every position. **Mixed-radix representations** let you use different bases in different positions. We do this routinely without even realising it whenever we talk about times and dates in mixed bases of 365/24/60/60/1000, depending on which combination of days, hours, minutes, seconds, and milliseconds we are talking about. The base in each position still determines the largest digit that can appear there. For example, since every hour has sixty minutes, we must say "one hour and fifteen minutes" instead of "seventy-five minutes" so that values that grow too big for their britches get carried over to the next column.

Encoding each permutation as a unique nonnegative integer should exploit the fact that exactly $n!$ permutations can be constructed from n elements. This can be achieved with the factorial number system, whose positions correspond to the factorials 1, 2, 6, 24, 120, 720, … (The linked Wikipedia page starts the factorials from zero, but that is redundant here because $0! = 1! = 1$.) The largest digit allowed for the position corresponding to $k!$ is k, since if you have the term $k!$ repeated $(k + 1)$ times in the sum, you already have the next factorial $(k + 1)k! = (k + 1)!$ in your hands without realising it. These factorial positions carry their spillage over to the next position, analogous to the positions in the ordinary positional number system in base ten and other bases.

The following methods implement the conversion between long integers and their factorial number system ("**factoradic**") representations. Since these methods are inverses, the JUnit fuzz test only needs to generate random long keys to verify that the functions cancel each other out.

```
public static void toFactoradic(long key, int[] coeff)
public static long fromFactoradic(int[] coeff)
```

Express the integer `key` as a sum of factorials so that the position that corresponds to $k!$ may be used at most k times. Note that the `toFactoradic` method does not return anything, but writes the answer into the array named `coeff` provided by the caller. (The caller is responsible for this array object having sufficient length to contain the answer.) This method should fill in the `coeff` array so that each `coeff[k]` tells how many times the factorial $(k + 1)!$ appears in the breakdown of the integer `key` as a sum of factorials. For example, 2168 breaks down to 2! + 3! + 3(6!) that you can verify is equal to to $2168 = 1 \times 4 + 1 \times 6 + 3 \times 720$.

key	Expected result (to be filled in the prefix of `coeff` array)
8	{0, 1, 1}
22	{0, 2, 3}
101	{1, 2, 0, 4}
2168	{0, 1, 1, 0, 0, 3}
61319700746071	{1, 0, 1, 1, 1, 6, 2, 7, 6, 8, 7, 4, 5, 13, 14, 2}

To see how we can use this representation to uniquely encode permutations of n elements as integers from zero to $n! - 1$, we need the concept of an **inversion** in a permutation. In your later courses on data structures and algorithms, this same concept will pop up again in analysis of the running time of the **insertion sort** algorithm, provably the best comparison sort algorithm in the situation where you are only allowed to compare and swap adjacent elements. A **right inversion** in the permutation `perm` consists of two positions `i<j` such that `perm[i]>perm[j]`. Intuitively, the elements in positions `i` and `j` are "out of order" relative to each other compared to the identity permutation, where all elements are in order, and the permutation has zero inversions. An n-element array in descending order has the largest possible number of inversions: $n(n - 1)/2$, the "n choose two" ways to choose any two positions among n positions without replacement when the order of the chosen elements doesn't matter.

The **inversion count array**, whose i-th element is the number of right inversions the element at position i participates in (that is, how many elements in later positions $j > i$ are smaller than the element at position i), uniquely identifies each permutation. Even better, you can reverse these inversion count arrays and convert them to integers with the previous two methods! The next two methods will perform the conversion back and forth between permutations and their inversion count arrays for you to write in the `Permutations` class.

```
public static int[] toLehmer(int[] perm)
public static int[] fromLehmer(int[] inv)
```

The following table contains some randomly generated permutations and their expected inversion count arrays. Since the last element of the inversion count array is always zero, we leave it out. Hence, the inversion count array is always one element shorter than the original permutation that generated it. For example, the permutation `{5,6,2,3,1,0,4,7}` has five elements after 5 that are less than 5, five elements after 6 that are less than 6, two elements after 2 that are less than 2, two elements after 3 that are less than 3, and so on.

perm	inv
{0, 1, 2}	{0, 0}
{0, 3, 4, 1, 2}	{0, 2, 2, 0}
{1, 3, 4, 5, 0, 6, 2}	{1, 2, 2, 2, 0, 1}
{6, 3, 2, 4, 1, 0, 5}	{6, 3, 2, 2, 1, 0}
{5, 6, 2, 3, 1, 0, 4, 7}	{5, 5, 2, 2, 1, 0, 0}
{3, 14, 13, 11, 4, 1, 5, 10, 9, 8, 12, 6, 2, 0, 7}	{3, 13, 12, 10, 3, 1, 2, 6, 5, 4, 4, 2, 1, 0}

Converting the permutation to the inversion count should be pretty simple with two nested loops, but reconstructing the permutation from the inversion counts is a nice little programming exercise in loops and arrays. Enjoy.

The last two methods in this lab combine these two operations into back-and-forth conversion between permutations and integer keys. Note that the method `fromKey` needs to know not only the key but the number of elements *n* in the permutation, since the same keys mean different permutations when applied to different values of *n*.

```
public static long toKey(int[] perm)
```

This method should first call the previous method `toLehmer` to compute the inversion count array for `perm`. It then turns this inversion count into a scalar integer value by applying the method `fromFactoradic` to the **reversed** inversion count array. (It is somewhat disconcerting to realise that after almost three decades of existence, the `Arrays` utility class in the Java standard library still does not offer us a utility method to reverse an array.)

```
public static int[] fromKey(long key, int n)
```

This method should use `toFactoradic` to convert the key into its factoradic form, then reverse it and pass it to `fromLehmer` to produce the original permutation.

Lehmer encoding maps the permutations of *n* elements into integers from 0 to *n*! – 1 in **ascending lexicographic order**. For example, when *n* = 4, the lowest permutation `{0,1,2,3}` maps to the

lowest key 0, and the highest permutation `{3,2,1,0}` maps to the highest key 23. Clever techniques seen in Lab 72, "Lazy Generation Of Permutations", iterate through *n*! permutations in lexicographic and other useful orders, spitting these permutations out one at a time so that you won't run out of memory even trying them all for some *n* for which the iteration itself would be an overnight run. On the other hand, Lehmer's correspondence between permutations and integers lets us quickly jump to any arbitrary position in the sequence of permutations, even though this sequence is longer than the proverbial year of famine.

Readers whose interest in permutations has been sufficiently piqued by these three labs might want to check out the author's old example class `Permutations` that produces permutations one at a time as a lazy `Iterator<List<Integer>>` with recursive **backtracking** converted to iteration so that each call to the method `next` continues this simulated recursion from the place where it left off in the previous call. To save memory, it returns the same `List<Integer>` object every time, with its elements rearranged. The object seen by the user code has been silently decorated with a `Collections.unmodifiableList` decorator to prevent the outside world from messing up the contents of this list. (That's another nifty little OOP trick that comes in handy whenever you need to expose internal implementation details to the cold, inhospitable winds of the outside world for efficiency.)

This generator can be given an optional `Permutations.Predicate` object that acts as a **filter** to let through only the permutations that satisfy that predicate. For example, we can generate all permutations of *n* elements where each element moves at most k steps from its original location. Or generate only the alternating permutations where the element values zigzag up and down, such as `{1,6,3,5,2,4,0}`. The backtracking algorithm generates each permutation only until the filter predicate rejects the current prefix, avoiding wasted time exploring completions that would still violate the filter.

Lab 48: The Curse Of The Clumpino

JUnit: ClumpsTest.java

All serious programming languages include the "A-list" data structures, such as sets and lists, either built into the language or, at the very least, in their standard library, since these structures are immensely useful for solving problems across all domains. However, not every data structure ever invented earns entry into this exclusive club. In this lab, we will implement the disjoint set data structure, one of the lesser-known "C-list" data structures that never got its big break to the major leagues, and then slowly drifted back to working at his old man's used car dealership back in his Rockwellesque hometown, every Friday night crying into his beer to anyone who would listen about how he could have been a *contendah* back when he was given the chance to speed up this more famous algorithm that *totally promised* to take him to the B-list as its plus one. The disjoint set data structure is still worth appreciating on its own, even if we never actually implement any of the classic algorithms or solve the myriad programming problems that benefit from this plucky little hometown champ.

Create a new class `Clumps` in your labs project folder. Each object of this class represents some **partition** of numbers from 0 to $n - 1$ into **disjoint clumps** so that at any moment, each number belongs to exactly one clump. Initially, every number belongs to its own singleton clump. Clumps are just disjoint subsets, but in this lab we'll use the more colourful word "clump" to emphasise how we crudely clump these subsets together to produce new, bigger clumps, as if they were made of clay. Before looking at the implementation details, here are the public methods that define what this data structure will do for its user code.

```
public Clumps(int n)
```

The constructor initialises the data structure for integers 0, …, $n - 1$.

```
public boolean sameClump(int a, int b)
```

Determines whether the integers `a` and `b` are currently part of the same clump.

```
public boolean meld(int a, int b)
```

If `a` and `b` already belong to the same clump, nothing happens. Otherwise, this method combines the two clumps that contain integers `a` and `b` into a single clump that contains all integers that were originally in the same clump as either one of the arguments `a` or `b`. This method should return `true` if the clumps of `a` and `b` were disjoint but became the same clump as a result of this operation, and return `false` if `a` and `b` were already part of the same clump so that nothing happened in this meld.

```
public int clumpSize(int a)
```

Counts how many numbers are currently in the same clump as the integer `a`, and returns this count. (Code inside this method will not actually be counting anything, but will simply look up the value from the `size` table that the other methods continuously update as they mutate the data structure.)

These clumps can only grow larger when they `meld`, since no clump ever breaks apart into smaller clumps. Eventually all numbers belong to the same clump, after which point nothing can ever change. This **monotonicity** often lets us take liberties in the data structure design, allowing a solution that would not tolerate **entropy-increasing** operations such as "remove" or "split". The disjoint set data structure allows the methods `sameClump`, `meld` and `clumpSize` to run in time that, for all purposes both practical and impractical, is a small constant independent of `n`. Even though one `meld` operation might theoretically combine two clumps of size $n/2$ into a new clump of size n, updating this data structure still takes constant time, independent of n.

It may seem strange that updating a data structure with one trillion elements needs no more time than updating a data structure with one hundred elements, so let us now pull away the cheap plywood and the black velvet cloth that has been to the cheap laundromat a couple of times too many to uncover the mundane secret inside this magic trick. To allow the previous operations to be implemented efficiently for arbitrary clumps, the current partitioning of elements into clumps will be encoded in two arrays

```
private int[] parent;
private int[] size;
```

We initialise these fields in the constructor once we know n and need arrays large enough to hold it. We initialise the actual elements by setting `parent[a]=a` and `size[a]=1` for each number `a` from zero to $n - 1$.

Every clump has exactly one **representative**, a kind of *primus inter pares* that could, in principle, be any number currently in that clump, chosen arbitrarily to shoulder the responsibility of representing the entire clump. Which number in that clump assumes this role depends on the precise order of `meld` operations that led the structure to its current state. The number `a` is the representative of its clump if and only if `parent[a]==a`. Otherwise, to find the representative of the clump that the integer `a` currently belongs to, keep looking for the representative of `parent[a]` the same way until you reach the representative of the clump, which you recognise because it equals its own parent. (And, like in the song, is also its own grandpa.)

This operation of looking for the representative of the given number inside its clump should be implemented as a private helper method

```
private int findRepresentative(int a)
```

that follows the parents all the way up to find the representative of the clump that contains the integer `a`. After this representative has been found, this method should perform **path compression**

as explained in the Wikipedia article about the disjoint set data structure, since this cleanup operation costs us essentially nothing while it will massively speed up future queries for numbers whose parent chain ever leads to the representative through this same waypoint number. To perform this path compression, simply follow the parents from `a` to its representative `r` the same way as before, and reassign `parent[b]=r` for every integer `b` along this path.

The method `sameClump` should at this point be a one-liner that merely checks that its parameters `a` and `b` have the same representative.

The array `size` tracks how many numbers belong to each clump. However, this `size` information is maintained correctly only for those numbers that are actually representatives of their own clumps. The value of `size[a]` for any other number `a` is never needed, so we don't waste time updating it.

The method `meld` should first check whether `a` and `b` are already in the same clump, and return `false` to indicate having done nothing to nobody in that case. Otherwise, find the representatives of `a` and `b` in the structure; let us call these representative numbers `ra` and `rb`. In principle, either one of the two assignments `parent[ra]=rb` or `parent[rb]=ra` would combine these two clumps into one. Still, it is more efficient to make the representative of the larger clump the parent of the representative of the smaller clump, to rein in the growth of the implicit tree structure in these `parent` values. Don't forget also to update the `size` information for the representative that you chose to make the parent of the other representative.

Once the `Clump` object for partitioning `n` elements has been created, each operation `sameClump`, `meld` and `clumpSize` works in **time that is, for all practical purposes, a constant** independent of the total number of elements `n`. There are some technical details best left for the second course on algorithms, such as the fact that this seemingly “constant time” is not actually a constant for all real-world cases, but a constant plus a certain really weird little function called the “inverse Ackermann" whose exact nature is somewhat difficult to explain at this point in your computer science studies. This function depends on *n* and grows without bound as *n* increases, yet it grows so excruciatingly slowly that even glaciers zoom past like light beams by comparison.

The author can guarantee without any hesitation that you will not find a slower-growing function than this one anywhere in any of the materials that comprise your undergraduate computer science education. Treating this function as an infinitesimally small rounding error in the constant running time will never cause any hassle in any possible counterexample, not just in our real world but in any imaginable one. When functions get weird, the weird turn pro.

Lab 49: Just A Minor Operation

JUnit: BitwiseStuffTest.java

♬ *So they know everything you code is* `true`,
Turn it on, don't turn it on, turn it on. ♬

— Float Division, "No Bits Lost"

Dealing with bits and bytes directly in Java gets us closest to the machine executing our code. We prefer to think about execution and a program's general meaning using higher-level languages and abstractions, so this lab is both character- and morale-building, even for students with no plans to take lower-level coding courses. Furthermore, knowledge of bitwise arithmetic in Java translates directly to every other programming language and environment, so students learn this once. They can use it productively no matter where their lives take them after this course. (The same principle of learning something once and then getting to use it everywhere also applies to **regular expressions**, equally indispensable in their own problem domain of text pattern matching.)

Depending on your attitude and the outcome of the random coin flips at the inflexion points of your future life, this lab could be either the most important lab you ever complete from this entire problem collection or the least important. There is no middle ground. Unfortunately, we don't yet know which extreme will play out, but having come this far, are you willing to take that risk and not spend all your time learning a killer's art? As you will learn in a future computer science course that you should take around the fourth year, whichever name that course might have in the institution where you study this, many computational problems have this kind of curious nature in that we cannot hurry up to get the result faster than helplessly waiting for the system to produce the result for us eventually. Such problems aren't limited to what we capture in computing; look at them through the eyes of someone standing outside, peering into the cage of a rare creature in a zoo. Even actually useful problems feel the pinch!

High-level languages such as Java and Python allow programmers to talk about integers and their arithmetic operations in their Platonic essence without knowing how these integers are actually stored, warts and all, inside a computer as aggregates of typically four bytes (`int`) or eight bytes (`long`), let alone requiring the programmer to perform bitwise arithmetic to juggle these bits and bytes around to achieve the desired results. However, this lab matters not just for general knowledge of our shared reality underneath, but also for the practical time and space optimisations this instrument allows in the hands of those who know how to insert it deep into the computer's memory. We should occasionally get our hands a little dirty in all walks of life, instead of always waiting for the tape to run by passing the buck to some public interface where magical elves do all the grunt work for us. Even if you got the spirit, don't lose the feeling.

The same integer can be represented in different bases, usually 10 or 2, and it is still the same mathematical object underneath, silently and eternally floating in the Platonic realm where all mathematical objects finger-quotes "exist" even before humanity entered the cosmic game. The

integer we normally write in base ten as 42 is still the same in binary as `0b00101010`, making it clear that 42 = 32 + 8 + 2, as you can see from how the expression `42==0b00101010` is `true` in Java. As a convenient shorthand, we say bits with a value of 1 are on, while bits with a value of 0 are off, like light switches or signal flags.

Since the binary notation tends to get cumbersome once we get past our built-in mental limitation of seven plus or minus two things to keep track of, the highly convenient **hexadecimal notation** groups four bits into one hexadecimal digit from `0` to `F`, where the letters `A` to `F` denote the six missing digits 10 to 15 that do not exist in base ten, which goes only up to nine. (In retrospect, perhaps those limeys are onto something with the base-twelve **imperial system**, with the nice bonus that the digits in that base literally go up to eleven!) Since an `int` consists of 32 bits and each hexadecimal digit encodes exactly four bits, every int value can be written as eight hexadecimal digits, such as the famous hexadecimal literal 0xDEADBEEF, along with more juvenile variations like `0x0B00B1E5`. The prefix `0x` tells the compiler to interpret the following digits as hexadecimal, even if they all fall in the range `0-9`, so we don't have to ask them whether decimal is the role they wanted to live. This way, there's never any ambiguity about which integer representation the source code uses. The 64 bits inside each `long` value can be encoded into sixteen snappy hexadecimal digits.

Again, all these integer literals could just as well have been written in base ten, since that integer itself is the same regardless of which one of the infinite names we happen to use to summon it this time, analogous to how "Clark Kent" and "Superman" are two separate names for the same actual object. Hexadecimal notation makes it clear that the integers should be thought of as bit patterns, with no direct interaction between bits in different positions, rather than as ordinary integers whose semantic purpose we normally associate with counting and arithmetic operations.

Bitwise arithmetic operators `&`, `|`, and `~` correspond to the logical operators `&&`, `||`, and `!`, except that the former operate on integers rather than `boolean` truth values. These operations work independently on each bit position, letting us achieve a kind of poor-man's parallelism by computing up to 32 logical operations in parallel, then reducing to a single result, instead of processing them one at a time with a for-loop. To paraphrase Voltaire's famous observation about human nature, we can make an integer tell the truth about itself and others by giving it a suitable mask.

The result of the **bitwise OR** operator, denoted x|y for two integers x and y, is on wherever **at least one** of the operands `x` and `y` is on. This operator can **turn on bits** of the first number `x` regardless of whether those bits were previously on or off, by making the second number `y` a mask that is on in the positions we want to turn on in the result.

The result of the **bitwise AND** operator, denoted by `x & y` for two integers `x` and `y`, is on in all positions where both operands `x` and `y` are on. This operator can **turn off bits** in the first number `x`, regardless of whether those bits were previously on or off, by making the second number `y` a mask that is on in the positions we want to turn off in the result.

The positions of bits inside a 32-bit `int` are numbered 0 to 31 from lowest to highest. Since all Java primitive integer types are signed, the highest bit is always the **sign** bit: 0 means the number is positive or zero, and 1 means the number is negative. The remaining bits in negative numbers don't have the same meaning as in positive numbers, but two's complement encoding ensures zero has only one representation (+0). This sign bit matters when the bit pattern is treated as an integer for arithmetic. Still, it doesn't show up in the hexadecimal notation for integer literals, since hexadecimal notation represents the underlying bit pattern directly rather than its semantic meaning as an arithmetic integer.

To create a mask whose *k*:th bit is on, and all other bits are off, use the expression `1<<k` where `<<` is the **left shift** operator for bits. This is not a **cyclic** left shift; incoming bits from the right are initialised to zeros, and outgoing bits to the left simply vanish. The **right shift** operator is also similarly not cyclic. Still, unlike the left shift, it offers two versions, `>>` and `>>>`, where `>>` preserves the highest-order bit (since that bit denotes the **sign** in **two's complement** signed integer representation). In contrast, `>>>` brings a zero bit into that position. Your choice of which one of these operators you need to use therefore depends on whether you are treating that bit pattern as a signed integer in the rest of your code. The choice is always yours. Bytes and their aggregations have no inherent semantics, since those same 32 bits inside a computer finger-quotes only "mean" whatever you want them to mean. If you treat those bytes as an unsigned integer, they are that; if you treat those same bytes as a signed integer, a floating-point number (but please don't when I am around), or a machine code instruction, they are that. By themselves, bits and bytes are just zeros and ones with no inherent meaning. (Made of ephemeral electric currents, even "zero" and "one" are words made up to bolster our intuition, and these vibrations of gums could as well have been "true" and "false", "black" and "white", or "foo" and "bar".)

You can use the left shift as a shorthand for multiplication by some power of two, 2^k, but the results can be a "bit" surprising if that highest sign bit of the result differs from the sign bit of the original number. This move lets us create masks for a single bit position, and we can combine more complex masks with the bitwise OR operator. To create an integer mask with the *k*:th bit off and all other bits on, use the bitwise negation of the previous mask: `~(1<<k)`. You can often think of such a mask as a spray-painting stencil. (This analogy holds at more levels than just the surface one.)

You can't rest on your laurels now, not when you've got none, so let us finally get on with the methods required in this lab. All the methods below have intentionally been specified to operate on 64-bit `long` parameters instead of 32-bit `int` values, since we now live in the science fiction age of 64 bits anyway, perhaps equipped with tail fins for additional "retro" appeal. Four, sixteen windows, ten in a row. Some art major who has stumbled this far to this bug-infested, dried-out forest when they originally just wanted to dip their toes in a computing course could perhaps turn this into one of those "Virgin vs Chad" memes, at the time of this writing arguably the second greatest artistic achievement of Generation Z. (The highest is the midwit meme, hands down.) The utility class Long contains a bunch of `static` methods for useful bitwise operations that did not make the cut to the

language itself, such as `bitCount`, `rotateLeft` and `parseUnsignedLong`, that you might find useful when writing the following methods in the class `BitwiseStuff` in your labs project.

```
public static int countClusters(long n)
```

Counts how many clusters of maximal consecutive blocks of ones the parameter value `n` contains, and returns that count. For example, for the value of `n` whose binary representation is `0b1001110111000111000000000110000100000110000000011111111110`, the expected result is 8, for the eight clusters of consecutive ones in the number. Note that each singleton one bit with zero bits on both sides is still a maximal cluster that should be counted. Make sure to add up the clusters at the very beginning and the end of the number correctly.

```
public static long reverseNybbles(long n)
```

The term **nybble** (or “nibble" depending on which side of the pond you grew up in) is used to denote the four bits that comprise each half of a single byte, but not, for example, the four bits around the middle of that byte. One **hexadecimal digit** therefore represents and describes exactly one nybble. Given a 64-bit long value consisting of sixteen nybbles, compute and return the long value that contains the same nybbles in reverse order. The four bits inside each nybble should remain unchanged. For example, if you call it with the long value whose hexadecimal representation is `0x63e821ceea5afcc6`, the result is `0x6ccfa5aeec128e36`, the original hexadecimal representation read backwards.

```
public static long dosido(long n)
```

Remain behind, go straight ahead, mess with your values and change odd to even by swapping each bit in every even position with the bit in the next higher odd position, so that positions 0 and 1 are swapped with each other, positions 2 and 3 are swapped, and the dance goes on up to positions 62 and 63. For example, the `long` with hexadecimal representation `0x8074db18c339bafb` would produce the result `40b8e724c33675f7`. For example, its lowest nybble `0xb (0b1011)` becomes `0x7 (0b0111)` in this operation.

```
public static int bestRotateFill(long n)
```

Even though Java doesn't have a cyclic left or right shift operator, these two operations can be handy in some combinatorial problems. These low-rent operations are assigned to play in the alternative venues since they won’t get past the velvet rope of the A-list lounge of the Java language to hang out with familiar superstars such as `+` and `*` so that, like Madonna or Cher in our world, they would be universally known for the character tokens that represent them. (Let alone being so important that, akin to The Artist Formerly Known As Prince, our rulers would add entirely new symbols to the Unicode character set specifically for them!) Here, everything is by design, so we implement these two as static methods, `rotateLeft` and `rotateRight`, in the `Long` utility wrapper class. This method should return the number of steps `k` to stretch all true realities by maximising the

`bitCount` of the **bitwise OR** operation `n|m`, where `m` is the result of shifting `n` cyclically `k` steps to the left. If several values of `k` are equally good, return the smallest such `k`.

Using the same ideas needed to implement the previous four operations, you can implement many other bitwise operations. In the real world, the need for bitwise arithmetic points to low-level programming close to the iron (hardware), where speed and efficiency matter most in this low and dark realm of spirits. Students interested in these sorts of clever optimisations can check out the site "Bit Twiddling Hacks", the classic book "Hacker's Delight", or the free online textbook "Matters Computational" for an encyclopaedic presentation of combinatorial and numerical techniques from the bit level up.

To wrap up this lab specification, we can point out that you can still use the one-letter operators & and | inside logical conditions. In that context, these operators are logical operators that do not short-circuit; that is, they don't skip evaluating the second operand when the first operand already determines the result of the entire expression. The Java language specification explicitly guarantees the strict **left-to-right evaluation** and the resulting **short-circuit** behaviour for the logical operators && and ||.

Otherwise, Java's evaluation order for subexpressions is generally left to the compiler and is not guaranteed to match left-to-right source-code reading. For example, the expression `rng.nextInt(42)*rng.nextInt(99)` is not guaranteed to produce the same result in every environment even under the same internal state of `rng`, since we cannot know which one of the two calls to `nextInt` will be evaluated first! To say nothing of Lovecraftian abominations such as `a=++a+a---a+++--a;` whose runtime behaviour the language can only leave undefined, even though the right-hand expression has a unique parse tree that defines its operations. To ensure your intended evaluation order, such expressions must be taken apart into multiple statements separated by semicolons that establish the sequential order in which these subexpressions are computed.

This invisible danger exists in basically all widely used programming languages, yet no textbook or other material meant for a first-year programming course seems to acknowledge it. For every computer program that is currently used to make decisions that affect us, the version of the compiler that was current at the time that program was compiled and released generated some evaluation order of subexpressions that passed not only the unit and integration tests of that project, but more importantly, the test of time in actual use. One day these programs will be recompiled with the current-year version of that compiler that will then emit some different evaluation order that *never* makes any difference to anything… at least until that initially adamant "never" once again turns out to be "hardly ever" in this creaky old Ship of Fools.

Lab 50: The Weight Of Power

JUnit: PowerIndexTest.java

Consider a weighted voting system, such as a corporate shareholder meeting, where participants wield different voting power, called a voter's **weight**. These voters convene to vote either “yes” or “no” on a motion. For a motion to pass, the weighted total of “yes” votes must be at least equal to the given **quota**. In a typical **majority vote**, more than half the votes are needed to pass the motion. However, in special situations, this quota may be something other than one half, such as when an organisation's rules require at least two-thirds of the votes for a particularly fundamental measure to pass. Sometimes even a unanimous decision is required, such as in some jury verdicts. (As many dramatic movies and special episodes of beloved television shows have taught us, being a one-person majority can be a pretty nefarious position!)

In a weighted voting system, each voter's actual power is not necessarily linearly proportional to their weight. For example, consider weights `[9,9,7,3,1,1]` with the quota of 16 votes required to pass the motion. In this setup, only two situations matter. Whenever the first two voters agree, that already determines the outcome regardless of the other four voters. When the first two voters disagree, the seventh voter acts as a **kingmaker**, determining which of the bigger guys gets their way and again rendering the three little guys irrelevant. The last three voters are **dummies** who might as well stay home, since the outcome tallied with their votes included can never differ from the outcome tallied without them. (The term “dummy” here doesn't mean some sort of imbecile, but a mute, inanimate doll that can't make decisions that affect its external reality.)

To see more clearly how actual voting power can differ from the pure percentage of total votes, look again at the third player, the kingmaker, who steps in when the two big guys disagree. From the kingmaker’s point of view, the bigger the kingdom, the better, since his ability to extract outside concessions in exchange for his cooperation can only grow juicier as the kingdom grows! Machiavellian operators in both fiction and real life throughout history have sought such leverage to move the world in ways Archimedes could only dream of.

In this lab, we compute two power-index measures for weighted voting: first the Banzhaf power index, then the Shapley–Shubik power index. Both computations involve recursing through the combinatorial possibilities relevant to each power index: all **subsets** for Banzhaf, and all **permutations** for Shapley–Shubik. Besides, the last names of these guys just sound so adorable, as if these gentlemen were rubbery characters in a *Mad Magazine* cartoon by Don Martin that depicts these algorithms as delightfully wacky machines with gears and whistles that sound like “Fwababa dabada fwababa dabada” and “Shtoink!” as the ballots chug through.

Both algorithms operate on the concept of a coalition, which is a five-dollar word for what we normally call a **subset** of voters. A coalition **wins** if the sum of its members' weights meets or exceeds the quota. In a winning coalition, an individual voter is **critical** if changing his vote would

make the coalition no longer winning. A winning coalition is **minimal** if every voter in it is critical, so it contains no superfluous voters who just run up the score.

For example, consider the coalition of voters `{0,2,5}` in the previous situation. This coalition is winning, since 9 + 7 + 1 = 17 votes is enough to reach the quota of 16. Voters 0 and 2 are both critical to this winning coalition, since removing either one makes the total fall below the quota. Voter 5, with his single vote, isn't critical to the win, since the coalition `{0,2}` would still win without him. That guy only gets the satisfaction of having rooted for the winning team this time… typically feeling even more triumphant the less important his vote was in achieving this outcome, a tragicomic psychological phenomenon often seen in the fandoms of movies, sports, and politics. The human condition is as mysterious as it is tragic.

The Banzhaf power index for an individual voter is defined as **the number of winning coalitions in which that voter is critical**. We usually normalise these numbers **by dividing by the total number of winning coalitions. Hence,** they add to 1, but in this problem we will keep these counts as integers for simplicity. The simplest way to compute the Banzhaf power index is to recursively generate all possible voter subsets. Whenever you find another winning coalition, increment the counter `out[m]` by one for each critical member `m`. Then recursively generate the coalitions that are supersets of the current winning coalition, since everyone gets points for all winning coalitions in which they are a critical member, not just the minimal winning coalitions. However, if none of the members of the winning coalition was critical for the win, you can save plenty of time by not recursing through the remaining supersets of such redundant winning coalitions.

Create the class `PowerIndex` in your labs project, and there the method

```
public static void banzhaf(int quota, int[] weight, int[] out)
```

that computes the Banzhaf power index of voters with given `weights`. This method does not create and return a new array, but writes the results in the array `out`, guaranteed to be of sufficient length and filled with zeros at the time of call. The recursion is best extracted into a helper method

```
private static void banzhaf(
  int quota, int[] weight, int[] out,
  LinkedList<Integer> coalition, int sum, int n
)
```

where the additional recursion parameter `coalition` contains the voters who have already been taken into the current coalition, the top-level call should initialise this to a new and thus empty `LinkedList<Integer>` instance. The parameter `sum` contains the sum of weights of the elements in the chosen `coalition`. (We could always compute this value by adding up the votes in the current `coalition`, but this way we have it available at each level in one constant-time lookup, much better than looping through the array each time.) Parameter `n` gives the length of the prefix of the `weight` array whose subsets this recursion should visit from the current level downwards.

The recursion that systematically iterates through all 2n possible subsets of the given set of n elements is essentially the same as the recursion used in the **subset sum** problem. The base case is $n = 0$, and nothing needs to be done. At each level in this recursion, if the current `coalition` is winning, so that its votes together add up to at least the `quota`, every critical voter of the `coalition`, without whom the rest of this coalition would not have reached the quota, gets its corresponding counter in the `out` array incremented by one. You must check the criticality of each member of the current winning coalition separately inside the loop where you hand out these points, since the winning coalition constructed so far is not necessarily minimal, but can theoretically contain any number of non-critical gnats.

In non-base cases with $n > 0$, regardless of whether the current `coalition` is winning, this recursion should branch in two directions: one for taking voter $n - 1$ into the coalition, and the other for leaving voter $n - 1$ out of the `coalition`. Once both branches have returned and silently updated the `out` array along the way, this method has nothing left to do but return to the previous level. Many legitimate recursions work this way: the recursive method returns nothing, but a data structure shared across every level of recursion gets incrementally updated as it systematically explores the branches of this implicit recursion tree. Once the recursion has systematically examined all possibilities, the final state of that shared data structure is our final answer to Regis.

The **Shapley–Shubik power index** is computed by summing over all n! permutations of these voters, rather than over their 2^n possible **subsets**. When you accumulate voters' weights in a given permutation from left to right, the voter who makes the cumulative sum reach the quota is the pivotal voter for that permutation. The Shapley–Shubik power index of that voter is simply the number of times that voter is pivotal over all permutations of voters. The straightforward recursive method to compute the Shapley–Shubik power indices uses the same parameters as the previous method for computing the Banzhaf power indices.

```
public static void shapleyShubik(int quota, int[] weight, int[] out)
```

The simplest way to visit all permutations is to define another private helper method that takes additional parameters describing the current subproblem. Such recursion generates each one of the $n!$ possible permutations by extending the current **prefix** in all possible ways to complete the permutation. However, to avoid explicitly visiting all $n!$ permutations inside this method, which would be prohibitive for large n, we can be a bit more clever with the pivotal voters.

```
private static void shapleyShubik(
  int quota, int[] weight, int[] out, int level, int last,
  boolean[] taken, int sum
)
```

The parameter `level` tracks the current recursion level. It starts at zero in the top-level call and increments with each recursive call. The current recursion level tells us how far along the current

permutation's prefix is. Unlike the Banzhaf recursion, which explicitly stores the voters in the current coalition in a list, the Shapley–Shubik recursion uses an array of truth values to track which voters have already been included in the current prefix. The parameter `last` remembers which voter was added to the prefix in the previous level, and the parameter `sum` contains the total weight of the voters in the current prefix.

The base case of this recursion is when `sum` is greater than or equal to `quota`, which must happen in every branch before running out of voters to add to the current prefix. Since this makes the `last` voter the pivotal element of all permutations starting with that prefix, we can add up these permutations in one swoop by incrementing `out[last]` by the number of possible ways to complete that incomplete but winning prefix into a complete permutation. This count is simply the factorial of how many unassigned voters remain, given by `weight.length-level`. You might want to **precompute** all factorials up to 17! that fit inside an `int` without overflowing into a `private static` array for quick lookup during the actual algorithm.

When the total weight of the current prefix is less than the `quota`, this method should use a for-loop to iterate through the voters who have not yet been `taken` into the prefix, and recursively complete all possible permutations that start with the previous prefix extended by the new voter with a recursive call with updated `level`, `last`, `taken`, and `sum` for that extended prefix.

The following table shows some `weight` arguments and the expected correct result from both methods. In all cases, the quota is half the sum of weights, **plus one**. The first row is a numerical example from the Wikipedia page on the Banzhaf method. The third row, constructed from the previous row by adding one more vote to the big guy, represents a dictatorship where one voter outweighs everyone else combined, so both metrics correctly assign all decision power to this dictator. The fourth row shows that equalising every voter's weight gives each voter the same voting power, as expected when nothing external breaks the symmetry between these voters. Note how the power indices for n voters must always add up to $n!$ in the Shapley–Shubik method, when the algorithm implicitly tallies up every possible permutation of these n voters.

`weight`	Banzhaf	Shapley–Shubik
`[4, 3, 2, 1]`	`[5, 3, 3, 1]`	`[10, 6, 6, 2]`
`[1, 2, 5, 8]`	`[1, 1, 1, 7]`	`[2, 2, 2, 18]`
`[1, 2, 5, 9]`	`[0, 0, 0, 8]`	`[0, 0, 0, 24]`
`[42, 42, 42, 42]`	`[3, 3, 3, 3]`	`[6, 6, 6, 6]`
`[2, 13, 17, 26, 30]`	`[2, 2, 6, 6, 10]`	`[8, 8, 28, 28, 48]`
`[10, 24, 24, 32, 40, 42]`	`[2, 10, 10, 10, 14, 14]`	`[24, 120, 120, 120, 168, 168]`

[3, 14, 24, 26, 27, 64, 67, 70]	[2, 10, 14, 14, 14, 58, 58, 62]	[288, 1824, 2592, 2592, 2592, 9888, 9888, 10656]

No matter how many orders of polynomials you shave off the exponential running time of your search, like carving ever thinner slices off an exponentially expanding turkey, you still have an exponential search left. If you previously had enough patience to solve the problem for n voters before this optimisation, you could probably solve the same problem for n + 2 voters after it. Meh. Inch-wormy increases rarely arouse much enthusiasm for the algorithm's future. For large n, randomised Monte Carlo sampling is best, evaluating each term only to the precision needed for the purpose we are grinding these power indices for. That’s the cost of doing business in any inherently exponential problem. To a first approximation, that's basically all of them.

Exponential haystacks can be a bitch to find even one measly little needle in, let alone carefully combing through every possible straw of hay inside that stack like some modern-day Rumpelstiltskin. It probably won’t be worth your while to speed up your computations in this lab by cleverly utilising the fact that two voters with equal weights must also end up with an equal voting power in both systems, since this special case will not appear in the automated tests often enough for this to pay off in total running time. (Some web examples that compute these values by hand use this trick to minimise work, in a similar spirit to integrals in calculus textbooks with carefully selected bounds and coefficients that make terms vanish.)

Interested students can easily find online material on both methods for computing power indices. This material is not necessarily educational only for math and computer science, but for understanding life and social reality in general. For example, C. S. Lewis's essay “The Inner Ring” uses no math, yet still makes important observations about this general topic. (Not everyone is doomed to talk nonsense even when they refuse to do arithmetic, whether in our real world or in Narnia.)

Lab 51: Twelve Angry Primes

JUnit: MillerRabinTest.java

"*The fact is, we all started out as someone's ideal little Platonic baby objects. And a space like natural numbers forces us to become primes or composites. Nothing in between can exist here.*"

— after *Shot Caller* (2017 film)

The **trial division** test used in an earlier lab to determine the primality of a positive integer *n* is a good exercise in basic looping, especially once we realize we only need to check primes up to the square root of n as potential divisors. However, this approach is not feasible for efficient primality testing outside a relatively small range of values of *n*. An important part of this course, as in the previous course CCPS 109 Computer Science I, is to make students stop thinking of one million as a large number and start seeing millions, or even billions, as more like Ralph Wiggum's idea of a large number. ("My daddy is at least a *million* years old!")

Number theory yields faster primality-testing algorithms by harnessing the predictably unpredictable power of randomness to navigate an exponentially hairy thicket of possibilities whose promises we can't evaluate before being forced to commit to exactly one. Suppose you don't even know yourself what you are going to do, like the main character in The Dice Man. No malicious **adversary** can exploit what you do by setting up the problem to make your choices seem reasonable at each turn, yet, in the aggregate, they are guaranteed to lead you to the worst possible outcome.

In this lab, you will implement the necessary ingredients for the Miller–Rabin primality test. Instead of randomization, we will implement a **deterministic** version of this algorithm using ideas from the Numberphile video "Witness Numbers," which you may want to watch before proceeding. Our deterministic Miller–Rabin algorithm is sufficient to conclusively determine the primality of all integers that fit inside a 32-bit integer variable, with no chance of a composite number sneaking by as prime. However, before we implement the primality-testing algorithm itself, we first need a non-exciting helper method that does the heavy lifting behind the scenes. In contrast, the front-man algorithm gets all the kudos and glory, as in so many other walks of life.

```
public static long powerMod(long a, long b, long m)
```

This method should compute and return the **modular power** given by the formula $a^b \bmod m$, where the **modulus** *m* is some positive integer greater than one. Since this method only needs to handle positive *a* and *b*, the modulus operator is the same as Java's integer division remainder operator (%). (For negative numbers, the modulus and remainder operators would give very different results. For example, `-7%3` equals `-1`, whereas -7 mod 3 would equal 2 if Java had a `mod` operator. But again, for this problem, that's moot.)

The massive advantage of computing all your integer arithmetic modulo some tactically chosen m is that the result of every intermediate arithmetic operation will always be less than m. This lets you compute humongous powers of integers without using the `BigInteger` type to represent all their millions of digits explicitly. The final result will still be correct, thanks to the formulas

$$(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$$
$$(a \times b) \bmod m = ((a \bmod m) \times (b \bmod m)) \bmod m$$

That works for any m, be it prime or composite. These formulas let us replace integers with their modular values at every step of evaluation. This way, the numbers won't grow too big, and the result will match the true result modulo m.

(Although this property isn't needed in this lab, it's interesting and important in general: whenever the modulus m is prime, every integer a in the range from 1 to $m - 1$ has a unique modular multiplicative inverse in that same range, denoted by a^{-1}. This multiplicative inverse satisfies the equation $(aa^{-1}) \bmod m = 1$ and therefore allows division within this field. You can compute this multiplicative inverse efficiently with the **extended Euclidean algorithm** without using literal division, which can cause problems with integers. In general, it's a good idea to use multiplications instead of division wherever possible.)

To make the modular exponentiation ridiculously fast even for large values of a, b and m, you should use the technique of exponentiation by squaring previously encountered in the labs that dealt with permutations of integers. When the exponent $b = 2k$ is even, use $a^{2k} = (a^k)^2$ to chop that exponent in half. When the exponent $b = 2k+1$ is odd, use the subdivision $a^{2k+1} = a(a^k)^2$ for the same outcome. Depending on how your logic runs, either as a while-loop or the equivalent **tail recursion**, either one of the equivalent subdivision formulas $a^{2k} = (a^2)^k$ and $a^{2k+1} = a(a^2)^k$ can always be used.

Unlike Python, whose dynamic nature doesn't require the behaviour of any function to be set in stone during compilation (which is why tail call optimizations cannot be done in Python while maintaining the original semantics of that program), the static nature of the Java language has the advantage of allowing the compiler to rewrite all recursive tail calls into equivalent iterative structures that do not allocate a new stack frame. This makes linear recursions effectively identical to iterative while loops. Since the stack size in this problem grows only logarithmically, as with any recursion based on repeated halving, even Python could handle this recursively within the usual recursion-depth limits out of the box, without any tail-call optimizations. The following table showcases some values of a, b and m along with their expected results.

a	b	m	Expected result
17	30	9	1
8	9	3	2
60	1909	56	32

255	428433	307	101
1656	3647549	928	640

Miller–Rabin primality testing relies on finding a negative witness for the positive integer n whose primality we want to determine. The existence of even one such witness… well, why don't we use a more colourful term, "papers check," inspired by this author watching one too many Big Herc videos on YouTube discussing the hard reality of prison life, to help and motivate him to stand up and "do his own time" during the pandemic lockdown. Once the lockdown is over, it's time to get some hard coding time in!

Imagine the number *n* as a hardened criminal who caught a case he couldn't beat, so he was sent up the river for a telephone number sentence. Having settled in, he cheerily greets his fellow crime lads, proclaiming himself to be in good standing in the cold and hard streets of the criminal underworld, meaning that he has never ratted on his fellow criminals, nor has he ever committed certain types of crimes that the men of conviction around him would not look kindly upon. Any failure in this paper check when demanded (and as the late great hardboiled crime novelist Andrew Vachss noted in one of his *Burke* novels, no matter how many years in your sentence, certain parts of it will never take long) reveals him to be nothing but a rat-faced punk who had better slink right into Punk City and never again show his mug in the mainline among the righteous cons.

However, this whole setup changes when n is an OP, a genuine Original Prime whose paperwork and shiv are hooped and ready to go ballistic at a moment's notice, safe in the knowledge that his name has already been carved deep into the cold, hard streets that raised him. No matter which particular underling number the shot caller with the keys to that particular modular pod will randomly send to do the heart check, *n* can keep his head up high, certain that he will stand up under any amount of pressure and interrogation under whatever bullshit charges they might fall behind. An entire train of such challengers can be thrown randomly at *n* without changing this outcome, since the genuine prime convicts will never break into smaller proper factors.

On the other hand, any composite number is guaranteed to eventually crack under the hot lights of factorization and tweet out its factors like a little finch. It is certainly in the criminal subculture's best interest to do its due diligence and quickly reveal any such snakes waiting in the grass. Any composite punks masquerading as unbreakable prime gangsters will soon realize that their life choices have led them straight into a Level Four yard that shows no mercy. A composite number trying to pass itself off as prime has about the odds of a parlay of two coin flips to survive even one heart check, let alone an entire gauntlet of checks coming randomly from every direction. If the number fails any of these tests at any time, every righteous con in the entire block will jump to split his wig with their Occam's shivs! Factors in, factors out.

Hell is other primes constantly staring at you, as a playwright philosopher might put it. In this spirit, the first part of the Miller–Rabin algorithm is the important subroutine

```
public static boolean isMillerRabinWitness(long n, long a)
```

that checks whether n survives the heart check from the challenger a. This method should first compute the two unique integers s and d so that $n - 1 = d2^s$, with d odd. This is easy: repeatedly divide $n - 1$ by two until the result is odd, and use s as a counter for how many times you divided by two. The real challenge begins by computing $p = a^d \bmod n$. If this p equals either 1 or $n - 1$, the convict n has passed the challenge on the spot with an instant knockout punch, and this method can return `true`. This challenger has nothing more to say. Note that n can still be composite even after this instant knockout; further challenges are needed to gain more confidence in n's primality.

Otherwise, the challenge continues with the following operation that will be executed up to $s - 1$ rounds. Recompute the quantity p from the formula $p^2 \bmod n$ for the current p. If the new value of p equals 1, the punk ass composite nature of n has been stripped bare for all to see, and this method can return `false`. If the new value of p equals $n - 1$ instead, the convict has passed the challenge, and this method can return `true`. For all other values of p, the current round is inconclusive, and the challenge continues with the next round of squaring modulo n. The challenged integer n must decisively prove its primality before $s - 1$ rounds are completed, as the prison yard rules declare the challenger the winner after $s - 1$ rounds.

Note the one-sidedness of these rules in that defeating any one challenger does not prove n to be a prime number, yet losing even one challenge conclusively establishes n as a composite. Miller–Rabin primality testing is a randomized algorithm that establishes the primality of the integer n by having it defeat k separate challengers randomly chosen from the range $2, \ldots, n - 1$. The randomness ensures that an adversary can't fool the algorithm by sending an undercover composite n that defeats all challengers from the fixed set of values hardcoded in the algorithm's cell block. Since any composite number trying to disguise itself as a prime has at most a one-in-four chance of defeating a randomly chosen challenger, we can make k large enough that the probability of a composite n sneaking through the entire gauntlet becomes vanishingly small. Once this probability is lower than, say, the probability that a meteorite destroys the computing device during the computation that assumes n is prime, it should satisfy even the most paranoid and ruthless shot caller!

However, as explained both on the Wikipedia page and in the Numberphile video linked above, ditching the randomness altogether by simply using the first twelve prime numbers 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31 and 37 as the fixed set of challengers is guaranteed to reveal the true nature of any composite number that fits inside the top bunk of the `long` integer type in Java. Only God may judge a true prime from the domain of `long`, but every composite is guaranteed to be soon carried by six after these twelve judge it. The method

```
public static boolean isPrime(long n)
```

should therefore send precisely these twelve angry primes to challenge the parameter `n`. If, and only if, these twelve challengers all get their asses handed out to them, the solid "rep" of `n` as a genuine prime number has been conclusively established. Of course, to make this method fast in practice, it

should first try trivial trial division of n with the first twelve primes before engaging in the more expensive Miller–Rabin primality test. For the vast majority of integers from the civilian world unaccustomed to prison rules, this simple check establishes their composite nature.

Once the above three methods pass the JUnit test, congratulations: your hard time is now considered fully served so that you can get back to The World again. This lab is complete, and you can move on with your life. The rest of this problem specification is just additional education for students who are interested in this sort of stuff. You know me, and how I like to ramble.

The randomized Miller–Rabin algorithm for primality testing can be used as a subroutine to generate a random prime number of b bits using rejection sampling so that every b-bit integer is equally likely to be chosen. Simply keep generating random b-bit integers from $b - 1$ random coin flips, with the lowest bit of each number hardcoded to 1. For each such integer, run the randomized Miller–Rabin primality test enough times to determine whether that number hit the jackpot; if so, return it as the randomly chosen prime. Otherwise, just keep generating more b-bit integers until you hit the pay dirt. Since the prime numbers are far more dense among all integers than most people might intuitively assume, this process is virtually certain to hit some prime number soon enough, even with b in the tens of thousands! We are not asking you to implement this method here, since it has already been implemented in the `BigInteger` class as a `static` method

```
static BigInteger probablePrime(int bitLength, Random rnd)
```

The parameter `bitLength` is the number of bits the desired prime number should have, and rnd is the random number generator used to make random choices during this method. (It is unclear to this author what `rnd` exactly stands for in this part of the Java standard library, since `rng` is more standard and clearly means "random number generator". Since the formal name of the parameter does not affect the caller anyway, this change would not break any existing user code.)

As the final philosophical point from this lab, we tend to think of randomness as an enemy whose effects we try to minimize and, preferably, avoid. But as you will learn in many later courses, randomness can also be a good friend and an ally in your corner, if you just allow it to be! Analogous to how we use fire properly down here in the physical world always as a servant but never a master, you should not let randomness wildly burn all over everything inside your program (or even worse, write your entire program randomly), but strictly limit its effects to rage inside some small, well-insulated part of your program while keeping the rest of the program predictably deterministic. This gives you the best of both worlds: the unpredictably raging fire of randomness takes apart what you need taken apart, no matter how tricky and convoluted your Adversary has tried to set it up to hinder your progress.

As you may appreciate by watching this method spit out random prime numbers with thousands of digits in less time than it takes you to snap your fingers for the gesture of such sorcery taking place, the sheer effectiveness of this random prime number generation algorithm might well be the closest thing to magic that we encounter during this course.

Lab 52: Linus Sequence

JUnit: LinusSequenceTest.java

This lab returns to the theme of interesting integer sequences where iterating a simple rule makes the sequence bounce around in a way that straddles order and chaos. The apparent incongruence between the rule's algorithmic simplicity and the almost lifelike complexity that emerges from repeated application can feel adorably comical. It is as if “The Little Sequence That Could”, delightfully portrayed in our imaginations by Zooey Deschanel, Anna Kendrick or some other lovely *ingénue* of equal stature, aspired to become something higher than it has any right to be. We instinctively root for lovable underdogs.

In this spirit, the definition of Linus sequence tries to be as unpredictable as possible, yet this very attempt makes it trivially predictable! This paradox echoes the Zen koan about the ox's tail tip inevitably getting caught in the gate. The sociological phenomenon is that every “hipster” rebel whose trendy, cookie-cutter opinions wouldn't raise an eyebrow in the HR department of any Current Year Fortune 500 company tries hard to be unique and mysterious, yet ends up as Yet Another Nonconformist, in lockstep with everyone else.

Before embarking on coding, this seemingly paradoxical sequence offers another, more general, and potentially useful life lesson. The argument Linus uses in the *Peanuts* comic strip behind the link follows the same basic train of thought that runs deterministically toward wherever its well-worn tracks carry it. As every mentalist knows, most people will unknowingly repeat the same flawed algorithm when asked to generate a “random” sequence of truth values purely mentally, without dice or other physical randomization aids. Somebody whose “random” sequence starts with four equal elements is quite a unicorn among us Earthmen, even though nearly one billion people on Earth ought to do exactly that if every human currently alive were somehow asked to generate four random bits mentally! In this sense, unlike Vulcans, humans are generally notoriously bad at both emitting and detecting randomness. Fortunately, algorithmic techniques exist to combine and convert biased sources of random bits, whether from a man or a machine, into higher-quality sources.

Hoping that the readers will excuse the slightly old-timey convention of using lowercase Greek letters to denote character strings (at least around your third year you should be taking a computer science course where this convention is still used, so that it is not “all Greek to you”), the **maximum repeated suffix** of the sequence s is its longest possible suffix β that allows the breakdown of sequence s into three component substrings $s = \alpha\beta\beta$ where α is the prefix of s before the duplicated suffix β. The maximum repeated suffix exists and is well-defined for any finite string s, since the empty string always works as the suffix β, forcing $s = \alpha$. For example, the maximum repeated suffix β of the string `"1100110101101"` is `"01101"`, where α is the leftover prefix `"110"` that could not be used to extend the repeated suffix.

Create the class `LinusSequence` in your labs project, and there the method

```
public static int maximalRepeatedSuffix(boolean[] bits, int n)
```

that computes and returns the length of the maximum repeated suffix of the sequence that consists of the first `n` elements of the boolean array `bits`. As seen earlier, it is often a good idea to have a method that operates on array arguments to take additional index parameters that make that method restrict its attention to some particular subarray. The following table showcases some *n*-bit `boolean` arrays, written below as character sequences of ones and twos instead of truth values for brevity and readability, and their corresponding maximum repeated suffixes. We might as well use this convention, since the Linus sequence consists of two possible elements that, although represented in our code as boolean values, are always called `1` and `2`.

`bits` (of length `n`)	Expected result
`2211212`	`2`
`1212121212`	`4`
`2112121112121`	`6`
`222222121222221212`	`8`
`2121112211122221121`	`0`
`1121111211121122111`	`1`

The first element of the Linus sequence is 1, and we construct the sequence one element at a time. For fun, let θ denote the Linus sequence of length k constructed so far. The *(k* + 1*)*-th element is always 1 or 2, whichever minimizes the length of the longest repeated suffix of the corresponding extended sequences θ`1` and θ`2`. (There will never be a need for a tie-breaker, since the maximum repeated suffixes of these two extended sequences will always have different lengths. DUCY?)

```
public static boolean[] linusSequence(int n)
```

Computes the first *n* elements of the Linus sequence and returns the sequence as an array of boolean values, where `false` stands for 1 and `true` stands for `2`. The first element of the sequence is in position zero, set to `true`. (Absolute position indices don't matter in this problem anyway, since the rule only talks about suffixes of the sequence generated so far and therefore takes the agnostic position in the Gulliverian schism between zero- and one-based indexing.)

The emergent behaviour of the Linus sequence is largely unknown, and chock full of mysteries far above this humble instructor's pay grade. Appropriately, your instructor first learned about this tricky sequence while browsing the Futility Closet, an excellent blog at the intersection of recreational mathematics and historical curiosities. The category "Science & Math" is probably the most relevant for both the aims and the spirit of this course.

Lab 53: Tchoukaillon

JUnit: TchoukaillonTest.java

Variations of Mancala have been played worldwide but remain relatively unknown in the West. The simple structure of such games, which gives rise to rich complexity in emergent situations, makes for many good exercises in integer lists and arrays. We implement the game's legal moves as methods that operate on arrays and lists representing the current position and the position that emerges after a move. The first course on artificial intelligence, usually taken around the third year, then covers the **minimax algorithm** to find the best move that thwarts the opponent's potential responses.

Until we have that algorithmic machinery, we have to settle for Tchoukaillon (bless you!), one of the simplest **solitaire** puzzle versions of this game. Conveniently for our coding purposes, especially compared to the more convoluted boards used in many two-player variations of Mancala (even ignoring the symbolic decorations and other chrome), the Tchoukaillon board is a simple one-dimensional row whose pits are numbered with natural numbers the same way as lists and arrays. Each position contains some number of discrete but identical seeds, coins, pebbles, men, or whatever word you wish to use to refer to your game pieces. In all methods in this lab, the game board is represented as a `List<Integer>` instance that stores the number of seeds in each position.

Position zero at the beginning of the row is the **safe home** where you try to bring as many of your coins as you can. A legal move in a state consists of choosing a position *k* that contains exactly *k* pebbles; for example, position five that currently contains five pebbles, no more or less. If no positions that fulfill this requirement exist on the board, it's game over, man, game over! Those *k* pieces of seed are sown to the *k* preceding positions, exactly one coin per such position. Each move maintains the total number of pebbles but increases the number safely home by exactly one; the game for *n* men reaches a terminal state after at most *n* moves.

Okay, I will stop that silliness now. From now on, we'll call the game pieces "seeds". Were you one of the lucky few who read the previous paragraphs too fast to notice? Either way, create a new class `Tchoukaillon` in your labs project, and in that class, its first method is:

```
public static boolean move(List<Integer> board, int k)
```

This method grabs the `k` seed from position `k` and sows that handful into the previous `k` positions, one seed each. Note that this method must update the state of the `board` argument object, shared by both this method and its caller the same way as all object arguments are shared in Java, to correspond to the new reality after this move. When called for `{3,1,2,0,4,5}` and `k=4`, the contents of that same list object would become `{4,2,3,1,0,5}`. Redundant zeros created from sowing the last position should be trimmed from the end of the list before returning.

The return value indicates whether the move from position `k` was legal and successful; that is, whether `board` position `k` contained exactly `k` seeds when the method was called. If position `k` contains any other number of seeds, this method should not modify the `board` and should return `false` to inform the caller that the attempted move was not legal.

```
public static boolean undo(List<Integer> board, int k)
```

This method is the **inverse** of the previous `move` method. Given the `board` state after performing the move at position k from some unknown previous state, restore the board to that previous state. The parameter `k` may be past the end of the board, which happens when you try to undo the sowing from the board's last position.

The return value indicates whether the arguments board and k represent an undo of a legitimate move under the rules of Tchoukaillon. The move is legal when the position `k` on the `board` contains zero seeds. Every position lower than `k` contains at least one seed (including position zero, which, for the JUnit test, must contain exactly one seed), so the undo can be performed by collecting these `k` seeds back to position `k` from the previous positions. Otherwise, this method should return `false` without modifying the board. For example, trying to undo move *k*=3 at position `{3,1,0,0,4}` would be a no-op, since position 2 doesn't have a seed to give back for the undo operation.

Your instructor certainly stands high and mighty on his bully pulpit every time he preaches a sermon about the fruits and blessings of immutable types in programming, and how accepting these types and their associated programming style in your heart will make your code ascend closer to its abstract form that resides eternally in the Platonic paradise that is fundamentally out of reach of our mortal minds. To ruin this pastoral idyll, the vile serpent Beelzebub will now slither in and point out the major downside of immutable types: iterating over many separate values with an immutable type requires creating a new object for each value. Furthermore, this functional approach is too alien and at odds with the usual way of thinking, even for seasoned professionals.

With mutable types, the same object or variable can track the current value and compute the next value from it. For example, we can write arbitrarily long for-loops that use a mutable primitive `long` value as their loop counters, and count up to millions of trillions without ever running out of space when the eight bytes of that variable act as our "peephole" to the long (in both senses of that word) sequence of values being iterated over.

Games that let `move` and `undo` operate in place on the same mutable game state, rather than creating a new object for each move, make recursive solution search much easier. If we imagine the Tchoukaillon `board` as a crudely simulated war zone, where these seeds have sprouted into an army of identical little army men, represented as tokens under your command, how many men could you bring home before running out of moves?

This problem, like other search problems that ask you to design a series of moves to solve a given puzzle, can be solved with a recursive search that takes the puzzle's current state as an argument.

The base cases are states with no legal moves, and their values are the number of safe men. Otherwise, the method should loop through the possible moves in the current state and recursively compute the values for the states resulting from each move, easily generated in place by using `move` and `undo` around the recursive call. Return the maximum value of these possible moves as the value of the current state. This recursion should be written as one method

```
public static int value(List<Integer> board)
```

that computes and returns the maximum number of men that can be brought safely home from the given starting `board`. Although it can branch exponentially in principle, this recursion usually runs quickly because most board states have only one or two possible moves, and most choices quickly lead to obvious dead ends. The following table displays some expected results for some initial distributions of these men in the front lines.

`board`	Expected result
`[0, 1, 2, 3, 4]`	9
`[0, 1, 2, 3, 3]`	5
`[0, 1, 2, 0, 3, 4, 4]`	3
`[0, 0, 2, 3, 4, 5, 6, 5, 3]`	16
`[0, 1, 2, 1, 4, 3, 6, 7, 8]`	27

When such a value method is available as a black box, we can easily use it to make the actual moves by consulting the value method for all possible successors of our current `board` and simply making the move the black box claims is best. We need not fear being led astray by the marching orders this box implies, since its individual evaluations will guide us to at least the best result possible, if not the hoped-for best possible result.

When it comes to the puzzle being fully winnable, Tchoukaillon is a rare bird among all such flightless waterfowl in that for every positive integer *n*, there exists **exactly one** possible way to place *n* men on the game board so that it is possible to bring all of them safely home using the moves of Tchoukaillon! Any other placement of these *n* men will eventually quiet the battlefield, leaving at least one man hopelessly stranded behind enemy lines, no matter how you finagle the moves in between.

Since each move will always decrease the number of men outside safety by exactly one, and these men must be arranged on the game board in a way that makes continuing to the solution possible, it follows that these unique solvable states form a linear chain that springs forth from the goal position where all pebbles are safe. From there, the chain reaches up towards infinity by moving from the unique solvable state for *n* men to the next higher unique solvable state for *n* + 1 men, easily generated by simply performing an `undo` to the leftmost empty position `k`, creating a new empty position at the end of the list when no earlier empty positions exist.

```
public static void previousSolvable(List<Integer> board)
```

This method should perform this operation on the given `board`. In the JUnit tests, this `board` is guaranteed to be in some solvable state. Furthermore, to canonize board representations, keep the number of men in the safe zero position at zero. Future actions depend only on men outside safety, never on the **sunk reward** of men who already reached it... and besides, what import are brief, nameless lives to Galactus anyway?

Lab 54: All Hail The Mighty Xor

JUnit: SigVectorTest.java

♬ Clean code, cool tools, and I don't know where I am going to,
Test suite, black box (black box), I don't need to read the do-o-cs,
Functions running as fast as they can
'Cause every core's crazy 'bout a bit wise man! ♬
— $\mathbb{Z}^2\top$, "Bit Wise Man"

The **caret** operator `a^b` has nothing to do with exponentiation in either Python or Java; it denotes the bitwise exclusive OR between two integers a and b when you think of them as bit patterns rather than ordinary integers that allow arithmetic and order comparisons. Unlike the irreversible **bitwise AND** and **bitwise inclusive OR** operators, which tend to erase information as they go, this delightfully symmetric and, most importantly, reversible bitwise operator can be thought of as taking a bit pattern and a stencil mask that flips the bits where the mask is set. This stencil intuition makes it easy to see why applying the same mask twice to the same bit pattern returns the original pattern. This intuition generalizes to show why successive operations with the same mask cancel each other out, even if other masks flip the same bits in the interim. This seemingly magical property of exclusive OR, performing tricks that apparently require memory without actually needing any, has several applications in computer science.

For example, it is occasionally convenient to compute and track some sort of checksum of a large data structure or data clump to speed up equality comparisons. If the checksums of two large objects differ, their payload data also differs. This is especially convenient when the data is **immutable** so that you can **cache** the checksum alongside the data in a separate field. For example, the Java `String` class computes each object's hash code lazily the first time it's needed and caches it so it can be looked up in constant time on future occasions. After paying the cost of each `String` object having another integer field, practically every future negative equality comparison resolves correctly in constant time by a single comparison of these cached hash codes.

For **mutable** data, updating the checksum on the fly at every mutation could be inconvenient. It seems especially wrongheaded to diligently perform some linear-time checksum recalculation after every little constant-time update! To avoid this silliness, this lab explains and implements **Zobrist hashing**, which uses the exclusive OR operator on a set of random integers to generate scrambly, chaotic checksums that update with simple constant-time operations at every data update! This ingenious technique is usually presented on chess and other exciting game boards made of two-dimensional arrays. For simplicity, we'll restrict ourselves to computing these Zobrist checksums for one-dimensional **bit vectors**. Higher-dimensional data, or data from domains larger than binary, can (and always *will*) be encoded into bit vectors anyway.

To kill two birds with one stone, we'll also finally explore Java's mechanism for cloning class instances. In fact, the automated fuzz test will clone your objects as if it were the mid-to-late

nineties again, with the cloning craze all over news and entertainment media. Create a new class with the following signature and the three data fields

```
public class SigVector implements Cloneable {
    private boolean[] bits;
    private long[] keys;
    private long sig;
```

The objects of this class will serve as bit vectors that maintain a running checksum of type `long` to describe their contents. The eponymous array field stores the `bits`, and the `sig` field contains the current checksum. Your class should have the constructor

```
public SigVector(int n, long[] keys) {
    this.bits = new boolean[n];
    this.keys = keys;
}
```

to initialize these fields to create a bit vector of length `n` for which we maintain a checksum.

Zobrist hashing uses a table of random integer `keys`. Since these random `keys` come from outside the class and its methods should never mutate them, we can share the same keys across all instances of this class, avoiding the expensive habit of creating a separate defensive copy for every instance.

To access the individual bits and the current signature, the class should have the methods

```
public long set(int i, boolean b) throws IllegalArgumentException
public boolean get(int i) throws IllegalArgumentException
public long getSignature()
```

The first two methods `set` and `get` the value of bit `i` in this bit vector. The `set` method should return the checksum after setting the bit `i`. Furthermore, whenever the bit `i` flips to a new value, this method should update the current checksum with the assignment

```
sig = sig^keys[i];
```

or more briefly with the shorthand notation analogous to `+=` and other such operators

```
sig ^= keys[i];
```

to mix into the checksum the random key that corresponds to the bit `i`. It is now easy to see that for any sequence of bit-flipping operations, flipping any bit an even number of times does not affect the

final checksum, regardless of the other bit-flipping operations. Especially if all bits are turned on and off in an arbitrary order, the checksum ends up the same goose egg as when it started.

With the checksum mechanism available, the following methods should now be embarrassingly easy to make both effective and efficient. For `hashCode`, you could simply extract any 32 bits of the `sig`, usually the bottom 32 since they tend to change the most erratically. Better still, use bitwise arithmetic to extract the top 32 bits and the bottom 32 bits into an `int`, then return the bitwise XOR of these two halves of a 64-bit `long`. The `equals` method should first compare the checksums, and only if they are equal, compare the actual contents of the bit vectors.

```
@Override public int hashCode()
@Override public boolean equals(Object other)
```

(In fact, the checksum inequality is always sufficient to infer content inequality for all the pseudorandom bit vectors generated during the run of the JUnit test! Still, you shouldn't rely on this happening in general. Even stars occasionally collide despite the vast emptiness of intergalactic space.)

```
@Override public String toString()
```

As so often in the real world, the `toString` method can return pretty much anything that aids debugging instead of hindering it. Finally, implement the important method

```
public Object clone()
```

This method should first call `super.clone` to duplicate the actual bitwise identical clone object, since this method has an **implicit contract** of never using the operator `new` for this purpose. Since the superclass version of `clone` inherited from `Object` performs a **shallow copy**, you need to perform the required **deep copy** by yourself and `clone` those freaking `bits` so that these bit vectors can later be safely mutated without silently affecting any other bit vectors that were supposed to be separate entities.

(Those among us who truly are of pure heart must also actually `clone` these `bits` instead of resorting to `Arrays.copyOf` or similar. Such methods will certainly use the operator `new` behind the scenes, causing us to inadvertently violate the sacred implicit contract passed from the subclass to the superclass without saying a word in the class itself.)

Lab 55: Accumulated Wisdom

JUnit: AccumulationTest.java

Numerical arrays allow constant-time **random access** to individual elements, regardless of position. Summation queries over arbitrary subarrays are less straightforward, since they require looping through the entire subarray to add its elements one at a time. However, with an **accumulation array** constructed from the original array, you can perform arbitrary summation queries over contiguous subarrays in guaranteed constant time, regardless of the subarray's width or position!

In fact, since the elements of the original array can be accessed as trivial summation queries for **singleton** subarrays, you can potentially save a bushel of memory by throwing out the original array once you have constructed its accumulation array. The accumulation array contains all the information in the original array, but in a form better suited for subarray summations. As long as there is no integer overflow, using more bits to store the individual array elements is a reasonable price to pay for this convenience. Now we are cooking with gas.

An accumulation array has the same dimensions as the original array, but each element equals the **sum of all elements up to that particular position**. For example, the accumulation array for the original array `a={4, -2, 3, 1}` would be `acc={4, 2, 5, 6}`. To construct this accumulation array in linear time, we simply need to realize that each element `acc[i]` can be computed with the expression `acc[i-1] + a[i]`, utilizing the fact that the sum of the elements up to the position uses the same terms as the accumulation up to the previous position, with only one more original element included. Set the initial element as the base case: `acc[0] = a[0]`.

In your labs project, create a new class `Accumulation`, and in it, two `static` methods

```
public static int[] accumulate1D(int[] items)
public static int subarraySum(int[] accum, int start, int end)
```

The first method creates and returns the accumulation array for `items`. The second method uses the accumulation array to sum the subarray from start to end quickly. (Yes, even though this method never even sees the original `items` array object!) Also note that these parameters follow the Pythonic convention where the `end` position is **exclusive**, while the `start` position is **inclusive**. So when both parameters are equal, they span an **empty subarray** whose sum is zero.

(After completing the logic for accumulation over one-dimensional sequence of data, the reader might take a breather to philosophize about the potential usefulness of computing a second-order accumulation array whose elements are the accumulated sums of the accumulations of the original elements. Elegant facts await us there.)

Next, we generalize this technique to **grids**, two-dimensional arrays where every row has the same number of columns. Hence, no row is ragged and leaves gaps or protrusions in the array's overall cleanly rectangular shape. Each element `acc[row][col]` of this accumulation array now equals the sum of elements in the rectangular subarray from the **origin** position `[0][0]` up to and including the position `[row][col]`. Note how the previous one-dimensional accumulation is merely a special case of this two-dimensional accumulation done to a two-dimensional array that contains only one row and is therefore effectively one-dimensional. The corresponding methods for you to write are, this time

```
public static int[][] accumulate2D(int[][] grid)
public static int subrectangleSum(
  int[][] accum, int row, int col, int h, int w
)
```

In the second method, `row` and `col` determine the top-left corner of the rectangular subarray of the grid whose elements are summed. Parameters `h` and `w` give the height and width of this subarray, measured in rows and columns. For example, if `h` and `w` both equal one, this sum equals the singleton element `grid[row][col]`.

As in the one-dimensional case, computing each element `acc[row][col]` should not require brute-force looping through the entire subarray. Instead, each `acc[row][col]` should be computed by adding and subtracting a fixed-size handful of previously computed values from the two arrays `acc` and `grid` in a more sophisticated manner that, as a nifty teachable moment for us, also showcases the inclusion–exclusion principle often seen in discrete math calculations!

(By the way, this is one of those problems where you should **literally draw a picture** to help you keep all those parameters and their meanings straight in your head. No rush. I will wait here.)

After these two methods, instead of mindlessly repeating this approach for three-dimensional cuboids, we'll finish this lab in flamboyant style by solving a classic coding interview chestnut. Like all the best problems, this one is simple enough to state in one sentence: Given a two-dimensional `grid` of `boolean` truth values, return the side length of the **maximal square** whose every element is `true`.

```
public static int largestTrueSquare(boolean[][] grid)
```

Every "Shlemiel" would solve this problem by throwing some horrendous amount of brute force at it, in this case **four** levels of nested loops. The outer two loops iterate over the positions of the `grid` that could potentially be the top-left (for a symmetric approach, the bottom-right) corner of the largest `true` square. For each potential corner, the two inner loops gauge the size of the true square emanating from that corner. For arrays randomly filled with truth values from coin flips, this technique's average running time would be pleasantly low. However, the worst case would

absolutely kill us. Simply imagine what would happen to the running time if every element of the original array were `true`!

Any attempt to make the method more clever so that it knows to terminate in this special case can always be thwarted by adding enough `false` values to tactically chosen positions to coax out the **quartic** worst-case behaviour. You must implement the method first; only then will the adversary choose the arguments your method receives. Since this nasty little imp, whom we already know from Murphy's law, will always make everything turn out the worst possible way for your algorithm, the average running time over all possible arguments is meaningless. When you fight against nature, whose randomness aims to help or hurt no one, you can rely on average-case algorithms. Still, when facing intentional hostility, the average is finite, and you must actively defend against the worst case.

We cry out for a more sophisticated approach to defeat the adversary by solving the problem with only two nested loops, whose running time depends only on the grid size and is oblivious to the exact values of the array elements! The adversary can then fill in this `grid` literally any way he wants, and still has no chance to gum up the works by making your algorithm run back and forth in some maximally inefficient and silly manner.

Even though the following approach technically requires the technique of **dynamic programming**, taught in some course normally taken around the third year in a typical computer science undergraduate program, this author hopes that completing the previous methods has psychologically primed the student for that one crucial flash of insight that is needed to implement the efficient and correct technique in this problem. Start by creating a two-dimensional integer array `int[][] s` whose size is the same as the size of the original truth value `grid`. Fill up this integer array so that each element `s[row][col]` gives the size of the largest `true` square whose bottom-right corner lies in that position. Once this array is filled and each element is correct, return its maximum value as the final answer.

The outer two loops are the same as in Shlemiel's solution. However, now ask yourself this: how could the three neighbouring values `s[row-1][col]`, `s[row][col-1]`, and `s[row-1][col-1]` adjacent to the north, northwest and west of the current position be used to compute the correct value of the current position `s[row][col]` without **any** inner looping, assuming that your two nested for-loops have already correctly filled the values of these three neighbouring elements? Once you see this, bang, that's all the fat lady wrote about this problem.

The following optimization is not required for your code to pass the JUnit test in reasonable time. However, observant viewers may still note that to save another big bushel of memory, the array `s` does not actually have to be the same size as the original `grid`. This array can act as a sliding window, keeping track of only two consecutive rows: the current row being filled in the inner loop and the previous row, whose values are needed to fill the current row correctly. Once the current row is filled, you never need its previous row again, assuming you track the overall maximum value seen across all previously generated rows, even those already memory-holed.) The current row

becomes the new previous row, from which the next current row (recycling the space of the now redundant previous row) is computed.

One final optimization lurks inside the previous one. There is no need to explicitly copy the elements of the current row to the previous row every time the sliding window is moved one notch downward. Once we realize that those things that walk, look, and quack like two-dimensional Java arrays are, under the hood, one-dimensional arrays whose elements are themselves one-dimensional arrays, swapping the references to these two rows achieves the desired effect in constant time, regardless of the rows' length.

One final thing that you might want to notice is that this very technique can be used to solve any well-formed recursive equation whose subproblem parameter domains are contiguous subsets of integers. You can even fill in the subproblem solutions in literally any order that guarantees that, at any position, the positions storing the recursive subproblem solutions needed to fill the current position have already been computed. Trivial array lookups can replace recursive calls to solve these subproblems!

A ravishing beauty of the technique, dynamic programming is truly a skeleton key that unlocks a thousand doors that Hercules himself could not pry open with sheer brute force. The hardest part of such problems is finding the recursive equation that describes the system. Once you have written down that equation, you should draw the picture of the subproblems and their recursive dependencies as a grid with arrows pointing from cells to other cells (even if only as a mental image), to make the actual coding stage basically a kabuki act.

Lab 56: `find()` Me A Find, `catch` Me A Catch

JUnit: MatchmakerTest.java

In the famous stable marriage problem in **combinatorics**, a group of *n* boys is to be paired one-to-one with a group of *n* girls. Each boy and each girl has privately **ranked** all the members of the opposite sex in some total preference ordering. Everyone is honest about their true preferences without shame, not trying to go after some hot babe just because that is what the mass media told them to do while every other guy also tries to follow suit, or in this case, a skirt.

An arrangement that pairs each boy to exactly one girl is called a **matching**. When both boys and girls are numbered from 0 to *n* – 1, such a matching can be expressed as an *n*-element array describing which girl each boy is paired with. Since this array must be a **permutation** of *n* elements, its inverse permutation describes which boy each girl has been paired with. This problem is completely symmetric between boys and girls, but is conventionally phrased in line with traditional gender roles so that the boys take turns to approach girls individually.

Suppose the matching contains two pairings, Chad–Becky and Norbert–Stacy, so that both Chad and Stacy would prefer to break off their current pairings and elope with each other, leaving Norbert and Becky to settle for each other. Even one such unstable pair makes the entire matching **unstable**. Note that Chad desiring Stacy (or Stacy desiring Chad) over his current partner does not make that pairing unstable unless Stacy (Chad) is equally willing to break away from her current pairing.

Regardless of how each boy and girl privately ranks the opposite sex, at least one stable matching with no unstable pairs always exists. In fact, depending on how these preferences mesh and conflict, even exponentially many stable matchings may exist! For example, consider *n* = 2 with two boys, Chad and Norbert, and two girls, Stacy and Becky. Chad prefers Stacy over Becky, whereas Norbert prefers Becky over Stacy. So far, no conflict. Unfortunately, the gals have opposite preferences, so Becky prefers Chad and Stacy prefers Norbert, setting the stage for a raucous nineties teen comedy. Both possible matchings are stable, as the reader can verify. However, these two matchings differ drastically in how fairly they satisfy preferences. One matching gives each boy his preferred girl and leaves both girls to settle for their lower-ranked boy, whereas the other matching gives each girl her preferred boy and leaves the boys to settle for their respective lower-ranked girls.

The famous Gale–Shapley algorithm can rapidly construct the stable matching that is optimal for boys and pessimal for girls. In the guaranteed result, each boy skips to the loo with his highest-ranking “best gal” he has any chance of pairing with in any possible stable matching. Conversely, each girl is paired with her lowest-ranking boy she could pair with without making the matching unstable. Since the roles of boys and girls are perfectly symmetric, we could run the mirror-image version to favour girls instead of boys. In our more enlightened times, since the

Swinging Sixties, back when this algorithm was first developed, it also serves as an example of **algorithmic bias** that systematically oppresses a particular subgroup of participants. (As in so many other walks of life, even those who instinctively and rightfully sense the entire system is rigged against them still see only the surface of how deep that rigging goes.)

The Gale–Shapley algorithm makes for an excellent exercise in handling two-dimensional integer arrays for our Java programming course. To visualize this algorithm, imagine boys and girls standing coyly near opposite walls of a high school gym, dressed and made up as if cast in a summer stock revival of *Grease*. After setting up the arrays to track the dance's current state, the algorithm is a single while loop that runs until every boy has been matched to a girl, which must happen when there are at least as many girls as boys.

On his turn, each boy approaches the highest-ranked girl he hasn't approached yet. Since the Bro Code is still decades away from being codified into law, the boy must approach that girl even if she's already been provisionally paired with someone else. If Stacy is still unpaired when Norbert approaches her, they are provisionally paired for now. However, if Stacy has already been paired with someone else, say, Melvin, one of the two boys currently vying for Stacy's hand must slink back to the opposite wall with his shoulders drooped. In contrast, the other boy can stay to twist his limbs in a victory pose, determined by Stacy's preference between her two suitors.

Whichever boy was sent back will get his turn again in the future, but may not approach Stacy again (as any repeated approach would be futile anyway and only deepen his potential future incel rage), so on his next turn, he will be approaching the girl next down the line in his preference rankings. Since each boy's preference ranking contains exactly n girls, and each boy can approach each girl at most once, every boy will be paired with a girl after at most n^2 total approaches, usually far fewer.

In your labs project, create a class `Matchmaker`, and there the method

```
public static int[] galeShapley(int[][] boysPref, int[][] girlsPref)
```

that receives the preference rankings of both boys and girls as *n*-by-*n* integer arrays so that the *i*:th row of each array contains the preference ranking list of the *i*:th boy or girl, respectively. This method should create and return a one-dimensional integer array `boysPair` whose each element `boysPair[boy]` shows which girl that particular `boy` has been paired with. During the execution, you will also need two other one-dimensional integer arrays, `girlsPair` and `boysPos`. The former tracks which boy each girl is paired with, which you need to decide whether to split an existing pair. The second array tracks how far down the preference ranking each boy has descended during the algorithm's execution.

Curiously, the algorithm's final result does not depend on the order in which the boys approach the girls. Confident of the outcome that is set in stone by the iron hand of mathematical necessity from the preference rankings of these individual boys and girls, Chad can be a suave gentleman and nonchalantly comb his hair in his best imitation of Kookie Byrnes. At the same time, he lets Norbert

first approach Stacy first, coyly waiting near the other wall, presumably wearing a pencil skirt and bobby socks, with starry-eyed visions of hunky Chad sweeping her away from chunky Norbert.

The bottleneck for this algorithm's execution time is quickly finding an unpaired `boy`, again, literally any such `boy`. The implementation details of this just-pick-any-one-I-honestly-don't-care-which-one selection are left to the reader; several good ways exist. Furthermore, since this algorithm needs to quickly determine whether Stacy prefers Norbert or Melvin, some preprocessing of the `girlsPref` array might help Stacy decide faster during the algorithm's main loop. The girl that the `boy` will approach in his current turn is then given by the expression

```
boysPref[boy][boysPos[boy]++]
```

that **post-increments** the `boysPos` counter for that particular `boy` so that the next time when his turn to approach a girl comes, he will approach the next girl down the line along his preference ordering. Since Java's pre- and post-increment operators are a highly convenient feature that Python lacks, we might as well smoke 'em since we got 'em, in this rare case where Java syntax is superior to Python's for the everyday programmer.

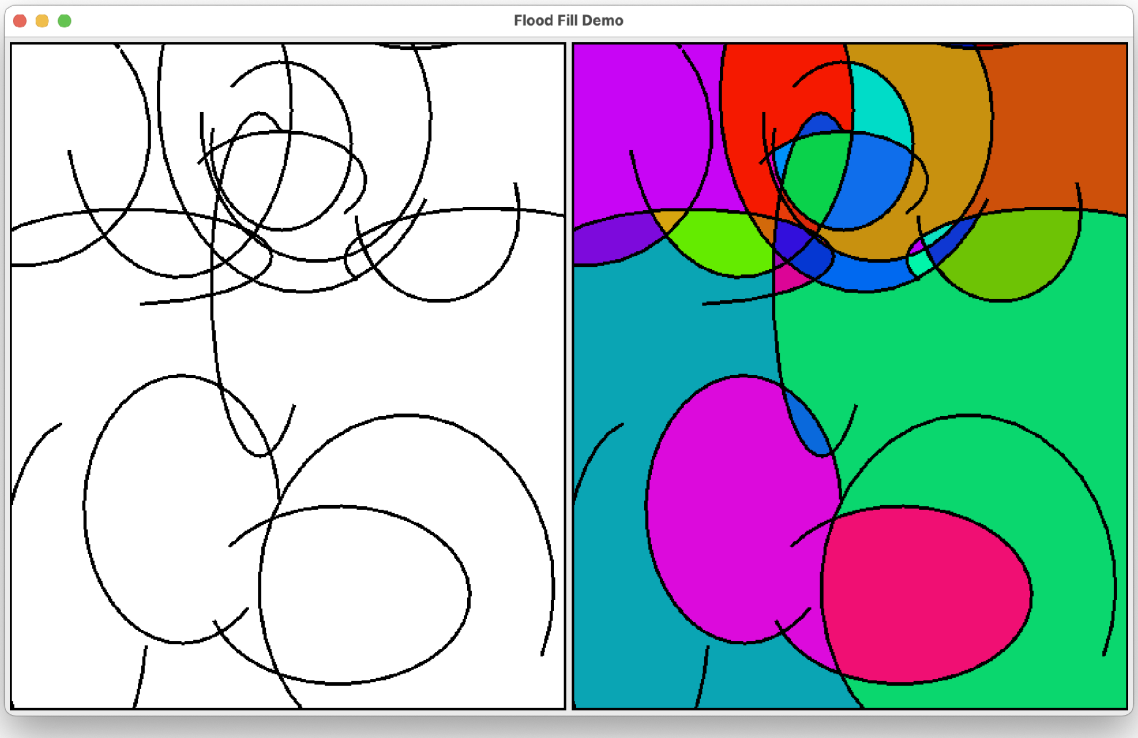
As discussed earlier, the Gale–Shapley algorithm produces an optimal result for the boys and a pessimal result for the girls. Advanced algorithms for stable marriages and other matching problems try to reduce this unfairness.

When *n* boys are to be paired with *m* girls, where *n* and *m* are not equal, we can always invite Rosie and her sisters to balance the ranks. Even so, the terms "boys" and "girls" are mere placeholder names for any **bipartite graph** whose nodes they comprise; for example, **hospitals and residents** in the most famous practical application of the stable marriages problem. This problem setting can be generalized further: instead of everyone meeting everyone else in the gym, each boy knows only a subset of girls, and each girl knows only a subset of boys. Hall's Marriage Theorem, a famous result in graph theory, shows that a matching exists if and only if every subset of boys knows at least as many girls as the size of that subset, and vice versa. (As usual, one of those ifs is trivial while the other one is world-changing.)

The real-world implications of this phenomenon are left to the reader. In the spirit of the Current Year's fluid modernity, this problem generalizes to the euphemistically named stable roommate problem, where participants are not divided into binary sexes, but anyone can be paired with anyone. Unfortunately, no algorithm can guarantee finding a stable matching in all situations, because some preference rankings simply do not allow one. The simplest example is when Andy prefers Bob, who prefers Carl, who prefers Andy, creating a non-transitive preference cycle. All three men equally dislike the leftover man, Dave, whose preferences don't really matter here. (Note the similarity to the famous Arrow's Impossibility Theorem for incoherent group preferences, even when each individual's preferences are perfectly coherent.)

Lab 57: Flood Fill

JUnit: FloodFillTest.java



Given a pixel image with a solid background colour (in the example on the left below, white) divided into disjoint areas by lines, curves, and other shapes in other colours, flood fill is a classic technique for painting all pixels in the same contiguous area. The flood fill algorithm carves out precisely the pixels that belong to the same contiguous area, regardless of the area's shape or the curvature of its boundaries. The area to be painted is defined by any given **seed** pixel (x, y) known to be inside that area. See the screenshot from the `main` method of the automated test for an illustration: the original image of random arcs on the left, and a bunch of flood-fill operations applied separately to its white areas, each using a randomly chosen colour.

The linked Wikipedia page explains the basic flood fill algorithm well enough in pseudocode; your task is to translate it into Java. For once, you don't need to invent the algorithm from the specification of what it is supposed to achieve! However, your flood fill method certainly should not be recursive, since a recursive flood fill that snakes through the white area in the most convoluted way will almost certainly require a large recursion depth and run out of the allocated stack space for recursion bookkeeping. Instead of heading to Stack Overflow with your hat in hand to ask how to increase the recursion limit in your Java Virtual Machine, your method should be completely non-recursive from the get-go and use a **queue** of pixels waiting to be processed.

In your project folder, create a new class `FloodFill`, and there the method

```
public static void floodFill(BufferedImage img, int x, int y, Color c)
```

Given an instance of `BufferedImage` and the pixel coordinates `x` and `y` of the pixel inside the current area, this method should paint over that entire area, and not one pixel more and no less, with the new colour `c`. Instead of the more general `Image`, this method expects its more specific subtype, `BufferedImage`, which provides `getRGB` and `setRGB` methods to read and write the colours of individual pixels. Note that these two methods expect colours as primitive int values, whose four bytes encode the ARGB components of the actual colour, one byte per component. However, this representation shouldn't be a problem, since Java Colour objects include the `getRGB()` method to extract this compact int representation. You don't need bitwise arithmetic to convert colours to bytes and vice versa.

The original colour of the seed pixel (*x*, *y*) becomes the background colour the flood fill paints over. In contrast, every other colour in the image is treated as a wall that prevents the current fill colour from spreading. (If the original pixel colour equals the colour `c`, the fill should therefore terminate immediately.) Note that this method returns `void`, so it should paint into the contents of the original `img` object shared by the caller and this method, rather than returning a new image to represent the result while keeping the original `img` intact.

After initializing your necessary variables and the queue data structure, the flood fill algorithm is another application of the classic breadth-first search, which you've seen a couple of times before in this problem collection. Initially, the **frontier queue** contains only the seed pixel (*x*, *y*). The algorithm's main loop keeps going as long as the frontier queue is nonempty. In each round of this main loop, pop out the next pixel to be processed from the front of the queue. If this pixel currently has a colour other than the background colour, ignore it and go to the next round. Otherwise, colour that pixel with the new colour `c`, and push its **four** neighbours in the main compass directions that are inside the image into the queue. The frontier queue becomes empty once we've processed and painted all background pixels reachable from the seed pixel with the new colour.

To simplify and streamline the translation of the algorithm from the Wikipedia pseudocode presentation to actual executable Java, it might be convenient first to define the utility method

```
public static boolean inside(BufferedImage img, int x, int y,
                             int bgCol)
```

that checks whether the pixel (*x*, *y*) lies inside the dimensions of `img` and that its colour is still equal to the given background colour `bgCol`. Note how the methods `getWidth` and `getHeight` of the `BufferedImage` class will provide you with the exact pixel dimensions of that image.

The `main` method of the automated test class tests your flood fill method on all areas of the pseudorandom image of black curves and lines shown in the screenshot above. Since your method should produce that same result when applied to the pseudorandom test image, you can use this `main` method to verify by eyeballing that your flood fill algorithm works as intended. The actual `testFloodFill` method does not display the thirty pseudorandom images of various dimensions that it creates and paints over using your flood fill method. Instead, the test method computes a checksum from all the pixels in each pseudorandom test image to ensure your flood fill algorithm fills precisely the same pixels with the same colours in every test image as this author's private model solution would.

Once they get the basic flood fill method working, interested students can also try implementing the more advanced, memory-efficient **scanline fill** techniques described on the Wikipedia page. Even though we are most familiar with the result of flood fill from the now mostly just meme-worthy MS-PAINT images, this algorithm allows many optimizations for both time and space that, back in the day, allowed it to run even on an eighties 8-bit home computer such as the Commodore 64 and Sinclair ZX Spectrum, both of which this author still has both fond and not so fond memories of.

Despite being adorably primitive from our point of view up here in the cyberpunk future, even the ZX Spectrum's BASIC included a flood-fill operation as a keyword, along with more elementary graphical primitives like drawing a line segment or a circle. However, the mighty one megahertz processing power of the Z80 processor could only execute this operation so slowly over the 256-by-192 screen resolution (and even that split into 8-by-8 regions with only two colours per region) that you could watch the algorithm paint the pixels, almost as if the whole thing were some animated YouTube demo of the algorithm.

(To see how far humanity has gradually advanced in this respect, and what the exponential growth of processing power means in practice, you can try out this author's hand-rolled and non-optimized Mandelbrot.java demo that not only flood fills the Mandelbrot fractal image with twenty times as many pixels, but determines the positions of the walls and the colour of each pixel on the fly from a complicated iterative process of high-precision complex number arithmetic!)

Lab 58: Block Cellular Automata

JUnit: BlockCellularAutomatonTest.java

Various cellular automata, most famously variations of Conway's Game of Life, produce surprisingly chaotic, emergent complexity and interesting patterns from local interactions when simple rules are applied uniformly over a grid of truth-valued cells. We have already seen Conway's Game of Life as one of the example programs `GameOfLife.java`, in this course material, and the Ulam–Warburton crystals as an earlier lab problem in this very document.

Block cellular automata are an interesting generalization of the basic idea of cellular automata. In a block cellular automaton, the local rules apply not to individual cells, but to a larger neighbourhood, in the simplest non-trivial case, a 2-by-2 square into which the grid is subdivided. For brevity, we call each such 2-by-2 square a **Margolus square**. Since the grid's width and height are assumed to be even, this subdivision places every cell into exactly one Margolus square. Since each `boolean` cell inside the same Margolus square can be in two possible states (as usual, these states are again vitalistically named "dead" for `false` and "alive" for `true`), that Margolus square can be in $2^4 = 16$ possible configurations.

As in all other cellular automata we have seen so far, a block cellular automaton executes forward in discrete time steps. At each time step, a hardcoded rule converts each Margolus square to its next configuration. To let these squares interact rather than remain small, isolated islands, we place the grid subdivision origin at (0, 0) at even time steps and at (1, 1) at odd time steps. To treat the cells on the grid edges properly at both odd and even time steps, we treat the grid as toroidal, meaning its east and west edges are logically wrapped together. The north and south edges wrap analogously. For a cell at the right edge of the grid, its immediate right neighbour is on the same row at the left edge, and vice versa.

The four truth values inside the Margolus square are treated as four bits of an integer's binary representation: the northwest cell adds 1, the northeast cell adds 2, the southwest cell adds 4, and the southeast cell adds 8 to the square's value. For example, if the northwest and southeast cells of some Margolus square are currently alive, and the other two cells are currently dead, that square as a whole will have the numerical value of 1 + 8 = 9. Another Margolus square with all cells dead would have a numerical value of zero, whereas a third square with all cells alive would have a maximum numerical value of 1 + 2 + 4 + 8 = 15.

Since the chosen rule for the block cellular automaton turns each Margolus square's configuration into another configuration of the same square, the entire `rule` can be compactly expressed as an array of 16 integers. Once you have computed the numerical `value` of some Margolus square, the numerical value of that square will simply equal `rule[value]` in the next time step. You can then convert this numerical value back into the individual dead and live cells inside that Margolus square by expressing that value in binary and making precisely those cells alive in the next time step for which the bits for the corresponding powers 1, 2, 4 and 8 are turned on in that number.

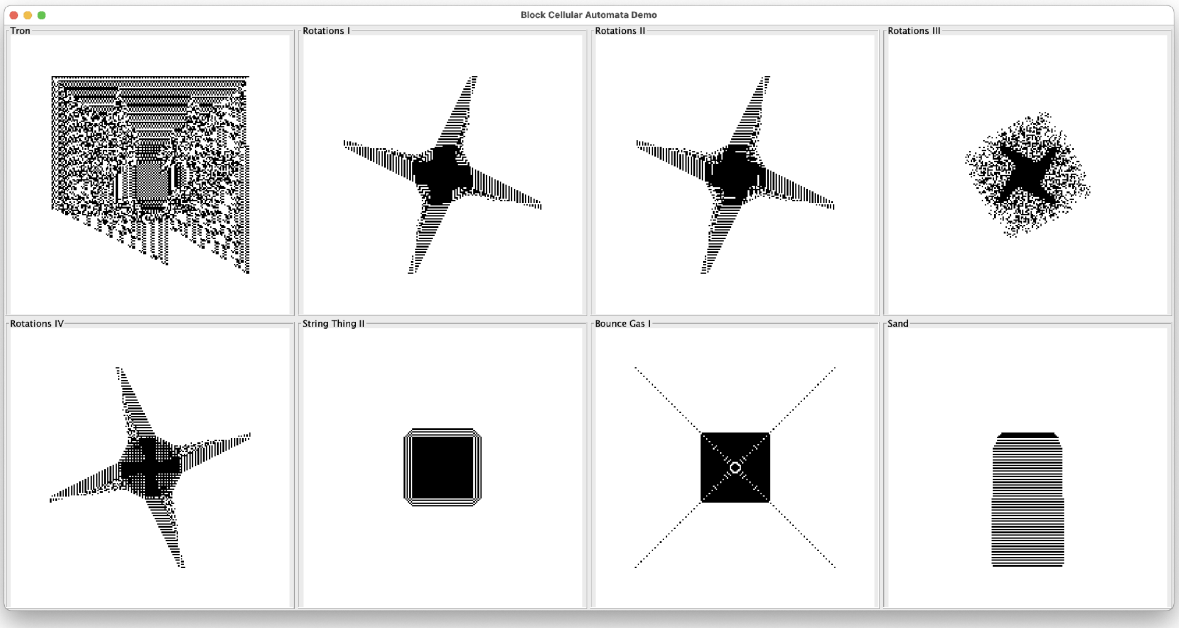
In your lab project, create a new class `BlockCellularAutomata`, and there the method

```
public static void margolusNextState(boolean[][] current, boolean[][] next,
int[] rule, int off)
```

This method takes the `current` state of the grid of cells and computes the `next` state using the given `rule`. The parameter `off` indicates whether the current time step is odd or even: 0 for even time steps and 1 for odd ones. (The method doesn't need to know the current time to work, only the parity of the current time.)

This method does not create and return the new `next` array object; instead, the caller provides this array object as a method parameter to be filled in. In the automated JUnit test, this `next` array is guaranteed to have the same dimensions as the current array from which the block cellular automaton computes the next state. This handy scheme eases the burden of the Java garbage collector when the execution thread of the JUnit tester (provided by the instructor here; what are you so worried about?) does not constantly have to be creating new grid arrays only to then throw these perfectly good brand new array objects into garbage at the very next time step! Instead, this animation logic can recycle the same array objects to track the automaton's current and next state, swapping their roles at each time step. (However, unlike with freshly created arrays, the recycled array is not guaranteed to be filled with false values at the call, so your method needs to compute and fill in every element.)

The JUnit test class for this lab also contains a `main` method that launches eight separate instances



of a block cellular automaton, each instance using eight aesthetically behaving rules copied from the

page "Reversible Cellular Automata". When you run it, the initial state of a solid block of live cells at the center of each grid should evolve toward the above screenshot.

Students interested in GUI programming with Swing should again note that, out of the box, the `JFrame` instances that represent the underlying GUI windows in Java and Swing do not obey the close button and go gently into that good night. Instead, they rage, rage against the machine, ignoring the order to ride into the valley of unreachable objects whose role is only to do or die without replying. Normally, we use the method `setDefaultCloseOperation` to tell a `JFrame` instance to become more respectful of the chain of command. However, we cannot do that in this example, since launching a new Timer instance creates a new execution thread that we must ensure also stops when the user closes the window. Otherwise, the Java Virtual Machine process couldn't terminate, since it would have to keep the timer ticking like a forgotten metronome nobody is listening to anymore!

As an interesting aside, all instances of the Swing `Timer` are managed by the same execution thread that internally keeps all active timers in a **priority queue** data structure (presumably implemented as a **binary heap**) to quickly find the active timer that currently has the least amount of time left until its next moment to step into the limelight. This common thread managing all timers repeatedly sleeps until the first timer in the priority queue is due. It then executes the action associated with that timer and pushes the timer back into the priority queue, where the binary heap logic quickly moves it to its correct place. Unlike actual execution threads, whose creation is expensive, you can easily launch thousands of timers to tick simultaneously in the background at arbitrary speeds! How wonderful it is that things that `sleep` quietly, so the processor doesn't have to check on them constantly, don't consume even one processor cycle just by existing in memory!

Beyond the eight example rules illustrated in the test class, this mechanism allows so many possible rules, even for 2-by-2 squares and boolean cells, that humanity will surely never explore the possibilities, but only a tiny portion of these $16^{16} = 2^{64}$ possible rules. Even if we restrict the rule array to a permutation of integers 0 to 15, we still have 16! = 20,922,789,888,000 possible rules to play around with. All such **bijective** permutation rules are guaranteed to be **reversible**; any such automaton can always be run backwards in time from any arbitrary future state all the way back to the starting state and even beyond that merely by inverting that permutation to use as the new rule! (Such an automaton therefore cannot have any Gardens of Eden.)

Further restricting these rules to be **symmetric** to all four compass directions will also drastically lower this count. Furthermore, the configuration where all cells are dead is usually restricted to stay dead (that is, a legal rule must map the Margolus squares of value zero to the same squares of value zero), as is the case with all of the rules used here. All this still leaves us a whole bunch of possible rules to explore. Without this natural restriction against something coming out of nothing, the entire automaton grid would immediately fill with nonzero stuff, creating an annoying flashing that could potentially harm students with epilepsy, as in the famous Critters rule.

The block cellular automaton mechanism also trivially generalizes to larger k-neighbourhoods, of which Margolus is a trivial special case of $k = 2$ (the neighbourhood shift offsets must then also use the same period of k), and a larger number of possible states for the individual cells (usually then called the **colour** of that cell), with the number of possible rules blowing up exponentially into unimaginable numbers and emergent complexity. Such automata can be used for generative and computational art; see the web page "6-Colour Block Cellular Automata" for illustrative examples.

Lab 59: Hitomezashi Golygons

For testing: HitomezashiMain.java

This problem was inspired by the Numberphile video "Hitomezashi Stitch Patterns," which you may want to check out first to get a quick idea of what these patterns are all about. (You should also note the clever mental technique used in the same video to create a random bit pattern!) The blog post "MathArtChallenge Day 14: Hitomezashi stitching" at Arbitrarily Close also succinctly explains the mathematics behind these patterns. Traditional Japanese stitching uses such patterns to create regular designs that are both useful and pleasant to look at. However, adding a pinch of randomness to this surprisingly simple process reveals emergent complexity that has been waiting beneath the surface to blossom.

A **hitomezashi** pattern is drawn on a rectangular area subdivided into a grid of squares. In the space between each row of squares, draw a horizontal stitching line so a line segment appears only under every other square, as if stitching fabric with a needle and thread. Each row is initially assigned a truth value to decide whether the squares in the odd positions (if `true`) or in the even positions (if `false`) get a visible line segment under them. Repeat this process for the vertical columns to complete the pattern.

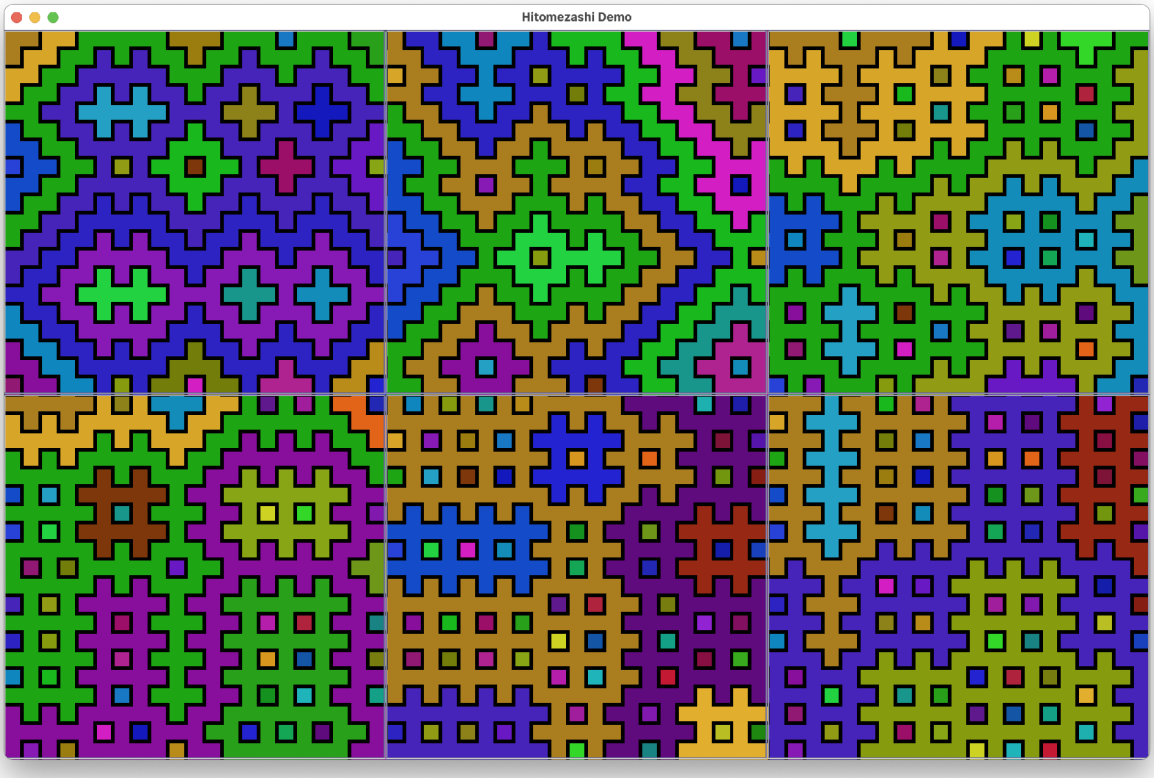
If the rows and columns are assigned truth values in some simple repeated pattern, the entire two-dimensional pattern will repeat uniformly with the same period. However, randomly assigning truth values to each row and column produces a chaotic fractal pattern of unit golygon shapes, each edge exactly one unit long. As long as there is room inside a shape, it will properly contain smaller fractal shapes tucked inside it, with none of these nested inner shapes ever abutting any of their brothers or the larger shapes that immediately surround them.

This lab is another exercise in how to create and draw on `BufferedImage` objects to create two-dimensional pixel raster images. Create a new class named `Hitomezashi` in your labs project folder, and in that class, the following `static` method

```
public static BufferedImage createPattern(
  int width, int height, int square,
  boolean[] horizontal, boolean[] vertical
)
```

The first two parameters, `width` and `height`, define the pattern dimensions in squares, and `square` gives each square's size in pixels. For example, if `width` and `height` both equal 25 and `square` equals 20, the pattern created by this method will be 25 × 20 = 500 pixels wide and tall. The bit vectors for `horizontal` and `vertical` determine the parity of squares that get a visible line segment, given separately for rows and columns.

From these parameters, this method should create a new `BufferedImage` instance that produces that pattern as line segments rendered in black against a white background. The instructor provides a main class `HitomezashiMain.java`, that calls this method with six different random horizontal and vertical bit vectors and displays the gallery of returned images inside a Swing `JFrame` window. The bit vectors that produce the six patterns in the screenshot below are created by setting each bit equal to the previous bit with a probability ranging from 30% to 80%, with a 10-point increment between successive patterns. To ensure your code treats width and height consistently, each of these six shapes is 21 squares wide and 20 squares tall.



Colouring the golygons is not required to complete this lab. You can just render only the black line segments that form the outline of that shape**.** Even so, make sure that your golygons end up having the same outline as those given in the example screenshot from the main class shown below. However, if you already completed the Flood Fill exercise earlier in this document, you might as well use its `floodFill` method to paint the insides of these contiguous golygons with random colours. As a result, they pop powerfully against their statistically similar surroundings.

At least to this author's untrained eye, these patterns evoke nostalgic visuals of Native American artwork, especially if the colour knob used to fill these golygons were adjusted toward more earthy, muted tones. Readers who have completed the Flood Fill project can experiment further with such tones. As explained in the Numberphile video above, this technique can also apply to other grids, such as triangular grids, not just the familiar integer grid.

You can generalize the technique further by using a more complex pattern to fill the line segments between rows and columns, instead of the regular on-off metronome stitch. If you render such stitches randomly using line segments aligned to the four intercardinal compass directions, you get a maze generator made famous in classic Commodore 64 BASIC one-liner 10 Print, arguably the spiritual great-granddaddy of all such later **generative art**.

Lab 60: Bit Deque

JUnit: BitDequeTest.java

As in other imperative programming languages, Java guarantees that array elements are stored in consecutive memory locations. Assuming these arrays are **homogeneous**, meaning each element takes the same number of bytes to store, such arrays benefit from being random-access **data structures**. This conceptual misfire of a terminological chimera has nothing to do with randomness and is better called an **arbitrary-access** data structure, but this linguistic hill has long since been turned into condos. In a random-access structure, unlike a sequential structure that must be accessed in order from beginning to end, you can access any element for reading and writing in constant time, regardless of earlier operations on that array.

The downside of such rigid memory ordering is that every array object must maintain the same length throughout its lifetime. This makes arrays inconvenient for storing dynamic data structures where the number of elements can increase and decrease during execution. At the time, older books and similar data-structures materials recommended using **linked lists** to represent such dynamic structures. Each element in a linked list is stored in a separate node object allocated on the heap, requiring heap management and garbage collection to happen behind the scenes.

Each node object must also store 8 or 16 bytes for the memory addresses of its successor and predecessor. This extra memory, in addition to the payload, lets you add and remove an element from a known position in guaranteed constant time, the upside of linked lists, whose nodes and chains of nodes can be extracted and inserted in arbitrary positions in constant time. Arrays can contain only the payload data bytes and don't need anything else to keep them in order, since the memory hardware already does that for free.

However, sometimes we know we will always add and remove elements at the end of the current structure. Consider the `StringBuilder` class in the Java standard library, where new characters and strings are appended piecemeal to the end of the string being built. A linked list would be overpaying for overkill. A superior technique uses an ordinary array to store the payload elements, with a separate integer data field (or two) on the side to keep track of how much of this array is actually being used to store the current payload cargo. The positions that do not currently contain any payload elements are **slack** available for future additions to the structure. The element values in the slack area of the array can be anything, since they will never be read out as actual elements.

Standard terminology defines the structure's capacity as the underlying array's length and its size as the number of actual payload elements stored in the array. This terminology can be seen in the public interface of the good old Collection workhorse `java.util.ArrayList<E>`, for example. The difference between these two quantities gives the slack space currently available. As long as some slack space exists, adding new elements is a simple constant-time operation, regardless of the structure's size or capacity. Once slack runs out when adding a new element, a new array twice the size of the previously used array is created to copy the existing elements into. The structure then

says *sayonara* to the old array, thanking it for its service. From that point on, it continues using the new, larger array as if it had been the original all along, now with plenty of slack space for future additions. (Convenience methods `copyOf` and `copyOfRange` of the `java.util.Arrays` utility class make this simulated resizing practically a one-liner.)

Primitive arrays often implement various forms of **queues**, a generic name for a data structure of which the "queue" in the everyday sense is just one possible special case. In data structures and algorithms, a queue lets you push elements in one by one and pop one out at any time. The queue's subtype determines which element comes out during a pop operation. A **first-in, first-out queue (FIFO)** behaves like a queue in the everyday sense, popping the element that has waited the longest. A **last-in, first-out queue (LIFO)** behaves like a **stack**, so it pops the element that has waited the least time. Both FIFO and LIFO queues have useful applications in many algorithms that we have already seen. For example, the **breadth-first search** algorithm used to solve the "Big Ten-Four" and "Flood Fill" exercises earlier in this document uses a FIFO queue to store the frontier of elements waiting to be processed.

Of course, the standard library of not just Java, but any programming language we could ever seriously consider for real work in this millennium, offers both of these queue structures, and more. Nobody should ever have to implement either one from scratch any more than, say, they would implement their sorting algorithm. (What separates computer science from most other fields is that no problem ever *needs* to be solved twice.) However, in the same spirit as the `NatSet` example class, which stores a monotonic set of natural numbers with one bit per element, we realize that generic queue classes that can store any type of element will be horrendously inefficient in memory when used to store primitive data types. You don't have to be Uncle Scrooge to shudder at the idea of wrapping each payload item, itself only a couple of bytes, in an object that, even in the best case, takes at least 24 bytes on the heap!

We implement an array-based **double-ended queue** data structure to store `boolean` values, using as little memory as possible. A double-ended queue, or just **deque** from now on, is a linear queue that lets you push and pop elements from either end at any time, with four operations: `pushFront`, `pushBack`, `popFront`, and `popBack`. If you have a deque available, you can always voluntarily restrict yourself to only the operations of a LIFO stack and a FIFO queue, making these structures special cases of a deque. The deque's added flexibility can come in handy in special situations… provided we first figure out how to implement all these deque operations in constant time, at least amortized over a long series of operations. If each deque operation needs linear-time reorganization after each operation, its user code might as well keep all the data in its own arrays anyway.

In your project folder, create a new class named `BitDeque`. This class will have the following six public methods for you to fill in once we get the preliminaries out of the way. The first four methods implement the deque operations, and the last two query the queue's current size and capacity.

```
public void pushFront(boolean v)
```

```
public void pushBack(boolean v)
public boolean popFront()
public boolean popBack()
public int getSize()
public int getCapacity()
```

To store the elements currently inside the deque, this class should first define the data fields

```
private boolean data[];
private int startIdx;
private int endIdx;
```

The elements are stored in the bit vector `data`, which has slack at both ends to allow constant-time pushing and popping from both ends. The indices `startIdx` and `endIdx` delimit the payload area in the middle. Conventionally, `startIdx` is inclusive, while `endIdx` is **exclusive**. For example, let `data.length==20` so that the array can store twenty elements. Suppose `startIdx` equals 5 and `endIdx` equals 13; the deque contains exactly eight payload elements, as given by the elegant formula `endIdx-startIdx`. (If the end index were also treated as inclusive "because *muh symmetry"* or some other sentimental *ad hoc* rationalization, you would always have to remember to add one to the result given by this formula. Other benefits of using an exclusive end index will follow until the morale improves.)

Once you understand how this structure works by packing everything into a simple bit vector, you can implement all four deque operations with just a couple of lines of code, with delightful symmetry to boot! For example, the `pushFront` method will first decrement `startIdx` and place the new element into the new `startIdx` position. Because the end index is exclusive, pushBack first places the new element at `endIdx`, which is the first position of the slack after the payload, and increments `endIdx` only after adding the element. (Knowing the distinction between the prefix and postfix forms of the operators `++` and `--` will now pay its dividend.) The methods `popFront` and `popBack` operate symmetrically to undo the corresponding push and, like the two push methods, differ by inserting the element and updating the index in the opposite order.

The class should also define the following constructor to initialize the deque. Since there is no point in being miserly with individual bits, the `data` array will maintain a minimum capacity of 512 bits throughout the deque's lifetime, regardless of how many payload elements it stores. The payload area in the middle is initially empty. Still, because we use `endIdx` exclusively, it's easy to recognize and handle this: the deque is empty if and only if `startIdx` equals `endIdx`.

```
private static final int MIN_CAPACITY = 512;
public BitDeque() {
    data = new boolean[MIN_CAPACITY];
```

```
    startIdx = endIdx = MIN_CAPACITY / 2;
}
```

As long as there is slack available in the push direction, everything is straightforward. But what happens when this slack inevitably runs out after enough additions? To get out of this predicament, your class should define a private utility method

```
private void expand()
```

that both methods call when no slack is available in the desired direction. This method creates a new bit vector **twice the size** of the current data array. After copying the existing payload elements to the center of this new vector, the garbage collector discards the previous data array. Update `startIdx` and `endIdx` to reflect the payload's new location in the larger array; then you'll have plenty of slack at both ends for future deque operations.

Note that the automated JUnit test class will enforce the fact that the threshold and logic of calling `expand` is symmetric in both directions, and your code will not pass this test unless it does this. (Sorry guys, I didn't make the rules; I am but a humble messenger of far higher forces responsible.)

In the common case where the deque is used as a FIFO queue, and the payload area always moves in one direction, plenty of slack will be available at the other end of the array, all sitting there doing nothing but twiddling its virtual thumbs. Instead of wasting perfectly good memory by allocating a new, bigger array and then using only one quarter of its space, you can implement a private utility method that moves all payload elements so they are centred in the array again. You should use the method `System.arraycopy` to perform this move efficiently in bulk, instead of blithely using a for-loop through the relevant positions and copying their elements inefficiently one at a time. This method is implemented **natively** in the Java Virtual Machine and is therefore significantly faster than any for-loop we could write at a higher level of abstraction in Java. Remember to also update the `startIdx` and `endIdx` indices yourself, though, since no method can magically update your fields and local variables for you.

(You may also note how `System.arraycopy` shows its age, dating back to Java's internal development versions, since its name doesn't follow the convention of being named `arrayCopy`. This method has been grandfathered in the language the same way as `Color.Black` and other named constants for the common primary colours that should have originally been named `Color.BLACK`, in accordance with the all-uppercase naming convention for named constants, and maybe to also remind us of a famous Chris Rock stand-up comedy skit.)

An alternative solution to the previous two methods, though more complicated to code, is to treat the data array as circular. So, any index that goes past the array's edge wraps to the other side. This scheme is more memory-efficient because it uses space on one side to push elements from the other. Data array expansion would be triggered by a push operation that makes startIdx and endIdx

equal, a situation that should mean the deque is empty. The expansion code also becomes more complicated. Students who implement their deque this way need to be mindful of (in this case, quite literal) edge cases where these indices point to the first and last positions of the array, which can happen either way during deque use. Even outside this edge case, the deque's contents are generally split into two contiguous subarrays, which the expansion must copy correctly into the newly allocated larger array.

The last situation you unfortunately have to consider in this lab is when you first push a large number of elements into the deque, then pop almost all of them. It seems very inefficient to use a giant array to store a handful of elements, with most of it ending up as slack that may never be filled again. Even if we knew for certain it would fill later, we might as well use those megabytes more productively before then! (A megabyte here and a megabyte there, and pretty soon you are talking about real memory.) An immediate idea is to define a utility method for contracting the deque as

```
private void contract()
```

Your `popFront` and `popBack` methods must call this method whenever the size of the payload block becomes too small compared to the overall capacity of the `data` array. The automated tester will enforce that, unless the deque's capacity is at its smallest possible value of 512 bits, the deque's capacity may not exceed four times the current number of elements. Again, be mindful of edge cases when implementing the `contract` method. Symmetric to the previous `expand` method to be able to undo its effect, `contract` should allocate a new array half the size of the original `data`, copy the existing elements around the center of this smaller array, and discard the previous `data` array, hoping that the bits and bytes that comprise it will be put to better use. However, realize that intuition about when to trigger the contraction can now easily lead you astray, especially if you again vigorously hand-wave something about symmetry!

The threshold for triggering a contraction operation absolutely cannot be that the data array is only half full, even if the automated tester doesn't enforce it. Following the general principle of hysteresis, this threshold must be **one quarter full**. Otherwise, a clever adversary could engineer a situation where, immediately after an expansion, precisely two elements are popped to trigger a contraction, followed by two elements being pushed to trigger an expansion. The malicious adversary could repeat this back-and-forth cycle any number of times, and the push and pop operations would no longer be guaranteed to run in amortized constant time, slowing the surrounding system that relies on this deque to a glacial crawl.

Lab 61: Save The Date

JUnit: CalendarQueueTest.java

Continuing the general theme of queues from the previous problem, this problem looks at priority queues, where elements pushed into the queue must be comparable with some total order that unambiguously determines, for any two elements, which is smaller. This ordering determines priority, so the pop operation always removes the element with the lowest priority among those currently in the queue.

The `PriorityQueue<E>` class in the Java standard library uses the classic **binary heap** to maintain the priority ordering behind the scenes. This class normally compares elements using their built-in `compareTo`, but you can also provide a `Comparator<E>` strategy object to compare elements based on any criteria you want. However, just as bees make honey and beavers build dams by instinct, humanity has developed many data structures that implement a priority queue with efficient operations, even for millions of elements. These data structures let the basic push and pop operations run much faster than keeping all elements in a messy, unsorted array and always searching for the smallest element with a linear loop every time the next element is popped. (Some advanced priority queue data structures also allow operations such as melding two priority queues into a single priority queue, or updating the priority of an element somewhere in the queue, to run much faster than they do with an unsorted array.)

This lab looks at a classic but now mostly forgotten data structure: the calendar queue, another has-been old wino from a simpler age that might still have some vinegar in its veins to save the day in situations that make more general structures grind to a crawl. Consider a typical algorithmic application of any priority queue in an **event simulation**, where each **event** includes a future time and payload information describing how it affects the simulation. The event simulator algorithm is a simple while loop that repeatedly pops the earliest event from the priority queue and executes its effect in the simulated world. Each event can and will produce new events to take place in the future. These new events are added to the priority queue during the execution of the event that begat them. In this technique, the event simulation algorithm can jump instantly to the next interesting future event, rather than waiting for something interesting to happen while advancing the simulation one time unit at a time.

If the world being simulated follows the usual laws of **causality** in that events taking place at present cannot affect the past by creating new events that do take place in the past (thus potentially creating ill-defined situations inside the entire simulation, such as whatever would be the equivalent of the famous grandfather paradox in the world being simulated), the event popped from the head of the priority queue can only cause later events to be pushed into the queue. Furthermore, in most simulations, most events will cause new events to take place in the very near future. Under these two reasonable assumptions about how events beget other events, a calendar queue can perform its basic push and pop operations in expected constant time, which is almost a cinch. The

associated algorithms are also among the simplest among the priority queue data structures linked above.

Like an ordinary desk calendar, the calendar queue assumes time proceeds in discrete years divided into **days**, the smallest unit of granularity available for scheduling these events. To get started with this lab, add the instructor's class `CalendarEvent` in your labs project. **Do not modify the contents of this class in any way while completing this lab**. Instead, read through this simple data class to see how its objects represent events that take place in a given year and day, with each event's payload as a message string and the `year` and `day` as integers. Unlike us glorified apes in trousers, who have trouble remembering anything symbolic better than a toaster could, the computer doesn't need the convenience of dividing years into months; each year has exactly `DAYS_IN_YEAR` days, with no leap years. Days are numbered from 0 to `DAYS_IN_YEAR-1`, after which the year increases by one and the day rolls back to 0 to begin a new year.

For automated testing of this lab, this named constant `DAYS_IN_YEAR` has been defined to equal ten inside the `CalendarEvent` class. (We can pretend this exercise takes place on the nanoplanets in The Little Prince, populated by various melancholic yet tragically colourful characters.) However, changing this named constant to 365, or any other positive integer, depending on which planet we live on, should not change the logic of your solution code. Never hardcode any **magic numbers** directly in your code, but always refer to such quantities using the named constants!

Internally, a calendar queue consists of an array with a slot for each of the `DAYS_IN_YEAR` possible days in our timekeeping. Each slot holds a singly-linked list of events that fall on that day. Events in each same-day list are sorted by the year they take place. For example, if the queue contains three events that all take place on day 7, but in three different years 2525, 3535 and 6565 as in the famous song of yore, these three events are guaranteed to be stored inside the list so that the event of the year 2525 is first, the event of taking some pills in the year 3535 comes right after that, and the event of picking your son and daughter from a long glass tube in the year 6565 will be the last. These events are distributed across these linked lists; finding the earliest event on any given day is now quick and easy, regardless of the total number of events stored in the calendar queue. (Readers who are familiar with the internal operation of **chained hash tables** may recognize this same basic technique being productively applied in a different context.)

For representing these chains, the Java Collection Framework offers the LinkedList<E> data structure for linked-list operations. However, because of the utterly idiotic design decision made in the pre-generics Java of making arrays covariant, which requires every Java array object to know its full element type at runtime, the implementation of Java generics under the hood with **type erasure** that throws out all this runtime information after compilation will by definition make it impossible to have arrays whose element type is any generic class, even if the type of that generic class is fully instantiated! However, this lab will be an exercise in implementing basic linked-list operations from scratch, as has historically been done in second programming courses like this one. We all have to eat a little dirt before we die, even if we aim toward the higher Platonic planes in our coding.

In your labs project, create a new class `CalendarQueue`. Inside this class, copy-paste the following nested utility class whose objects represent individual **nodes** of a **singly linked list**. Each node contains the `event` payload and a `next` reference that, as the name suggests, points to the next node in the linked chain.

```
private static class CalendarNode {
    public CalendarEvent event;
    public CalendarNode next;
    public CalendarNode(CalendarEvent event, CalendarNode next) {
        this.event = event;
        this.next = next;
    }
}
```

In the last node object currently stored in a chain, the `next` reference equals `null`, indicating the absence of a successor node object. To implement all the operations of a calendar queue, we only need the operations of a singly linked list, so these node objects don't need to waste memory on a reference to their predecessors in the chain.

Note also that this nested class is not an **inner class**, since it is defined as `static`. Unlike the non-static inner classes whose objects cannot exist on their own but must always exist in the context of some mothership of an outer class object (this allows the inner class objects to access all public or private members of this mothership object directly, exactly as if these members were their own, without even any special syntax), objects from static nested classes don't need anything from the outer class and may therefore exist on their own, completely independent of whatever outer class objects have come into existence so far. The advantage of using static nested classes whenever feasible is that they save eight bytes of heap storage per object compared to inner classes, because inner class objects must store a hidden reference to their mother object, and inner class objects can have different motherships.

(Students who have been paying attention during this instructor's lectures might at this point object that this nested class violates the proper principles of **encapsulation** by defining its two members `event` and `next` to be `public`! Hold your horses; no noble engineering principles are actually being violated here, since this nested class is defined `private` inside the outer class. No information about implementation details leaks out to the outside world any more than would leak from using local variables inside some method.)

After this nested class, define the following fields inside the class `CalendarQueue`:

```
private CalendarNode[] head;
private int size;
private int currentYear, currentDay;
```

The `head` array contains a total of `DAYS_IN_YEAR` elements; each element is a reference to the first node of the linked list hanging from that slot. For days that have no events scheduled to happen during that day, this slot is initially `null`. The field `size` contains the current number of events stored in this calendar queue, updated inside every `push` and `pop` for quick constant-time lookup in the `getSize` method. The calendar queue object must also keep track of the current year and the current day of the simulation, here stored in two fields: `currentYear` and `currentDay`. As the last piece of code this specification will give you before unleashing you to complete the rest yourself, here is the `CalendarQueue` constructor to properly initialize these fields.

```
public CalendarQueue(int currentYear, int currentDay) {
    head = new CalendarNode[CalendarEvent.DAYS_IN_YEAR];
    this.currentYear = currentYear;
    this.currentDay = currentDay;
}
```

The field `size` is already zero, so it needs no initialization. (Of course, it never hurts to be explicit.) The three `public` methods left for you to implement are

```
public int getSize()
public void push(CalendarEvent event)
public CalendarEvent pop()
```

The automated JUnit test for this problem guarantees that it will never `push` events that take place before the current year and day of the calendar queue, and will never try to `pop` an event from an empty queue. Nobody can reasonably be expected to achieve anything logically impossible, after all.

The `push` method should first access the day of its argument event to decide which slot the new CalendarNode object containing that event as a payload will be linked into, and then, from that chain, find the position where this new node should be linked. There are essentially three possible cases to get right. First, if the linked list hanging from that slot is empty, or if the event stored in the chain's first node takes place later than the event currently being pushed in, the new event becomes the first node in the chain. (Your code must correctly break the ties between two events with the same year and day by the message component. This happens automatically if you use the `compare` method of `CalendarEvent` to perform this comparison.) You can achieve this by assigning the reference in the proper head slot to point to this new node, whose `next` reference then points to the node that was previously the first node in that chain. That previous first node therefore becomes the second banana, properly positioned as the new node's successor.

(Note how the same unconditional logic for assigning the `next` reference works for both cases. It is not okay to follow a null reference to get to an object, whereas assigning a null value to a reference is hunky dunky.)

Otherwise, if the chain hanging from that day already contains at least one node, loop through the nodes of this chain by repeatedly following their `next` references to step to the next node in the chain. Keep going until you find the **predecessor node** whose successor the new node should become to maintain the sorted order of the events stored in that chain. Reassign the `next` reference of this found predecessor node to point to the new node, and assign the `next` reference of the new node to point to the earlier successor of that predecessor node. (Again, the same unconditional logic works even when the predecessor is the last node of the chain.)

The `pop` method should first look at the slot of the `currentDay`. If the year of the event stored in the first node of that chain equals `currentYear`, remove that node from the chain using a symmetric approach to the one used to insert a node, and return the event stored in that node as the answer. Otherwise, keep incrementing `currentDay`, rolling back to zero and incrementing currentYear every time you go past the end of the array, until you reach a slot whose first node satisfies the previous condition, allowing you to terminate. Assuming a reasonable distribution of event times in the near future, this approach will usually return the next event faster than you could say boo to a goose, regardless of how many events are currently stored in that priority queue.

To appreciate the calendar queue approach's simple elegance, remember that this search algorithm only looks at the first node in any slot! Even if a billion other future events that take place that same day in some far future year (perhaps something funny has already been destined to take place at The Restaurant at the End of the Universe) are hanging from every chain, these unseen nodes do not affect the time it takes to find the event to take place next. More remarkably, these unseen nodes also do not affect the speed of updating the priority queue data structure after extracting the first event! This is truly unusual not just for priority queues, but for all data structures this author has ever seen. Typically, even if you can find and extract that first element in constant time, updating the rest of the structure to reflect the new state usually takes at least logarithmic time with respect to the overall size of the data structure. But not today, Satan. Today we will feast and rejoice in the gift we have received.

Lab 62: Worley Noise

For testing: WorleyNoiseMain.java

The advantage of using pseudorandom number generators instead of true randomness is that any algorithm that uses the same RNG, initialized with the same seed, is guaranteed to produce the same results every time. Since the golden days of Elite, many computer games have used this approach, using procedural generation to build game levels. Even when confined to 64 kilobytes of memory for literally everything in the game, such games can pack truly enormous worlds into that tiny space. Only the level-generation algorithm needs to be stored, and its RNG is seeded each time with, say, the level number or the coordinates of the level being generated for the player to enter. Of course, such levels can't remember any changes the player made, but are generated afresh each time the player returns.

In general, using raw randomness to decide what goes where in each level usually isn't the best way to achieve an aesthetically optimal outcome. Imagine playing a game of *Civilization* where the type of each tile is randomly chosen from all possible tile types in that game, independent of the tiles surrounding it! As amusing as playing in this kind of *Secret Wars* patchwork battleworld might be, even if only as a thought experiment, we normally want to place our thumb on the scale to produce structures whose complexity falls somewhere around the sweet spot in the continuum between the two end points of the rigorous uniform determinism that allows no variety, versus the boiling pot of random noise that doesn't allow permanent meanings to coalesce.

If the randomly generated structure is supposed to resemble something familiar to us that has an organic existence in our physical reality, such as an island, patterns on granite or other types of stone, or biological cells, finding the right balance between chaos and order by computational means might at first seem an impossible task. Fortunately, many techniques for creating procedural noise have been discovered, so tweaking the noise-generation parameters and the palette used to render the result can produce surprisingly natural-looking patterns. Even better, the algorithm can produce the result image in any desired resolution, without any pixelation! In this lab, we implement a famous procedural noise generation algorithm called Worley noise.

To understand how Worley noise works, we should first look at a classic computational-geometry mainstay: the Voronoi diagram. For a given set of points *P* (typically placed pseudorandomly on the image to be generated), the Voronoi diagram divides the image into disjoint convex polygons, each containing precisely the points that share the same nearest point in *P*. Once you construct the Voronoi diagram for the given set of points, you can compute many other important computational geometry properties from it. However, colouring all pixels within the same polygon with a single, flat colour does not create an aesthetically pleasing image, which is our goal in this lab.

Worley noise takes a similar approach by considering not just the point in *P* closest to the pixel being coloured, but potentially all points in *P*. The colour of each pixel is computed from a weighted sum of its sorted distances to all points in *P*, although in practice most weights are set to zero. For

example, a pixel's colour might be determined by its distances to its second, fifth, and seventh nearest points in *P*, and the rest of the points do not matter for that pixel's colourization.

Since weighted distances can become arbitrarily large and we want to normalize the colours to [0,1], we first need a function that maps any nonnegative real distance to [0,1], preferably in an efficiently computable but generously aesthetic way. Even at the slight risk of giving readers painful flashbacks to high school trigonometry, we recall that *sin* and *cos* not only oscillate between –1 and 1, but are smoother than a baby's bottom in all their derivatives. A slight adjustment to also consider the desired frequency gives us the utility function

```
private static double wave(double x, double a, double f) {
    return (1 + Math.sin(Math.pow(Math.abs(x), 0.95) / f)) / 2 * a;
}
```

where a is the amplitude and f is the **wave frequency** that maps the real number x into the interval $[0, a]$. To make this transformation a bit less boring, first raise x to some fractional power before the transformation, so the frequency changes slowly as x increases. (After completing this lab, the student may wish to change the exponent from its present value of 0.95 to something larger or smaller to observe the effect that this exponent has on the shape of the noise.)

Next, we need to refresh our high school algebra on formulas for computing the distance between two points (x_0, y_0) and (x_1, y_1) on the two-dimensional plane. **Euclidean distance** comes from the Pythagorean theorem, but **Manhattan distance** and **chessboard distance** are also alternative (and mathematically sound) distance metrics, among infinitely many others.

```
private static double euclidean(
  double x0, double y0, double x1, double y1
) {
    return Math.sqrt((x1-x0)*(x1-x0) + (y1-y0)*(y1-y0));
}

private static double manhattan(
  double x0, double y0, double x1, double y1
) {
    return Math.abs(x1-x0) + Math.abs(y1-y0);
}

private static double chessboard(
  double x0, double y0, double x1, double y1
) {
    return Math.max(Math.abs(x1-x0), Math.abs(y1-y0));
}
```

With these preliminaries out of the way, we now have everything needed to generate Worley noise. In your project folder, create a new class, `WorleyNoise`, and in it, the following method, which probably takes the largest number of parameters of any method in this specification.

```
public static void compute(
  double[][] out, double[] xs, double[] ys, double[] weight,
  double a, double f, String distance
)
```

Note again that, for efficiency, this method does not create a new array to return each time; instead, it fills the computed result into the given array object argument `out`. The raw numerical values computed into this `out` array will later be turned into a `BufferedImage` whose height equals `out.length` and width equals `out[0].length`. As always, the caller is fully responsible for ensuring that the dimensions of the array object passed in as the `out` argument are large enough to contain the result. As contract law dictates, your method can fail however it wants if the caller doesn't hold up their end of the contract. This is not the time to be defensive.

The arrays `xs` and `ys` contain the *x*- and *y*-coordinates of the points *P* from which the value of each element of `out` is calculated, both arrays containing the same nonzero number of elements. Your method should loop through all the positions of the `out` array and fill in the value of each `out[y][x]` that corresponds to the point (*x*, *y*) on the plane in the following manner. (The swap of coordinate roles is done here because Java arrays are stored in row-major order.) First, calculate the array of distances from the point (*x*, *y*) to all points of *P*. The value of the `distance` parameter determines which distance metric you should use for this calculation, for the three possible **option constant** values of `"euclidean"`, `"manhattan"`, and `"chessboard"` that the automated tester will ever give it. (A more modern implementation would use an **enumerated type** here instead of using String as a fake enumerated type, like the early nineties.)

Next, sort this array of distances in ascending order, without caring which distance corresponds to which point in *P*, since we need only these distances themselves to compute the value of the element `out[y][x]` currently being filled in. Then, calculate the weighted sum of these sorted distances so that the *i*:th distance in this sorted order is weighted by `weight[i]`. Finally, transform that weighted sum of distances into a value in the interval [0, *a*] using the previous `wave` function, where the remaining parameters a and f come into play.

Note that the `weight` array can be shorter than the number of points in *P* given in the arrays `xs` and `ys`, in which case the weights past the end of the `weight` array should simply be considered to be zeros. For example, should the `weight` array happen to equal `{1, 0, -0.5, 0.5}`, this would mean that the distance of the closest point is given the weight 1, the distance to the third closest point is given the negative weight of –0.5, and the distance to the fourth closest point is given the weight 0.5. Other points in this ranked order are ignored in this calculation.

This is it for this lab: the `main` method of the WorleyNoiseMain.java test class takes care of converting these noise values into greyscale pixel values to be proudly displayed as images inside a new `JFrame` instance.

In practice, the points of *P* aren't thrown onto the image like a blind man throwing darts; instead, the image is first subdivided into a smaller grid, and each cell gets the same number of points, typically just one. Such a horrendous zero-variance deviation from true randomness, with probability close to one, is guaranteed to create clusters and clumps somewhere, ensuring that pixels near the edges of the image don't all behave the same way as the pixels in their local neighbourhood. Still, the noise remains pleasantly detailed and varied even near the edges. (As discussed in the earlier Linus Sequence problem, the easiest way to detect human-generated fake randomness is in the notable lack of such clustering. If you divide the image into a 5x5 grid of 25 cells, most people would automatically place one point in each cell when asked to simulate throwing darts randomly, even if they fancy themselves especially devious!)

To create even more natural results, multiple images generated with either Worley noise or other suitable types of noise, for example Perlin noise, can be combined in layers to produce **fractal noise** to smooth out the result further and make it seem more natural without merely making it blurry by losing all the interesting little detail. To achieve this outcome, compute these layers with different amplitudes and frequencies: the higher the amplitude, the lower the frequency and the number of points in P used to generate that layer. The high-amplitude layer therefore defines the landscape's main features, and the lower-amplitude layers add detail at smaller amplitudes and higher frequencies without distorting the general features too much.

If you choose each layer's amplitude and frequency accurately, zooming in on this landscape reveals its fractal nature: it looks similar at every scale, as fractals are expected to do. Of course, as in the real world, zooming in deep enough will eventually reach the final layer, after which the microstructure will look very different from the whole thing at the macroscopic level. Once again, as above, so below: repeatedly subdividing the coastline of England can make it as long as you want. Still, eventually you reach the quantum subatomic level, and the whole approach breaks down.

Practically, suitable noise parameters and the associated colour palette can make this fractal noise pass the Pepsi Challenge when paired with real photos of mountain landscapes or other natural and organic entities. However, the parameter space has enough degrees of freedom to simulate not just alien landscapes, but realistic biological textures and structures of animal, vegetable, or mineral.

Lab 63: Recursive Subdivision Of Implicit Shapes

For testing: ImplicitShapeRendererMain.java

The earlier lab about Lissajous curves demonstrated the important technique of rendering one-dimensional **parametric curves** on the plane. A parametric curve is defined by two functions $x(t)$ and $y(t)$ of one parameter variable t so that the curve consists of all points $(x(t), y(t))$ produced by taking the parameter t for a walk through some range of values, typically from 0 to 1. To render such a continuous curve as a discrete pixel raster, subdivide the curve into smaller pieces and render them piecemeal, typically as quadratic or cubic segments. Each piece should smoothly connect with the previous and next piece around it without any visible discontinuities in either the curve or its tangents. Viewed from a distance, the pixel representation of that continuous curve will appear as that continuous, smooth curve, within the limitations of pixel-raster granularity.

In this same spirit, implicit functions define two-dimensional **implicit shapes** on the plane. One function f receives as its arguments both the x- and y-coordinates of an arbitrary point (x, y), and returns a numerical result $f(x, y)$ whose sign indicates whether that point lies inside the shape (negative), on the shape boundary (zero), or outside the shape (positive). For example, the implicit function of the unit disk centred at the origin is $x^2 + y^2 - 1$, which the reader can verify is negative for all points (x, y) inside the unit disk, zero for all points on the circle wrapped around that disk, and positive for all points outside the disk. (The implicit formula for the general case of a disk centred at the point (x_c, y_c) with the radius of r would be $(x - x_c)^2 + (y - y_c)^2 - r^2$, of course.)

To render a continuous two-dimensional shape as a discrete pixel raster, a subdivision scheme samples points on the plane and uses that information to approximate the unsampled continuum. The marching squares technique subdivides the plane into a regular grid of uniform boxes. We fill each box with triangles of foreground colours based on which of the four corner points lie inside and which lie outside the implicit shape. We resolve ambiguous ties around saddle points by letting the box center determine the dominant direction.

At most five points sampled from the infinite continuum inside the box hopefully reveal the entire continuum with reasonable accuracy. As with any numerical method based on finite sampling, we assume the implicit function given as a black box is sufficiently well-behaved that we can reasonably estimate the shape's global and local structure from such point samples. Of course, a crafty adversary who knows our sampling scheme could design this black box as a fractal made of holes, islands, and tendrils, making any such sampling result arbitrarily far from reality. No algorithm based on finite sampling of a black box can fight back against this… at least unless we again resort to randomness to keep the adversary from predicting which points we will sample!

This lab concerns another rendering technique that also subdivides the implicit shape into rectangles. However, unlike marching squares, we subdivide recursively, so the depth isn't uniform

across the plane; it automatically becomes finer near the shape's edges, where more precision is needed. In your project folder, create a new class named `ImplicitShapeRenderer`, and there the recursive method

```
public static int render(
  DoubleBinaryOperator f, double xStart, double xEnd,
  double yStart, double yEnd, Graphics2D g
)
```

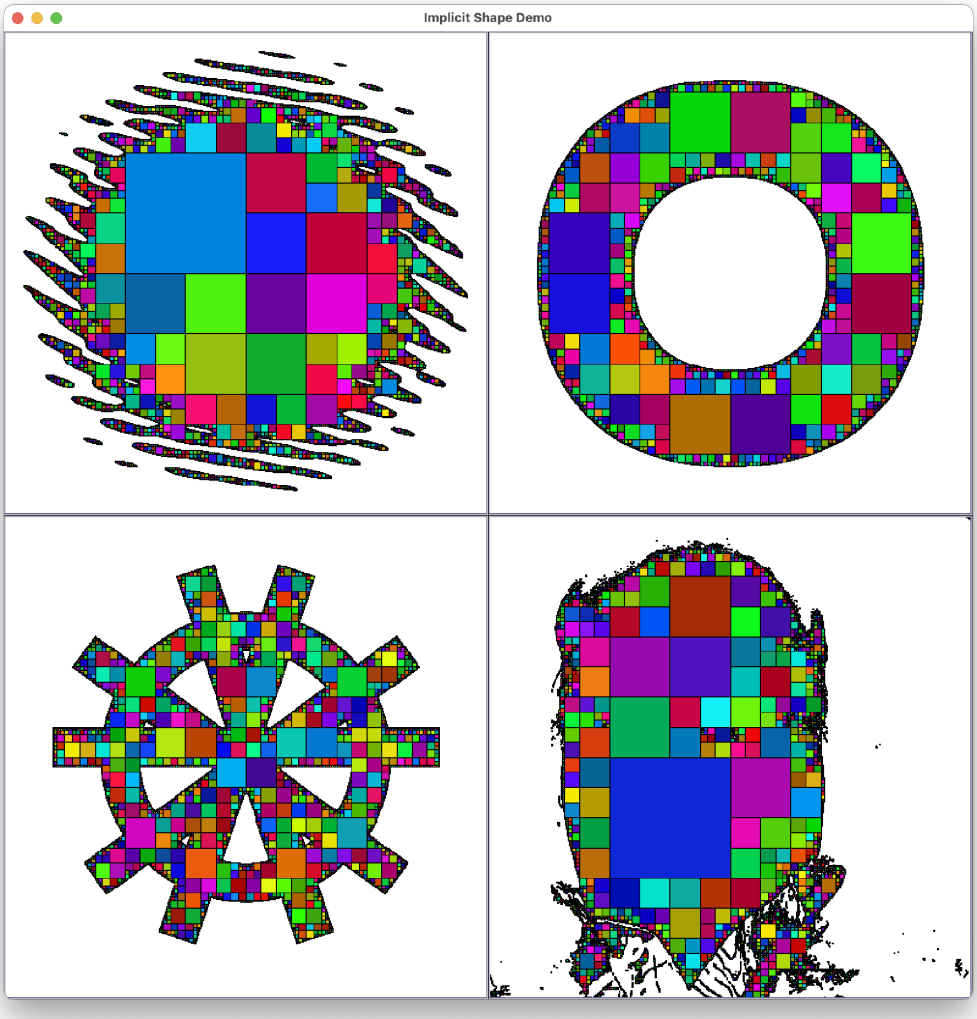
The first parameter is the implicit function *f* that determines the shape rendered through recursive subdivision. Since functions are not **first-class objects** in Java, they cannot be passed back and forth in method calls; instead, the implicit function is wrapped in a strategy object of type `DoubleBinaryOperator` defined in `java.util.function`, and this strategy object is passed as an argument to this method.

The next four parameters define the rectangular box whose contents are rendered in this method call into the `Graphics2D` object `g` given as the last parameter. This box is an axis-aligned rectangle that covers all points whose *x*-coordinate lies between `xStart` and `xEnd`, inclusive, and whose y-coordinate lies between `yStart` and `yEnd`. The width and the height of this box are given by formulas `xEnd-xStart` and `yEnd-yStart`.

This method chooses a random colour for each box it renders, creating the psychedelic sixties effect seen above. To create a random colour, it is bad form to choose random red, green, and blue components, because the palette created this way usually ends up looking like something you would expect to find hanging from the rack inside a Goodwill store. A better technique, as illustrated in the example program `Sliders`, first chooses a random hue from the visible spectrum, then adjusts it with random **saturation** and **brightness**, each between 0.5 and 1.0. Fortunately, the `Colour` class in the AWT package can convert between the RGB and HSB colour spaces.

As you can see from the trippy screenshot, we aim to subdivide the implicit shape by making each box as large a rectangle as we can, and have the subdivision recurse deeper to produce smaller boxes only near the edges of the shape. This approach closely resembles the idea we saw in the earlier lab 31, where a binary grid of pure black and white cells was subdivided into a **region quadtree** whose branches were at different depths depending on the uniformity of the shape represented by that branch.

In the top-level call for each of the four example images in the screenshot, the box is 500 by 500 pixels, with the origin at the top-left corner. The shape depicted in the top left is a circle whose *x*- and *y*-coordinates have been perturbed with some trigonometry before being passed to the circle formula. This example illustrates one big advantage of implicit shapes in that they can be easily **geometrically transformed** by *any* invertible transformation (not merely by the affine transformations of **translation**, **rotation** and **scaling**, as when the shape must consist of a **triangle**



mesh) by first applying the inverse transformation to the point (*x*, *y*) whose membership inside the implicit shape is to be determined.

The shape depicted in the top-right image is an **annulus** created by subtracting a small **superellipse** from inside a larger one, with both superellipses using the exponent 2.3. This example shows another nifty advantage of implicit shapes: their **constructive geometry** operations, **union**, **intersection,** and **difference** (symmetric or not), become trivial by combining the results of two implicit functions with appropriate logical operators. The resulting shape appears seamless when a subdivision algorithm treats the combined shape as a black box.

The bottom-left shape illustrates the `Area` decorator, which can be applied to arbitrary instances of `Shape` to give them superpowers beyond constructive geometry: arbitrary affine transformations. The function to determine whether the point (*x*, *y*) lies inside the shape then translates into a one-liner that asks the `Area` object the very same thing, and converts the received `boolean` answer into a signed `int`. The bottom-right shape uses an implicit function that determines

whether point (*x*, *y*) is in the area from the colour of that pixel in the underlying image; here again, the example image `ilkka.jpg`, with this colourful subdivision transforming your instructor's ugly mug into something more enjoyable.

To achieve this fabulous effect, the `render` method's integer return value should always be one of three possibilities: –1, 0, or +1. A return value of –1 means the entire box lies inside the implicit shape, with no holes inside it, at least to the extent our finite sampling of the infinite continuum can detect. (The Adversary is assumed to be a gentleman who plays ball with this assumption.) Symmetrically, the return value of +1 means that the entire box lies outside the implicit shape. A return value of 0 means some parts of the box are inside the implicit shape, and some are outside.

The base cases for this recursion are when either the box's width or height is less than 1.0, the size of a single pixel. In this base case, the method should pass the box's center coordinates to function *f* and return –1 if that center point lies inside the shape, or +1 if it does not. (In this finite sampling of the continuum, we treat the vanishingly small subset of points that lie precisely on the edge of the shape as being inside.) Note that nothing gets rendered on `g` at these base cases, not even if the center point is inside the shape! After all, we cannot know whether this small base case box will eventually become a part of some larger box whose points also all lie inside the implicit shape.

This method should subdivide boxes too large to qualify as a base case into four equal-sized quadrants, then recursively call itself on each quadrant. The four answers from the NW, NE, SE, and SW quadrants determine whether the method renders at this recursion level and what answer it returns to the caller at the previous level. If all four quadrants return the same answer, either –1 or +1, this method should also return that answer without rendering anything. This is exactly what happened in the base case, since the entire box lies fully inside (–1) or fully outside (+1) the shape.

However, if any of the four recursive calls returns zero, or if any two of the four quadrants return different answers, this method should return zero; before that, render a filled rectangle over precisely those quadrants whose recursive call returned –1. First `fill` the rectangle with a randomly chosen colour as explained earlier, then `draw` a thin black outline around it. Pay attention to calculating the arguments for the `Rectangle2D.Double` the object, since this constructor expects the top-left corner coordinates, width, and height, whereas the method parameters define the box's top-left and bottom-right corners. You need to be careful when calculating the arguments for the rectangle constructor from the box-corner coordinates.

Nothing is rendered for quadrants whose recursive call returned +1, since those quadrants told us their contents lie fully outside the shape currently being rendered. Most importantly, we also render nothing for quadrants whose recursive calls returned zero, since that answer indicates the recursive call already properly rendered the contents of those quadrants! Anything we could render on top of that work would, at best, be superfluous and, at worst, mess up the perfectly good rendering those recursive calls achieved.

Lab 64: Learning The Ropes I: Composition

JUnit: RopeTestOne.java

The `String` data type in Java is powerful and useful, especially because it guarantees **immutability**. Since `String` is essentially a Potemkin village draped over an ordinary character array stored privately inside the String object, you can access individual characters with the charAt method by position in guaranteed constant time. However, string concatenation and substring slicing always take linear time with respect to the string length. Both operations require creating an entirely new `String` object, since the rigidity of the underlying character array requires creating a second character array to store the result.

However, raw character strings aren't the only gunslinger in this dusty backwater town for representing immutable character sequences we can concatenate and slice as needed. This lab and its follow-up introduce a more flexible representation of character sequences: a data structure called a rope. Just like the ordinary `String` type, each instance of `Rope` represents a linear sequence of Unicode characters. For example, we could start with two ropes that represent the character sequences `"hello"` and `"world"`. The truly astonishing property of the rope data structure is that any two instances of `Rope` can be concatenated into a new rope `"helloworld"` so that both time and memory use are guaranteed to be constant, regardless of the length of the two ropes being concatenated! Even more amazingly, the same guarantee of constant time and memory use holds for slicing out a subrope from any existing rope.

Since there is no free lunch, there must be a hidden catch somewhere. Otherwise, why should the ordinary `String` data type even exist in Java, if these ropes could just waltz onstage at any time and belt out, "♫ Anything strings can do, we can do better ♫," to lasso not just our minds but also our hearts? The hidden disadvantage of ropes is that accessing individual characters by position with the charAt method can no longer be guaranteed to take constant time. Still, the time required will be **logarithmic** in the rope's length. Even that guarantee assumes an industrial-strength implementation that can **rebalance** behind the scenes whenever the structure gets too lopsided.

In this lab, we implement a slightly simplified version of ropes that do not perform any complicated rebalancing, since the student will surely agree that even the toy version presented here is already amazing not just in its algorithmic elegance, but as an example of good object-oriented design that we are supposed to be learning in this very course!

Unlike a string object `"helloworld"`, a rope structure isn't necessarily a single object, but a tree made of several rope **nodes**. All these nodes are instances of subtypes of the abstract superclass `Rope` that defines the public functionality common to all ropes.

Start by adding the following implementation of that superclass in your labs project.

```
abstract public class Rope {
    abstract public char charAt(int index);
    private int len = -1;
    public final int length() {
        if(len == -1) {
            // Call the template method to initialize len.
            len = computeLength();
        }
        // Either way, we can now return the cached value.
        return len;
    }

    // Subclasses must override the actual length computation.
    abstract protected int computeLength();
}
```

Each concrete subclass of `Rope` must implement the public method `charAt` to return the character at the given position `index`, in a way that depends on how the subclass stores its data. If the given `index` is out of bounds for the sequence of characters represented by that rope node, this method must throw an `IndexOutOfBoundsException` instead of returning anything. Note that because this exception type is unchecked, it does not appear in the `charAt` method signature.

Each instance of any `Rope` subtype can also tell you its `length`. However, because all `Rope` instances are immutable, their length cannot change after creation, so computing it repeatedly each time user code requests it would be redundant. Therefore, the `Rope` superclass implements `length` as a final method to cache the computed length in the private data field `len`, a classic space-for-time trade-off. Since no rope can have a negative length, this field is initially set to `-1` to indicate that its true value has not been computed yet.

The first time someone requests the length of this rope, the method `length` calls the `protected` **template method** named `computeLength` that each concrete subclass must implement to compute the length of that particular rope instance correctly. The template method design pattern lets both the `Rope` superclass and all user code remain blissfully ignorant of the gory details of how each instance's length is computed in the smoke-filled sausage factory tucked hidden behind the encapsulation interface.

The first concrete subclass of `Rope` represents a simple rope constructed from a given `String` instance. Create the class `StringRope` in your lab project, with the signature

```
public class StringRope extends Rope
```

This class should have a public constructor that receives a `String` parameter that it simply stores into some `private` data field. Both `charAt` and `length` can then call the same methods on the privately stored string and return whatever those calls return.

As with the other subclasses we extend from the abstract superclass `Rope` in this lab, the `StringRope` subclass must also override `toString`, which converts the contents of the rope instance back into a Java string used by the rest of the user code. Inside this class, this method can simply return that privately stored string.

Instances of the second concrete subclass written in this lab represent ropes that have been constructed by concatenating two existing ropes. In your lab project, create the class

```
public class ConcatRope extends Rope
```

whose polymorphic constructor receives two parameters, `left` and `right`, both instances of the type `Rope`. As `StringRope` did with the `String` parameter it received, the `ConcatRope` constructor should store these parameters in data fields with the same names.

The method `computeLength` should call the method `length` of its `left` and `right` pieces and return the sum of these lengths. (Note that you need to specifically call the method `length`, not the method `computeLength`, to enjoy the benefits of caching the length for these pieces!) The `charAt` method should decide which piece to consult recursively for the character at the given index, based on whether that index falls in the left piece or the `right` piece. You can make this decision by comparing the `index` to the `length()` of the `left` piece. Continue from the left piece if the `index` falls in there. Otherwise, continue with the right piece, but first subtract the length of the left piece from the index you pass on!

The third concrete subclass in this lab represents ropes constructed by slicing a subrope from an existing rope. In your lab project, create the class

```
public class SubRope extends Rope
```

The constructor of this class should expect three parameters: the `original` rope being sliced, and the `start` and `end` indices that define the `start` and `end` position of the subrope inside the original. To match the behavior of the `substring` method in the Java String class that this operation mimics, the start index is inclusive, but the end index is exclusive. Also, again in the spirit of the `substring` method in `String`, the constructor of this method must immediately throw an `IndexOutOfBoundsException` if the `start` index is negative or greater than the `end` index (these indices can also be equal, to denote slicing off the empty substring), or if the `end` index is greater than the length of the `original` rope. (The corner case where both `start` and `end` equal the `length` of the string returns an empty rope.)

The `charAt` method can now consult the original rope's `charAt` method, but must remember to shift the `index` right by adding the `start` value. The `length` method simply returns the value of `end-start`. Without access to the original type (which the method shouldn't care one whit about anyway, following the bumper sticker version of the Liskov substitution principle), the toString method has no choice but to create a new StringBuilder and append each character, extracted with `charAt`.

In `SubRope` construction, it is possible to flatten out the resulting tree structure by noticing that if the `original` rope is a `ConcatRope` so that the subrope defined by the parameter indices `start` and `end` falls entirely within either its `left` or `right` part, the newly created `SubRope` can bypass this redundant middleman and point directly to its `left` or `right` part, whichever is appropriate. Similarly, if the `original` rope is itself a `SubRope`, the newly created `SubRope` can always unconditionally bypass this redundant middleman. These optimizations can then descend the tree structure as deep as possible. When these ropes are used to construct complex strings by chopping and gluing together existing ropes, making the resulting tree structure as flat as possible also makes the `charAt` operations performed on those ropes significantly faster.

The JUnit test class RopeTestOne.java uses these three subtypes to build up and slice out ropes of various lengths and contents, including one giant instance of rope that consists of over one billion characters, far too many to be represented as an ordinary `String`.

One major criticism of this design remains that user code must be aware of all the subtypes, `StringRope`, `ConcatRope`, and `SubRope`, since rope operations involve explicitly creating instances of these subtypes with the operator new. However, we can easily remedy this by defining appropriate methods in the `Rope` superclass. For example, concatenating two existing ropes could be defined as a one-line **factory method**

```
public final Rope concatenate(Rope other) {
   return new ConcatRope(this, other);
}
```

to allow the user code to say simply

```
Rope result = r1.concatenate(r2);
```

to concatenate their two existing `Rope` instances `r1` and `r2`, without knowing or caring about the type of the subclass that represents this concatenation. A superior alternative to using a constructor and operator `new`, these factory methods can also enable other optimizations. For example, when extracting an empty subrope where `start == end`, the subrope extraction utility method could always return a reference to the same `Rope.EMPTY` object, instead of redundantly creating a new

object for the empty rope. When you create new objects with the operator `new` instead of using a factory method, you always create a new object, and the type of that object is set in stone at compile time. However, a factory method isn't under those constraints and can return an existing object when one already fits. Since the `Rope` type is immutable, sharing the same `Rope` objects across multiple parts of the user code causes no harm.

Because the rope data structure is immutable, deleting a subrope from an existing rope doesn't modify that rope; instead, it creates a new `Rope` instance representing the result. We don't even need to define a new subclass of `Rope` to represent instances created in this fashion, since even this operation can be made to work in guaranteed constant time and memory once we realize that it is merely **syntactic sugar** for concatenating two subropes tactically sliced out of the original rope, implemented in `Rope` as something like

```
public final Rope delete(int start, int end) {
    Rope left = new SubRope(this, 0, start);
    Rope right = new SubRope(this, end, original.length());
    return new ConcatRope(left, right);
}
```

Another common and useful operation on a text string is reversal. The `String` class in Java doesn't have a `reverse()` method, so we typically reverse with a `StringBuilder`. For ropes, we can define a class `ReverseRope` that holds a reference to the original rope and reverses it. The `charAt` method in this class mirrors the given position index relative to the rope's length and consults the original rope for the character at that mirrored position. If you construct `ReverseRope` via the factory method `reverse` defined in the `Rope` superclass, this method can optimize the case where the rope being reversed is itself a reversal of some rope, and return the original unreversed rope without creating any new objects.

This lab is followed by Part II right around the corner, but before that, you can take a quick side tour to put your ropes to good use in bonus lab 104, "Beklemishev's Worms".

Lab 65: Learning The Ropes II: Comparisons

JUnit: RopeTestTwo.java

After completing the basic rope operations for character sequences in the previous lab, it's time to implement the core methods that make the Rope data type a first-class citizen among other classes in Java and its standard library. That is, these ropes need a meaningful `equals` method for equality comparisons, a `hashCode` method to calculate an integer hash code needed to store arbitrary ropes in a `HashSet<Rope>` for fast lookup and retrieval, and a `compareTo` method to compare two `Rope` objects in lexicographic order.

We'll take the bull by the horns and start with hash codes, since we need to write them anyway and can later use them to speed up equality comparisons greatly. Since `Rope` instances are immutable, their hash codes can never change. We can therefore cache the hash codes the same way we did with the lengths. Inside the abstract superclass `Rope` that was given to you in the last lab, define a new data field

```
private int hash = -1;
```

that will cache the computed value of the hash code the first time that someone requests to see it, available in constant time for all future hash code queries after that. The special value `-1` again denotes that the actual hash code has not yet been computed. Then, write the method

```
@Override public int hashCode()
```

that computes and caches the hash code whenever `hash==-1`, and returns the value of the cached `hash`. The simplest way to turn a character sequence into a reasonably chaotic hash code, so every character affects the computed hash code, is to use a for-loop to iterate through the characters of the sequence from beginning to end. For each character `c` encountered along the way, update the `hash` with the assignment

```
hash = 33 * hash + (int)c;
```

After the loop has terminated, return the accumulated hash. However, you also need to ensure that in the rare case where the `hash` ends up being equal to the special value `-1`, the method returns some other fixed value instead. (Pick any value, for example 17 or 42 or 1234567, since any fixed value is as good as any other value.) Alternatively, if you dislike negative numbers and want the hash code always to be nonnegative, if only to make the job a bit easier for those guys who thanklessly

implement the data structure to use these hash codes, the fast and correct way to do so is to use **bitwise arithmetic** to chop off all but the lowest 31 bits of the signed integer with the expression

```
hash = hash & 0x7FFFFFFF;
```

After implementing `hashCode`, the next job in our TODO list is to allow lexicographic ordering comparisons between arbitrary ropes. Modify the signature line of the entire class to say

```
public class Rope implements Comparable<Rope>
```

to promise that the order comparison is a meaningful operation between any two instances of this type. Implementing the interface `Comparable<E>` then requires you to implement one method

```
@Override public int compareTo(Rope other)
```

whose return value should be `-1` if `this` rope is lexicographically smaller than the `other` rope, `+1` if `this` rope is lexicographically greater than the `other` rope, and `0` when both ropes are equal. To perform this order comparison, loop through the positions of `this` and the `other` rope in lockstep from beginning to end, comparing the two corresponding characters in each position. As soon as those two characters are in any way different, or if this loop runs past the end of either rope, you have your answer and don't have to even glance at the rest of the characters in either rope. You can conclude that both ropes are equal only if you run past the end of both ropes simultaneously, and the method can return 0 at the end.

We intentionally left the equality comparison method for last, since it greatly benefits from the existence of `hashCode` and `compareTo`. However, remember that the equality comparison method

```
@Override public boolean equals(Object other)
```

must accept any `Object` whatsoever as its argument, since in the Java type system, equality comparison is a perfectly meaningful concept between any two objects. (Even if the answer to a question happens to be some laconic variation of "No way, José", this by itself doesn't render the question itself in any way meaningless or malformed!) After using `instanceof` to verify that the other instance is also some kind of a `Rope`, create a new properly typed reference to the argument object with a **downcast**, to allow you to access the `Rope` functionality of that `other` object.

Since the inequality of the hash codes entails the inequality of the objects themselves, this method should start by first checking if both `this` and `other` ropes have their cached `hash` values set to something other than `-1`. If that is indeed the case, and those cached hash values are different, you can smoothly return `false` without further ado. Otherwise, call the previous `compareTo` method

to find out how `this` and `other` ropes relate to each other in lexicographic order, and return `true` if and only if these ropes are equal in that sense, as this can only happen when the ropes themselves represent equal sequences of characters. The two ropes can have the same hash code even if they represent different character sequences. As unlikely as this may be, it is far more likely to happen over time than most people would intuit, so your code needs to be prepared for it.

Lab 66: Slater–Vélez Sequence

JUnit: SlaterVelezTest.java

The lab problem 17, "Diamond Sequence", as well as the two more recent sequences implemented in problem 0(L), "I Can, Therefore I Must", were based on the idea of always continuing the given positive integer sequence with the smallest previously unseen positive integer that does not create a violation of the sequence-defining rule with the earlier values in the sequence generated so far. This lab continues the same grand theme by generating the elements of the remarkable Slater–Vélez sequence, with this author learning of this lively sequence only from the Code Golf problem the link leads to.

Each element of the Slater–Vélez sequence is the smallest positive integer *n* that has not appeared so far in the sequence, under the constraint that the absolute difference between *n* and its immediate predecessor in the previous position of the sequence also has not previously appeared as the absolute difference of any two consecutive elements in the sequence. This definition makes the sequence unique, and the reader can verify that the Slater–Vélez sequence starts as

1, 2, 4, 7, 3, 8, 14, 5, 12, 20, 6, 16, 27, 9, 21, 34, 10, 25, …

with the corresponding sequence of absolute differences between consecutive elements being

1, 2, 3, 4, 5, 6, 9, 7, 8, 14, 10, 11, 18, 12, 13, 24, 15, …

It follows trivially from the rule used to construct this sequence that no element can ever repeat. However, as in the diamond sequence, this sequence leaves no holes on its journey toward the omega; it will eventually visit every positive integer, as Slater and Vélez proved in 1979. This sequence is therefore an infinite permutation of the positive integers. However, its cycles are not as tight as those in the diamond sequence, where every cycle has length two.

As in the diamond sequence lab, we implement this sequence as a virtual iterator that computes the next element lazily on demand. Unlike Python, Java doesn't have built-in lazy sequences or generators. The **iterator framework** is the next-best way to achieve the zen bliss of enlightened laziness. In your lab project, create the class

```
public class SlaterVelez implements Iterator<Long>
```

Since the sequence is infinite, the method `hasNext` should always unconditionally return `true`. We will tacitly assume that the sequence won't be generated far enough for our values to get their heads chopped off by spilling outside the Procrustean bed of 64-bit `long` integers.

Since the sequence generator needs to track the numbers and their absolute differences it has already seen, we can again follow the lead of the diamond sequence problem and use two instances of the `NatSet` utility class. Inside your class, define three data fields

```
private NatSet seen = new NatSet();
private NatSet diffs = new NatSet();
private long previous = 0;
```

The two instances of the `NatSet` utility class, named above `seen` and `diffs`, keep an eye on the positive integers and their absolute differences that have already been `seen` in the sequence generated so far. The class must also remember the previous number in the sequence; initially, set it to zero to indicate that nothing has been generated. To complete the obligations of implementing the interface `Iterator<Long>`, write the method

```
public Long next()
```

to virtually extend the sequence by one more element that becomes the new `previous` number and gets returned from this method. This method should also update the `seen` and `diffs` sets to correspond to the new sequence that is now one element longer than it was before this method call.

Since the `NatSet` instances keep track of natural numbers that include zero, unlike the positive integers that start from one, make sure to add zero as a member to both `seen` and `diffs` sets in the constructor of this class, before you start generating the actual values. Furthermore, to speed up the generation of the `next` element of the sequence, you should note that the `NatSet` utility class comes with the convenience method

```
public long allTrueUpTo()
```

that returns the smallest natural number that is not a member of this set. With this invaluable information about the numbers you have already seen in the sequence, you know right away where to start looking for the next number, instead of always starting from one. By cleverly using also the information about the `previous` value and the smallest difference that is not yet a member of the `diffs` set, you can sometimes jump even higher for your starting point to look for the next element, without any risk of jumping over the smallest positive integer that would have satisfied the rule of this sequence. With these optimizations, your code should be able to produce the first half a million elements of the Slater–Vélez sequence in a couple of seconds.

Lab 67: Lemire Dynamic Frequency Sampling

JUnit: LemireFrequencySamplingTest.java

To sample a random element fairly from the given array `arr` so every element has an equal chance of being chosen, use rng.nextInt(arr.length) to get a random position to fetch the element from, where `rng` is your random number generator. However, sometimes we don't want all elements to have the same uniform probability; instead, we want the random choices to follow the given frequency array, which has the same length as `arr`.

Each element of the frequency array gives the **probability** that the corresponding original element will be chosen in the long run. For example, if the original array `arr` has five elements, the frequency array `{3, 4, 0, 1, 4}` would indicate that the element in position 0 with frequency 3 would get sampled roughly three times as often as the element in position 3 with frequency 1, and roughly 3/4 as often as the element in position 1 or in position 4. As you can see, some frequencies can be zero, so those elements should never be sampled.

To sample the next element from the given frequencies, each element must have a probability *of f/s* of being sampled, where *f* is that element's frequency, and *s* is the sum of all frequencies in the frequency table. For example, for the above example frequencies, the probability for the element in position zero to get sampled would be 3 / (3 + 4 + 0 + 1 + 4) = 3/12 = 1/4.

Note that the choice of each random element must be done independently of the history of the past selections, with no attempt to "correct" the frequency of some element that so far has not appeared as much as its desired frequency would predict. In the short run, all random walks exhibit variance and clustering, so the sampled elements' frequencies will rarely match the frequency table exactly. In the long run, the random sampling algorithm should see the proportion of each sampled element converge to its given frequency divided by the sample size *s*, in line with the frequentist interpretation of probabilities.

The simple algorithm that samples a random element according to the given frequencies starts by creating a random number *r* between 0 and *s* – 1. It then adds element frequencies from the beginning until r becomes strictly smaller than the cumulative sum up to that position. The element at that position is then returned as the answer. In Java, the method might look something like this. (Note the zero-based counting used in both quantities `total` and `pos`.)

```
public int frequencyChoice(int[] arr, int[] freq, Random rng) {
    int s = 0, total = 0, pos = 0;
    for(int f: freq) { s += f; }
    int r = rng.nextInt(s);
```

```
    while(total <= r) { total += freq[pos++]; }
    return arr[pos - 1];
}
```

This algorithm works well for sampling a single element from a small set. However, if you need to sample many elements one at a time, the first obvious optimization is to compute the sum of frequencies s once and pass it to this method as an additional parameter, rather than recomputing it in the first for-loop each time. If the array `arr` is large, the next opportunity to optimize is to precompute the **accumulation array** for these frequencies, as seen back in Lab 55, "Accumulated wisdom". Using this accumulation array, the binary search algorithm finds the position pos where the sum first exceeds `r` and runs in logarithmic time, rather than using a for-loop to find that position in linear time.

The previous discussion assumed sampling with replacement, so sampling a random element does not affect the frequencies used in future sampling. However, the problem becomes more interesting when adapted to sampling without replacement: each time we sample the element at position i, its associated frequency `freq[i]` should decrease by one! It is not immediately obvious how to do this efficiently. However, in this lab we shall implement a slight variation of the ingenious technique to achieve this goal, as explained by Daniel Lemire in his blog post "Sampling efficiently from groups".

In your lab project, create a new class named `LemireFrequencySampling`, in which you will be writing the following two static methods.

```
public static int[][] buildSamplingTable(int[] freq)
public static int updateSamplingTable(int[][] table, int r)
```

Given an arbitrary frequency array `freq`, the first method builds and returns a two-dimensional sampling array. The topmost (zeroth) row of this sampling array has the same elements as the original frequency array `freq`. After this zeroth row, the length of each following row is always exactly one half of the length of the row immediately above it. The *i*:th element of each row equals the sum of the two elements in the positions 2*i* and 2*i* + 1 in the row immediately above it. For simplicity, this method may assume the frequency array has a length that is an exact power of two. (Any frequency array of some other length can always be **padded** with zeroes until its length becomes some exact power of two, without affecting the probability of any element being sampled.)

For example, given the frequency array {`4, 0, 1, 5, 2, 2, 3, 4`}, the two-dimensional sampling array would have four rows. Row 0 would have eight elements, matching that frequency array. The next row (row 1) would have length 4, and its elements would be {`4, 6, 4, 7`}, the sums of pairs of elements in the previous row. The next row (row 2) has length 2, with elements {10, 11}, since 4+6 = 10 and 4+7 = 11. The bottom row 3 has one element, {21}, which is always the sum of the original frequencies.

The technique introduced in Professor Lemire's blog post cleverly performs these accumulations **in place** in the original frequency array. However, this compact representation also makes the random sampling-without-replacement algorithm slightly trickier. To sidestep these pitfalls, we trade space for clarity, a tacit tradeoff similar to its more famous cousin, **trading space for time,** but surely more common in practice.

The second method takes as input the sampling table constructed in the previous method and a nonnegative integer *r*, which serves the same purpose as the local variable r in the earlier linear-time algorithm for frequency sampling. This method should find and return the position in the current frequency array (that is, the current zeroth row of the given sampling `table`) where the sum of frequencies up to that position becomes strictly greater than *r*. Before returning that position to the caller, decrease the frequency at that position by one, since the randomly sampled element is not returned to the population.

Since our automated JUnit tester will repeatedly try this method with frequency tables of over a hundred thousand elements, updating the entire sampling table to reflect the new situation must be blazingly fast for the tests to finish in an acceptable total time. Fortunately, the ingenious construction of the sampling table lets us do this in guaranteed logarithmic time with respect to each sample's frequency table length. The sampling algorithm is simplest to implement as a while loop that starts at the bottom row of the sampling table and keeps climbing until it reaches the zeroth row. During this climb, the body of the loop will update exactly one element of the current row, and look at exactly one element in the previous row to decide which direction to continue. The loop's total work is therefore linear in the number of rows in the sampling table. This quantity is, in turn, logarithmic in the length of the original array and its frequency table.

The while-loop in this method should track the `row` and `col` position of its traversal up the sampling table, starting at the lone element in the bottom row. The loop body should first decrement the element at the current row and col, then look at the element e at `[row-1][2*col]` to decide which direction to continue. If `r<e`, continue left to the column `2*col` in `row-1`, keeping the value of `r` as it is. Otherwise, continue right to the column `2*col+1` in `row-1`, but also decrease `r` by `e`. Once you reach the zeroth row of the sampling table, decrement the frequency stored in the column that you ended up at, and return the index (not the value!) of that column to the caller.

Lab 68: Interval Sets I: Dynamic Set Operations

JUnit: IntervalSetTest.java

Bit vectors can store a set of natural numbers compactly, using only one bit of memory for each element in the appropriate type and range that could potentially be in the set. However, this representation can still use space very inefficiently for **sparse** sets, where most bits are false and only a few true bits are scattered throughout the vast emptiness. In situations where both the members and the nonmembers of the set tend to form contiguous **clusters**, a vastly more efficient representation for such sets can be achieved with ideas adapted from the technique of **run-length encoding** by storing not merely the individual elements, but entire consecutive **intervals** of contiguous elements that are members of the set.

This lab has you implement a rudimentary version of this approach to represent clustered sets of natural numbers in a highly compact way, so their basic dynamic set operations, add, contains, and remove, are as efficient as the student will hopefully become effervescent while writing this code. This lab also serves as an exercise in using the `LinkedList<E>` subtype from the Java Collection Framework along with its **list iterators** that not only allow the **bidirectional traversal** back and forth through the elements of the list one step at a time, but also constant-time modifications with the operations `add` and `remove` performed at the current location of the iterator, as opposed to the eponymous list methods that operate on that list as a whole.

Compared to the more commonly used `ArrayList<E>` collections, linked lists are appropriate for situations where we require constant-time **random access** to the elements in arbitrary positions, but always traverse the elements sequentially from left to right, with the option to occasionally update the contents of the list in constant time at the current position of the iterator traversal. For the `ArrayList<E>` collections, the rigidity of the underlying array makes such local modifications inefficient, since the elements that follow the current position all have to be moved one step forward or backward after the modification to either make room for the new element that was added, or fill in the gap left behind by the element that was removed.

The best way to understand the behavior of a `ListIterator<E>` instance acquired from the underlying `LinkedList<E>` object is to think of it as a **cursor** that points not at an element directly, but at the invisible space that separates every two consecutive elements of the list. For example, the underlying linked list of integers might contain the following elements, with the current location of the iterator cursor shown as a vertical bar between the elements 42 and 333:

```
17   99   42   |   333   81   4   87
```

Calling `next()` on the iterator skips the element immediately following it and returns the value of the element it skipped. You should generally call `hasNext()` before this move to determine whether the cursor has passed the end of the list, to avoid throwing NoSuchElementException when trying to skip over a nonexistent element.

For example, calling `next()` in the situation depicted above would return 333, with the new cursor position inside the list becoming

```
17    99    42    333   |     81    4     87
```

Since each **node** of the underlying linked list contains references to both its successor and predecessor nodes, the `ListIterator<E>` can support bidirectional traversal. The methods `previous` and `hasPrevious` operate symmetrically to the methods `next` and `hasNext`.

The `remove` method of the `ListIterator<E>` cursor removes the element that the cursor stepped over during its most recent `next` or `previous` operation. The iterator cursor remembers only the most recent element that was stepped over. Therefore, if you call `remove` twice without calling `next` or `previous` in between, the second call to remove will fail and throw an `IllegalStateException` to indicate that the iterator is not in a proper state to perform the requested operation.

For example, calling `remove` after stepping over the element 333 with the previous call to the method `next` would create the following situation, with the cursor position unchanged:

```
17    99    42    |     81    4     87
```

The iterator cursor's `add` method adds a new element before the cursor position. Unlike the `remove` method, this method can be called any number of times regardless of whatever calls to `next` and `previous` have taken place in the interim. For example, calling the method `add(38)` for the iterator cursor would produce the new situation

```
17    99    42    38    |     81    4     87
```

Be sure to appreciate what a different thing it is to call the method `add` for the iterator to add the new element to the current position of the iterator cursor, as opposed to calling the method `add` in the list object as a whole to add the new element at the end of the list, regardless of any iterator cursors that might exist at that moment.

These preliminaries give us enough to begin implementing the set of intervals. In your labs project, create a new class `IntervalSet` whose instances represent sets of natural numbers stored as **intervals** of consecutive integers that are members of that set. For example, an `IntervalSet`

instance that will print out as `"[4-10, 15, 20-27]"` would contain all integers from 4 to 10, the integer 15, all integers from 20 to 27, and no other integers. Each interval is considered to be inclusive at both ends. **Singleton** intervals whose `start` and `end` values are equal are printed as the starting value only, without the separator dash and the end value.

You should start by first defining the following **nested class** inside the `IntervalSet` class to represent an individual interval:

```
private static class Interval {
```

This class should have two public integer data fields, `start` and `end`, and a constructor to initialize these fields from its parameters. Note that we are not designing this nested private class to be immutable, but allow the methods of the surrounding `IntervalSet` to modify the `start` and `end` values of any given `Interval`. You should also override the `toString` method of the `Interval` class to return the string of the form `"start-end"` whenever `end>start`, and just `"start"` when these two values are equal.

Each `Interval` instance represents the fact that the set contains all elements from `start` to `end`, inclusive. Each `IntervalSet` is therefore a collection of disjoint intervals stored in a linked list. Each `IntervalSet` instance should therefore contain the field

```
private LinkedList<Interval> intervals = new LinkedList<>();
```

in which the intervals that form the interval set are kept in strictly increasing order. During execution, we will ensure that all intervals are maximal, so no two can be combined into a single larger interval that contains both. For example, instead of storing two intervals `10-20` and `21-30`, the list should contain only one interval, `10-30`, with the same information in a more compact form.

To help debug the dynamic set operations that you will be writing in this class, you should first override the `toString` method for the `IntervalSet` class to return a string that contains square brackets as its first and last characters, followed by the individual intervals as produced by the `toString` method of the `Interval` class, consecutive intervals separated by exactly one comma and one whitespace character. Make sure to also not have any silly trailing commas or whitespace at the end. Once you have completed this method, you can start filling in the bodies of the required dynamic set methods

```
public void add(int start, int end)
public void add(start)
public boolean contains(int start, int end)
public boolean contains(int start)
public void remove(int start, int end)
```

```
public void remove(int start)
```

Since the operations on **singleton** intervals whose `start` and `end` are equal would presumably be common in the user code of this data structure, we will provide the convenience versions of the dynamic set methods that take only one parameter each. All three convenience methods should be simple one-liners that call the more general two-parameter version of that same method by duplicating that parameter. The student should read through the explicit test cases of our JUnit test class to get an idea of how these methods are used, and what their expected results are in the provided test situations.

The `add` method should first create a new `Interval` object with the given `start` and `end`, and ask the list of `intervals` for a list iterator to traverse through the existing `intervals` in the list. For each existing interval, check whether it could be **absorbed** into the new interval that is being added to the set. To absorb an existing interval, `remove` it from the list, and update the `start` and `end` values of the new interval accordingly to make the new interval fully contain the absorbed one. For example, adding the interval `10-20` to the set could absorb an existing interval `5-15` and become `5-20`, so the now-redundant interval `5-15` can be removed from the list. Note that an interval can also absorb another interval if either one of these intervals `start`s at the value that is one larger than the `end` value of the other interval. For example, the new interval `4-9` can absorb an existing interval `10-15`, and become a single interval `4-15`.

As the traversal through the existing `intervals` removes intervals absorbed into the new interval being added, one of two things must eventually happen. Once the traversal reaches an existing interval that starts after the end of the new interval being added, add the new interval at that position and terminate. If the traversal reaches the end of the list before finding such an interval, the new interval is appended to the end of the list.

Because we store intervals in a maximal, non-redundant way, the contains method that checks whether all numbers from start to end are currently in the set is straightforward. This method only needs to iterate through the list of existing intervals to find one that fully contains the interval being searched for. If no such interval exists, the method can return `false`, knowing that at least one number from start to end is not in the set.

As usual with dynamic sets, the `remove` method is the most complicated of the three dynamic set operations. (Removing an element from any data structure decreases its entropy, so the operation must work against the natural flow of the universe.) The `remove` method should iterate through the existing `intervals` in the list and update the list and the intervals stored therein based on how the existing interval overlaps the interval of the values being removed. To illustrate the different cases and how to handle them, assume, for simplicity, that we are removing the interval 10-20 to make it easier to visualize what to do for a given existing interval in the list.

- If the existing interval is fully inside the interval being removed, such as `10-14`, `15-18`, or `13-20`, simply remove that existing interval from the list.
- If the existing interval contains the interval being removed so that their endpoints coincide, such as `10-30` or `5-20`, update the opposite endpoint of the existing interval, and terminate.
- If the existing interval properly contains the interval being removed so that the endpoints do not coincide, such as `7-30`, modify that existing interval to end one step before the removed interval begins, and add a new interval for the values that follow the interval being removed. For example, removing `10-20` would split the existing interval `7-30` into two separate intervals `7-9` and `21-30`. Again, you can terminate the method after this operation.
- If the interval being removed partially overlaps the existing interval from the left, such as with the existing interval `15-40`, update the `start` value of that existing interval to one past the `end` of the interval being removed. For example, removing `10-20` from the existing interval `15-40` makes that interval `21-40`.
- Symmetrically to the previous item, if the interval being removed partially overlaps the existing interval from the right, such as with the existing interval `0-14`, update the `end` value of that existing interval to one less than the `start` of the interval being removed. For example, removing `10-20` from the existing interval `0-14` makes that interval `0-9`.

The mass test of these three operations creates many pseudorandom intervals to add to and remove from the current `IntervalSet`. You can uncomment the lines that print out the current contents of the interval set in the mass test method of the JUnit test class to watch the results that your methods produce for all these pseudorandom additions and removals.

Lab 69: Interval Sets II: Iteration

JUnit: IntervalIteratorTest.java

The three dynamic set operations `add`, `contains`, and `remove` by themselves are not enough for the common use case of **iterating** through the elements of that dynamic set so that each element is visited exactly once through this traversal. To follow the lead of the classes in the Java Collection Framework, we shall adapt the `IntervalSet` class from the previous lab to also allow this. Since it must be possible for multiple iterators to simultaneously exist and point to different locations inside the same collection, the collection object itself cannot act as its own iterator. The iterator objects are constructed from a separate class that has enough access to the collection's private internal details. An **inner class** is exactly what we need for this purpose. Therefore, inside your `IntervalSet` class, define the inner class

```
private class IntervalIterator implements Iterator<Integer>
```

whose instances act as independent iterators on the particular `IntervalSet` instance from which they were originally acquired. Because `IntervalIterator` is an inner class, not a static nested class, its instances cannot exist independently of other objects; they can only exist in the context of an IntervalSet object. Making `IntervalIterator` an inner class gives its methods direct access to all members of the outer class, even those that are `private`. This doesn't require any special syntax, but it works the same way these methods access their own members.

Your `IntervalIterator` objects should themselves contain a data field that is a reference of type `ListIterator<Interval>`, to keep track of which `Interval` in the list of `intervals` in the outer class object their traversal is currently at. Another `int` data field named `nextValue` will then keep track of the particular integer value inside the current `Interval` the iterator's traversal is currently at. After initializing these fields in the default constructor of `IntervalIterator`, you have everything you need to implement the two required methods of the `Iterator` interface

```
@Override public boolean hasNext()
@Override public Integer next()
```

Although we could make iteration more general by defining it as a `ListIterator` with `hasPrev` and `prev` methods symmetric to the two methods above for forward iteration, we will keep this lab simple by allowing iteration only in the forward direction. The method `hasNext` should return `true` if the previous `ListIterator<Interval>` reference is non-null, meaning at least one value remains to be visited during the current iteration, since we never store empty intervals in the `intervals` list. The method `next` returns the current value of `nextValue` in the current Interval

and also increments nextValue, telling the list iterator to step to the next Interval in the outer intervals list whenever nextValue exceeds the current Interval's end value.

To keep things simple, we also choose not to allow mutation of the `IntervalSet` directly through the iterator methods `add` and `remove`, so we don't have to implement these two methods in the inner class. Once you have completed this inner class, you can add the following instance method to your `IntervalSet` class

```
public Iterator<Integer> iterator() { return new IntervalIterator(); }
```

to allow the outside user code to acquire iterator instances to the particular `IntervalSet`, and finally change the signature of your `IntervalSet` class to the form

```
public class IntervalSet implements Iterable<Integer> {
```

to proudly come out of the closet and announce to the whole world the good news that it has now become possible to iterate through the instances of your `IntervalSet`! Even the **for-each loop** will now work for the `IntervalSet` instances out of the box, just like it does for all those snobby Boston Brahmin array and collection types who landed on Plymouth Rock back then, when the Java language and its standard library were first founded.

The iterators in the Java Collection Framework and elsewhere vary widely in how tolerant they are of their underlying collection being mutated during iteration. In one extreme, some collection subtypes are hardened to tolerate **concurrent** access and mutation from multiple execution threads, without interfering with iteration over the contents of the set at the time the iterator was created. However, in this lab we will make `IntervalSet` iterators the other extreme by being fail-fast; any mutation of the collection while it is being iterated makes any active iterators fail the next time their `hasNext` or `next` methods are called.

This design decision not only makes our lives simpler, but also allows us to demonstrate the useful programming technique of **timestamping** that allows our iterators to immediately detect any mutations done on the underlying collection, without these iterators actually having to look at the elements of the underlying collection to determine whether some stealthy mutation has occurred since that iterator was last accessed. In the outer class `IntervalSet`, define a field

```
private long timestamp = Long.MIN_VALUE;
```

and modify the `add` and `remove` methods of the `IntervalSet` class to increment this timestamp every time that they are called. The inner class `IntervalSetIterator` should then also have a new data field

```
private long initialTimestamp;
```

initialized in the constructor of `IntervalSetIterator` to the current value of the outer class `timestamp` field. The iterator's two methods, `hasNext` and `next`, should first check that the initial timestamp stored in that iterator object still equals the outer class's timestamp; if not, throw a `ConcurrentModificationException` instead of performing the requested operation.

Since the timestamp needs to be stored only once per `IntervalSet` instance and once per iterator object, the use case of millions of iterators being vanishingly unlikely in practice, we might as well use the eight-byte `long` integer type to keep track of the timestamp values to ensure that they can't realistically overflow during the use of this data type. ("Four billion mutations on a set should be enough for anybody" just somehow sounds like famous last words, even if given in the voice of Carl Sagan, unlike the same sentence starting with the more impressive "Sixteen billion billion mutations…")

Lab 70: Cup Tournament Survival

JUnit: CupSurvivalTest.java

After these commercial interruptions, we resume the live national sportsball championship tournament with n teams numbered 0 to n – 1, where n is guaranteed to be a power of two. Each round, pairs of teams play a match that determines which team advances to the next round. The number of teams is halved each round until the final match determines the tournament winner. For example, if n = 8, and the teams are numbered from 0 to 7, the first round has matches between teams 0–1, 2–3, 4–5, and 6–7. The method that you write in this lab will compute the exact probability for each team to win the tournament, using an iterative process described below to build up these probabilities bottom-up, using the `Fraction` type to represent the intermediate and final probabilities exactly.

In your lab project, create a new class named `CupSurvival`, and in it, the static method

```
public static Fraction[] computeSurvival(Fraction[][] winProb)
```

that returns an array of n fractions that together add up to one, giving the winning probabilities for each team. The parameter `winProb` is an n-by-n array of fractions, where `winProb[i][j]` is the exact probability that team `i` wins against team `j`. (Perhaps these probabilities were computed by the famous sportsball handicapper "Ilk the Finn".) Since sportsball has no ties, each match guarantees exactly one participant advances to the next round. The two symmetric probabilities `winProb[i][j]` and `winProb[j][i]` always add up to exactly one.

Instead of estimating the winning probability for each team in a **frequentist** spirit by simulating the entire tournament a large number of times and estimating the winning chances of each team as the percentage of these simulated tournaments that the team ended up being the top dog, we will rather compute these probabilities as exact fractions from the bottom up without any randomness. The summations in the **marginalization** formulas for these probability calculations can get pretty hairy, but that is why we have bred our machines to do these calculations, instead of expecting us slow, hairless apes to grind through them by hand.

For the bookkeeping needed to compute the winning probabilities from the bottom up, the method should define as a local variable a two-dimensional array

```
Fraction[][] survival = new Fraction[k][n];
```

where k is the number of levels in the tournament cup tree. For example, k = 3 when n = 8. Each quantity `survival[r][i]` gives the probability that team i wins its match against its opponent in round r. Once the method has filled this table, it can return the entire row k – 1 as the final answer,

since the probabilities in that row are exactly the probabilities that each team wins the entire tournament.

This method should start by filling all elements of the bottom row `survival[0]` with values equal to one. Before you start coding this method, you might want to define two named constants

```
private static final Fraction ZERO = new Fraction(0, 1);
private static final Fraction ONE = new Fraction(1, 1);
```

Define these inside your class to make your method code easier to read.

After initializing the bottom row with ones, you can compute each probability `survival[r][i]` in the higher rows using a recursive formula that involves the probabilities in the previous row `(r-1)` and the winning probabilities for the individual matches between the two teams, given as a parameter to this method. To avoid exponential time from redundant recomputation, your method should fill the survival array in a dynamic-programming style using three nested for-loops, with no recursion used anywhere. The outer loop will count through the rows from 1 up to $k - 1$. For each row, the inner loop will count from 0 up to $n - 1$ to fill in the survival probabilities for the teams during that round. This computation requires another loop inside, so the algorithm has three levels of nested loops. (Hey, it's action, isn't it?)

The probability that team `i` wins its match in round `r` is the product of two probabilities that represent the **parlay** of two events that must both occur for team `i` to win its match in round `r`. First, team `i` must reach round `r` to play there and have a chance to advance; that is, team `i` must win its match in the previous round `r-1`. Conveniently, this value was already filled in the `survival` table in the previous outer-loop round! Having crossed the tall hurdle of showing up, the team `i` must win its match against the team that becomes its opponent in round `r`. Marginalization of the combined probability into the sum of its disjoint elementary possibilities expresses the desired probability as the sum of simple terms of the form `survival[r-1][j]*winProb[i][j]`, where the innermost loop variable `j` counts through all the teams that could end up becoming the opponent for team `i` during round `r`.

The only remaining problem is to determine which teams this innermost loop must go through to cover precisely the teams that can meet team `i` in round `r`. We start by noting that if team `i` starts at position `i` in the cup tournament tree and wins its first match, it will be in position `i/2` in the next round, where `/` is **truncating integer division**. Continuing this reasoning, team `i` reaches position $\texttt{i/2}^{\texttt{k}}$ of that level in the cup tournament tree after winning `k` matches in a row. Although Java does not have an integer exponentiation function, you can compute powers of two easily with bit shifts, since `(1 << k)` always gives 2^k.

To determine team `i`'s potential opponents in round `r`, first compute the position `p` where team `i` would end up in round `r-1` if it won all its matches to reach that far. If position `p` is odd, the opponent would play in position `p-1` in that round; if position `p` is even, the opponent would play in position `p+1`. To traverse down the cup tournament tree back to the original teams that could end up in that opponent's position in round `r-1`, simply multiply the opponent's position `p` by the factor 2^{r-1} to find the first team in the bottom row that could become the opponent for team `i` in that round. For round `r`, there are exactly 2^{r-1} such teams, and since they are all consecutive in the original bottom row, it is easy to loop over them. For example, if either team 0 or team 1 wins the first match, its potential opponents in round two are teams 2 and 3, and in round three, teams 4, 5, 6, and 7.

(Readers who will next take a course on data structures and algorithms may note the similarity between the previous formulas for moving up and down the cup tournament tree and the formulas used to move up and down the positions of a binary heap data structure, to implement an efficient **priority queue**.)

Even when the original winning probabilities are simple fractions such as one half or one third, the summation formulas over all possible opponents for round `r` quickly produce complicated fractions with dozens of digits in their numerators and denominators. To make these results exactly reproducible across past, present, and future computer systems so we can write a meaningful JUnit test for them, we cannot compute them in floating point; we must use `Fraction` objects. To paraphrase the famous slogans of yesteryear, winners don't use floating point. Floating point — Just Say No! Friends don't let friends use floating point. Dare to keep kids away from floating point. It's float o'clock — do you know where your kids are? A friend with floating point is no friend indeed. For most programs on this computer, the days of having a floating point in every register are gone. Are you better off now than you were four floating-point operations ago? In your heart, you know `int`'s right!

You starting to get the floating point?

Lab 71: The Gamma And The Omega

JUnit: EliasCodingTest.java

Positive integers are simple enough to encode and process as **binary numbers** using sequences of zeros and ones. However, encoding not just a single integer but an arbitrary sequence of integers requires a way to recognize where each integer's binary representation ends, and the next begins. For example, would a given sequence of 32 bits encode a single 32-bit integer, two 16-bit integers, one 23-bit integer followed by a 9-bit integer, or some other way to partition those bits into separate pieces? We have to resolve this without any hand-waving, such as adding separator symbols between the binary numbers to make the payload-bit boundaries stand out.

Of course, we can agree that each number is stored in a fixed number of bits, typically 16 or 32, or these days, actually 64. (As an aside, whenever you need to make up a **magic number** to limit something in computing, make that limit some power of two, or some power of two minus one, and suddenly nobody will question even the existence of your made-up limit, but will imagine the implementation artifact that created this limit to be some kind of immutable law of the universe.) However, not only would such a policy set a hard upper bound to the magnitude of the integers that could be encoded in this system, but also end up transporting air by stretching all those itty bitty numbers to fill up this Procrustean bed by padding them from the front with redundant zero bits that convey no additional information.

We already saw one way to solve this problem back in lab 15 with the Zeckendorf encoding. Break down the given positive integer into the unique sum of nonconsecutive Fibonacci numbers, then use the bits to indicate which Fibonacci numbers appear in the sum. The impossible bit sequence `11` separates the encodings of two adjacent integers. In Zeckendorf encoding, the separator pattern 11 appears at the end of each integer's encoding, like the period that ends each sentence in English. In this lab, we look at two alternative ways to solve the same problem by prefixing each integer with an unambiguous indicator of its bit length. We start with the simpler but less efficient technique of Elias gamma coding, then move to its more advanced, tighter cousin, Elias omega coding.

To encode some positive integer *x* whose binary representation consists of *N* + 1 bits (that is, *N* is the highest power of two that fits inside *x*), the Elias gamma coding prefixes the length *N* in **unary** by using exactly *N* zeroes, followed by the *N* + 1 bits of the integer *x* itself from highest to lowest. For example, consider the integer 13, which is `1101` in binary, so *N* + 1 = 4. This number would be gamma encoded as the bit sequence `0001101`. Note that since the highest bit of any positive integer is 1, the **1** in the middle cleverly plays a dual role as both the terminator of the length prefix of unary zeros and the highest-order bit of the number *x* itself.

In your lab project, create a new class named `EliasCoding`, and in it implement the following two methods to encode and decode a list of `BigInteger` values into a seamless sequence of zeros and ones using the Elias gamma coding. Of course, an adult, real-world implementation would emit

actual bits and bytes instead of Unicode characters for maximal compression. However, this lab exercise is a proof of concept for learning and demonstration, so the resulting bit sequences are expressed as strings containing only 0 and 1 to simulate the bits that would actually be emitted.

```
public static String encodeEliasGamma(List<BigInteger> items)
public static List<BigInteger> decodeEliasGamma(String bits)
```

In the spirit of "**design by contract**", the encoding method can assume that all `items` are positive, and the decoding method may assume that `bits` is a complete and legal Elias gamma coding of some list of `BigInteger` values. Neither method has to handle and recover from errors when this is not the case. As the party in complete control of the material facts that affect the preconditions, the caller always bears the fault for the consequences of passing method-argument values that violate the method's preconditions.

Instead of performing all the bitwise computations by yourself, it pays to remember that the people who created the Java standard library did not choose the classes and methods included in this club as some sort of dadaist abstract performance art event, but to serve programmers by giving them operations that were not important enough to gain entry to the inner circle of the language itself, but are still common enough to show up in all problem domains that it would just be wrong to expect every programmer to roll their own versions of those operations.

Therefore, before starting to hammer in the above two methods, you should take a moment to appreciate how the constructor of the `BigInteger` class has been overloaded to accept the **radix** as its second parameter, so you can have the first parameter string be in binary instead of base ten just by using the radix of two. Similarly, `toString` has been overloaded to accept the desired radix as the second parameter. Finally, `bitLength` returns the length of the binary encoding of the positive `BigInteger`, without constructing that string representation. Subtracting one from the result of `bitLength` therefore gives you the highest power of two less than or equal to that `BigInteger`.

The Elias gamma coding is simple enough to understand and implement. However, a moment's thought reveals its inefficiency: it almost doubles the number of bits needed to encode each integer! We can do *much* better. By skipping intermediate Elias delta coding and going straight to the more advanced Elias omega coding, we avoid prepending each integer's bit length to its binary representation in such a wasteful splurge; instead, we encode it recursively using the same Elias omega coding!

This clever idea has an obvious chicken-and-egg problem: how can we tell whether the integer we just decoded is actually a member of the sequence being encoded, or whether it denotes the length of the integer whose encoding follows it in the bit sequence? Since the highest-order bit of any integer is 1, this bit can serve a dual role: it is both the highest-order bit of that number and the marker indicating whether you need to continue decoding after the current integer. Once this

highest bit is zero, the decoding of the preceding integer is complete, and you can start decoding the next integer in the sequence after that zero bit.

```
public static String encodeEliasOmega(List<BigInteger> items)
public static List<BigInteger> decodeEliasOmega(String bits)
```

The computational steps for encoding and decoding any positive integer in Elias omega coding are given on the Wikipedia page, and we leave translating them into higher-level Java syntax as an exercise for the reader. We still note that, instead of using recursion, this method is more stylishly written as a while loop where N decreases each round as we prepend the length of the current number to the encoding. The loop terminates when $N = 1$, meaning that the length of the next block of bits plus one can be expressed in two bits.

Anyway, this recursion can't realistically get very deep. Because an integer's bit length grows only logarithmically with its absolute magnitude, the extra zeros and ones needed to encode its length are a rounding error of a rounding error compared to the integer itself. The Elias omega coding becomes even more efficient as the numbers in the sequence grow astronomically large. For example, to quote the example given on the Wikipedia page, the Elias omega coding of 10^{100}, one **googol**, requires only 15 additional bits for the length header before the 333 bits that encode the actual one googol in binary, plus that extra zero bit tacked to the end to indicate that one googol was the integer that was actually encoded, instead of giving the decoder the bit length of the actual integer that follows it. (Such an unimaginably huge integer wouldn't even fit inside our physical universe, even if we could somehow carve each bit into an elementary particle!)

As the encoded integers grow even larger, the extra bits for the recursive length header become utterly insignificant. For example, according to the same Wikipedia page, encoding the behemoth 10^{10000} requires only 22 additional header bits for its length (and the zero marker bit at the end), plus the 33,243 bits needed to encode the hundredth power of the already enormous one googol. We will gladly pay this chump change to reliably determine, without ambiguity or error, where one integer's encoding ends and the next in the sequence begins. Boundaries and their maintenance matter in all walks of life, since ambiguity usually benefits only the dishonest man.

Lab 72: Lazy Generation Of Permutations

JUnit: PermutationGenerationTest.java

We already thoroughly examined properties of individual permutations in labs 45 to 47, where we implemented methods for algebraic operations, cyclic forms, and the Lehmer coding of permutations. This lab examines the extremely important related combinatorial problem of systematically **generating** all permutations of the given length *n*, one permutation at a time, so that each one of the *n*! possible permutations is visited exactly once during this process.

Algorithms to systematically generate permutations are well-known classics in computer science and combinatorics. However, most online material on these algorithms, such as the Steinhaus–Johnson–Trotter plain changes algorithm and Heap's algorithm, presents them recursively for simplicity. However, this recursive form is impractical for generating permutations or similar combinatorial sequences with an exponential number of elements. Instead of wasting memory by first collecting all permutations into a list for the user code to iterate over, we want the permutation generator to produce permutations lazily, one at a time.

The permutation generator should track only the current permutation, plus any auxiliary data whose exact nature depends on the permutation-generation algorithm. This information should be enough for the generator to swap the correct elements in the current permutation to produce the next permutation in the sequence, making only local decisions based on this information without using recursion. This way, the outside user code can tell the generator when it finishes the current permutation and needs the next one. The generator fills it into the same permutation array that held the current permutation. Instead of constructing the entire sequence of permutations into a list, which would soon become prohibitive when iterating through billions of permutations, the generator needs memory only for the current permutation and the auxiliary data needed to take the next step in the sequence.

To allow different algorithms for permutation generation to exist within the same class hierarchy, we make each of these algorithms a subclass of an abstract superclass `PermutationGenerator`, as provided by your instructor. You should not modify this class in any way whatsoever, but implement the following two algorithms as concrete subclasses of this abstract superclass. Although it is abstract, the superclass defines a constructor that lets the user code provide an integer array that both the user code and the permutation generator share to represent the current permutation of the sequence. The user code is responsible for not mutating the contents of this shared array object when computing the next permutation from the current one.

Most importantly, this abstract superclass defines the **template method**

```
protected abstract boolean next(int pos, int[] perm)
```

that the `public` method `next` in the abstract superclass calls to ask the algorithm implemented in the concrete subclass to rearrange the elements of the current `perm`utation to turn that array into the next permutation in the sequence. This method also receives the position of the current permutation in the sequence as the parameter `pos`, since some permutation generation algorithms need this information to decide what to do next. This method should return `true` if it produces a new permutation, and `false` when the current permutation is the last in the generated sequence.

We start with the classic algorithm that produces permutations in lexicographic order: first swap two tactically chosen elements, then reverse the suffix of the array after the second swapped position. The section "Generation in lexicographic order" on the Wikipedia page "Permutation" explains these steps, but we can illustrate them here with an example. Assume the current permutation is `{1, 5, 0, 3, 4, 2}`. This is the successor permutation in the sequence we are trying to determine.

Counting down from the second-to-last position before the end of the array, find the first position `k` in `perm` that satisfies `perm[k]<perm[k+1]`. If no such position exists, this permutation is the lexicographically largest permutation that ends the sequence, and the method `next` should return `false`. In this example permutation, this is the position marked with a green background. Then, again counting down from the last position of the array, find the first position `l>k` that satisfies `perm[k] < perm[l]`. In this example permutation, this is the position marked with an orange background. Swap the elements at positions `k` and `l`, then reverse the subarray from position `k + 1` to the end. In our example permutation, swapping the elements 3 and 4 changes the array into `{1, 5, 0, 4, 3, 2}`, and reversing the subarray from position `k + 1` to the end turns this into `{1, 5, 0, 4, 2, 3}`, the next higher permutation that follows the previous permutation in lexicographic order.

To implement this algorithm, create a class named `LexicographicPermutation` in your lab project, and have this class extend the earlier `PermutationGenerator`. To allow creation of objects of this concrete subclass, it also needs to have the constructor

```
public LexicographicPermutation(int[] perm) { super(perm); }
```

that passes the argument array `perm` on to the superclass constructor.

Zaks' algorithm for prefix reversals, discovered in 1984 by Shmuel Zaks, is less well known than its more famous cousins, such as the **plain changes algorithm** or **Heap's algorithm**. However, Zaks' algorithm is interesting because it always performs exactly one prefix reversal on the current permutation to produce the next permutation in the sequence. A precomputed scheme quickly calculates the length of the prefix to reverse to turn the current permutation into its successor. Repeatedly applying this scheme generates the entire sequence of $n!$ different permutations, each permutation visited exactly once as if by magic.

Create another subclass of `PermutationGenerator` named `ZaksPermutation`, with a similar constructor as the one seen in the class `LexicographicPermutation`. Before implementing the template method `next` in the `ZaksPermutation` subclass, you need first to write the following `static` utility function used to determine the length of the prefix to be reversed in the current permutation, based on the position of that permutation in the generated sequence:

```
public static int zaksSequenceElement(int n, int pos)
```

This utility method takes the permutation length `n` and the position `pos` of the current permutation in the permutation sequence, with position counting starting from zero. This method should compute and return the value at position *pos* of the prefix length sequence S_n. Each sequence S_n is defined to contain exactly $n! - 1$ positive integers. As a base case, the sequence S_2 contains one element, which is 2. Reversing the prefix of length two turns the first permutation `{0, 1}` into its successor permutation `{1, 0}`, which ends that sequence.

After this base case, each sequence S_n is a concatenation of exactly n copies of S_{n-1}, with the number n tacked in as a separator between these concatenated subsequences. For example, S_3 consists of three copies of the singleton sequence S_2 with the new element 3 between each S_2, giving us the five-element sequence 2, 3, 2, 3, 2. The sequence S_4 consists of four copies of the sequence S_3 with the new element 4 inserted as a separator between them, which makes this sequence 2, 3, 2, 3, 2, 4, 2, 3, 2, 3, 2, 4, 2, 3, 2, 3, 2, 4, 2, 3, 2, 3, 2. Note how the length of S_3 equals $3! - 1 = 5$, and the length of S_4 equals $4! - 1 = 23$.

To keep the method efficient as n grows, it should not attempt to construct the sequence S_n explicitly. Compute the element at the requested position using only integer arithmetic, leaving the details for the reader to discover. It would also be useful to precompute factorials from 2 to 13 in a static array for quick lookup, since you need them at every step during generation. Once this utility method is working, as confirmed by the JUnit tests, you can finally implement the method

```
protected abstract boolean next(int pos, int[] perm)
```

in this subclass to compute `zaksSequenceElement(perm.length, pos)`, and use that result to decide the length of the prefix of `perm` that needs to be reversed to take us to the next permutation. Before doing this, the method should check whether pos plus one is greater than or equal to `perm.length` factorial, which happens when the entire sequence of permutations has been generated, and then return `false`.

Zaks' permutation generator will work just as well if you reverse the suffix of the array each time instead of the prefix. This sequence will then go through the permutations in a different order, but finish with the same lexicographically largest permutation as the prefix version.

Readers who have come this far can check out Combinatorial Object Server, an interactive website that produces colorful visualizations of various algorithms for generating permutations and other combinatorial objects. The article "A Unified Framework to Discover Permutation Generation Algorithms" introduces a general template for permutation-generator algorithms that includes all these classic permutation generators as special cases.

Lab 73: Newton–Raphson Fractals

For testing: NewtonRaphsonMain.java

This problem was inspired by the video "Newton's Fractal" by the popular mathematics channel 3Blue1Brown and is, for many reasons, suitable for computer science students interested in complex numbers and their functions. Even though neither man could have possibly anticipated them, these so-called Newton fractals naturally emerge out of the classic Newton–Raphson method, a famous iterative numerical approximation method to find a root for any real-valued function that is differentiable everywhere and is, in a certain sense, sufficiently well behaved, the tacit assumption that underlies all numerical methods. The initial guess determines not just how quickly the method converges to a root, but also which root it converges to when the function has more than one root.

As explained in the video, Newton's method works just as well for functions of complex variables, needing only a simple substitution of a complex variable *z* for the real variable *x*. However, when the iteration of some function is not confined to the narrow straitjacket of the real line, but takes place on the two-dimensional complex plane that allows things to go around other things in infinitely many squiggly ways without bumping heads with each other all the time, the complex plane gets partitioned into disjoint areas depending on which root the iteration converges to when started with a guess from that area.

Even for simple polynomials, these areas aren't boring Voronoi diagrams, but fractals with infinitely deep detail along their boundaries. Once again illustrating the principle of how the number three is fundamentally different from the number two in computer science, a moment's thought reveals that between any two points in two different areas, there must exist a point between them where the invisible forces pulling towards those two roots are in perfect balance so that the iteration has no choice but to lead into some third root. Since you can repeat the same reasoning using the third point as one of the two endpoints, this detail will keep on going like the Energizer bunny, no matter how deep you zoom in!

Since Java does not have a `Complex` number type, our first task for this lab is to define one. In your labs project, create a new class (or these days in a more modern Java 21+ fashion, a **record**) named `Complex`, whose instances represent immutable complex numbers made up of `real` and `imag`inary parts, each of which is, unusually for these labs and the overall worldview of this author, a double-precision floating-point number. We will not zoom in far enough into the resulting image for the limited precision of the double type to matter; besides, no matter what finite precision you use, every image of a fractal of this nature is doomed to be inexact anyway because of the inaccuracies of floating-point iteration. (Curiously enough, any inaccurate rendering is still the correct rendering of *some* fractal; we just can't control which one. Furthermore, rendering fractals with woefully insufficient floating-point precision can create intentionally dramatic glitch-art displays.)

This class should have a constructor to initialize these fields from its arguments, the **accessor** methods `getRe` and `getIm` to read their values, and a `toString` method to produce a human-readable description of that complex number to help you in debugging the following methods for arithmetic and the **absolute magnitude** of the complex number, analogous to the four methods for the arithmetic operations that were seen in our lecture example class `Fraction`:

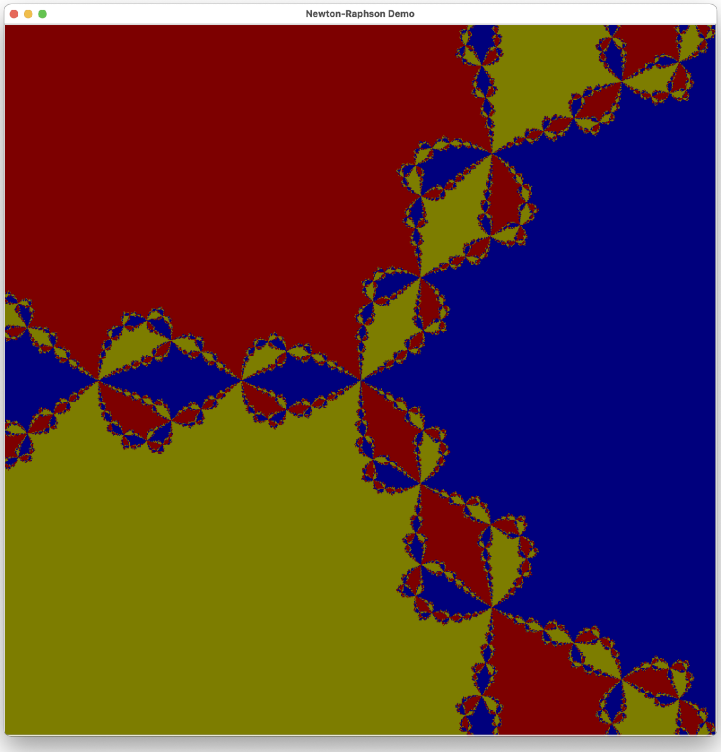
```
public Complex add(Complex other)
public Complex subtract(Complex other)
public Complex multiply(Complex other)
public Complex divide(Complex other)
public double abs()
```

Consult any introductory online or printed material on complex numbers for the formulas to perform arithmetic on complex numbers (especially the division), and how to calculate the absolute value of a complex number. (Instead of computing the absolute value itself, it might be better to define the abs function to return its square, to avoid the expensive square root calculation that usually isn't needed in many practical use cases.)

Once you have defined the class for `Complex` numbers, create a class `NewtonRaphson` in your labs project. This class needs to have just one method that computes the fractal. It receives seven parameters: those that define the finite square **peephole** into the infinitely large complex plane; the function `f` whose root we are looking for; the first derivative `fp` of that function; and finally a `hueFunction` that maps `Complex` points to the hues used to color that partition with. (The last function lets us render this fractal without first solving all those roots.)

```
public static BufferedImage createNewtonRaphson(
  Complex top, int pixelN, double pixelSize,
  Function<Complex, Complex> f,
  Function<Complex, Complex> fp,
  Function<Complex, Float> hueFunction
)
```

The first parameter, `top`, gives the top-left corner of the peephole rendered into a new `BufferedImage` instance created by this method. Both the width and the height of this square image are `pixelN` pixels. The parameter `pixelSize` gives the size of the square on the complex plane represented by each pixel. The `Function` object `f` is a black box that contains the function whose root we are numerically iterating, and the similar black box `fp` gives its first derivative. Once the iteration from the given starting point z_0 converges numerically to some z_n, the pixel corresponding to z_0 is colored using the hue obtained by applying `hueFunction` to the convergence point z_n.



This method should start by creating the new `BufferedImage` instance that it will return at the end. Then loop through the pixels `(x,y)` of that peephole (with modern Java, this **embarrassingly parallel** part could even be parallelized to increase the rendering speed massively), and compute for each pixel which complex number z_0 on the underlying infinite complex plane it corresponds to. Iterate the Newton-Raphson formula $z_{n+1} = z_n - f(z_n)/f'(z_n)$ until convergence, using the methods of your `Complex` class to perform the arithmetic operations in the absence of **operator overloading** that, for some mysterious reason, the Java language still does not have, at least as of this writing. After convergence, color that pixel `(x,y)` with a color whose **hue** comes from applying the given `hueFunction` to whatever point this iteration ended up, whose **saturation** is `1.0f`, and whose **brightness** is computed from the number of iterations needed to reach convergence within some reasonably small `EPSILON` that you have defined as a named constant in your class. Note the method `HSBtoRGB` in the `Color` class that makes this color computation a breeze.

The class `NewtonRaphsonMain` uses your method to plot the Newton–Raphson fractal for the simple example polynomial $z^3 - 1$. By the Fundamental Theorem of Algebra, this polynomial has exactly three roots: one real root and two complex conjugates with nonzero imaginary parts. Running the `main` method of this class should therefore soon pop up the result window that looks like the screenshot above. If the colors of your result image end up being mirrored or rotated, remember that the y-coordinates are mirrored inside a `BufferedImage` so that they increase downwards, as is conventional in computer graphics. Therefore, use the correct sign in the formula

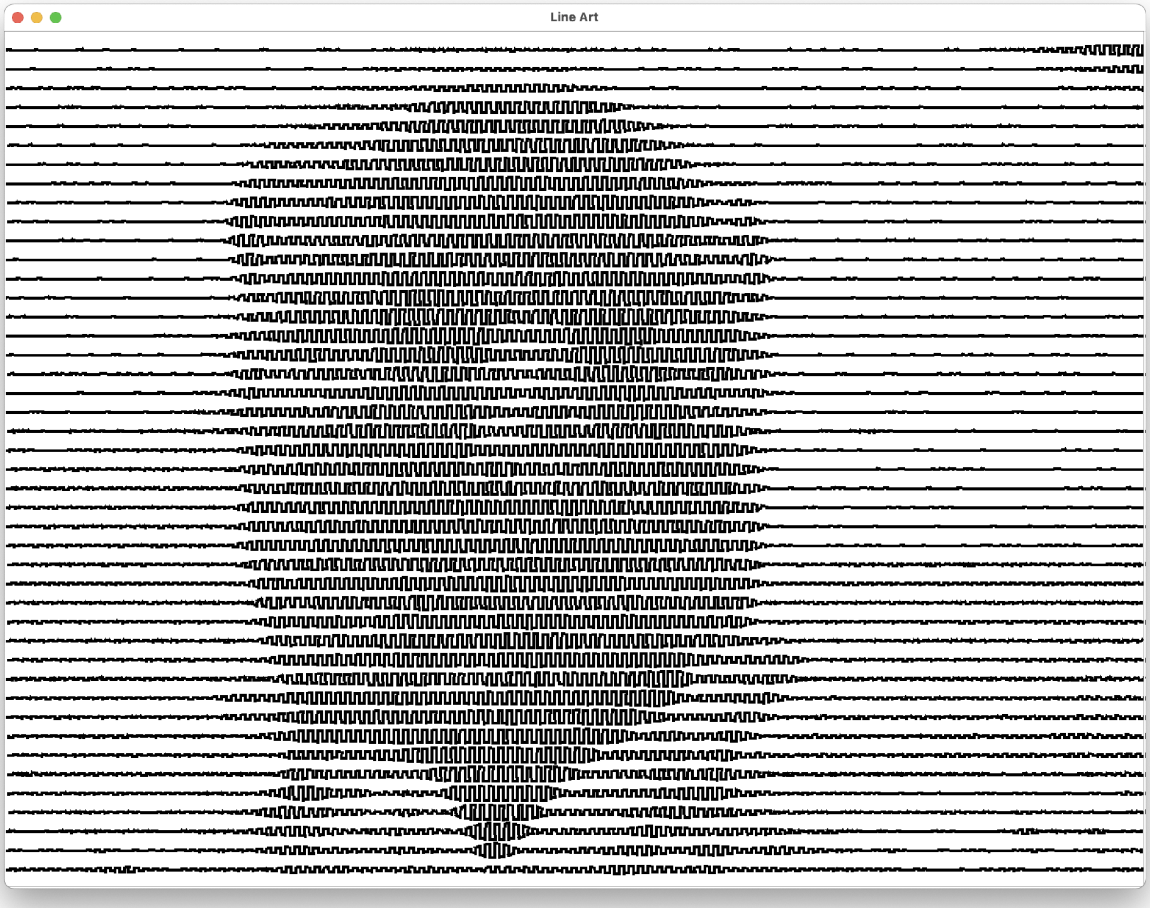
to calculate the starting point z0 for each pixel `(x, y)` from the given top-left corner point top, which should map to the top-left pixel `(0, 0)`.

The Wikipedia page and countless other places on the web showcase pictures of Newton–Raphson fractals computed from the roots of more complex (heh) functions than the mere three **roots of unity**. As that same Wikipedia page explains, the iterative formula can also be generalized to produce wackier results that no longer correspond to actual root-finding, but still prove their worth by being pretty to look at, which is always a good plan B in life.

Lab 74: Between The Lines

For testing: LineArtMain.java

Op art exploits the human visual system's inborn tendency to extrapolate patterns from simple but



suggestive cues. Complex three-dimensional shapes emerge from the interplay of simple two-dimensional shapes, usually rendered in black and white to make the effect even more dramatic. As seen in the example screenshot below, the method written in this lab can convert the given color pixel image into a bunch of jagged horizontal curves drawn with a black pen on a white background, so that squinting at this black-and-white op art picture from a sufficient distance will make the visual sensation of the original image pop out in almost three dimensions.

In your labs project, create a new class named `LineArt` where you will first write two brief utility methods followed by the actual method to perform the above conversion to the pixel image. The first utility method calculates the **luminance** of the given RGB color, that is, how bright this color will appear to the human eye compared to other RGB colors.

```
private static double getLuminance(int rgb)
```

Given an RGB color as a single int value `rgb`, whose bytes from low to high encode the blue, green, red, and alpha components of that color, respectively, this method should return a `double` between 0 and 1 that estimates its luminance. You should use **bitwise arithmetic** to extract the individual color components from the `rgb` integer (you can consult either one of our two example programs, `ImageOpsDemo.java` and `FloydSteinberg.java`, for the formulas to do this), and from these use one of the possible luminance formulas given in the Stack Overflow page "Formula to determine perceived brightness of RGB color", or some other equivalent online resource. Then, you should write the second required utility method

```
private static double averageLuminance(
  BufferedImage source, int x, int y, int r
)
```

that calculates the average luminance of the pixels inside the square box in the `source` image whose *x*-coordinates start and end at `x-r` and `x+r`, and whose *y*-coordinates start and end at `y-r` and `y+r`. Be careful at the edges of the image where the boundaries of this box would reach outside the boundaries of the `source` image. With this utility function available, the actual conversion of the pixel image into the black-and-white op art image will be done by the method

```
public static BufferedImage lineArt(
  BufferedImage source, double density,
  double frequency, double amplitude
)
```

This method should start by creating the `result` image as a `BufferedImage` with the same width and height as the `source`. Get the `Graphics2D` object needed to render into the image, and fill it with a white background to act as your canvas. Then set the stroke to a solid black pen with a width of `0.15*density`, converted to a `float`.

Each curve starts at the left edge of the image and corresponds to some *y*-coordinate baseline. The `density` parameter defines how many pixels apart the *y*-coordinate baselines of these curves are spaced vertically. Render each curve with a loop that starts with *x* equal to zero and terminates once *x* reaches the right edge of the image. Every other round, increase *x* by frequency for the horizontal curve segments; every other round, keep *x* the same for the vertical curve segments.

The loop should also maintain another local variable `int sign`, initially set to +1. This `sign` should alternate between +1 and –1 every two rounds, flipping only in the rounds when the x-coordinate stays the same. Each round of the loop, draw a line segment (for nicer results, use `Line2D.Double` and don't forget to turn on anti-aliasing before drawing) from the previous point to the new point

whose *x*-coordinate is given by your variable *x*, and whose *y*-coordinate is computed with the formula

```
y + sign * amplitude * (1-luminance)
```

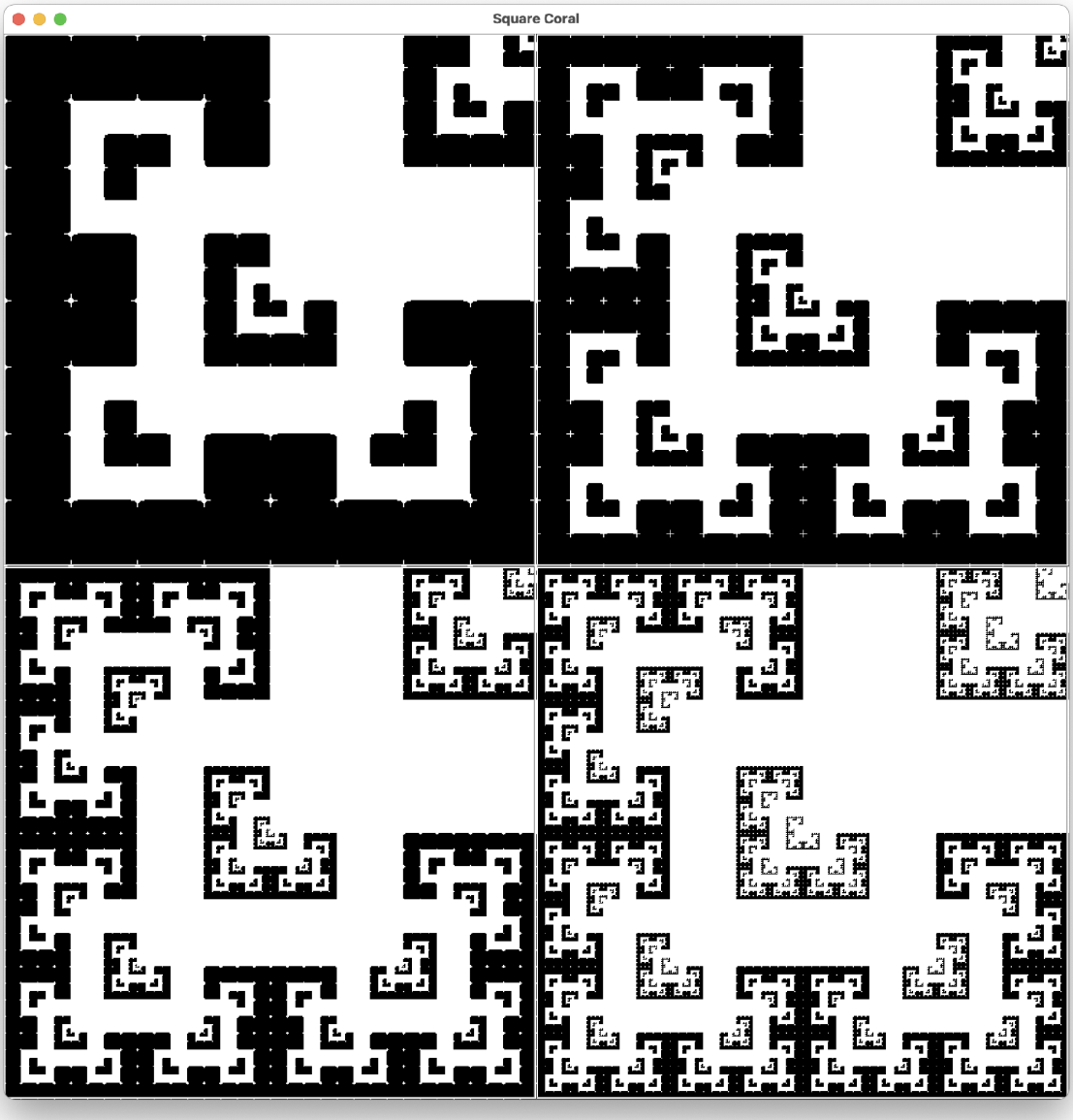
where `y` is the curve's baseline coordinate, and `luminance` is the average luminance computed over the box centered at the pixel (x, y) with r equal to half the `density`. Once you have drawn this line segment, the new point becomes the previous point for the next round of this loop. Finally, to give your result some kind of artisanal hand-drawn feel of the quaint past when everything was analog and had a constant component of noise, you should also perturb each corner point of your jagged curves by some small random amount. Use the method `nextGaussian` of your random number generator multiplied by a suitable amount to get a good distribution of perturbations.

Building on this idea, the lines covering the image plane don't need to be horizontal; for example, they could form a spiral emanating from the image center, or even a space-filling curve such as a Peano curve or Hilbert's curve. No matter how your chosen curve meanders and squiggles, the displacement at each point always points along the curve's normal vector at that point.

Lab 75: Square Coral

For testing: SquareCoralMain.java

The page "Fractals, Chaos and Self-Similarity" by Paul Bourke offers a wealth of formulas and



visualizations of exotic fractals to explore after all those Sierpinski gaskets and Dragon curves start to feel a bit stale. Most of the fractals there emerge from iterating all kinds of inventive formulas on a complex plane around simple poles, as in the earlier lab on Newton–Raphson fractals. However, "Square Coral" is a rare example in that collection of a **subdivision fractal** in which some simple geometric shape is recursively cut into smaller pieces whose areas together comprise the original

shape, and some smaller pieces are systematically discarded during this process. As a result, the shape meanders around fractal-shaped holes.

The earlier lab problem of imitating the famous **Mondrian art** also concerned subdividing a rectangle into smaller rectangles. However, in that lab, the cutting points inside the rectangle were chosen randomly, and none of the smaller rectangles were thrown away, so the distinction between these pieces was made with black outlines filled in with simple primary colors. The Square Coral fractal begins with a square subdivided into four square quadrants; each quadrant's side length is exactly half that of the original square. However, depending on the current square's orientation, one quadrant shrinks to half size, so its side length is one quarter of the original, revealing the background color behind the shape.

Each of the four quadrants also gets an orientation that depends on the quadrant's compass position within the current square and the current square's orientation. This orientation affects how the quadrant is recursively subdivided. By choosing quadrant orientations according to the rule illustrated in the example on the linked page, the resulting fractal has the overall coral shape shown in the example screenshot.

In your lab project, create a new class named `SquareCoral`, and there the recursive method

```
public static void render(
  Graphics2D g2, int depth, int x, int y, int width, int orientation
)
```

This method takes the *x*- and *y*-coordinates of the current square's top-left corner, along with the desired width of the result image in pixels. The `depth` parameter specifies the resulting fractal image's depth. The base case is when `depth` equals zero, indicating the shape is small enough and should not be subdivided further, but should be rendered as an instance of `RoundRectangle.Double`. (For a more aesthetic appearance, let's give its corners both horizontal and vertical roundness of `width/5.0`.) Otherwise, depending on the `orientation` of the current square, this method should recursively render the four smaller squares in their correct positions at depth 1. (To simplify these formulas and make them more readable, first define local variables `h` and `q` to denote the one half and the one quarter of the given `width`, respectively.)

The screenshot above shows the expected visual outcome of performing the Square Coral subdivision at depths 3 through 6. To ensure our `SquareCoralMain.java` test bed matches this screenshot when executed with your implementation of the recursive render method, orientation 0 should mean the northeast quadrant contains the quarter-sized square; 1 means southeast; 2 means southwest; and 3 means northwest. Again, make sure to pass on the correct orientation to each of the four recursive calls for the subdivided quarters. To simplify your coding and debugging, you should note that for each of the four orientations, exactly two of the subdivided quadrants will have that same orientation, and the quadrant that was shrunk to half the size should be one of these

two. When a shape has underlying symmetry, the rules that define it do too, even if abstraction layers hide that symmetry.

Lab 76: Sultan's Daughter

JUnit: SultansDaughterTest.java

Students whose first programming course was in Python and whose second is in Java have already realized how different these two major programming languages are, not merely at the syntactic level but in their deeper foundations. Even so, these two languages are still relatively close in the space of all possible Turing-complete programming languages. As a crude analogy, consider how our Old Blazey is far too distant from its nearest neighbor, Proxima Centauri, for practical human space travel, and for all other species who must do what all that eat and drink must do. Yet even this immense distance becomes meaningless compared with the galactic superclusters truly out there.

Along most dimensions where programming languages can differ, Java and Python turn out to be practically identical. We just tend to miss this, since most of those dimensions seem virtually nonexistent from our everyday point of view, as if their width were zero, so some kind of law of nature dictates that all programming languages must behave in a certain way along that dimension. However, even a casual perusal of esoteric programming languages ought to cure any sweet summer child of this misconception.

Esoteric programming languages are not intended as serious tools for working programmers to solve actual problems. They are meant to be works of **abstract performance art**, appreciated for opening your mind to esoteric possibilities that otherwise remain hidden from the masses. (You know, all those sheeple around you who don't have a clue what's really going on and what's coming to them.) The very existence of a good esoteric programming language is a comically hubristic affront, and a slap in the face, not just to what is good and ordered in humanity itself, but also to the Janus faces of Nature and Nature's God. Don't worry, they won't get angry for this insult. By its very existence, an esoteric programming language makes a poignant point about how many assumptions and ideas about programming language design turn out to be far less solid than we thought.

To explore this concept, this lab has you implement an evaluator for expressions written in Sultan's daughter, an esoteric programming language dreamed into form by no less of a tinkerer in the kingdom of the abstract than Raymond Smullyan himself. Better known for his logic puzzles set on islands populated by deeply obsessive-compulsive characters such as knights who always tell the truth and knaves who always lie, Smullyan also created other whimsical works that illustrate the surprising powers of various formal logics. This little language mixes ideas from combinatory logic and the string-rewriting paradigm, both ancestors of actual programming languages, more suited to elegant salons and tea rooms. *Ad fontes!* Come, come, the game is afoot.

Expressions written in Sultan's daughter are strings that consist only of digit characters 0 to 9, with no other characters allowed. (If this seems somehow restrictive, you should remember that inside your computer, all your Java programs, just like this PDF document and everything else that exists inside your computer, consist of sequences of nothing but virtual zeros and ones somehow encoded

as physical atoms!) To evaluate an expression of the form `dX`, where `d` is some leading digit character and `X` some string of digit characters (possibly empty), the first digit `d` determines which operation is applied to the recursively computed value of the rest of the expression `X`.

The evaluation rules for the possible values of `d` are listed on the Wiki page for Sultan's daughter at esolangs.org for the reader to consult. For example, since `19872` evaluates to `987`, `319872` evaluates to `987987`, and `4319872` evaluates to `789789`.

In your labs project, create a new class named `SultansDaughter`, and there, the first method

```
public static String sultansDaughter(String expression)
```

that evaluates the `expression` according to the rules of Sultan's daughter. Our automated tester is guaranteed only to give this method expressions whose evaluation terminates within reasonable bounds of both time and space. If the `expression`'s value is not defined under the rules of Sultan's daughter, such as for expressions `143` and `81932`, this method should return `null`.

Once you get the previous method working so that it passes the instructor's JUnit tests, you can use it as a subroutine in the second method

```
public static int iterateSultansDaughter(String expression)
```

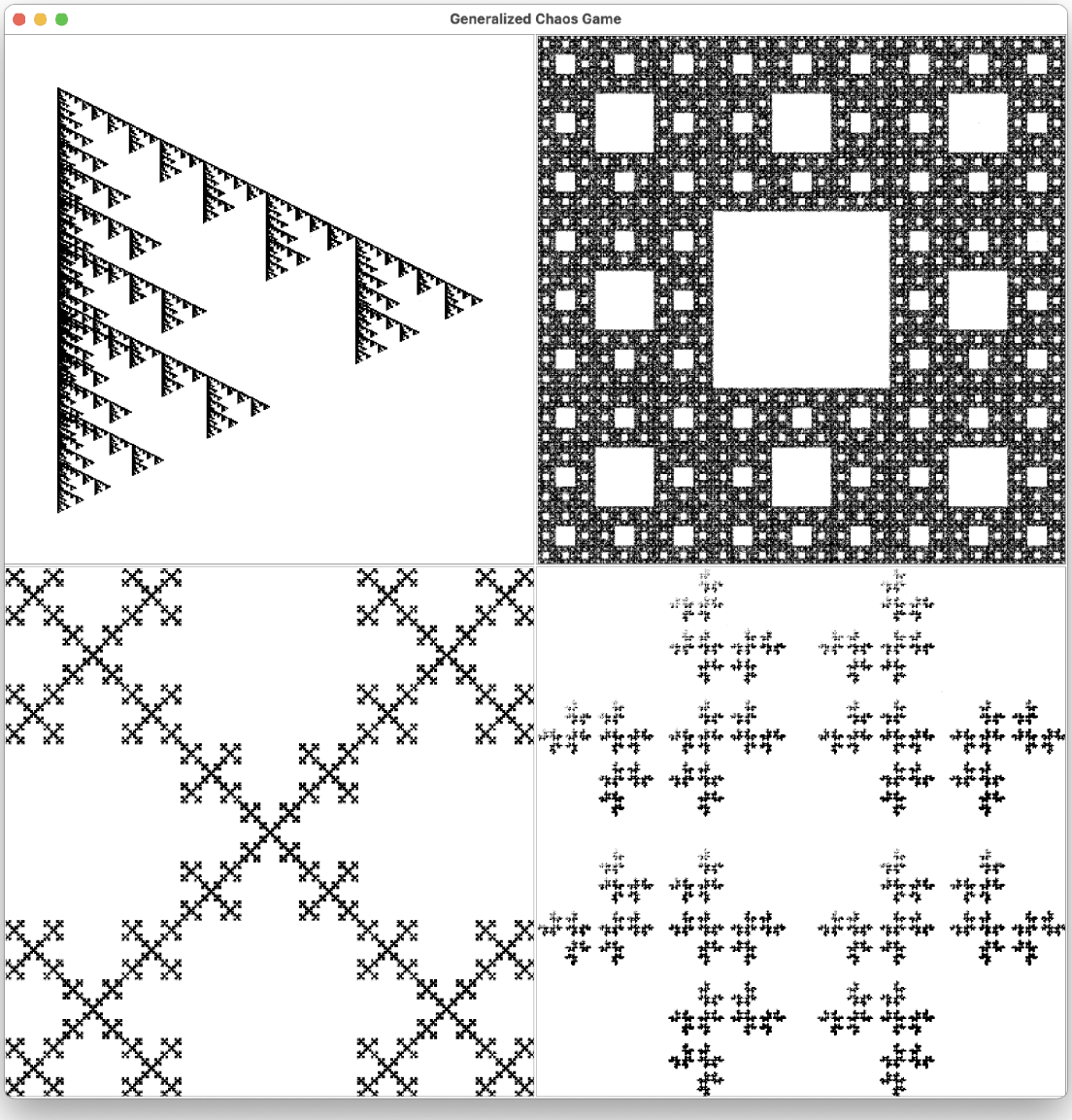
that iterates the evaluation of the starting `expression` until the value of the current expression is undefined so that its evaluation produces the result `null`. This method should return the count of how many steps of iterated evaluation were needed to reach the result `null`. For example, since the starting expression `1161175032222` evaluates to `16117503222`, which evaluates to `611750322`, which evaluates to `1175032`, which evaluates to `17503`, whose evaluation is then undefined, this method should return the answer 5 when starting from this `expression`.

One extra kink that you unfortunately have to consider in writing this method is that the iterated evaluation of some starting expressions will never terminate! For example, consider the **fixed point** expression `47431474312`; inside the microcosm of Sultan's daughter, this expression is a quine that evaluates to itself, as the reader can readily verify. Your method must somehow detect such situations where the iterated evaluation eventually loops back to itself through one or more steps, and return `-1` as the artificial "stuck in an infinite loop, will never terminate, kthx bye" answer for all such expressions.

Lab 77: Generalized Chaos Game

For testing: ChaosGameMain.java

With enough discipline to constrain its noisy excesses, shapes created by jumping randomly across



the pixel image can end up far more regular than one might initially assume. The Chaos game is a classic general mechanism for generating all sorts of nifty fractal shapes by randomly jumping around, as if you were a headless chicken stuck in a *House of Pain* music video. Each jump goes to a point chosen randomly from the given set of fixed points. The initial placement of these fixed points determines the fractal's overall shape and the Swiss cheese holes that this jumping around always avoids, no matter how far you let it run. Starting from some arbitrary point on the image, choose

one of these fixed points randomly. Move the current point halfway towards your chosen fixed point, and increment the visitation counter for the pixel that the current point lands on. Keep going until you have visited enough points to expose the image enough that it stands out clearly from its white background.

In this lab, you will implement a slightly generalized version of the chaos game that our automated tester then uses to render some classic fractal shapes seen in the previous screenshot. In this generalization, the fixed points are not all equal; some are more likely than others, and each point can be chosen with a different **frequency**. Each point can also have a separate attraction weight that determines how close the current point moves toward that chosen point in each jump.

In your lab project, create a new class named `ChaosGame`, and there, the method

```
public static BufferedImage playChaosGame(
  long rounds, int n, int[] x, int[] y,
  int[] freq, double[] weight, boolean canRepeat
)
```

The first parameter, `rounds`, determines the total number of jumps made to generate the image, and `n` gives the width and height of the resulting image in pixels. The parameter arrays `x` and `y` contain the *x*- and *y*-coordinates of the fixed points to jump towards.

The array `freq` contains the intended frequencies that determine how often each point is chosen at random in the long run. For example, if there are four fixed points for us to jump towards randomly, and the `freq` array equals `{2, 3, 1, 4}`, the first point will be chosen as the target roughly 20% of the time, the second point will be chosen roughly 30% of the time, and so on. This proportion is statistical, not exact, since each jump must make a random choice between the target points independently of past choices. As explained in the introduction to the earlier problem about Lemire dynamic frequency sampling, to choose the random fixed point with respect to these given frequencies, you should generate a random integer d from 0, …, m – 1, where m is the sum of frequencies of the fixed points, and then find the first fixed point for which the sum of frequencies up to and including that point is *strictly greater* than d. (Since these images will use only a handful of fixed points, you can do this with a single for-loop without any sophisticated optimizations by first once suitably preprocessing these frequencies.)

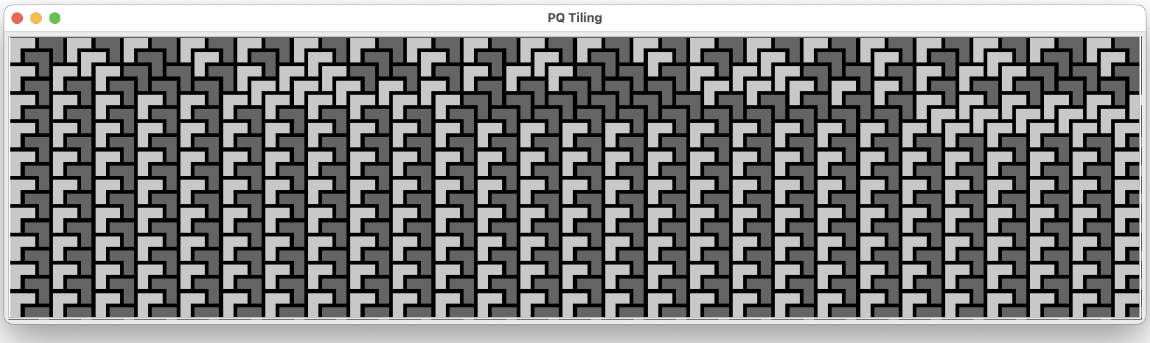
The array `weight` contains the attractive weights of each fixed point so that when jumping from the current point (x, y) towards the chosen target point (x', y'), this jump will land on the point $((1 - w) \times x + w \times x', (1 - w) \times y + w \times y')$. For example, if $w = 1/2$, as in the ordinary chaos game, the jump lands exactly halfway between the current point and the chosen target point.

The parameter `canRepeat` determines whether the target point from the previous jump can be chosen again as the target point for the current jump. If this parameter is set to `false`, the choice of the current fixed point to jump towards must be rerun if it would lead to the previous target point.

The colors of the result image should be as in the above screenshot, with the fractals drawn in greyscale against a white background. To compute each pixel's darkness, treat every pixel visited eight or more times as fully black, and every pixel with a count of zero as fully white. For pixels whose counts fall between those extremes, linearly interpolate their greyscale values between full black and full white.

Lab 78: Mind Your P's And Q's

For testing: PQTilingMain.java



Of all the recreational math books this author has consumed over the past quarter century or so, "Mathematical Puzzles" by Peter Winkler is the absolute unit and *numero uno*, earning a blurb to that effect from no less a luminary than Donald Knuth himself! The problems and their elegant solutions cut through all kinds of seemingly unbreakable Gordian knots like a chainsaw through butter, using simple flashes of insight that seem obvious in retrospect. (This is, of course, how you recognize genius ideas in all walks of life: they seem self-evident and hardly worth mentioning once pointed out, even though they sure as hell weren't that self-evident five minutes before that!)

This lab has you implement a curious **tiling** of one quadrant of the infinite two-dimensional integer plane, using two types of **bent trominos** composed of three unit squares. In computer graphics, the *y*-coordinate conventionally increases downward, opposite the usual mathematical convention. We therefore present this problem vertically mirrored from Professor Winkler's problem book. The origin of the infinite plane quadrant is at the top-left corner of the image, and the horizontal and vertical coordinates increase to the right and down.

The two types of bent trominos available for filling the infinite plane quadrant are called P's and Q's. In the screenshot above, the P-tiles are rendered in light grey, while the Q-tiles are rendered in dark grey. These two types of tiles are the only types that you are allowed to use here. If the other two possible rotations of these tiles, the L-tiles and J-tiles, were also available in unlimited amounts, there would be infinitely many ways to tile the plane quadrant without overlaps or holes. However, when restricted to only these two types of P- and Q-tiles without rotations, we must place them carefully to avoid painting ourselves into a metaphorical corner. Under this restriction, only one arrangement works. Furthermore, this tiling has unexpected, counterintuitive properties that make it worthy of inclusion in Professor Winkler's book.

First, we quickly see that the first row must alternate P- and Q-tiles. Placing two consecutive P-tiles or two consecutive Q-tiles anywhere on the first row would leave a unit gap between the legs of these tiles in the row immediately below. Such a gap would be impossible to fill with P- or Q-tiles,

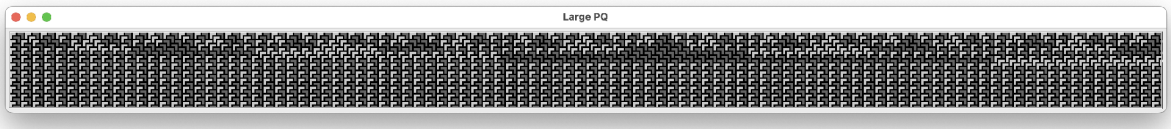
since we cannot rotate these tiles to point their legs upward to fill the unit gap. The first row's contents are therefore forced by the iron hand of this game's rules.

After filling the first infinite row, each gap of length two in the second row must be filled similarly with either P- and Q-tiles. In general, when filling row *n*, the choice of tile for each gap is forced by the same reasoning as in the first row: you may not leave a unit gap anywhere in row *n* + 1 to be filled next.

If we were tiling a half-plane that was infinite to both north and south, every column could consist of a straight alternation of P-tiles and Q-tiles. However, for the quarter plane, even though the first row is as simple and periodic as it can be, the top edge of the quadrant breaks this symmetry. It forces the entire tiling to become aperiodic, even though the top row is perfectly periodic! This illustrates how sneaking a small pebble of asymmetry under the stack of mattresses the princess sleeps on can cause uncontrollable chaos to emerge from the initial perfect order. However, as both screenshots show, each vertical column eventually becomes periodic. Hence, this aperiodic tiling is not quite as exciting as it could be in the spirit of, say, Rule 110 in elementary cellular automata.

In your lab project, create a new class named `PQTiling`, and in there, the method

```
public static BufferedImage render(int rows, int columns, int tile)
```



that creates and renders a view of this infinite tiling for the first `rows` and `columns`. The parameter `tile` gives the pixel width and height of each unit square on the plane. You should draw a black outline of width 30% of the `tile` width around each tromino, and fill the P-tiles with light grey and the Q-tiles with dark grey. The easiest way to do this is to use the `Path2D.Double` shapes from the `java.awt.geom` package. Build each shape object piecemeal from consecutive straight-line segments, then use `fill` and `draw` to render the interior and outline of the completed closed shape at the desired position in the result image.

You might want first to define two private subroutines to render a P-shape and a Q-shape at the given location, to simplify the logic of your actual rendering. When you then compute the contents of the image with two nested loops, you should use a two-dimensional `boolean` array to keep track of the positions that some tile has already filled. You should also make this two-dimensional array slightly larger in both dimensions than needed, to avoid accidentally stepping out of the bounds of this array at the right and bottom edges of the image.

Lab 79: Die Hard With A Variance

JUnit: BaysDurhamShuffleTest.java

To ensure portability of results of any integer computations in all computing devices that have enough moxie to run the Java Virtual Machine, the class `java.util.Random` provided in the standard library to generate all kinds of pseudorandom quantities has been standardized to use the classic linear congruential generator algorithm with hardwired fixed parameters. This allows our automated JUnit tests used in these labs to generate the pseudorandom test cases with `Random` and expect to receive consistent hardwired checksums for all methods, regardless of which hardware and operating system these results are computed on, and whether these results are computed from the model solutions from the private top shelf reserve of your 'umble, very 'umble narrator, or from the solutions put together by all of you bratties and devotchkas toiling diligently at your desks.

Linear congruential generators are sufficiently indistinguishable from true randomness for games and other applications that aim to give the human user the illusion of a larger reality on top of what is really nothing but a simple deterministic state machine. However, the classic 1968 article "Random numbers fall mainly in the planes" by George Marsaglia, the late author of the famous DIEHARD test suite for measuring the quality of pseudorandom number generators, not only established that punny titles for scientific articles well predate the snark of the modern Internet, but also that the statistical randomness properties of linear congruential generators leave a lot to be desired for the needs of serious simulations that in this day and age may require even trillions of random samples. Any systematic bias in the pseudorandom generator that produces these samples will then translate into systematic bias in that simulation.

For this course, the class Random also works as a wonderful example of the template method design pattern; all its methods for generating random quantities internally use the same protected method next as the source of random bits, which they refine into more complex random quantities. The template method pattern lets you create subclasses of Random that use a more advanced, higher-entropy algorithm to generate pseudorandom bits, since the remaining methods, such as nextDouble or nextGaussian, inherited from Random will work right out of the box.

This lab has you implement two ways to produce statistically superior streams of pseudorandom numbers. We use techniques most famously discussed in "Seminumerical Algorithms", Volume 2 of the classic doorstop "The Art of Computer Programming" by the starry dedoochka Donald Knuth himself. First, we will extend `Random` to create a subclass whose template method `next` uses the Lagged Fibonacci generator to produce its random integers. Second, we will create a general-purpose decorator subclass that can wrap any past, present, or future Random number generator and use the ingeniously simple Bays–Durham shuffling algorithm to scramble the numbers it returns, greatly improving the sequence's statistical properties. To get started, create a class in your labs project with the signature

```
public class LaggedFibonacci extends Random
```

As explained on the Wikipedia page linked above, the Lagged Fibonacci generator is initialized with two integers `k` and `j` such that k > j, and a seed array Y containing exactly k integer values that form the generator's initial state. (Even though Java variable naming conventions reserve words with initial capital letters for classes, we shall follow the names used in TAOCP to make things easier for readers accustomed to mathematical notation.) The caller is responsible for filling in the initial contents of the seed array `Y` however they wish, except for the small requirement that at least one of the values in this array must be odd. (If all values in `Y` are even, the additive update formulas show that the generator will produce only even numbers from that point onward, not the uniform distribution we hope for.)

The Wikipedia page above also lists some possible values for `k` and `j` that make the lowest-order bits of the generated pseudorandom numbers behave especially chaotically. Since the buffer size `k` can be arbitrarily large, the state space and the resulting **period** of the pseudorandom generator can become truly astronomical, so the remaining lifetime of our physical universe until its inevitable heat death would not suffice for this pseudorandom number generator even to come close to repeating itself. The class should have a constructor

```
public LaggedFibonacci(int[] Y, int k, int j)
```

that initializes the corresponding data fields with the constructor parameters given. However, since we index the arrays starting from zero instead of starting from one as done in TAOCP, you should decrement the quantities `k` and `j` by one before assigning their values to the corresponding data fields. Also, to ensure you understand the difference between an object and a reference to that object, this constructor should create and store a defensive copy of array `Y`, rather than storing a reference to the original array object passed as the first argument.

After completing this constructor, you only need to override the `protected` template method

```
@Override protected int next(int bits)
```

To keep things simple, this method should ignore its `bits` parameter and always return a 32-bit integer result as follows. Initialize your local variable `int result` from the result of evaluating `Y[k]+Y[j]`, and update the value of `Y[k]` also to equal this result. Decrement both indices `k` and `j`, and if either index becomes zero or less, set that index to the last position of the array `Y`. Then return the stored result to the caller. And that's all she wrote; you can now use the first half of the combined JUnit tester BaysDurhamShuffleTest.java to verify that your generator works as required.

The template method design pattern meshes especially well with the decorator design pattern, since the former eliminates the most annoying part of the latter in Java: overriding methods that

only call the underlying client with the same arguments and then pass the result back to the caller. Create a new class

```
public class BaysDurhamShuffle extends Random
```

Just like every decorator class everywhere, this class should contain a `private` reference to its underlying `client`, given to this decorator as a constructor parameter. The Bays–Durham shuffling algorithm maintains an internal buffer `V` that has room for `k` integers, initialized with the first `k` results from calling `client.nextInt()`. In addition, this algorithm maintains a `private` integer variable `Y`, initialized with one more call to `client.nextInt()`. Write the constructor

```
public BaysDurhamShuffle(Random client, int k)
```

to perform these initializations. Next, override the template method

```
@Override protected int next(int bits)
```

to again completely ignore its `bits` parameter and always return a 32-bit integer result computed in the following fashion, slightly modified here from the "Algorithm B" given in TAOCP to simplify the integer arithmetic involved. Start by calculating the index `j` to the buffer `V` using the expression

```
int j = (Y & 0x7FFFFFFF) % V.length;
```

where the strange-looking **bitwise arithmetic** formula chops off the highest sign bit from Y to ensure the result is nonnegative and can therefore be used as an array index. Knuth's original discussion of Algorithm B scales `Y` to fit within the range of legal buffer indices, 0 to k – 1, which requires care to avoid overflows when implemented with only integer arithmetic.

(Since the signed integers represented in two's complement encoding necessarily contain one more value in the negative direction than in the positive direction due to the number zero being the odd man out in the middle, the expression `Math.abs(Y)` to achieve the non-negativity of the produced random number will silently fail when `Y` equals `Integer.MIN_VALUE`, the smallest possible negative integer. This edge case can cause weird bugs in some applications that manifest once in a blue moon and are nearly impossible to debug unless you understand the two's complement encoding of fixed-size signed integers.)

Having computed the index `j` from `Y` to access the buffer `V`, you next assign `Y` to have a new value equal to `V[j]`, and then assign the element `V[j]` to equal the result of again calling the method `client.nextInt()`. Return the new value of `Y` from its safekeeping as the next random number produced by the method `next`. With that, the fat lady has sung you the song of her people again, so

you can run the JUnit acceptance tests to verify that your implementation of the Bays–Durham shuffle works as required.

Neither the laws of nature nor man restrict the Bays–Durham shuffle to pseudorandom numbers. When applied with a buffer width `k` equal to 3, the perfectly orderly sequence of natural numbers 0, 1, 2, 3, … gets scrambled into 0, 4, 1, 6, 5, 2, 9, 8, 10, 7, 13, 14, …, and so on. The value 3 is missing because the algorithm never returns the initial value of `Y`. However, you can easily modify this algorithm to not drop any values by initializing `Y` from an external random source.

Bonus Lab 100: The Buddhabrot Is Silent

For testing: BuddhabrotMain.java

The famous Mandelbrot set is defined as the set of initial points z0 on the complex plane for which the iteration zn+1 = zn^2 + z_0 does **not** escape to infinity. Because the Mandelbrot set is a black hole, infinitely hairy around its edges, rendered images are usually more interesting when you color the points outside the Mandelbrot set based on how long it takes them to escape past a certain distance threshold from the origin.

This instructor can still fondly remember the halcyon days of Fractint running on his first PC, which could spend a better part of an hour rendering a simple zoom near the high-level view of this fractal. With the exponential speedup in computing available to Joe Everyman, today's off-the-shelf desktop computers can render deep, complex parts of this fractal in almost real time. The instructor's own example program Mandelbrot.java uses multiple Java execution threads to carve out, classify, and color the escaping points outside and around the Mandelbrot set, so that no execution time is wasted iterating the sequence of complex points from any internal point of this black hole, except for the fencepost pixels at the edges of this set.

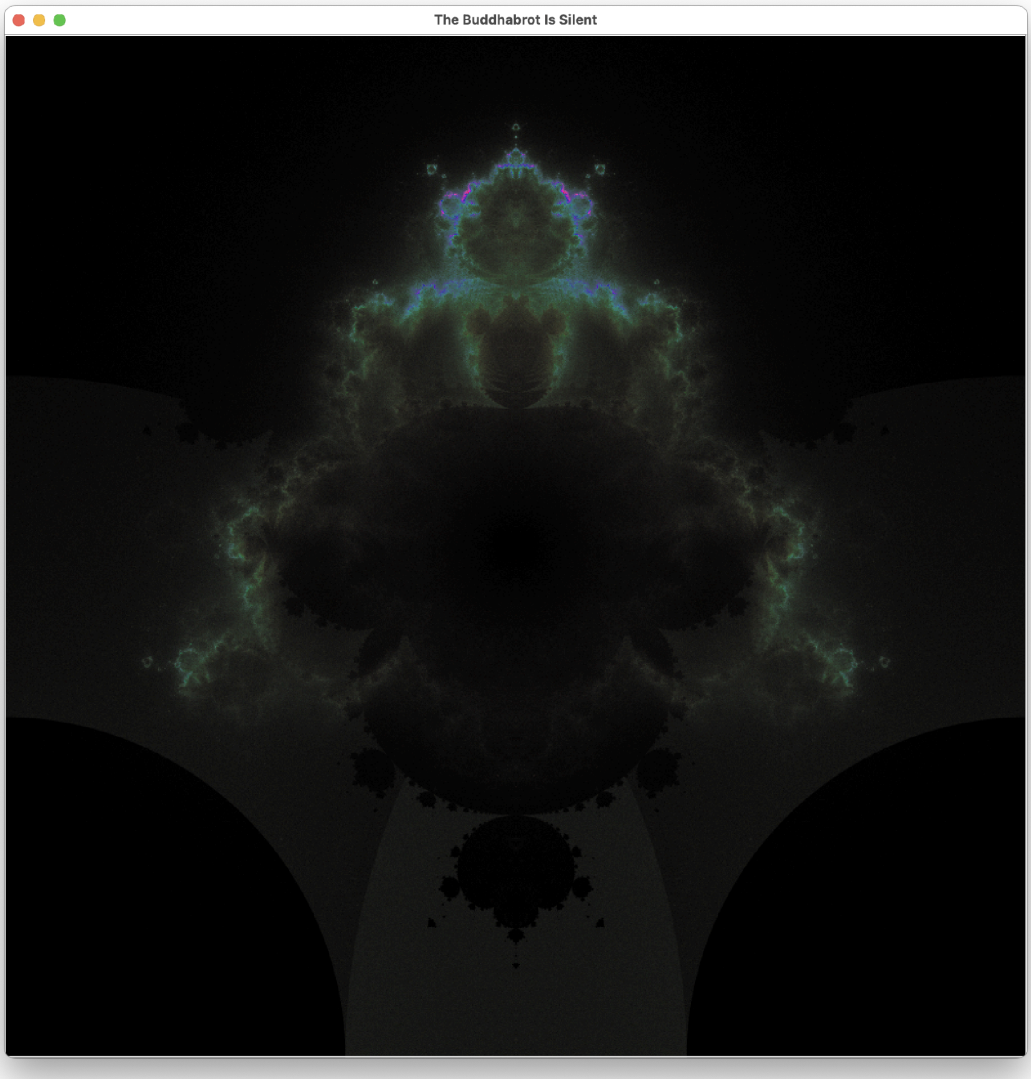
Buddhabrot is a variation of the Mandelbrot set that is significantly more computationally intensive than rendering the plain old Mandelbrot set. Buddhabrot uses the same iterative formula for each starting point z_0 on the complex plane to generate a sequence of points, then investigates their escape properties. However, it does not use the sequence z_n from the starting point z_0 to color the pixel where z_0 falls. Instead, if the sequence z_n escapes within a preset iteration threshold, it slightly brightens the color of every point the sequence visits along the way. Starting from a black canvas and iterating through millions of sequences from randomly chosen starting points, the canvas slowly brightens to display a meditating Buddha, bringing out the energetic ahhs and ohhs of a mystically inspired audience of bohemians and similar sophistos. To make the Buddha figure sit up properly, the rendered image is canonically rotated 90 degrees counterclockwise; the simplest way to achieve this is to mirror and swap the real and imaginary axes of the complex plane.

To make this fractal's fine structure pop, a simple false-color technique uses the length of the sequence zn to decide which of the three color channels to brighten for each pixel visited in the sequence. Instead of the classic red, green, and blue color channels, you can also generate a false-color image using the hue, saturation, and brightness of the HSB color model. Using the modified **quartic** sequence formula $z_{n+1} = z_n^4 - z_0$ with the threshold limits set to 50 for the hue, 500 for the brightness, and 5000 for the saturation of the HSB color of the pixel in the main class BuddhabrotMain.java, along with the instructor's private model solution for a hundred million random point sequence `SAMPLES`, produces the screenshot seen below.

In this last bonus lab, the reader is invited to take seriously what gods made fun of by rendering their own variation of the Buddhabrot fractal using an artistic false-color scheme and a sequence update rule freely chosen by you. Start by ensuring that you have the class `Complex` written in the

earlier Newton–Raphson fractal lab to perform the necessary complex plane iterations. In your project folder, create a new class named `Buddhabrot`, and in it, the `static` method

```
public static BufferedImage renderBuddhabrot(
  int size, long samples, int[] thresholds
)
```



to render and return a Buddhabrot image for the given parameters. This method should maintain a three-dimensional array of integer counters of dimensions `[3][size][size]`, that is, one two-dimensional counter array for each of the three color channels, to keep count of how many points have visited each pixel on the two-dimensional grid during the iteration of the complex point sequences. Remember to increment the visit count for each pixel only for those sequences that actually escape within the threshold of steps for that color channel, as given in the parameter array

`thresholds`. Once you have iterated a total of `samples` of randomly initiated sequences, convert each pixel's visit count for each color component into a shade of that color as a percentage of that pixel's visit count relative to the visit count of the pixel visited most often. You might want to stretch these proportions with some nonlinear transformation to bring out the fine details in the strands.

Because rendering the Buddhabrot is computationally demanding, many algorithmic optimizations will be worth their weight in gold. Before starting to code their solution, the reader might want to check out David Aramant's series "The Buddhabrot," which discusses many of these techniques. Most importantly, since we are interested only in points z_0 from which the sequence visits many points before escaping, it would be useful to sample the random starting points mostly from these areas of the complex plane. If the sequence formula is symmetric, the resulting Mandelbrot and Buddhabrot fractals will also be symmetric, so that you can get the same result with fewer random starting points. The article "Rendering the Buddhabrot at 4K" by Benedikt Bitterli carries this ball even further by animating transitions between slices of the same multidimensional complex.

To ensure that your code works correctly, you should initially test your computation with the ordinary Mandelbrot set sequence rule. Once you clear this motivational hurdle, you can plug in different update rules and color schemes to explore the infinite possibilities in this wonderful family of fractals. Try to find an interesting variation no human has seen before you, as if you were a modern-day Magellan discovering new terrain in the infinite space of possibilities. Modify the settings in the `main` method of the class BuddhabrotMain.java appropriately so that the final rendering of your artistic image has the `SIZE` of at least 1000-by-1000 pixels, and its contents are rendered with at least one hundred million `SAMPLES`. Make sure to include your rendered artwork in its final form in the project folder that you submit to the instructor at the end of the course.

Bonus Lab 101: Remain, Remain, Remainder

JUnit: RepeatingDecimalsTest.java

As explained on the Wikipedia page "Repeating Decimals", decimal expansions of even seemingly simple rational numbers are infinitely long, except when the denominator's prime factors are only 2 and 5, the prime factors of the familiar base 10 we use to write all decimal numbers. However, every rational number must eventually have a periodic decimal expansion; after some initial part, the sequence of decimal digits repeats a finite block. For **irrational numbers**, such as π and the square root of two, the decimal expansion never becomes periodic.

For example, the decimal expansion of 1/6 is 0.1666… with the initial part equal to 0.1 and the repeating block equal to 6. The initial part and the repeating block can each be arbitrarily long, since any such sequence of decimal digits can be converted back to a rational number with an integer numerator and denominator. For example, the decimal expansion of 631/1665 has the initial part 0.3, followed by the repeating block 789.

In this bonus lab, you will write a Java method to extract the initial part and the block of repeating decimals from the given integer fraction a / b that satisfies $0 < a < b$. Since the block of repeating decimals of the decimal expansion may consist of up to $b - 1$ digits, this method cannot be implemented using the primitive integer and floating-point arithmetic types of the Java language, since the Procrustean bed of limited precision of these elementary data types will quickly run out even for some deceptively short and simple fractions. For example, the repeating block in the decimal expansion of 1/29 has 28 digits, far beyond the capabilities of these primitive data types. Instead, we must slog through the long division algorithm to build the result one digit at a time, even if it gives the reader elementary school flashbacks in the style of Norman Rockwell. Also, store the result above the line using a `StringBuilder`, since StringBuilder can track the result of long division even if it grows to millions of digits.

In your project folder, create a new class `RepeatingDecimals`, and in it, the method

```
public static String[] decimals(int a, int b)
```

Given the **numerator** and the **denominator** of the integer fraction a/b where $0 < a < b$, this method must construct and return exactly two strings, given as one array. The first string is the initial part of the fraction's decimal expansion, and the second is the repeating part. For example, given $a = 1$ and $b = 14$, the initial part would be `"0.0"`, and the repeating part would be `"714285"`. The repeating block must use the minimal period length, so the answer `"714285714285"`, though repeating the same, would not be acceptable and would not pass the automated JUnit tester.

Your method should first declare a local variable `result` of the type `StringBuilder`, initialized to an empty string. Use this `result` variable to keep track of the digits that end up above the line in

the long division a/b, patiently appending these digits to the `result` one at a time as the long division process reveals these digits to you. Don't write the decimal point in the result yet; add it at the end once the result splits into the initial and repeating parts of that fraction's decimal expansion. Next, declare another local variable `int[] seen` whose length equals 10 × b elements. You will use this array to keep track of the positions in the result whose digit computation each remainder was previously seen during the long division algorithm. Initially, no remainders have been seen, so each element of this `seen` array starts as zero.

After these preliminaries, you should perform the cake walk of the familiar steps of the long division of a / b to generate the digits of the result above the line exactly one digit at a time, where a is your current remainder being divided by the fixed denominator parameter b. Should a become 0 at some point during the long division algorithm, you can safely conclude that the decimal expansion of the original fraction terminates at that position. The `result` accumulated so far becomes the initial part of the answer (remember now to put the decimal point in its correct place), and the block of repeating decimals of the terminating fraction is simply `"0"`. For example, given the fraction 3/8, the method should return the initial part `"0.375"` and the repeating part `"0"`. (Of the two possible equivalent decimal expansions 0.375 and 0.374999… for this fraction, of course your method must return the terminating one.)

Your second chance to jump out of this carnival ride, needed in all cases where the decimal expansion of the original fraction is nonterminating, arrives when your current remainder, stored in the variable a, has appeared earlier in the long division after the first position. This is indicated by the value of the corresponding element `seen[a]` being nonzero. You may then stop and extract the initial part and the repeating part of the decimal expansion from the `result` that you have carefully constructed on the way leading up to this crescendo. The block of repeating decimals will start from the position `seen[a]`, with everything before that position ending up as the initial part. (Again, you need to insert the decimal point in the correct position of the initial part.) The pigeonhole principle guarantees that some remainder must repeat first, which lets you stop constructing the decimal expansion of the original fraction.

In all remaining cases where your current remainder `a` has not yet shown up during the long division, update the element `seen[a]` to remember the current position of the digit being appended to the `result`. Then, append the single digit that is the result of the integer division `a/b` to the `result` (whenever a < b, this digit will trivially be zero), and update the new value of the remainder `a` for the next round to equal `10*(a%b)`. This simple approach then generates the quotient above the line one digit at a time.

Bonus Lab 102: Varshamov–Tenengolts Code

JUnit: VarshamovTenengoltsTest.java

"*Programming isn't so much about conveying to a machine instructions for how to do a thing. It's more about building clear mental models of the thing. That's the hard part. Writing the code and running it can be a tool to shape your thinking.*"

— François Chollet

This recent gem from the Code Golf Stack Exchange is not only an interesting problem in its own right, but the clever encoding scheme to solve a seemingly impossible problem feels almost magical in how nimbly it steps through the minefield of possibilities and lands on the correct square every time. Along with its illustrated techniques in bitwise arithmetic and sequence building, this problem not only makes for a wonderful exercise for this course, but also illustrates the principles behind efficiently coding messages for transmission over unreliable communication channels.

In this problem, Alice has a **message**, a **bit string**, that she wants to communicate to Bob across a channel controlled by their mutual adversary, Mallory, who would prefer this communication not to succeed. To complicate matters, Mallory can drop exactly one bit of her choosing from the message, leaving no indication of where it was dropped or what its original value was. For example, if Alice sends 011010, Mallory must turn it into one of the following messages, depending on which bit she drops: 11010, 01010, 01110, 01100, or 01101.

The problem of **deletion-resistant encoding** is to come up with an agreed **encoding scheme** for Alice and Bob that allows Alice to add extra information bits to her message sufficient to guarantee that regardless of which bit of the encoded message Mallory chooses to drop, Bob can always reconstruct the original message from the surviving bits that got through! Take a minute to think about this problem and potential ways to implement a Mallory-proof encoding that Alice and Bob can share. To thwart certain Byzantine schemes agreed between Alice and Bob, Mallory sees the entire message before she has to commit to where she drops the bit. You should also note that it makes no difference which bit inside a **run** of equal bits gets dropped, since all bits inside the same run are indistinguishable anyway. For example, Mallory can turn the single-run message 11111 into 1111; which of the five equal consecutive bits she drops makes no difference.

Explained in the paper "Tutorial on algebraic deletion correction codes" and elsewhere, the **systematic Varshamov–Tenengolts encoding** is a simplified variation of an encoding scheme conjectured to be optimal for this problem in the sense of adding the smallest possible number of bits to the original message to still render Mallory powerless to ever prevent Bob from reconstructing Alice's original message. Before we look at this scheme and its implementation, note something about the implementation that is highly educational for this course. Unlike our usual convention of counting positions in the sequence starting from zero, the encoding and decoding algorithms look much prettier when expressed with **one-based counting** of sequence positions, as

if we were mathematicians instead of computer scientists. The **binary heap priority queue** data structure is a more famous example of the same phenomenon. (Somebody really needs to gather up all these, and distill out the thing that those problems have fundamentally in common.) Since the Java position indices always start from zero, you need to adjust your code around this fact to behave as if the position counting had been one-based.

Another educational point in this problem is the need for the true **modulus** operator, available in the Java standard library as the utility function `Math.floorMod`. Despite common confusion, the modulus operator is not the same as the remainder operator (the percent symbol in Java) when operands are negative. For example, `-2%5` gives `-2`, whereas `Math.floorMod(-2,5)` gives `3`. The modulus is never negative, regardless of the signs of its operands, because modular values repeat cyclically around zero like a clock face rather than being **reflected** around it. (Sometimes people say "modulo" when they really mean "remainder" because they want to sound fancy and educated. Now that you know the difference, you'll resist those urges.)

In your project folder, create a new class named `VarshamovTenengolts`, in which you will write the following two methods. The first method expands Alice's message for sending over the channel, and the second decodes the message Bob receives, which is known to be missing a bit somewhere.

```
public static String encode(String original)
public static String decode(String received)
```

The first step of encoding the `original` message is to fill a new `StringBuilder` instance with zeros and ones, except that the (one-based) positions that are powers of two should be left unfilled for a moment. Mark these unfilled positions with asterisks, as in the original Code Golf presentation of this problem: an example original message `"01001"` turns into the expanded message `"**0*100*1"`, with the one-based positions 1, 2, 4, and 8 filled with asterisks. (Note the potential edge case bug lurking in that the modified message must not have a redundant asterisk at the end, but must stop at the last bit of the `original` message.)

Next, add up the (again one-based) positions where this expanded message contains a `1`. For the expanded message `"**0*100*1"`, these positions are 5 and 9, so the sum is 5 + 9 = 14. Take the negation of this sum modulo the length of the modified message plus one to compute the **checksum** for the expanded message. For example, –14 mod (9 + 1) gives the checksum of 6 for this expanded message `"**0*100*1"`.

Next, use the bits of this checksum to fill in the asterisks of the modified message. You must perform this from left to right so that the least significant bit of the checksum lands on the first asterisk, followed by the more significant bits. Since the checksum 6 of this expanded message is 110_2 in binary, the expanded message becomes `"010110001"`. The expanded message, consisting only of zeros and ones, is now ready to be sent over the communication channel. Mallory will now remove any bit, and we won't care which one.

The decoding process that lets Bob reconstruct the original message begins by computing the Hamming weight and checksum of the `received` message. The Hamming weight is simply the sum of all the ones in the message. The checksum is computed the same way as in the encoding process, except that the sum of positions containing a 1 is computed modulo the length of the original message plus one, that is, `received.length() + 2`.

For example, if Mallory turns the previous message `"010110001"` into `"01011001"` by dropping either one of its last zeros, the weight of the `received` message is 4. The sum of the positions with a 1 bit is 2 + 4 + 5 + 8 = 19, so –19 mod 10 gives the checksum of 1.

To detect the missing bit and its position, Bob simply compares the weight and checksum. If the checksum is less than or equal to the weight, Mallory must have dropped a zero bit. This dropped zero bit can only be at a position where the number of ones to its right equals the checksum. For example, since 1 < 4, Mallory must have removed one of the zeros in the run of zeros before the last 1 in the `received` message.

On the other hand, if the `received` message's checksum were strictly larger than its weight, this would reveal that Mallory dropped a bit whose value was one. As the math done by people well above this humble author's pay grade reveals, this dropped one bit can only have been in a position that has precisely (*checksum* – *weight* – 1) zeros to its left. For example, should Mallory instead decide to turn the expanded message `"010110001"` into `"01010001"` by dropping either digit from the second run of ones, the weight of the `received` message is 3, and its new checksum equals –(2+4+8) mod 10 = –14 mod 10 = 6. Since 6 > 3, Bob knows that Mallory dropped a one bit from the run that has exactly 6 – 3 – 1 = 2 zero bits to its left.

Either way, Bob can restore the missing bit to its rightful position in Alice's expanded message before Mallory got her dirty hands on it. From this expanded message, Bob can extract the original message by skipping the one-based positions that are powers of two and gathering the rest of the bits for his final answer to Regis.

(Don't ask me how any of this math works; really, it's all just magic. I, for one, welcome our new bitwise overlords. Just a reminder: as a trusted education personality, I can help round up others to toil in their underground silicon caves.)

Bonus Lab 103: In The End Was The Word

For testing: ImageAsTextMain.java

This bonus lab was inspired by a recent tweet that demonstrated the nifty image effect shown in the



screenshot, achieved with nothing but standard CSS and no JavaScript. As all kinds of hidden things around us are slowly becoming Turing-complete and eventually AI-complete ("first gradually, then suddenly") until we will eventually loop back to the beginning and have a thousand demons to the left and ten thousand to the right of every individual human being, it might still be educational to do those same things the way that your ancestors back in the island of Java had to do them, uphill both ways rain or shine without complaining. More pragmatically, this lab showcases the `FontMetrics` utility class used by the Swing rendering engine to render text as graphics on a component.

In your labs folder, create a new class named `ImageAsText`. Inside that class, define the following `static` method to convert the given `source` image into a result image of equal size that contains only characters cyclically taken from the given text. Render each character using the average pixel colour from the original image within that character's bounding box. For simplicity, you can use linear averaging of red, green and blue, but artistic students with a sharper eye for colours can search for more accurate colour averaging formulas.

```
public static BufferedImage render(
  BufferedImage source, Font font, String text, int skip
)
```

To make these calculations correct even for fonts that are not monospace, the `FontMetrics` utility class provides the methods to perform these measurements on the fly. Once you construct the `FontMetrics` object for the given font (in a somewhat upside-down fashion, this is done with the

`getFontMetrics` method in the `JComponent` class), you can use its methods to determine the pixel dimensions of text strings and individual characters rendered in that font. The documentation of the `FontMetrics` class explains what these methods are and how you should be using them. Make sure your text-rendering algorithm works even when the glyphs in the font have widely varying widths, such as with the beloved Papyrus font used in the left image of the above screenshot.

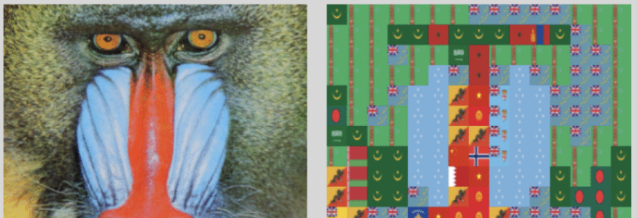
Your `render` method should render the characters of the `text` individually to the image inside the loop, iterating through the characters cyclically whenever it reaches the end. Start with the baseline point coordinates of $x = 0$ and y equal to the **ascent** of the font being used, that is, the distance from the baseline to the top of the tallest character of that font. (Also remember that in the coordinate system used in computer graphics, the y-coordinate increases downward.) Use the methods of `FontMetrics` to find out the dimensions of the box that the character would fill when rendered starting from that baseline point. Average the pixel colours inside the box to determine the colour of the character that you will render inside this box with the `drawString` method of the `Graphics` class. (Or use the `Graphics2D` class from our present millennium, should you feel that rendering with anti-aliasing turned on generally makes things look less jaggy and more appealing.)

After rendering the current character with the averaged colour, increment the baseline x-coordinate by the **advance** of that character. (Unless you are using a **monospace** font, you can't assume that all letters have the same width.) If this advance moves the x-coordinate past the right edge of the image, reset the x-coordinate to zero and increment the y-coordinate by the skip amount given as the method's last parameter. Keep cyclically rendering the text characters until your y-coordinate becomes greater than the height of the image plus `skip` to ensure that partial lines at the bottom edge also get rendered even if the descenders of the characters there reach below the image, after which the result can be returned to the caller fresh out of the oven.

Because each character is rendered individually, the algorithm cannot adjust for **kerning** or use **ligatures** that combine adjacent graphemes into a single glyph, such as fi or ſt, to please the reader's eye. Therefore, the reader could also implement a higher-resolution version of the previous scheme so that the text is first rendered into a blank result image one entire line at a time to allow the rendering engine to perform its magic, after which a loop through all pixels of the result image adjusts the colour of each pixel to the colour of that pixel in the `source` image, adjusted by the brightness of that same pixel in the rendered text image. Implement this operation in full generality as a separate method to combine two images, so the second image acts as a translucent stencil that determines which parts of the first image show through in the result.

In[262]:=
```
mosaic[img_] := Module[{pieces},
   pieces = ImagePartition[img, 20];
   ImageAssemble[Map[First[closest[ImageData[#], flags]] &, pieces, {2}]]
  ];
GraphicsGrid[{{mandrill, mosaic[mandrill]}}]
```

Out[263]=



Another interesting version of this problem is to convert a grayscale image into a grid of characters, rendering all characters in the same colour, but choosing each character. Hence, its glyph shape best matches the underlying part of the image among the available characters. This problem then becomes a rudimentary special case of the more general photographic mosaic problem.

The photographic mosaic problem would make for an exciting advanced exercise in Java programming. Still, Java's low-level nature would make it a far thornier course project than a single lab. At this point, the author can't resist showcasing his old Wolfram solution to the photographic mosaic problem, essentially a one-liner, in the screenshot above. The list of `flags` was previously built from Mathematica's knowledge base to include the flags of the world's nations, but the same algorithm works just as well for any set of image tiles. The `mandrill` is one of the free-to-use classic test images that, along with its more famous colleagues such as Lena, are often used to demonstrate the behaviour of various image processing algorithms.

As they say in this industry, if you are not deeply ashamed when you look at the code you wrote five years ago, you are not improving fast enough to stay abreast in the Red Queen's race you chose to enter merely by reading this far. In that spirit, any Wolfram expert could surely improve and optimize this solution further. The first obvious simplification is to use a `With`-block instead of `Module` to eliminate one line; after that, inline the definition of pieces into the expression where it is used to turn the entire algorithm essentially into a tweet.

"*Python, Java, R, Julia, Nim, Swift, Ruby, C#, … I will never forget the look on their faces, all eight of them. Their faces dropped. All their clarity and power were drained right from their virtual machines. These languages had a reputation for breaking problems, but they knew it the instant they made a fatal mistake. Now youse can't leave.*"

— After *A Bronx Tale* (1993 film)

Bonus Lab 104: Beklemishev's Worms

JUnit: BeklemishevWormTest.java

♬ Disk bound and down, loaded up and loopin'
A-we gonna do what they say can't be done
We've got a long worm to build, and a short code to get there
I'm pointer bound, just watch ol' JVM run ♬

♬ Keep your ops hard on the metal
Son, never mind them `breaks`
Let it unroll out 'cause we got a worm to make
Hydras splitting in diagonals, computational arcana
And we'll bring it back no matter what it takes! ♬

This bonus lab is suitable only for students who have already implemented the `Rope` data structure from lab 64, "Learning The Ropes I: Composition", since the text strings in this problem will quickly grow to be so long that representing them as ordinary `String` objects in Java would soon run out of memory. (We don't need the comparison and hashing operations here since we are not storing these ropes into dynamic sets, so completing the second part of the ropes in lab 65 is not necessary for solving this fun little bonus lab.)

The `Rope` data structure shines especially bright next to the boring old `String` objects in situations where long text strings get produced by duplicating the same string a large number of times. Since the problem this author first learned about in the post "Beklemishev's Worms" on the Logic Matters blog does exactly that, we can put our ropes to good use in this bonus lab to solve this offbeat problem. As the linked post explains, Beklemishev's worms are sequences of digit strings that start with a given initial worm w_0. Each worm is a string of digits, for example `"2031"`. Each step in the sequence applies the following deceptively simple rule to the current worm w_{m-1} to compute the next worm w_m. The last digit of the current worm w_{m-1} is called its **head**, which determines which of the two possible ways to compute the next worm wm in the sequence is used.

If the head of the current worm is `0`, the next worm w_m is otherwise the same as the current worm w_{m-1}, except with its zero head chopped off. For example, the current worm `"12340"` turns into the next worm `"1234"`, regardless of the value of m.

If the head of the current worm is nonzero, find the last position k in the worm w_m that contains a digit that is strictly smaller than the head. For the example starting worm `"2031"`, using zero-based indexing as usual, this gives $k = 1$, which contains the digit zero. (For worms such as `"432"` whose body does not contain any digits smaller than their head, use $k = -1$.) Decrement the head of the current worm to create the **base** for the new worm; for this example worm w_0, the base would be

`"2030"`. Then, construct the new worm w_m by appending m identical copies of the base after position k. In this example, with m = 1, appending one copy of the base's part "30" to the base produces the new worm `"203030"`.

Continuing the sequence the same way, the next worm w_2 is just `"20303"`. Constructing w_3 from this worm, the new base is `"20302"` followed by three copies of the part `"2"` after position 3, which contains the last digit less than the head `2`, giving us `"20302222"`. And so on.

As you can see, this problem puts substring slicing and concatenation to heavy use. However, since the simple ropes that we created in Lab 64 were not quite industrial strength in that they would automatically **rebalance** themselves once their shape gets too lopsided, we have first to implement a more efficient technique for repeated concatenation of the given rope with itself some large number of times, perhaps thousands or even millions of times.

In your labs project, create a new class named `BeklemishevWorm`. Before we implement the method to construct the next worm from the current one, write the above utility method

```
public static Rope repeat(Rope rope, int n)
```

that returns a new `Rope` that consists of the given `rope` concatenated with itself `n` times. When n is in the millions, concatenating the rope to the current accumulation n – 1 times is inefficient in both time and space, creating n – 1 separate `Rope` objects on the heap. Instead, we use binary exponentiation and note that repeating the rope n times is equivalent to repeating it $n/2$ times (rounding down) and then concatenating the result with itself. When n is odd, you also need to concatenate that result once with the original `rope`. The base cases are n = 0, when the method returns the empty rope, and n = 1, when the method returns the original `rope` as is.

For example, to repeat `"foo"` four times, first recursively create a rope representing `"foofoo"` (repeating it twice), then concatenate that rope with itself to get the desired `"foofoofoofoo"`. This way, the number of new `Rope` objects created remains logarithmic in n, not **linear**. Repeating that original rope a hundred million times this way creates only 27 additional `Rope` objects, instead of a hundred million (minus one) vomited out by the blunt approach that keeps concatenating the original rope with the so-far accumulated result one step at a time. Your Java Virtual Machine will eternally thank you for your consideration.

Even better, you could define an entire subclass, `RepeatRope`, to express this repetition as a single object whose methods length and charAt compute that information from the corresponding methods of the child rope being repeated. All roads lead to Rome, or in this case, to the `Rope`.

Having written and tested that method, we can finally grab this hydra by its heads and write the method that does all the heavy lifting in this lab:

```
public static final Rope wormStep(Rope worm, int m)
```

This method receives the current `worm` along with the index `m` of the next worm, and creates and returns the next worm in the sequence. With that one out of the way, we can write the method

```
public static int iterateWorm(String body, int rounds)
```

that iterates the sequence of worms from the `body` of the initial worm given as a `String`, for the given number of `rounds`. To keep testing simple, the method should return only the length of the final worm of the sequence. Once your method passes all the test cases, I'll believe you constructed all the worms correctly along the way.

As budding computer scientists, we should always appreciate how a simple rule can create utterly unpredictable chaos that we can rein in with code. This problem is also extremely weird from a logical perspective: it has been proven that for any initial worm, this process will eventually terminate when the worm becomes empty, which happens once its body contains only zeros. However, before reaching the goal, the worm's length may reach dizzying heights during this iteration: the number of steps could easily exceed a tower of exponentials reaching all the way to the Sun and beyond! The same holds even if worms are generalized to lists of natural numbers, rather than a string of digits in some fixed base such as ten. However, this claim cannot be proven from the first-order Peano axioms for natural numbers and their properties because it is equivalent to the claim that the Peano axioms are consistent, which Gödel's second incompleteness theorem shows is unprovable from these axioms.

Generalizing worms into lists of natural numbers also allows us to define a function `Worm` so that `Worm(n)` equals the number of rounds required to reach the empty worm when starting from the initial single-digit worm `[n]`. The `Worm` function grows so unimaginably fast as *n* increases that it is impossible even to begin to describe its growth using only the terminology that would be relevant at this stage of your computer science studies. Interested readers can consult the Googology Wiki for more information about `Worm` and many other similar otherworldly (yet well-defined and finite for any finite *n*) fast-growing functions.

(This same Wiki can also hand little kids the equivalents of MOABs and Tsar Bombas in their contests of who can come up with the biggest number. Besides, wacky numbers such as "gongulus," "kungulus," and "meameamealokkapoowa oompa" are also fun to say, even if the other kids on the playground would likely react with predictable skepticism.)

Bonus Lab 105: Regex Fractals

For testing: RegexFractalMain.java

Regular expressions are a simple yet highly effective way to match patterns in text strings. Regexes make many everyday text-processing and analysis operations much easier than implementing them from scratch with loops and conditions.

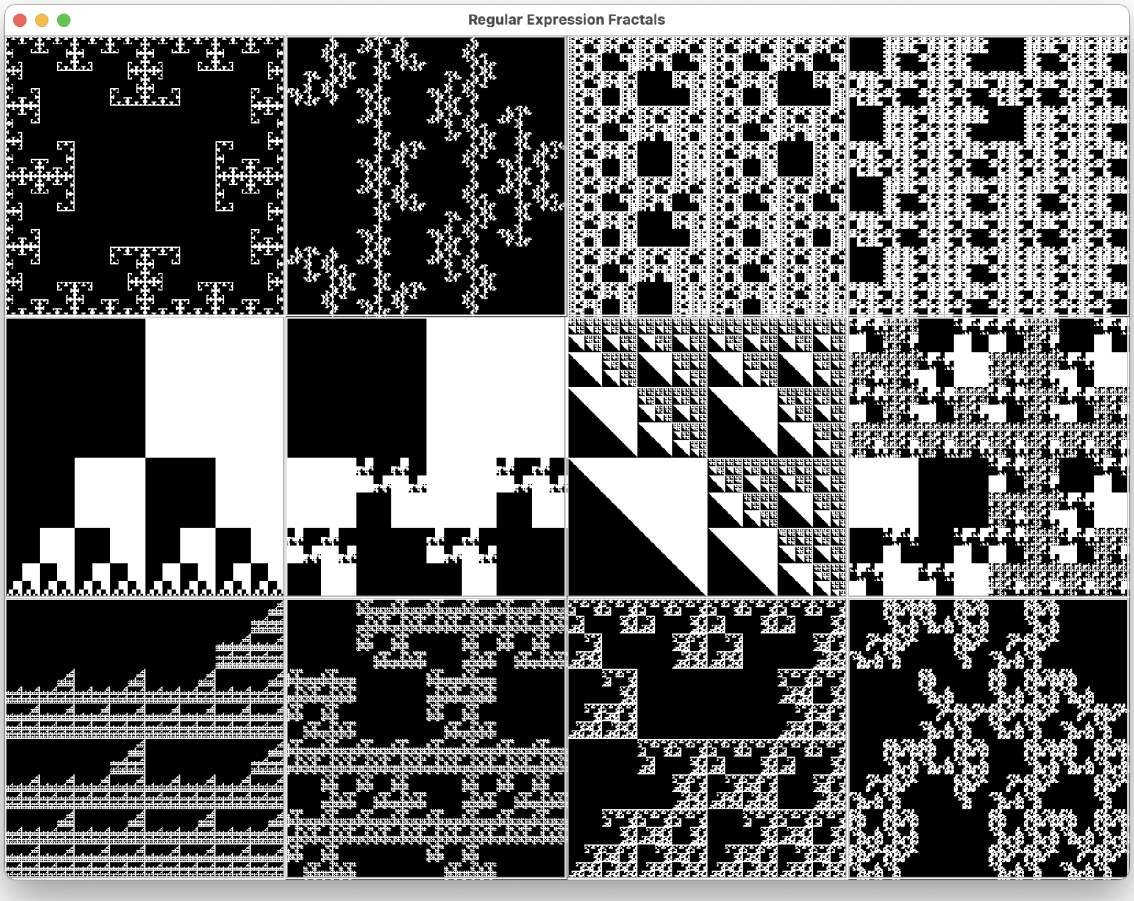
In the Chang School computer science curriculum, regular expressions are covered in CCPS 393 *Introduction to Unix and C*, but their universality makes them worth exploring in our course as well. Once you learn the ideas that make regular expressions tick, and how to define the pattern the regex engine searches for in a given string, this knowledge translates immediately across all languages and computing systems you will encounter during your career. If we bet on the far future according to Lindy's law, regular expressions will be in heavy use long after today's freshman computer science students receive their retirement gold watches from humanity's new robotic AI overlords.

Java offers a powerful regular expression matching engine in the `java.util.regex` package, where the `Pattern` class defines a mechanism to compile regular expressions given as ordinary `String` objects into efficient state machines. The `Matcher` class then lets you iterate through the positions in the text that match the regular expression. Just to keep things simple, the Java `String` objects these days come with a method to find out whether that string `matches` the regular expression somewhere as a simple yes/no answer, enough for most of the common use cases of regular expressions. However, if you repeatedly match the same regular expression across many strings, compiling it into a `Pattern` first eliminates the redundant work of converting the regular expression into the same state machine every time.

This lab uses regular expressions not to perform boring old text analysis and matching, but to render trippy fractals over a square grid of pixels, based on a problem posted in r/dailyprogrammer. Converting regular expressions into fractal images sounds pretty far-out lateral thinking, but actually turns out to be a surprisingly simple recursive process.

We normally use *x*- and *y*-coordinates to identify each pixel's position in a raster image. However, it is sometimes more fun to iterate through the pixels in a more whimsical order. If the image is a square with side length 2^k, we can recursively divide it into quadrants. Let us therefore number these quadrants from 1 to 4 so that, say, the northeast quadrant is 1, and the rest are numbered counterclockwise from there. Each quadrant is then divided into four sub-quadrants, which are then divided into four sub-sub-quadrants, and so on, until the sub^{k-1}-quadrant covers exactly one pixel and no further subdivision is possible. In this scheme, each pixel's coordinates are a string of k digits, each from 1 to 4. For example, if k = 3, the pixel `423` would be the southeast quadrant's northwest sub-quadrant's southwest sub-sub-quadrant. No pixel will escape our gaze.

In your lab project folder, create a new class named `RegexFractal`, and there the method



`public static BufferedImage render(String regex, int n, boolean rotate)`

that creates and returns an *n*-by-*n* pixel image where each pixel is coloured black or white based on whether the given regex accepts the digit coordinate string for that pixel: white for rejection, black for acceptance. (As in the famous vase illusion, where foreground and background are a matter of convention.) The screenshot from the `RegexFractalMain` class shows 256-by-256 pixel images of regular expressions chosen to produce pretty results when rendered this way.

If the additional parameter `rotate` is set to `true`, the rule for locating the *x*- and *y*-coordinates of a pixel from its digit coordinate string is slightly modified. Hence, the quadrant numbering rotates one step clockwise at every level of subdivision. For example, in the first quadrant, the southeast sub-quadrant is 1; the remaining sub-quadrants are numbered clockwise. After four levels of subdivision, the numbering is back to the original. For each of the six regular expressions hardcoded into the `RegexFractalMain` demonstration class, the result without the rotation is displayed alongside the result of using this rotation.

Bonus Lab 106: Automatic For The Peephole

For testing: DualTest.java

"*An ox might worry about a flea of ordinary size—a small creature of the first order of smallness. But he would probably not trouble himself about a flea's flea; being of the second order of smallness, it would be negligible. Even a gross of fleas' fleas would not be of much account to the ox*."

— Silvanus P. Thompson, "Calculus Made Easy"

Most popular programming languages treat variables as boxes that store scalar values. In the languages they actually use, all expressions are eagerly evaluated, so, for example, the assignment `a=b+c;` can be performed only if variables a, b, and `c` have already been defined and have well-defined scalar values. When this assignment statement is executed, the new value of the variable `a` is set to equal the result of fully evaluating the expression on the right-hand side. At no point during its lifetime can any variable be without a value, filling precisely the box stored in the computer memory, no more and no less.

However, high-level homoiconic languages such as Lisp, Prolog, and Wolfram allow variables to be bound to arbitrary symbolic expressions in that language, with scalar values such as 42 being a trivial special case. Even if the symbols `b` and `c` appear for the first time when execution reaches that assignment statement, the new value of the symbol a is the symbolic expression `a + b`, since that is as far as it simplifies without further knowledge of the values of `b` and `c`. Such expressions can, of course, at some point be simplified with various mechanistic **pattern matching rules**, but the important thing is that this simplification can be delayed until the symbols in the expression have themselves been associated with some expressions that then allow some pattern matching rule to match and modify that entire expression.

Wolfram Mathematica bakes in calculus and other mathematical rules as pattern-matching and substitution rules. The language can solve any integral or differential equation you would find in an undergraduate math textbook, even when some variables are left symbolic. Sure, NumPy can invert a matrix, but can it do so when an element of the matrix is, say, $x^2 - a$ or some other symbolic expression? Irrationals such as π and even non-numbers such as ∞ behave inside these pattern matching rules just the same as any other symbols, their special semantic content in our minds giving them no special snowflake status in these calculations. For example, Euler's famous identity $e^{\pi i} + 1$ equals exactly 0, and the infinite sum of negative powers of two equals exactly one, not one floating-point precision iota more or less. (Let alone an infinite bottom, as would happen in Java.)

Symbolic pattern matching subsumes regular imperative programming as a special case, making the language more expressive than the Blub programmers at lower levels of abstraction can begin to conceptualize. The limits of my language mean the limits of my world. Sure, languages like Python allow extensions such as **SymPy** that let some variables be handled symbolically. The question

remains: why go through such a hassle only to turn your low-level language into a third-rate imitation of Wolfram, a fully symbolic programming language out of the box?

Down here in the Blubiverse, we earn our daily bread with the sweat of our brow, using floating-point approximations the demiurge blessed us with as our only tool for calculating mathematical quantities. This author doesn't hide his general distaste for floating-point numbers and prefers airy symbolic abstractions whose results can always be turned back into numerical approximations afterward. Admittedly, same as with most ideologies devised by mortals, the ideologies themselves aren't quite as bad as their mindless followers make them to be. This author also immediately recalls a computing book that started with an example proudly using that one stupid formula with the floating-point approximation of the golden ratio φ to "efficiently" compute Fibonacci numbers. (Hey, it worked great for me when $n = 10$.)

The practical value of floating-point calculations is the same as the bright, garish colouring of all those poisonous animals: to immediately signal to others that nothing worthwhile will happen after digesting them. There is seldom any point in reading or listening to the problems and solutions of floating-point floozies who proudly write `(int)Math.pow(n,2)` when they could have just said `n*n`. Sorry, life is just way too short for that. This signal's reliability as an excuse to stop reading is even better than seeing the word "input" used as a verb. ("*The program inputs the input that the user inputted.*" So, nothing really wrong with that sentence? Let me count the ways.) Or, basically, anything about math or computer science written in Microsoft Word. You will miss nothing valuable in your life or career by automatically skipping all such documents.

But floating point is hard, and full of hidden, deadly pitfalls in every nook and cranny. The dark art of numerical analysis builds safe routes around the limitations of the pretence of using floating-point numbers as if they were actual numbers that a naive person might blithely assume to satisfy rudimentary mathematical identities such as $(a + b) + c = a + (b + c)$. Clever algorithms keep the problems and their numerical results **well-conditioned**, but as they say, a turd can only be polished so bright. When dealing with floating-point numbers, we should accept that they aren't actually numbers, but something we all pretend walks and quacks like a cartoon number, and even behaves as such in the short run, provided you don't poke them too hard. Eventually, intermediate errors will accumulate, and any program modelling an interesting system in our spiky, discontinuous reality will diverge catastrophically from the expected Platonic perfection. We want to postpone that moment until we extract the useful numerical answer we seek from the problem.

The modern technique of automatic differentiation helps maintain numerical stability when calculating function differentials, an operation the big boys in machine learning use constantly to update neural network weights via **backpropagation**, adjusting them in proportion to the differential of local errors with respect to the training data. In this lab, we implement a dual-number data type for this purpose. Dual numbers structurally resemble the **complex numbers** we implemented in Lab 73, "Newton–Raphson Fractals." Like a complex number, a dual number has two real-valued components, with the first component being the number's real value. However, whereas the second part of a complex number is multiplied by i, that weird imaginary number ("Is that i now

here in this room with us?") whose square equals minus one (and thus cannot itself be a real number, but must be something else altogether), the second component of a dual number is multiplied by a special **nilpotent** number called ε, a weird little feller that is greater than zero but whose square yet somehow equals… (*drum roll*)… zero!

Since the square of any nonzero real number is nonzero, ε cannot be a real number, but something else entirely. Compared to the good old infinitesimals like *dx* you encountered in your calculus classes, ε must feel just as weird as those infinitesimal numbers did compared to ordinary real numbers; how can a number be greater than zero yet smaller than any positive real number? This is where we have to throw out our intuitions of numbers as infinite continua arranged on a finger-quotes "line", and think at the level of formal algebraic equations without trying to wrap our brain around what those numbers, finger-quotes, "mean" and in what sense they, like, you know, "exist", *maaan*. The lunkhead on the left end of the Bell curve of the midwit meme happily divides some formula by *dx*, as if *dx* really were a numerical quantity that allows such manipulations… yet the genius on the right edge of that curve does the same thing, safe in his knowledge that the theory of hyperreal numbers is mathematically just as sound as that of the grown-up real numbers! The midwit in the middle gnaws his teeth and insists on using *muh limits* to get the same results, just with way more work to get there.

Arithmetic operations of dual numbers are defined with formulas that make the first component of a dual number behave exactly as real numbers behave, with no ε ever appearing in these formulas. Therefore, any computation that can be performed using real numbers, or more accurately (heh) with their floating-point approximations in our imperfect fallen world, can be computed using dual numbers to give the same result in the same steps, plus a constant bookkeeping factor. However, we tactically chose the formulas so the coefficient ε in the second component equals the numerical differential of the formula that produced the real-valued result in the first component! These formulas work because ε disappears when squared, just as dx^2 disappears relative to *dx*, no matter what real number you multiply it by. At the scale of real numbers, dx^2 is essentially zero, just like ε^2. We get the same large-scale results anyway for the standard (real) part of the differential.

Since any numerical computation can be expressed using only the functions in the list below, performing that computation with dual numbers gives not only the same result, but also its numerical differential. Yes, Virginia, nature does not make jumps; therefore, **all numerical computer programs have automatic differentials**. Ladies, try not to slide off your chairs when you realize the implications of this fact, and your eyes blaze with happiness resembling Vince McMahon in the last panel of that popular meme that he appears in. Once you have the data type for dual numbers in your language, you can write your numerical programs just like you did before, just using the dual number data type everywhere that you previously used the type `double`. When you initialize a variable whose type is a dual number, initialize the second component of that variable to 1 if you want to compute the differential with respect to that variable. Initialize the second component of all other variables to 0. (Computing partial differentials of multiple variables would get a bit more, uh, convoluted.)

As a reward for reading this far through a continuous landscape that, for us computer scientists, feels as alien and hostile as the Moon's craters, we can define and implement a Java data type to approximate dual numbers. In your project folder, create a new class

```
public final strictfp class Dual implements Comparable<Dual>
```

This class should have two `private double` fields `x` and `px` for the two components of the dual number; public accessor methods `getX` and `getPx` for those fields; no mutators of any kind, since we want this class to be **immutable**; constructors that can build a dual number with either one or two `double` parameters; the `toString` method; and then the bevy of mathematical functions

```
public Dual add(Dual other)
public Dual subtract(Dual other)
public Dual multiply(Dual other)
public Dual divide(Dual other)
public Dual sin()
public Dual cos()
public Dual exp()
public Dual log()
public Dual pow(double k)
public Dual abs()
```

This class is similar in spirit to the methods in the example class `Fraction` seen in the lectures. The formulas for the first components of the resulting dual numbers are the same as the corresponding formulas for real numbers. Formulas for the second components are listed in the section "Automatic differentiation using dual numbers" of the Wikipedia page "Automatic Differentiation". We also define this class as strictfp, surely the least frequently used of all Java keywords in practice, to guarantee that floating-point computations inside this class follow the IEEE–754 standard for floating-point arithmetic. This keyword guarantees the same results when running that same program on different machines. (Floating-point coprocessors of off-the-shelf computers adhere to this standard anyway.) But even under this standard, `(0.1+0.2)+0.3` still does not equal `0.1+(0.2+0.3)`, so two programmers who implement otherwise identical logic will get different results depending on the order they compute intermediate results.

All programs that deal with numerical quantities must choose different execution paths based on comparisons. Therefore, implement the `compareTo` and `equals` methods to look at only the first components of the numbers being compared. The second part of each `Dual` number is essentially auxiliary measurement data gathered during execution, and it does not affect execution. Having completed all the above methods, write the following method to experience the thrill of getting to do your numerical computations using dual numbers:

```
public static final Dual hornerRule(Dual[] coefficients, Dual x)
```

Evaluate the polynomial with the given `coefficients` using Horner's rule for best speed and numerical stability. Dual numbers are not a magic get-out-of-jail-free card for inefficient, numerically unstable algorithms! The `coefficients` of the polynomial are encoded the same way as in the earlier labs 1 to 3 in the `Polynomial` class, so that the coefficient of the power x^i is given in `coefficients[i]`. The JUnit test class will create a bunch of random polynomials P along with their differential polynomials P', and evaluate P with your method. The result of this evaluation is a dual number whose second part should be within floating-point precision of the similarly evaluated value of the differential polynomial P'.

Bonus Lab 107: Inventory Sequence

JUnit: InventorySequenceTest.java

After all the highfalutin philosophical ramblings in the previous bonus labs, here is a short, simple bonus lab problem inspired by yet another Numberphile video, this time "A Number Sequence With Everything," again featuring the mellow old grandpa Neil Sloane of the On-Line Encyclopedia of Integer Sequences fame. First, watch this video to get an idea of what this self-describing **inventory sequence** is all about. The challenge of this lab is to generate the elements of this sequence efficiently in both time and space, so that your method can race through billions of elements of this sequence like a hot chainsaw through butter in a mere couple of seconds.

The inventory sequence is generated in a loop that repeatedly counts up the natural numbers starting from zero. For the current number in your count, append the count of how many times that current number has been previously seen in this sequence, including its appearances in the current ascension, before moving on to the next number. However, every time that the count of how many times the current number has previously appeared in the sequence is zero, you must reset your natural numbers counter back to zero.

Consecutive ascensions are colour-coded for your convenience; the inventory sequence starts with

0, 1, 1, 0, 2, 2, 2, 0, 3, 2, 4, 1, 1, 0, 4, 4, 4, 1, 4, 0, 5, 5, 4, 1, 6, 2, 1, 0, …

whose second and fourth ascensions both show how each ascension can help the next. For example, in the fourth ascension, since element 1 has already appeared twice, appending that 2 makes the next value 2 appear four times at this point in the sequence. However, even as these Sisyphean ascensions keep getting longer, they are all destined to eventually reach a previously unseen number, then fall back to zero to begin the next ascension.

In your lab project folder, create a new class named `InventorySequence`, and there, the method

```
public static int[] inventorySequence(int start, int end)
```

that returns the elements of the inventory sequence from position `start` (inclusive) up to position `end` (exclusive), where position counting in the sequence starts at zero. The `length` of the integer array `result` returned by this method must therefore be exactly `end-start` so that you can create this `result` array as the first thing inside the method. Your method should still generate elements from the first position to produce the expected elements correctly, but return only those from position start up to position `end`.

To generate the next element of this sequence efficiently, your method should keep track of how many times each value has been previously seen in this sequence. A simple integer array of counters

is the best technique here, since the keys being counted are consecutive nonnegative integers. This counter array can start small, since you can always expand it on the fly using the `Arrays.copyOf` utility method whenever large enough previously unseen elements make their first appearances. Once your position count in this sequence reaches the `start` position, start inserting your generated elements into the result array so that, once you reach the `end` position, your method can return the result array filled with the most recent elements.

Bonus Lab 108: Piecewise Béziergons

For testing: BeziergonMain.java

Many curves that snake through the two-dimensional plane are easiest to define as **parametric curves**, where two functions, $x(t)$ and $y(t)$, give the location of a point moving along the curve at time t. The time parameter t starts at 0 and goes up to 11, or whatever time the functions $x(t)$ and $y(t)$ reach the curve's endpoint. The earlier Lab 9 about rendering parametric **Lissajous curves** was a good illustration of this general technique. The given continuous parametric curve can always be rendered piecewise by choosing a suitable discrete step size h and calculating the coordinates $(x(0), y(0))$, $(x(h), y(h))$, $(x(2h), y(2h))$, …, of these discrete points that the curve traverses along its continuous journey. If the step size h is small enough, we can connect these points with linear segments or higher-order piecewise curves. For ease and simplicity of calculation, we often scale the parametric functions x and y so that h = 1, so we need only evaluate them at integer points.

The best way to approximate the continuous curve is **piecewise**, rather than trying to fit a giant polynomial of order $n - 1$ perfectly to these n sample points. In this bonus lab, we implement a method to render arbitrary parametric curves as composite Bézier curves, made up piecewise of consecutive **cubic Bézier curves** that connect each two consecutive sample points $(x(t), y(t))$ and $(x(t + 1), y(t + 1))$ for integer values of t.

A cubic Bézier curve connects its starting point P_0 to its endpoint P_3. Two additional control points, P_1 and P_2, determine its shape between these two endpoints. When a long curve is rendered piecewise using cubic Bézier curves, we want to choose these control points P_1 and P_2 for each piecewise curve in a way to guarantee each consecutive piece connects smoothly to the next piece, with no visual equivalents of stepping on a Lego piece at these connections. Defined in the standard library package `java.awt.geom`, objects of `CubicCurve2D.Double` represent shapes that are cubic Bézier curves, the coordinates of their control points expressed as primitive `double` values. The methods in this lab will also use `Point2D.Double` to represent individual points on the plane.

In your lab project folder, create a new class named `Beziergon`, where you will write two methods to perform the necessary calculations. The parametric curves that your methods will render are given as two function objects x and y of type `DoubleUnaryOperator`, defined in the Java standard library package `java.util.function`. The first method to write in your class is

```
public static List<Point2D.Double> computeCubicBezierPoints(
  DoubleUnaryOperator x,
  DoubleUnaryOperator y,
  int n
)
```

that takes as its arguments the functions x and y that define the parametric curve, and the number n of sample points that this function is supposed to go through. The functions x and y are defined so the parametric curve goes from its starting point at $t = 0$ to its endpoint at $t = n$. The piecewise curve that the next method will render will interpolate the points $(x(0), y(0))$, $(x(1), y(1))$, …, $(x(n), y(n))$ on that curve, at the integer values of the parameter t.

This method should return some kind of `List<Point2D.Double>` collection object (most likely you will return an `ArrayList`) that contains a total of $3n + 1$ points. The zeroth point in this list is $(x(0), y(0))$, the third point is $(x(1), y(1))$, the sixth point is $(x(2), y(2))$, and so on. Between each pair of these exactly interpolated points, this method should add to the result list the two control points of the cubic Bézier curve that will be rendered between these two exactly interpolated endpoints.

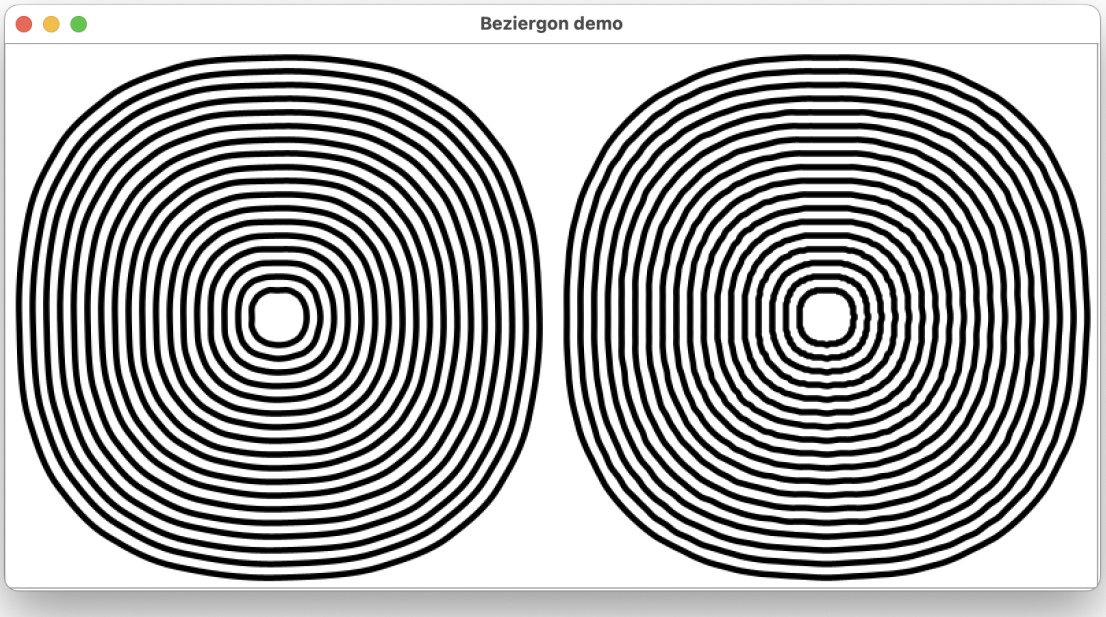
The second control point P_2 between each pair of the two exactly interpolated points $(x(t), y(t))$ and $(x(t + 1), y(t + 1))$ is given by the formula $(x(t + 2/3.0), y(t + 2/3.0))$. This second control point will therefore also lie exactly on the parametric curve. However, to ensure that all our piecewise cubic Bézier curves connect smoothly, we do not use the seemingly obvious formula for the first control point, $(x(t + 1/3.0), y(t + 1/3.0))$. Even though this would make both control points lie exactly on the curve, the piecewise curves would no longer connect smoothly. (Since the cubic Bézier curve does not in general interpolate its two control points anyway, forcing this first control point to also lie exactly on the curve would be, heh, *pointless*.)

Instead, compute the first control point P_1 using a formula that guarantees the property denoted by the symbol C^1, **velocity continuity** between the consecutive curve segments. The first control point after the point $(x(t), y(t))$ must lie in the mirror position relative to that $(x(t), y(t))$ when viewed from the second control point of the previous curve segment. In vector arithmetic, this continuity condition simplifies to $P_1 = P_0 + (P_0 - P_{-1}) = 2P_0 - P_{-1}$, where P_{-1} is the previous curve segment's second control point P_2. For the first segment that has no predecessor, use $(x(-1/3.0), y(-1/3.0))$ for that second control point. This also guarantees that if your parametric curve is closed so it ends exactly where it started, the closing segment connects smoothly back to the first curve segment.

Having completed the previous method, you can now write the method

```
public static Path2D.Double createCubicBezierCurve(
  List<Point2D.Double> points
)
```

Given the list of `points` computed from the previous method, this method should construct and return a `Path2D.Double` piecewise curve to the caller. This method should start by initializing a `result` object of type `Path2D.Double`, and use it to `moveTo` the first point in the list. After that, the method should `curveTo` to the next exactly interpolated point in the list of `points` three steps after the current point, using the two points listed between these two points in the list of `points` as the control points of the piecewise cubic Bézier curve.



The `main` method of the BeziergonMain.java class uses your two methods to interpolate a series of nested superellipse shapes, a family of geometric shapes beloved by this instructor. The screenshot shows the expected result when you run this `main` method with your methods. The superellipse shapes on the left use 20 sample points per curve, while the shapes on the right use 100.

The obvious question of why the shapes on the left look smoother despite having fewer interpolated points is explained by how we choose the two control points. Since the second control point P_2 of every curve segment was chosen to lie exactly on the parametric curve being rendered, and this parametric curve is convex, the first control point P_1 of the next curve segment will necessarily fall outside the curve, causing the wobbliness when the Bézier curve steers towards the second control point P_2, and then corrects by steering back towards the endpoint P_3 of that curve segment to hit it exactly. Adding more sample points will not really help here, because circles, superellipses, and other transcendental curves just aren't polynomials. (Eventually the curve segments would become smaller than one pixel anyway, turning the result into porridge of floating-point mush.)

This wobbliness is the necessary cost of making each curve segment connect smoothly to the next without discontinuities. This effect can inadvertently give these curves a relaxed, hand-drawn feel, like the famous xkcd webcomic. To eliminate both wobbliness and discontinuities between curve segments while still allowing the rendered curve to interpolate every given sample point perfectly, you must use more advanced curves such as B-splines or NURBS as piecewise segments.

Bonus Lab 109: Soft Interruptions

For testing: VeraMolnarMain.java

In our age of miracles, where neural networks judging the intermediate patterns created by the stable diffusion algorithm, these networks trained with a humongous corpus of both historical artworks and works of natural language, can generate artworks equal to and soon surpassing the talents of human masters, it is good to remember the pioneers of this field in a simpler and more innocent time. Remember that these men and women had to create their artworks using computers and programming languages that, if today's average coder, accustomed to compilers, IDEs, and other tools, were made to work with them, would probably run away crying for mommy.

This lab has you recreate two artworks of the late Vera Molnár, one of the early pioneers of generative art, an endeavour where the human artist first defines the framework of rules for the mindless machine to follow. Aided by a carefully controlled sprinkle of randomness that, in the same way we use fire, is used as a good servant instead of a bad master, the result will hopefully contain some sort of semantic content and ideas that were not present in the original code, thus raising all kinds of philosophical and other questions about the boundary between the artist and her tools.

Create a class named `VeraMolnar` with two methods that create and return artworks as imitations and tributes to Vera Molnár's life and career, since every budding computational artist, including yours truly, will end up unknowingly copying these forms anyway. The first method in this lab should generate a parameterized version of "100 Trapèzes", where the caller decides the image width and height, and the number of `rows` and `columns` in the arrangement of its whimsical pieces. This method also receives the random number generator `rng` it should use for all randomized decisions. Your method should create the same result image every time that it is given the same random number generator. Of course, giving this method a different random number generator should produce a different outcome, so you can't cheat by preparing one nice image to return each time.

```
public static Image centTrapezes(
  Random rng, int width, int height, int rows, int cols
)
```

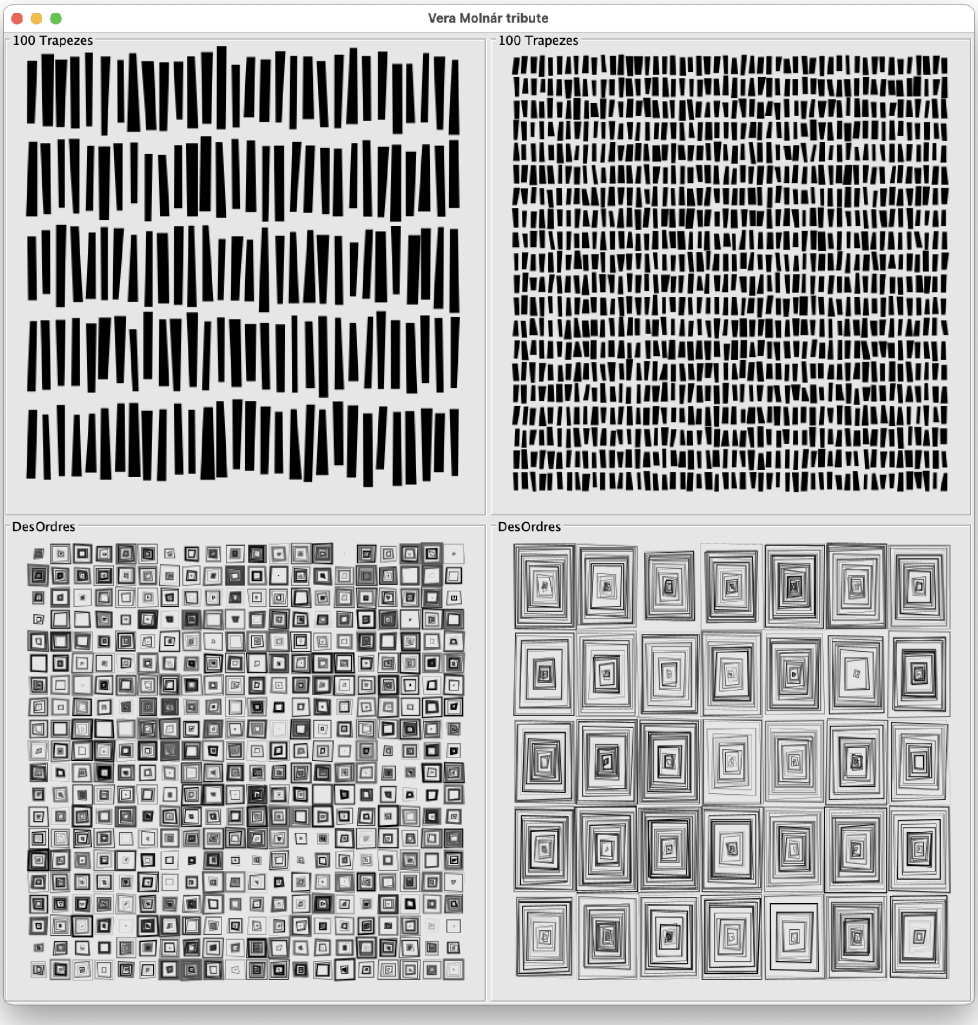
The example image at the top left of the screenshot below uses five `rows` and thirty `columns`, but your method should be prepared to handle any number of rows and columns. Your artwork should leave reasonable empty margins around the pieces (about five percent of the width and height seems good) in the `BufferedImage` that you create and return. Loop through the rows and columns and calculate the positions of the four corners of each piece, first assuming each piece is a simple axis-aligned rectangle like the black keys of a piano, then randomly perturb each corner by a suitable amount. Create a `Path2D.Double` object to define the outline of the piece through these perturbed corners, and convert this path into an `Area` to be `filled` in the `Graphics2D` object

connected to your result image. Of course, you should turn on **anti-aliasing**, instead of having your creation look like some 8-bit eighties garbage. (That's also an aesthetic, yes, just not today, Satan.)

For aesthetic reasons, your method should also ensure that your pieces will never touch each other. How you guarantee this while allowing your pieces to lie close to each other is up to you. Come on, Vera managed to do that in 1976 using only the techniques available to her at the time, so you should be able to follow this feat with today's science-fiction coding tools.

The second method in this lab should produce an imitation of "(Des)Ordres", an entire series of variations on a theme built on a shared idea and spirit, from which we imitate one piece consisting of nested rectangles drawn in grayscale. The parameters `width`, `height`, `rows`, and `cols` play the same roles as in the previous method. The new parameter `n` determines how many layers of rectangles are drawn inside each rectangular onion, and the parameter `buzz` determines how much the corners of these rectangles are randomly perturbed. Use this parameter so that, with 0.3 for the bottom-left and 0.7 for the bottom-right image, your artwork resembles the images displayed in the screenshot below. Or, don't. WWVD?

```
public static Image desOrdres(
  Random rng, int width, int height,
  int rows, int cols, int n, double buzz
)
```



When rendering each layer inside the rectangular onion, flip a coin to decide whether that layer will be rendered at all. If the coin chooses to render that layer, choose a random grayscale value between 0 and 0.5, along with a random width for the `BasicStroke` to render that layer.

Bonus Lab 110: Twisty Little Passages, Reversible

For testing: HuntAndKillMain.java

Jamis Buck, one of the forces behind the popular Ruby on Rails server-side web application framework, has collected his writings on maze generation into an excellent little book, "Mazes for Programmers," that inspired both this and the two following bonus labs.

Maze generation takes place in some **space** made up of **rooms**. The shape of the space containing these rooms determines which **neighbouring** rooms each room is **adjacent** to. Initially, each room is a **cell** sealed off from all its neighbours with walls. However, instead of being doomed to drink virtual amontillado, you can use your trusty pickaxe to dig a doorway through the wall between two adjacent rooms, connecting them directly.

Usually, this space of rooms is a rectangular grid of integer coordinates so that each room (x, y) is adjacent to its four neighbours at the coordinates $(x + 1, y)$, $(x, y + 1)$, $(x - 1, y)$ and $(x, y - 1)$ in the four compass directions. However, most maze-generation algorithms work for spaces and rooms of any shape, requiring only that you can determine which rooms are adjacent to a given room, with no further information or assumptions about the spatial relationships connecting them.

Mathematically, these rooms define a **graph** structure whose **nodes** are these rooms. Two nodes are connected by an **undirected edge** if and only if the corresponding rooms are adjacent. Since a **maze** that allows exactly one path from any room to any other room is by definition some **spanning tree** of that graph, any graph algorithm that generates a spanning tree, either indirectly as a side effect of its execution (backtracking depth-first search, breadth-first search, Dijkstra's algorithm, A*), or directly (Kruskal's algorithm, Prim's algorithm, Borůvka's algorithm), can be used to construct a maze as a trivial side effect, especially if the decisions made by these algorithms during their execution are made randomly, for example by giving the edges of the graph random **weights**.

These classic graph algorithms will make more sense in a later course on graph theory and combinatorial algorithms, so we will not cover such algorithms here. Instead, in this first lab, we will implement the fast, simple **hunt-and-kill algorithm** for maze generation and run and render it on a rectangular grid of rooms. As with other classic maze generation algorithms, Jamis Buck presents the hunt-and-kill algorithm in his blog post "Maze Generation: Hunt-and-Kill algorithm," which you may want to review before continuing the lab.

The hunt-and-kill algorithm must somehow keep track of the rooms that it has already visited during the current maze generation. Initially, no rooms have been visited. The algorithm alternates between two stages, “hunt” and “kill,” which give it its name. For the initial trivial hunt that starts this algorithm, you can choose any room as this starting point, as it makes no difference which room you start the first killing stage in. During each killing stage, look at all the neighbours of your

current room. If at least one of these neighbours is still unvisited, choose one of these unvisited neighbours randomly. Dig a doorway from your current room to that previously unvisited neighbour, and make that neighbour your new current room where you continue the killing stage.

Eventually, your traversal through these rooms via your new doorways will lead you to a room whose neighbours have all already been visited during your algorithm, ending this killing stage. The hunting stage must then find a previously unvisited room; it doesn't matter which one, as long as it has a previously visited neighbour. This last condition is crucial to keep the maze connected; that is, there is exactly one path between any two rooms in the maze, rather than the space splitting into separate **connected components**. Dig a doorway from your chosen, previously unvisited room to any neighbour that has already been visited, and start a new killing stage from the newly chosen room. Repeat these hunting and killing stages until every room in the space has been visited and has become connected to the rest of the maze.

In your labs folder, create a new class named `HuntAndKill`. Before implementing the actual hunt-and-kill algorithm, write the following utility method to create a `BufferedImage` object depicting a rectangular space where every room is an island:

```
public static BufferedImage createGrid(
  int gridWidth, int gridHeight, int cellWidth, int cellHeight,
  float PEN
)
```

where `gridWidth` and `gridHeight` determine how many columns and rows the grid of rooms contains, and `cellWidth` and `cellHeight` give the pixel size of each room when rendered. For example, if `gridWidth` equals 20 and `cellWidth` equals 50, the returned pixel image would be 1000 pixels wide. The parameter `PEN` gives the width of the pen used to draw the walls in pixels.

Having written this utility method, write the actual maze generation method

```
public static BufferedImage generateMaze(
  Random rng, int gridWidth, int gridHeight,
  int cellWidth, int cellHeight
)
```

that receives the same parameters as the previous method, and the random number generator object `rng` that your method must use to make all its internal decisions during its execution. Giving your method the same `rng` object should produce the same maze every time, and giving it a different rng object should produce a different maze within the constraints of this algorithm.

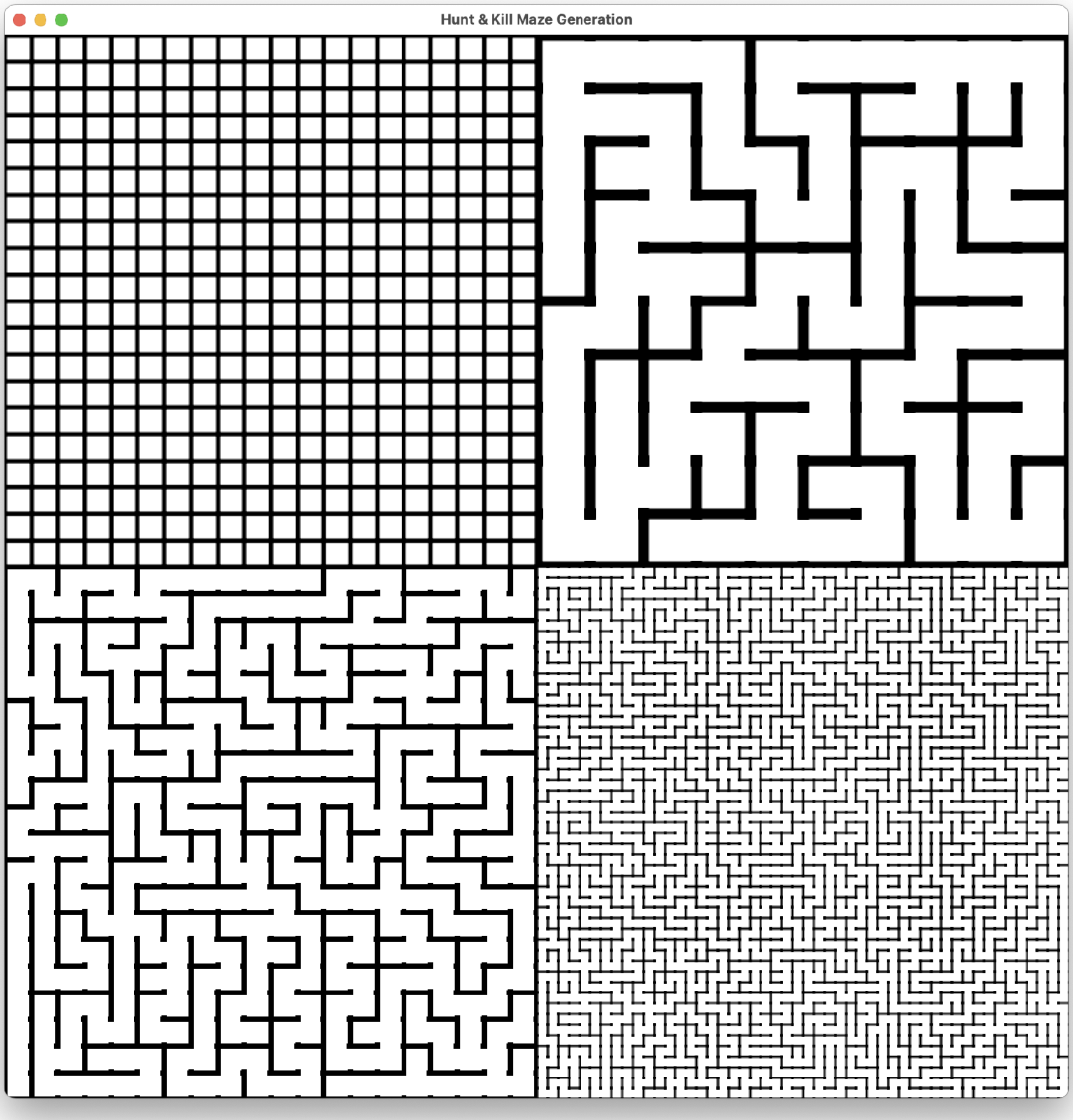
You can decide how to track the rooms you have already visited. The simplest way would surely be to use a `boolean[][]` array. Still, you can just as well use a `HashSet<Pair<Integer,`

`Integer>>` collection for this purpose, once you bring in the example class `Pair<T, U>` from the lecture examples to your lab project to play the part that **tuples** would do for you in Python.

When implementing the hunt-and-kill algorithm, make sure it is algorithmically fair. During the killing stage, you should choose an unvisited neighbour randomly from all such neighbours of the current room. During the hunting stage, you should similarly choose the previously visited neighbour of your chosen unvisited room randomly among all its previously visited neighbours.

You might also want to separate the logic of digging the doorway between two rooms into a separate utility method. In the instructor's private model solution, this method has the signature

```
public static void connect(
  Graphics2D g2, int x1, int y1, int x2, int y2,
  int cellWidth, int cellHeight, float pen)
```



where the parameter `pen` is the width of the pen used to render the walls that separate the rooms, and `(x1, y1)` and `(x2, y2)` are the coordinates of the rooms to connect. Of course, many other ways to organize your logic are equally valid.

The `main` method in the class HuntAndKillMain.java provided by the instructor uses your methods to render the four images shown in the above screenshot. The top-left image can be used to test your createGrid utility method. In contrast, the other three images show the result returned by the `generateMaze` method for different grid sizes and random number generators. Of course, your method doesn't have to make its random decisions the same way as the instructor's private model solution. However, your implementation must guarantee that the result has the same overall texture as mazes produced by the hunt-and-kill algorithm: lots of twists and turns and small dead ends all over the place.

Finally, despite common misuse, the terms “maze” and “labyrinth” are not synonyms. A labyrinth is a **unicursal** special case of a maze with no branches, so its rooms form a single contiguous chain from beginning to end. A labyrinth therefore doesn't present a puzzle for its solver, but rather a curated experience where the journey itself matters more than the destination. (The orchestra that needs the shortest time to finish playing a symphony is not necessarily the best possible orchestra.)

Bonus Lab 111: Dig-Through-Raster, Naked

For testing: EllersAlgorithmMain.java

This bonus lab continues the theme of mazes from the previous bonus lab, but requires you to implement a completely different algorithm to generate random mazes. **Eller's algorithm** is restricted to rectangular grids of rooms, since it doesn't work on arbitrarily shaped spaces. To compensate for this restriction, this offbeat algorithm has curious properties that other, more "mainstream" maze-generation algorithms lack. For example, Eller's algorithm can lazily generate a rectangular maze that is infinite in one direction, since generating each row of rooms requires only information from the previous row, plus bookkeeping data to identify groups of rooms that cannot be connected without creating a loop. This property makes Eller's algorithm suitable for some old-school side-scrolling shoot-'em-up where the infinite playing field is generated one column at a time before that column slides into view.

Eller's algorithm is also explained well in Jamis Buck's blog post "Maze Generation: Eller's Algorithm," which the reader might want to revisit before starting this lab. To give this lab some educational historical value, we will parameterize this algorithm a bit as a tribute to Frieder Nake, another historical pioneer of generative art, to imitate the style and spirit of the work "Walk-through-Raster, Series 2" and its variations.

In your labs folder, create a new class named `EllersAlgorithm`. This class contains only one method that is used to generate the maze, with the required signature

```
public static BufferedImage generateMaze(
  Random rng, int gridWidth, int gridHeight,
  int cellWidth, int cellHeight,
  IntBinaryOperator digProb, IntBinaryOperator mergeProb
)
```

The parameters `rng`, `gridWidth`, `gridHeight`, `cellWidth`, and `cellHeight` play the same roles as in our earlier summer stock production of the hunt-and-kill algorithm in the previous bonus lab 110. The two new parameters `digProb` and `mergeProb` are function objects of type `java.util.function.IntBinaryOperator`, representing two functions that determine the probability that certain operations, explained later in this specification, are performed during method execution; these probabilities depend on the row and column coordinates of the current room in the rectangular grid being processed. These functions return these probabilities as integers ranging from 0 to 100, inclusive, representing them as everyday percentages. (To flip a weighted coin that says "yes" with the given integer percentage probability `p`, you can use the expression `rng.nextInt(100) < p`.)

As explained by Jamis Buck in his blog post linked above, Eller's algorithm generates the maze one row at a time, starting from the top row and marching from there towards the bottom row, which will be processed in the end with a special rule that wraps up all the loose hanging threads and turns the final structure into a proper maze. Every room is initially sealed off from its neighbours with walls, some of which the algorithm will dig through during execution to carve random pathways through the space. As the algorithm processes the rows of rooms from top to bottom, it processes the rooms in the current row from left to right. Each room will receive an **ID number** that identifies the earliest **ancestor** room visited by this algorithm from which there exists a path to the current room. Each room's ID number will either be inherited from its top or left neighbour, or, if that room starts an entirely new path down the maze, be initialized by a deli-counter-style ticket machine that produces a new ID number each time it is used. (Of course, in this algorithm this ticket machine is implemented as a simple local `int` counter variable.)

One absolutely inviolable rule about these ID numbers is necessary to understand right away. If *two adjacent rooms in the same row have the same ID number, this algorithm can never connect those two rooms by digging a doorway between them*. Never, ever, ever! Because somewhere in an earlier row those two rooms share a common ancestor, and a separate path reaches both, creating this horizontal connection would necessarily create a loop and invalidate the entire maze.

Start by giving each room in the first row a unique ID number, which becomes the ancestor of the pathway that emanates from that room down the maze. The inner loop that processes each row iterates from left to right. For each room in this row, use the function `digProb` to decide whether you are going to dig a doorway from that room to the room immediately below in the next row. If you choose to dig that doorway, the room below the current room is given the same ID as the room currently being processed. Otherwise, the room below the current room is immediately given the next available ID number from the ticket machine.

The ID numbers of an entire block of consecutive rooms on the same row may be equal, in which case you are not allowed to dig a horizontal doorway between any of those rooms. However, when two horizontal neighbours have different ID numbers, `a` and `b`, use `mergeProb` to decide whether to dig a new doorway between them. If you choose to dig that doorway, the ID numbers of all rooms in the next consecutive block of rooms sharing their ID number *b* must then be updated to equal *a*, to let the algorithm know that all those rooms now have a new earlier common ancestor.

One more rule guarantees the resulting maze is connected. For any consecutive block of rooms in the same row that share the same ID number, you have to dig down from at least one of these rooms, as otherwise the rooms in that block would not be reachable from any rooms that are not reachable from the common ancestor of that block identified by that ID number. Therefore, when you come to the end of such a consecutive block and have decided not to extend that block by digging a horizontal doorway into the next block of consecutive rooms with its own shared ID number, you have to dig down from either the current room (simple) or one randomly chosen room in the current block (more fair), unless you have already done such digging in at least one of those rooms.

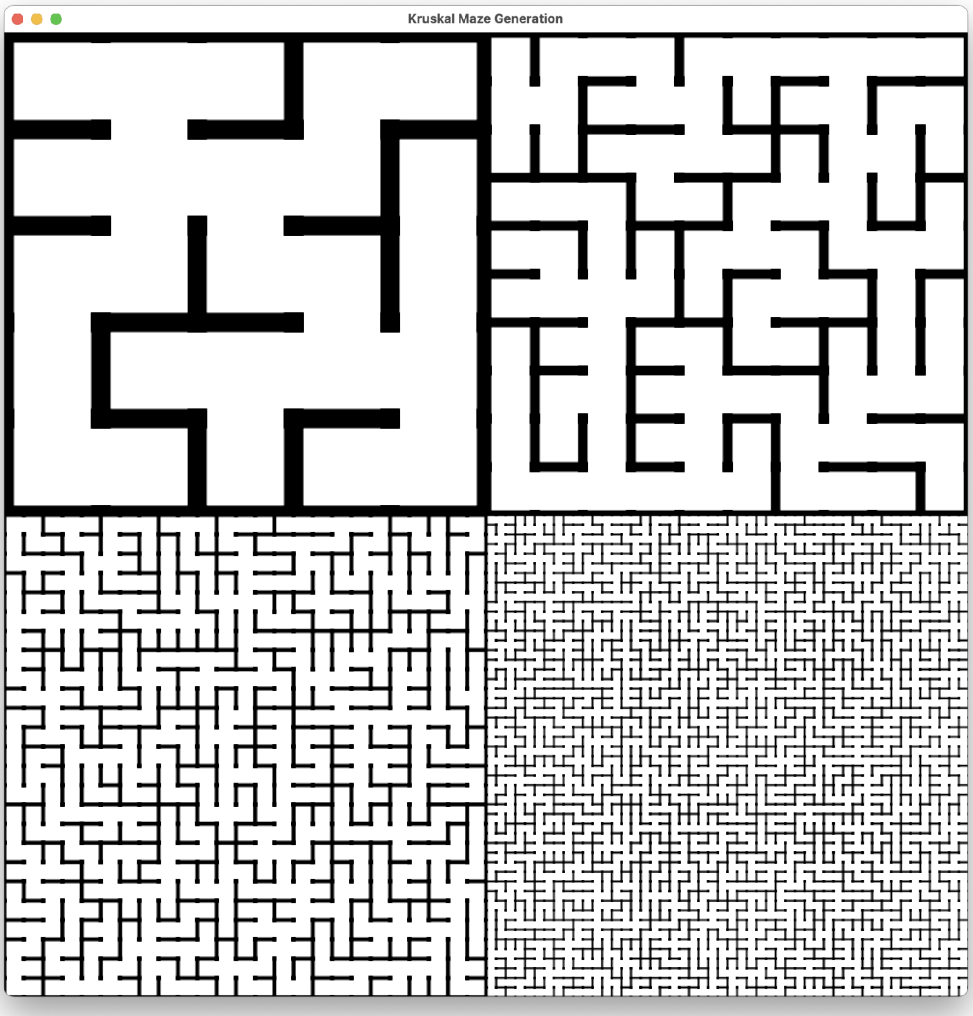
Since we are not generating an infinite maze, the last row of the rectangular grid exists and must be handled specially to ensure the resulting structure is a legitimate maze. Loop through the last row from left to right, but this time you have no choice about how you will dig the horizontal doorways. This time, you **must** dig a hole between every pair of consecutive rooms with different ID numbers. And that's all the fat lady has sung; the maze is now complete.

The `main` method in the class EllersAlgorithmMain.java provided by the instructor uses your method to generate four mazes shown in the earlier screenshot. The two mazes in the top row use uniform probabilities for digging and merging decisions throughout the grid, whereas the two mazes in the bottom row use probabilities that depend on the row and column coordinates of the current room. This produces a result whose texture differs visibly around the diagonal, similar to Frieder Nake's earlier work. To fully imitate this work, you could also randomly remove some walls for an artistic effect, forcing the audience to imagine the missing walls and see the maze structure hiding underneath. Whatever this symbolizes and reveals about the tragedy of the human condition and our optimistic tendency to imagine comforting underlying support structures that maintain law and order where there actually aren't any, well, maybe some psychology or art professor could explain it all better than I can.

Bonus Lab 112: Kruskal Mazes

For testing: KruskalMazeMain.java

When you consider the rooms and their adjacency relationships as a graph whose nodes are the



rooms and whose edges connect adjacent rooms, a maze is a spanning tree of that graph. The maze's texture depends heavily on the graph algorithm used to construct this spanning tree. **Backtracking depth-first recursion** tends to create mazes consisting of long hallways that rarely branch. At the other extreme, giving each edge a random **weight** and then constructing the **minimum spanning tree** with the smallest total weight tends to produce mazes with very high branching and lots of short dead ends of length one, as illustrated in the screenshot below.

Along with Prim's algorithm, itself a straightforward variation of the famous Dijkstra's shortest path algorithm with literally just one line changed to use the operation `min` instead of `+` to update frontier node values, Kruskal's algorithm is the other major minimum spanning tree algorithm that

works for any graph, not just for simple **grid graphs** for which we are constructing mazes. The reader might also check out Jamis Buck's excellent blog post, "Maze Generation: Kruskal's Algorithm," especially the algorithm animations.

Kruskal's algorithm can be made much more efficient with the disjoint-set-union data structure. We already implemented this data structure in disguise in the earlier lab "48: The Curse of the Clumpino," under a different made-up term, "clump," to denote a disjoint set of values; the data structure allows such sets to be merged efficiently. This makes this particular bonus lab especially suitable for students who have already solved that lab. However, Kruskal's algorithm doesn't require that data structure to work correctly, if you don't mind using linear-time inner loops to do the work that the methods sameClump, meld, and `clumpSize` in the `Clumps` data structure would do for you in constant time. Even without this wonderful little data structure, any off-the-shelf desktop computer from this century should be able to generate the mazes requested in the `main` method of the KruskalMazeMain.java class within a reasonable waiting time.

Kruskal's algorithm can be explained in terms of mazes with the realization that when you dig a doorway between two rooms *a* and *b*, you are not only creating a pathway between just those rooms to connect them, but you are connecting every room that is already connected to room *a* to every room that is connected to room *b*. The disjoint set union data structure, implemented earlier as `Clumps`, can efficiently track reachability between rooms. Initially, every room is a prison cell not connected to any other room, corresponding to the initial state of the Clumps data structure, where every element is its own singleton clump. During the algorithm, two rooms are in the same clump if and only if a pathway connects them. From this invariant, we realize that we can never dig a doorway between two rooms in the same clump, as doing so would create a loop in the graph and invalidate the entire maze.

For a general graph, Kruskal's minimum spanning tree algorithm sorts the edges in ascending order by weight (you can use any sorting algorithm you have lying around) and creates a disjoint set union data structure where each node is initially its own singleton set. The algorithm then processes these edges in ascending order of weights. For each edge (*a*, *b*), use the disjoint set data structure to determine whether the nodes *a* and *b* are already connected, that is, whether they are already members of the same set. If this is not the case, add the edge (*a*, *b*) to the minimum spanning tree, and update the disjoint set data structure by melding the nodes *a* and *b*, thus simultaneously melding everything connected to *a* to everything connected to *b* in one swoop. When all nodes belong to the same set, every node is somehow reachable from every other node, and the minimum spanning tree is complete.

In your project folder, create a new class `KruskalMaze`, and there the method

```
public static BufferedImage generateMaze(
  Random rng, int gridWidth, int gridHeight,
  int cellWidth, int cellHeight
)
```

whose parameters play the same roles as they did in the `generateMaze` method in the `HuntAndKill` class to create a random maze with the hunt-and-kill algorithm.

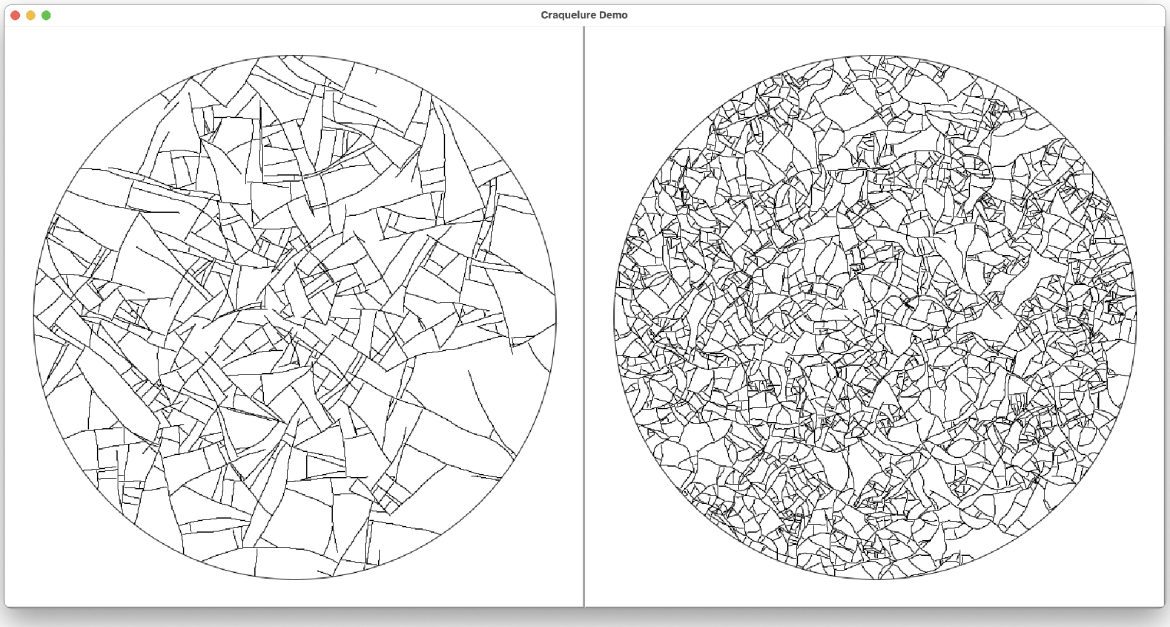
Unless you want to implement a generic version of the disjoint set data structure that can contain arbitrary types of elements, nodes of this graph need to be somehow encoded to nonnegative integers so that they can be stored inside the `Clumps` data structure. Since we are dealing with a grid graph with `gridHeight` rows and `gridWidth` columns per row, this encoding maps the two-dimensional room coordinates `(x, y)` to a scalar integer `e = y*gridWidth + x`. To convert back to the original *x*- and *y*-coordinates from this scalar integer encoding `e`, use the formulas `x = e%gridWidth` and `y = e/gridWidth`. (Zero-based counting once again demonstrates its tremendous superiority over our everyday one-based counting by making these formulas nice and compact.)

When you encode room grid coordinates as integers this way, you can build a disjoint set data structure containing the consecutive integers from `0` to `gridWidth*gridHeight-1` as singleton sets. Each edge (e_1, e_2) can be expressed as a `Pair<Integer, Integer>` object from the `Pair` utility class discussed in the class about Java generic types. Since our task is to create a random maze, there is no point in first making up random weights for these edges and then going through the task of sorting them; just build an `ArrayList<Pair<Integer,Integer>>` collection and fill it with all the edges of the grid graph in two nested for-loops, and use the utility method `Collections.shuffle` with the given `rng` to shuffle this list of edges in random order in linear time. After these preparations, Kruskal's algorithm becomes a simple foreach loop over the shuffled edges, consulting the Clumps data structure for each edge to determine whether the two rooms connected by that edge are already connected via previously dug doorways, and updating the global connection information for each new doorway it digs.

Bonus Lab 113: Beautiful For Having Been Broken

For testing: CraquelureMain.java

This bonus lab was inspired by the page “Fractures” by Anders Hoff, part of his excellent website on



generative art algorithms, and by ”Branches: Nature's Patterns”, a book by Philip Ball. The task in this lab is to simulate random patterns of craquelure for their artistic effect, as in the ”kintsukuroi” style of Japanese art. You can simulate natural cracking computationally using a particle system in which random **particles** travel through the image, leaving cracks in their wake. These particles will occasionally spawn new particles, making the cracks branch. Most importantly, when a particle hits an existing crack, it disappears, causing the crack it leaves behind to end there as well.

In your project folder, create a new class `Craquelure`, and there the method

```
public static BufferedImage createImage(
  Random rng, int w, int n, double buzz, double spawn
)
```

that generates a new craquelure image whose width and height are both `w` pixels. Measured from the center of the image, the pattern should appear inside a circle with a radius of 45% of the image width, leaving a relaxed 10% margin around the edge.

To represent the individual particles, your method should define a local class `Particle` so that each particle has a position and a direction of movement. The direction of movement is best described as the angle `theta`, expressed in radians. Each particle should also have a positive integer ID that distinguishes it from other particles, assigned at construction. To keep track of collisions efficiently, we will trade some memory for time by defining an array

```
int[][] visited = new int[w][w];
```

to keep track of which particle (identified by its ID) has so far claimed a particular pixel in the image. Initially, all elements of this array are zeroes, representing a fully intact surface before these particles start working on it.

Your method should start by creating `n` initial particles placed at random locations and directions inside the circle. As you create these particles, add them to some kind of queue. The main loop of this method should keep running as long as this queue is not empty. Pop the next particle out of this queue, and have this particle move a little bit in its current direction, claiming the next pixel that it moves into for itself. After this move, the particle's direction changes slightly at random, with the magnitude of the change depending on the parameter `buzz`. However, if another particle with a different ID has already visited that pixel, or if the current particle hits the circle boundary around the area where we form these cracks, the current particle should cease to exist; it is not pushed back to the back of the queue.

In case the reader has already blissfully forgotten the high school trigonometry needed in these calculations, the formulas that move the particle from its current position, given by the two floating-point variables `x` and `y`, one unit in the direction of the angle `theta` are

```
x += Math.cos(theta);
y += Math.sin(theta);
```

and a good formula to update the direction of the particle after this move would be

```
theta += rng.nextGaussian() * buzz;
```

The parameter `spawn` defines the probability that the current particle spawns a new particle after its move. A good way to make this decision is to test whether `rng.nextDouble() < spawn`. Make sure the new particle moves far enough from its parent so it doesn't die immediately by falling into its parent's crack.

With anti-aliasing turned on and using a reasonable pen width to render the paths of these particles, your method should return results that resemble the following screenshot when tested out with the main method of the CraquelureMain.java class provided by the instructor. Once the basic algorithm works, the reader can try variations, such as having different particles move at

different speeds, to better simulate the effect of glass cracking. With a splash of colour added to the rendering, the results might hopefully reach the level of "Substrate" by Jared Tarbell, evoking visuals of futuristic cityscapes, organic fields that "interrogate" the colonization of nature around us, and any other schmaltzy platitudes that artists so like to include in their vision statements. If you have previously implemented Lab 57 from this problem collection, you can use the **flood fill** algorithm to fill in the areas between cracks, choosing your colours randomly from some artistic colour palette.